

PITANJA I ODGOVORI ZA INTERVJU

Sadržaj:

SOLID principi.....	3
Design patterns.....	6
C# Teorija.....	19
Operatori.....	33
Stringovi.....	35
Propertiji.....	36
Polimorfizam.....	37
Delegati.....	39
Datatypes - tipovi podataka.....	40
Funkcije.....	42
Nasledjivanje.....	44
Strukture.....	45
Namespace.....	46
Interfejsi.....	47
Kontrola instrukcija.....	49
Klase i objekti.....	52
Nizovi.....	54
Enumeracije.....	55
Rukovanje izuzecima.....	56
Collection classes(klase kolekcija).....	57
Generics.....	58
API/REST API/SOAP.....	60
Strukture podataka.....	65
Sortiranje.....	76
Algoritmi.....	78
OOP, Koncepti OOP.....	80
Relacione baze podataka.....	87
SQL.....	89
NOSQL.....	99
Web.....	100
CORS.....	114
HTML.....	121
CSS.....	124
Tipovi dijagrama definisanih UML-om.....	126
Mreze.....	131
IKP.....	141
HTTP.....	155
Azure.....	167
Testiranje softvera.....	169
GIT.....	172
OSTALO.....	176
Literatura.....	182

cetiri osnovna principa OOP, solid principe, dizajn paterne, relacione baze sa primerima, sql upite, razliku izmedju liste i niza, hes mape

za prakse sta smo radili, rekao je ako imam neki problem da zapucam da li cu traziti pomoc lol

pretrage, algoritam za sortiranja hmm, da mu ispricam algoritam za ffaktorijal preko rekurzije

unit testiranje, white box black box

Sealed klasa

Interfejs i apstraktna klasa

za zadatak konkatenciju stringova a teorijski 40 pitanja ukupno, js i c#

Pitali sta je potpis (mislili su na polimorfizam to ni ja nisam znala da se to zove potpis), abstraktna klasa i

interfejs, abstraktna metoda i virtuelna, SOLID principe , REST, Entity, Repository Layer,

Async kako funkcioniše u java scriptu

enkapsulacija vs polimorfizam

method overloading

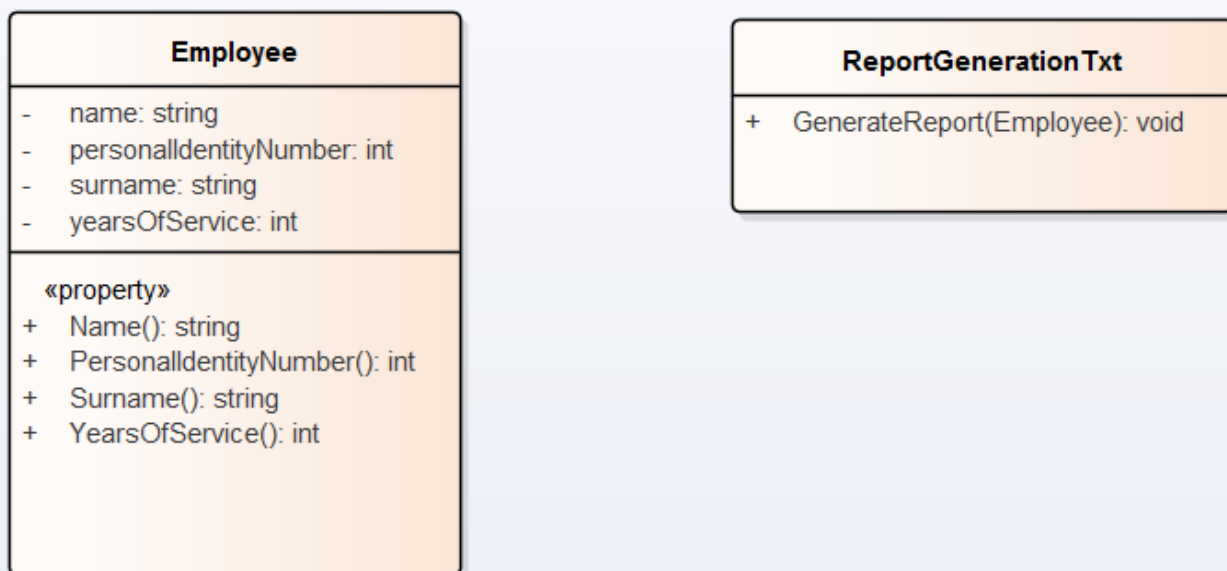
primarni ključ, normalne forme sta su cemu služe, tipovi joina

SOLID principi:

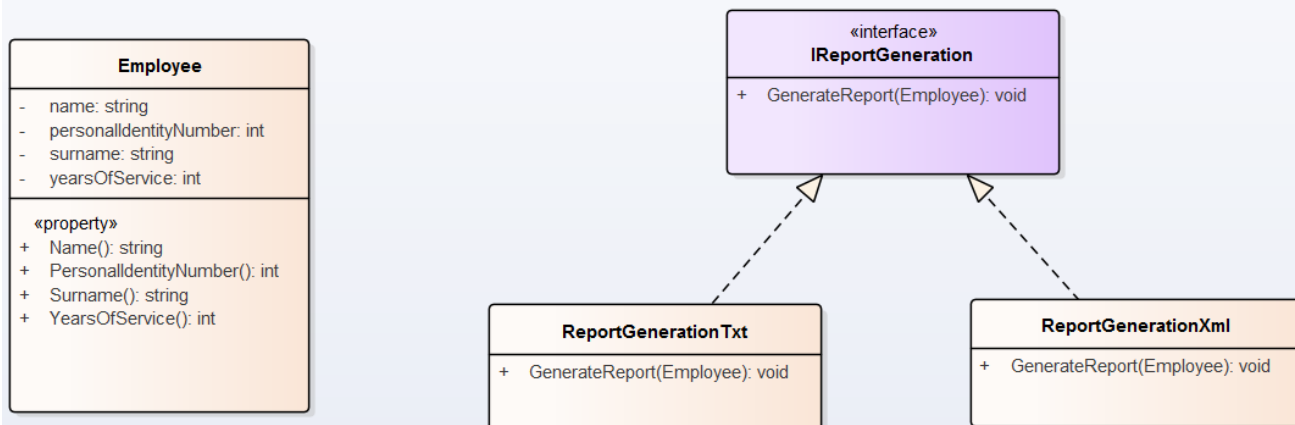
Koriste se za dizajniranje klasa.

1. svaka klasa radi jednu stvar
2. novi zahtev, nov kod, a ne menjati postojeći(napravi klasu, dodaj interfejs)
3. interfejsi imaju minimalni broj metoda(po 1)
4. kada nastavljas klasu, child klasa u potpunosti se ponasa kao bazna to je u kodu(base.Send(message))
5. zavisi od apstrakcije, ne on konkretne implementacije(pogresno bi bilo da list ima vezu 2 koraka gore)

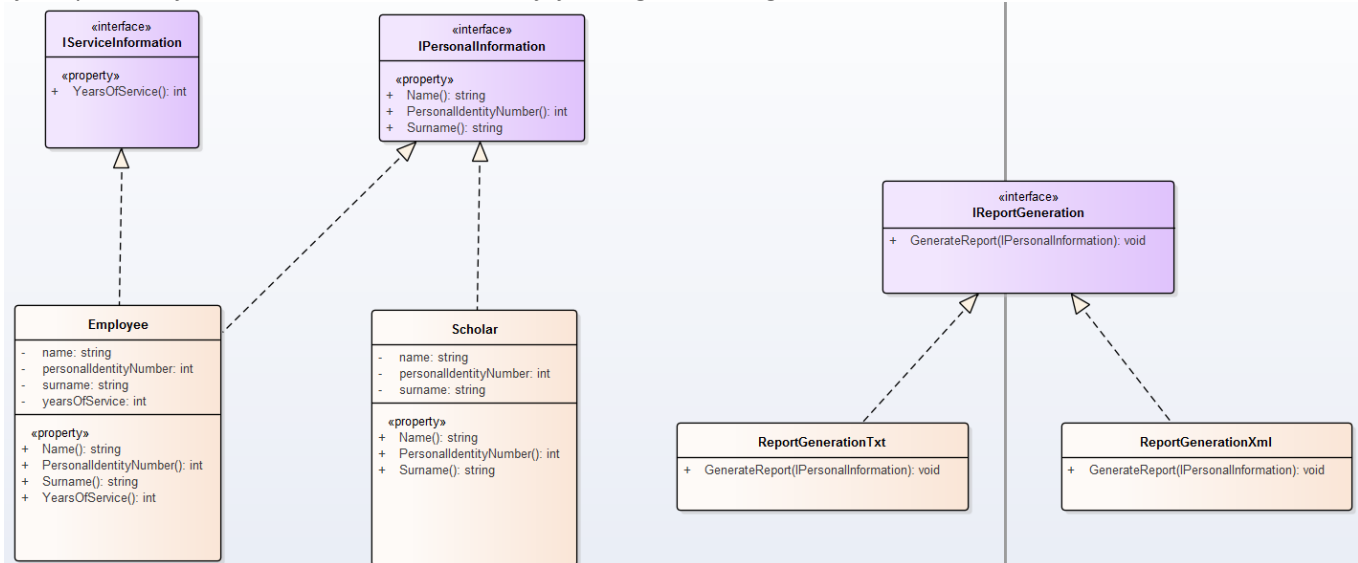
1. SRP(The Single Responsibility Principle) - klasa treba da bude odgovorna samo za jednu stvar.



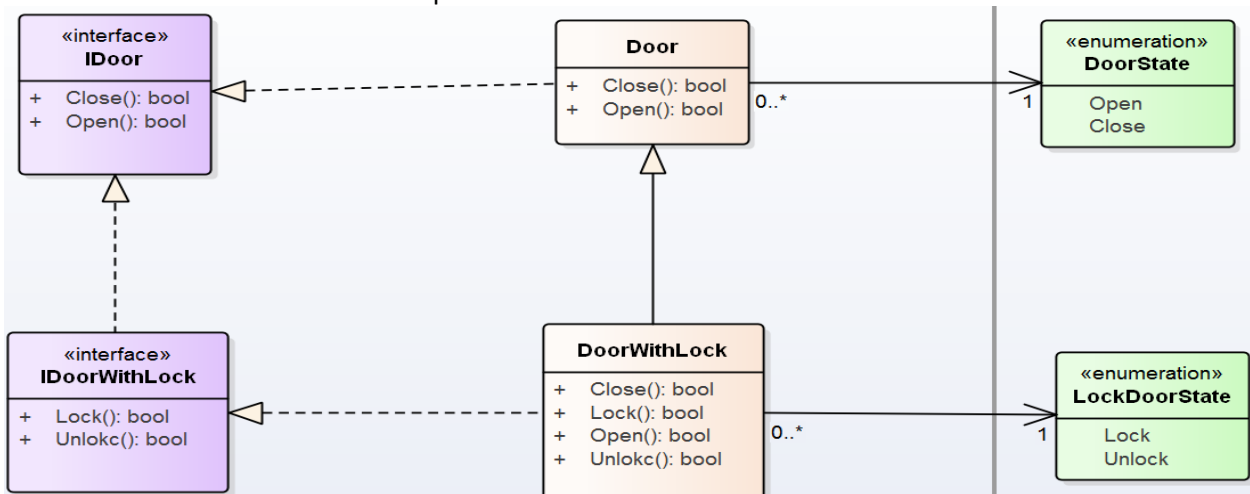
2. OCP(The Open Closed Principle) - klasa treba da bude zatvorena za izmene, ali otvorena za prosirenje tj. klasa se moze prosiriti bez modifikacije. U sustini znaci da se kod ne moze menjati sa promenom zahteva vec je moguće koristiti postojeće tehnike prosirivanja. U C# to uključuje nasledjivanje ili polimorfizam. Primena ovog principa omogućava da novi zahtevi donose novi kod, a ne menjaju postojeći kod koji radi. Apstrakcija i polimorfizam su ključni. Uobicajeni medoti Template method i Strategy obrazac.



3. LCP(The Liskov Substitution Principle) - izvedena klasa treba da moze da zameni baznu klasu. Izvedena klasa ne treba da narusi funkcionalnost bazne klase. Ako za novi objekat mozemo da kazemo da u potpunosti ispunjava stari objekat onda mozemo da kazemo da njegova klasa moze biti izvedena iz klase starog objekta. Izvedena klasa treba da prosiruje osnovnu klasu, a ne da narusava njeno ponasanje. **Primer:** klasu 'Oblik' nasledjuju 'Krug' , 'Pravougaonik'.

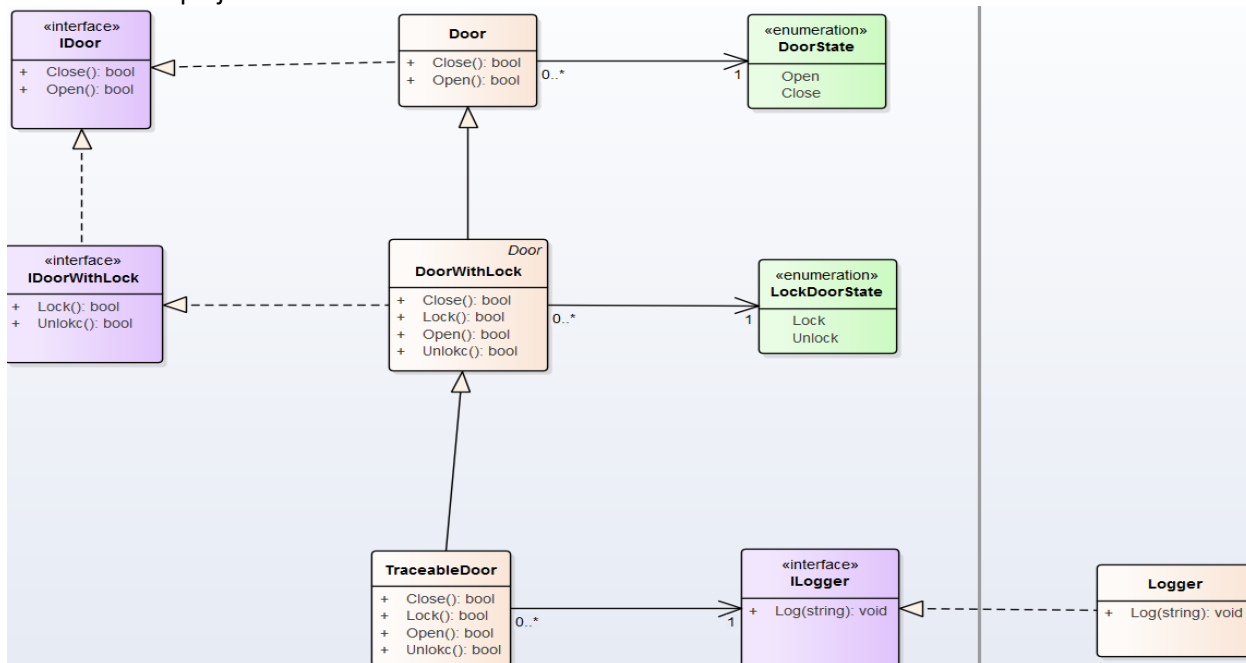


4. ISP(The Interface Segregation Principle) - ne treba forsirati klase koje implementiraju interfejs da implementiraju metode koje im ne trebaju. Umesto jednog interfejsa koji sadrzi veliki broj metoda, uvek je bolje deklarirati vise interfejsa sa manjim brojem metoda tako da klase koje ga implementiraju mogu izabrati samo ona ponasanja koja ze da implementiraju. Interfejs treba da pruži samo ono sto je potrebno njegovim korisnicima. Klasa ciji su interfejsi nekohezivni ima 'debele' interfejse. Interfejs moze da sadrzi metodu koju klasa nuzno mora da implementira iako joj implementacija nije potrebna(sta vise nije deo funkcionalnosti klase). Ovo je cest slucaj u jezicima kao sto su C# i Java i naziva se 'Interface polution'.



5. DIP(The Dependency Inversion Principle) - klasa treba da zavisi od apstrakcija, a ne od konkretnih implementacija. Klase, moduli i funkcije ne trebaju biti cvrsto spregnute. Moduo sa viseg nivoa ne treba da zavisi od modula sa nizeg nivoa. Oba treba da zavise od apstrakcija. Apstrakcija ne treba da zavisi od detalja. Detalji treba da zavise od apstrakcije. Ako su zavisnosti invertovane imamo objektno orjentisan dizajn, a ako nisu imamo proceduralni dizajn. Fokusiramo se na to da apstrakcije

omogućavaju modulima viseg nivoa da budu nezavisni od nizeg nivoa, što olakšava održavanje i testiranje koda. Moduli su nezavisne celine koda koje obavljaju određene funkcije i sastoje se od varijabli, funkcija i/ili klase. Klase viseg nivoa ne znaju koje klase nizeg nivoa se koriste. Na primer, možda ćete morati da koristite ime osobe u drugoj klasi, ali ne morate znati da se ime skladišti u **ime** polju.



6. Information Expert(Expert) - kojoj klasi da dodam novu metodu.

UML je grafički jezik za vizuelizaciju, specifikaciju, konstruisanje i dokumentovanje. Iskazuje sistem kroz odredjena pravila.

Agregacija - automobil se sastoji od tockova i sadrzi ih. Odnos deo-celina. Mogu postojati kao nezavisni elementi.

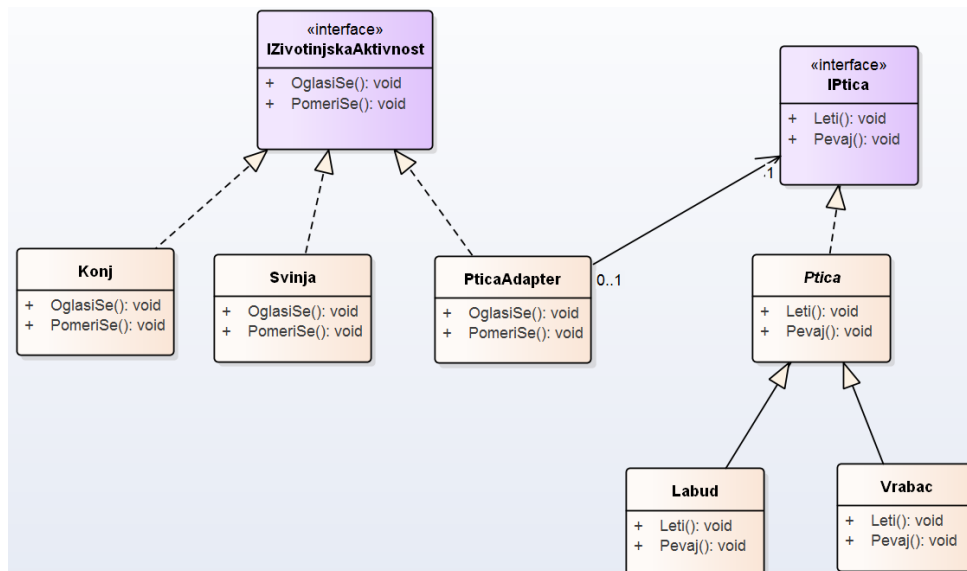
Kompozicija - deo i celina su cvrsto povezani. Ako zapalimo knjigu, zapalili smo i stranice.

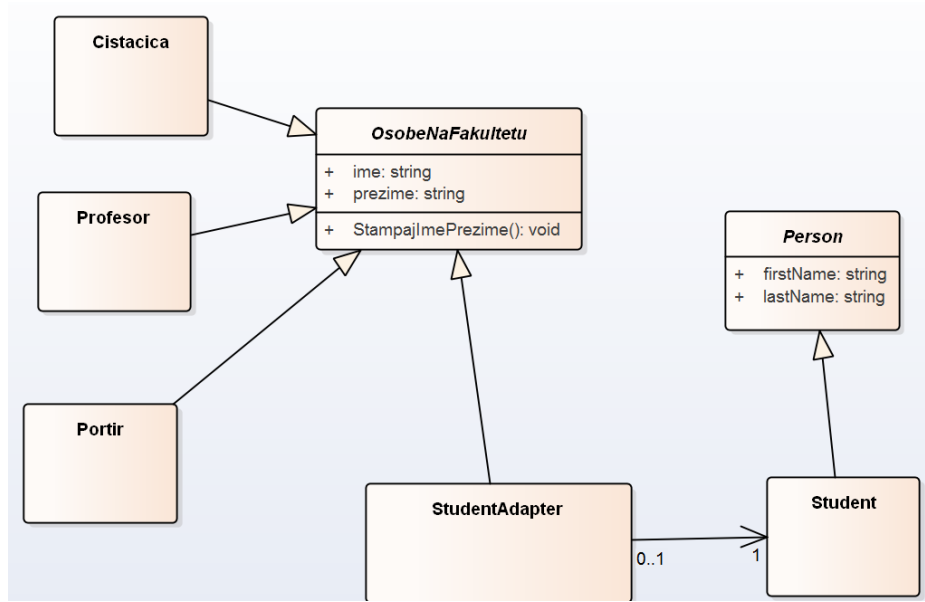
Design patterns(opisi i nacrtaj primer za interfejs i klasu):

Kada koristimo design pattern, mi zapravo koristimo testiran i dokazan način rešavanja problema koji se javljaju u različitim situacijama u razvoju softvera. Ovi patterni se razvijaju na osnovu iskustva programera u rešavanju problema, a zatim se dokumentuju i dele sa drugima kako bi se drugi programeri lakše snašli u sličnim situacijama. Koristimo design patterne kako bismo unapredili proces razvoja softvera i obezbedili kvalitetan i održiv kod.

1. Adapter

Kada je potrebno iskoristiti vec postojeću klasu čiji interfejs ne odgovara potrebama - primenjuje se naknadno. Kada zelimo da napravimo klasu koja ce u buducnosti saradjivati sa nepovezanim i nepredvidjenim klasama u sistemu(čiji interfejsi ne moraju biti kompatibilni). Kada imamo 2 sistema i potrebu da u sistemu pokazemo na jedinstven način stvari oba sistema tj. adaptiramo jedan sistem u drugi.

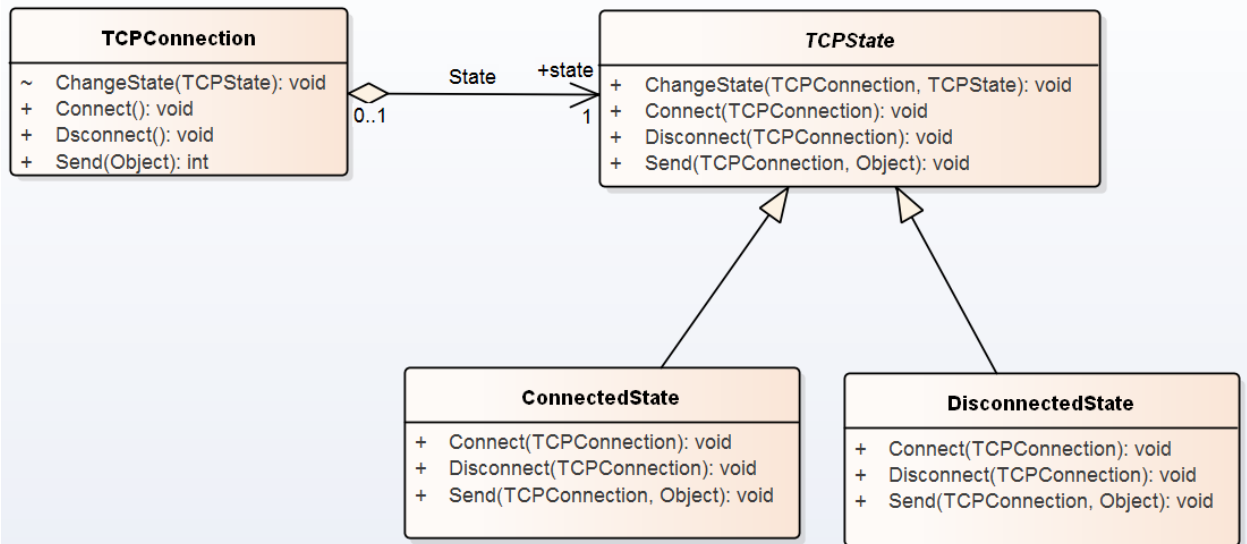


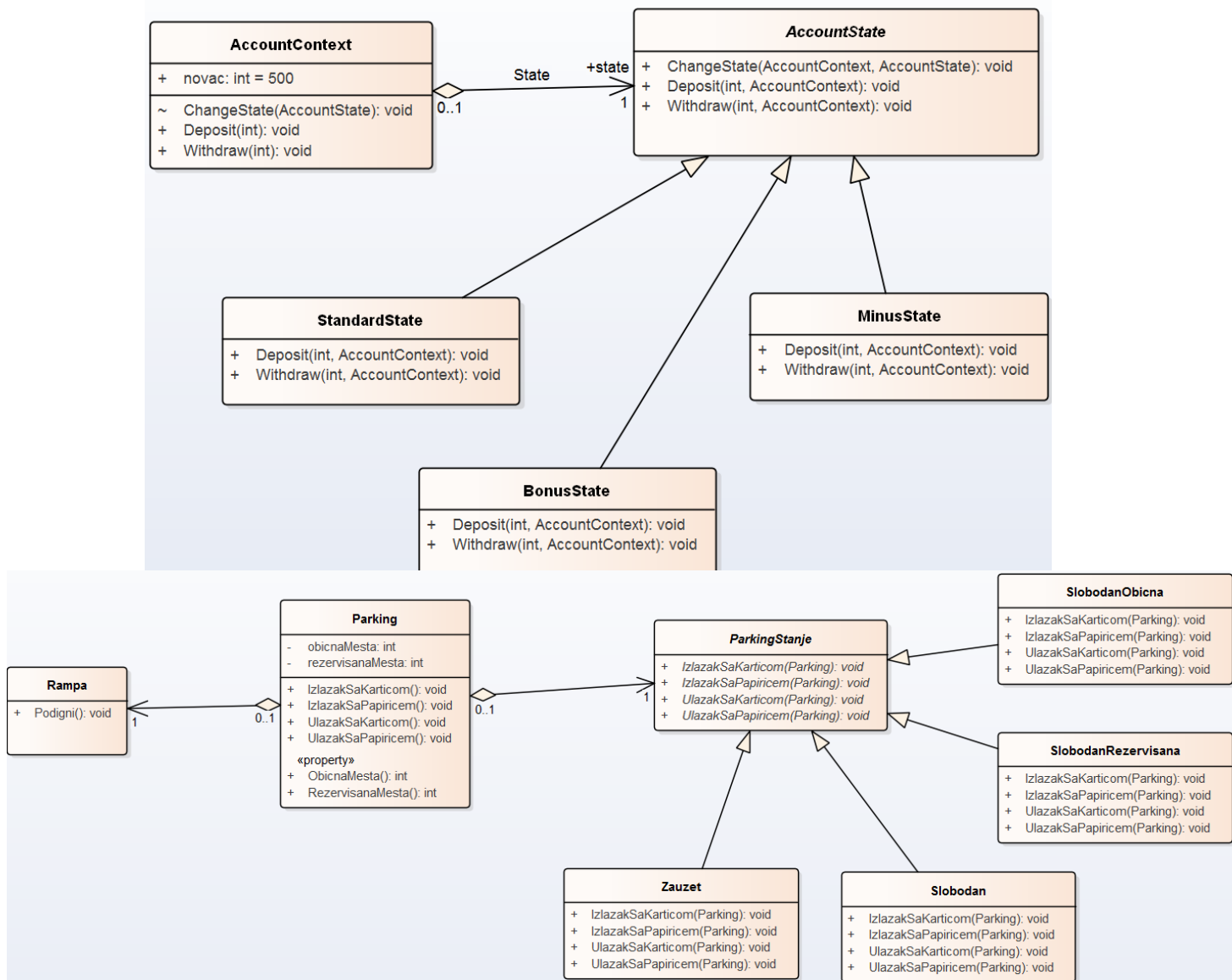


2. State

Ponasanje neke klase se modeluje u odnosu na stanja necega. Razlika izmedju state i command je kako se menja neki objekat i kako se ponasa neki objekat.

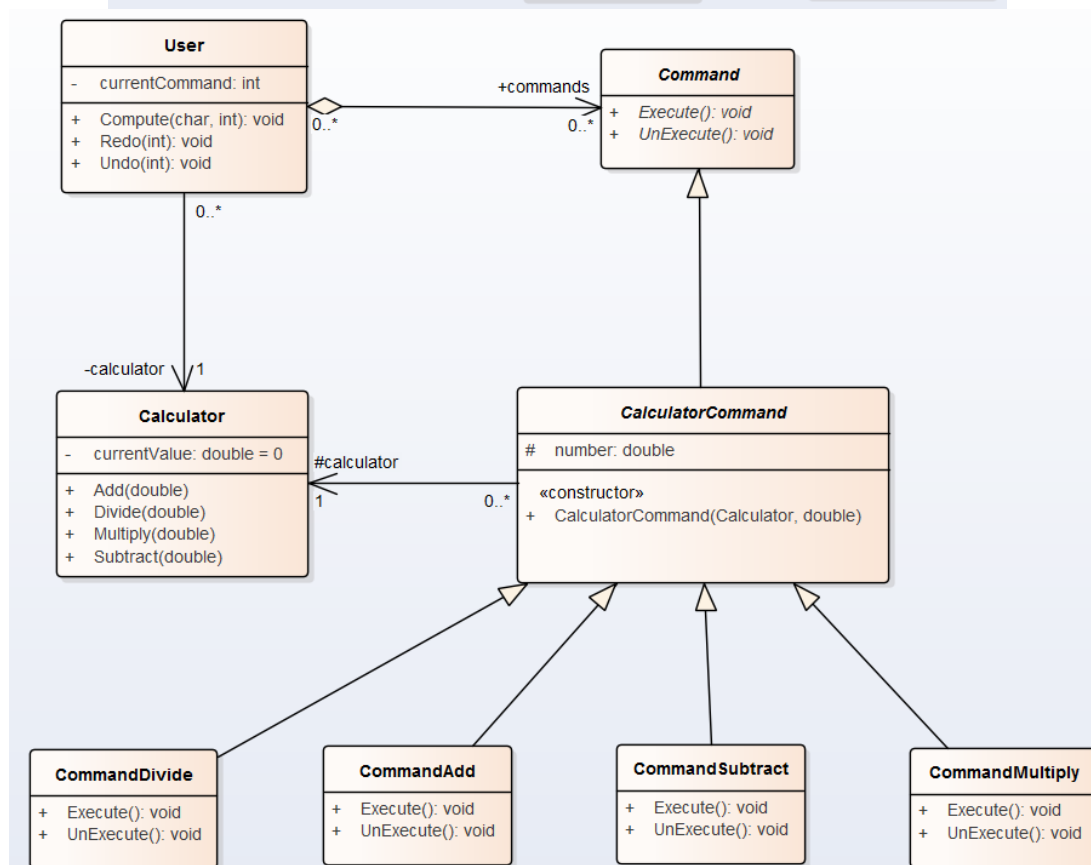
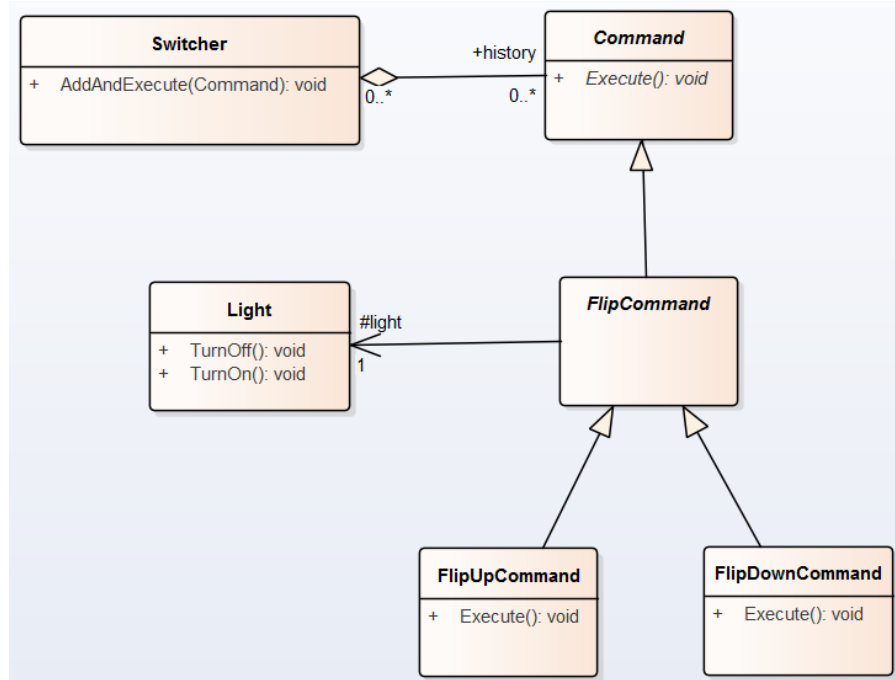
Omogucuje objektu da menja svoje ponasanje u zavisnosti od stanja u kom se nalazi. Ima skup stanja i menja ih kao odgovor na spoljasnje ulaze. Dodavanje novih stanja moze biti otezano.





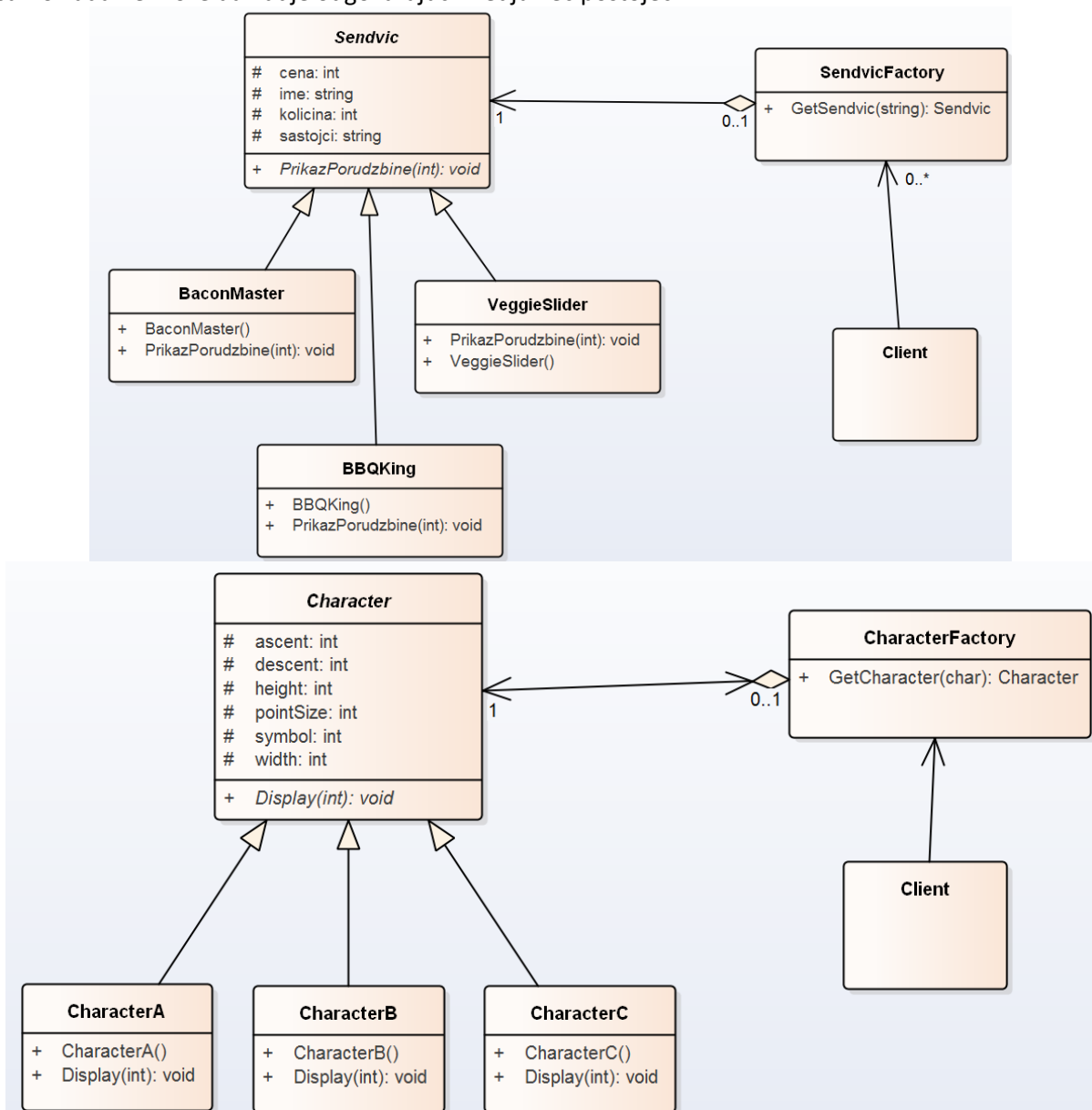
3. Command

Promene neke klase i potreba da se te promene ponistavaju(recimo undo i redo je skoro uvek command). Umesto da se direktno izvrši određena operacija, kreira se objekat-komanda, koji se potom prosledjuje na izvršenje. Prednost je to razdvajanje objekta koji poziva radnju od objekta koji je izvršava, olaksano proširivanje jer se nove funkcionalnosti dodaju bez menjanja starih, moguće izvršavanje niza komandi odjednom.



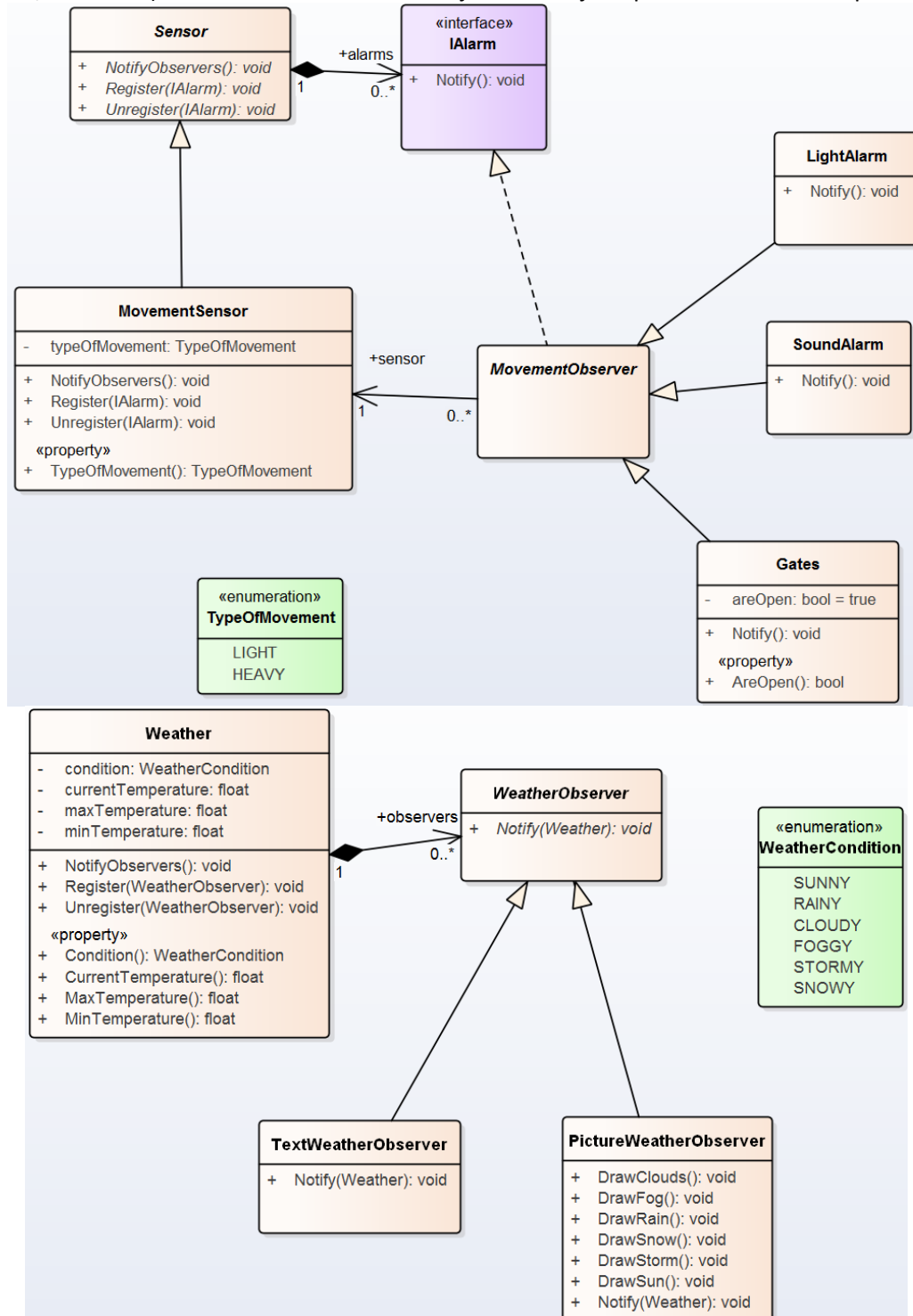
4. Flyweight

Minimizuje upotrebu memorije deljenjem podataka drugim, slicnim objektima. Predstavlja nacin za upravljanje velikom kolicinom objekata, u situacijama gde konstantno kreiranje novih objekata nije prihvatljivo zbog memorijskih ogranicenja(primer navijaci na stadionu u igrici). Kreira novi objekat samo kada ne moze da nadje odgovarajuci medju vec postojećim.



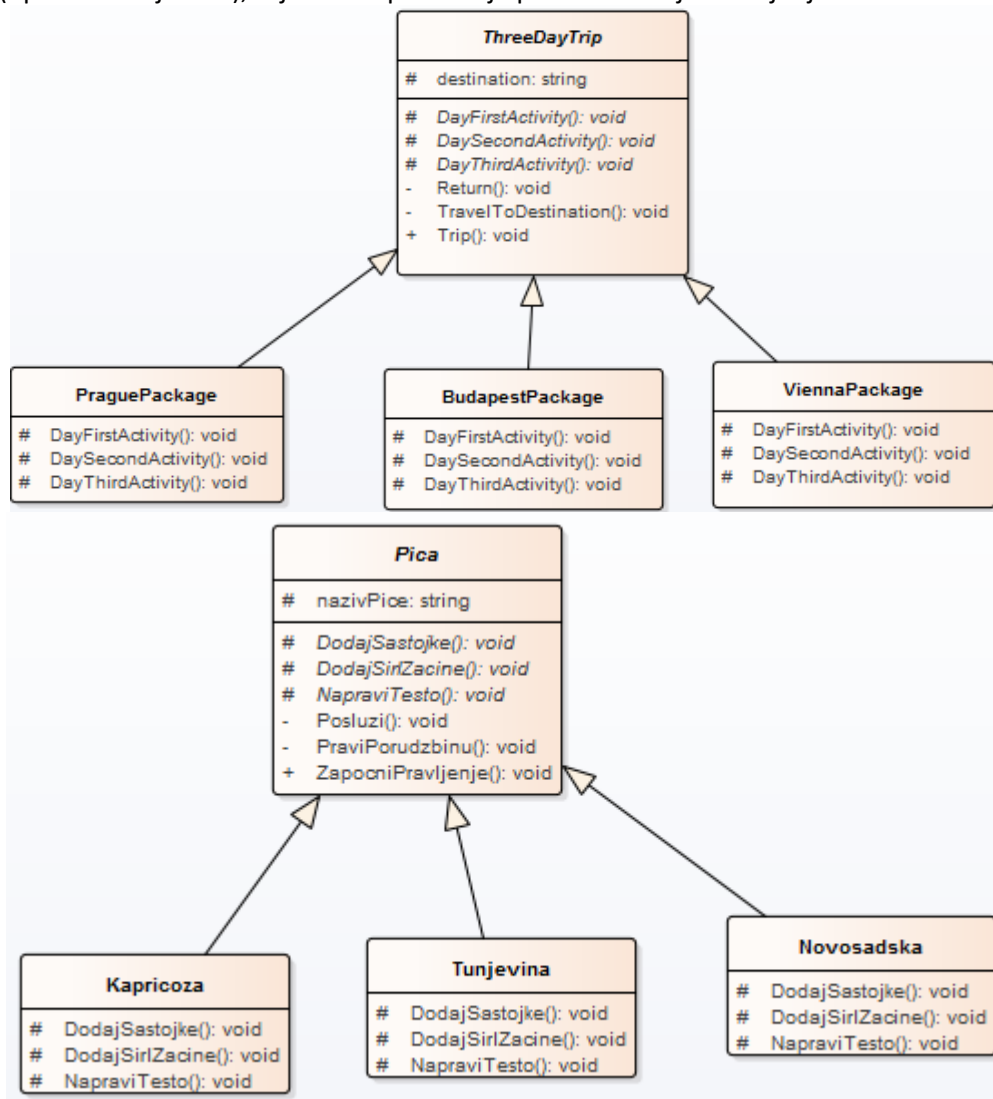
5. Observer

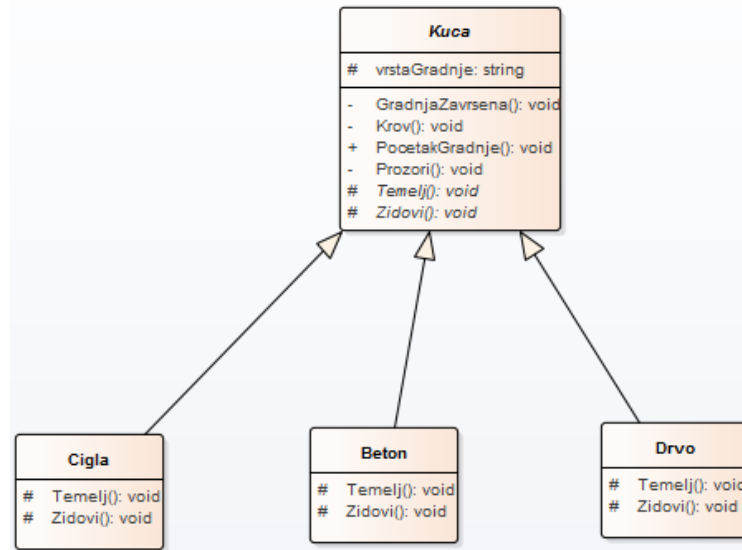
Ako imamo notifikacije, neko slusa, neko ne mora da slusa. Promena sadrzaja u jednom elementu se odmah pojavi i u svim delovima programa koji dati element prikazuju u nekom obliku. Obezbedjuje mehanizam pomocu kojeg se izmedju zavisnih delova(posmatraca: dijalog, tabela, grafova) automatski azurira promena stanja koja se desila nad podacima koje prikazuje(promena nad subjektom, modelom). Koristimo za svako azuriranje na odredjenu promenu. Observer=posmatrac



6. Template method

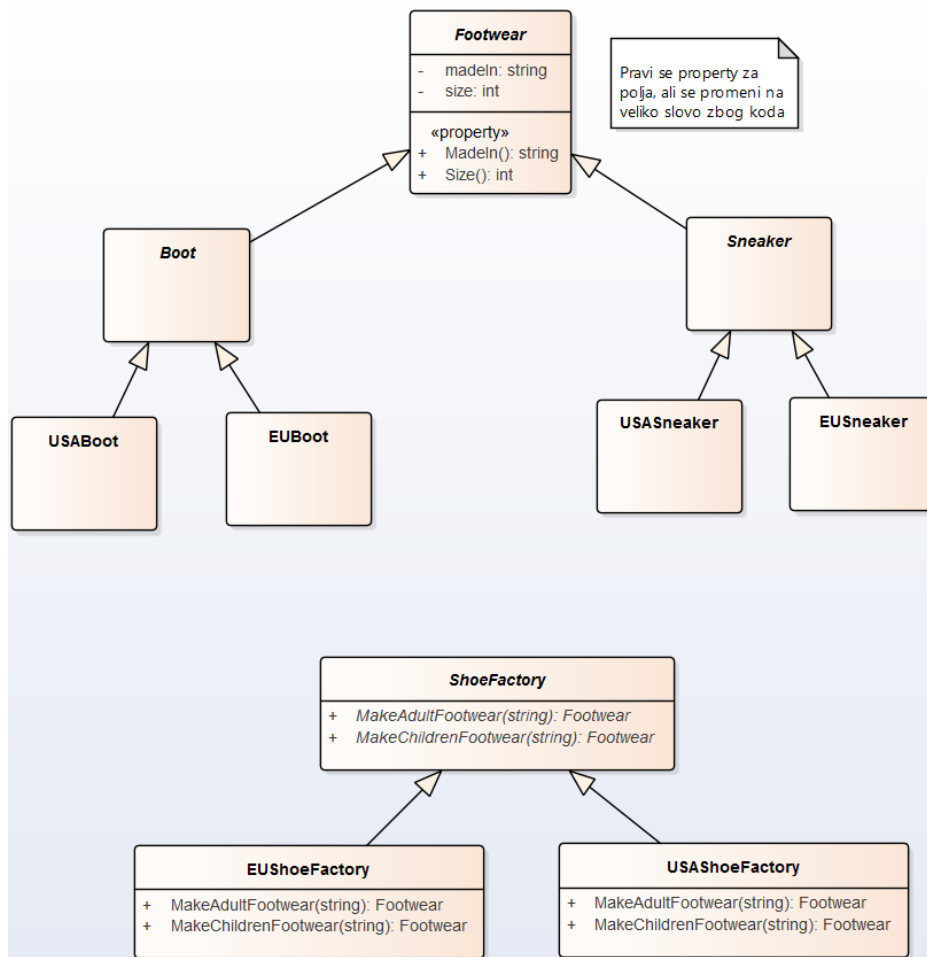
Sablon ponasanja, snimanje na disk, neki koraci, otvori fajl, snimi. Pripada grupi projektnih obrazaca ponasanja. Definise kostur nekog algoritma i omogucava podklasama da redefinisu korake algoritma. Za potrebe implementiranja delova algoritma koji su isti u svim slucajevima samo se sami koraci razlikuju(npr. sortiranje niza), zajednicko ponasanje podklasa u cilju smanjenja koda.





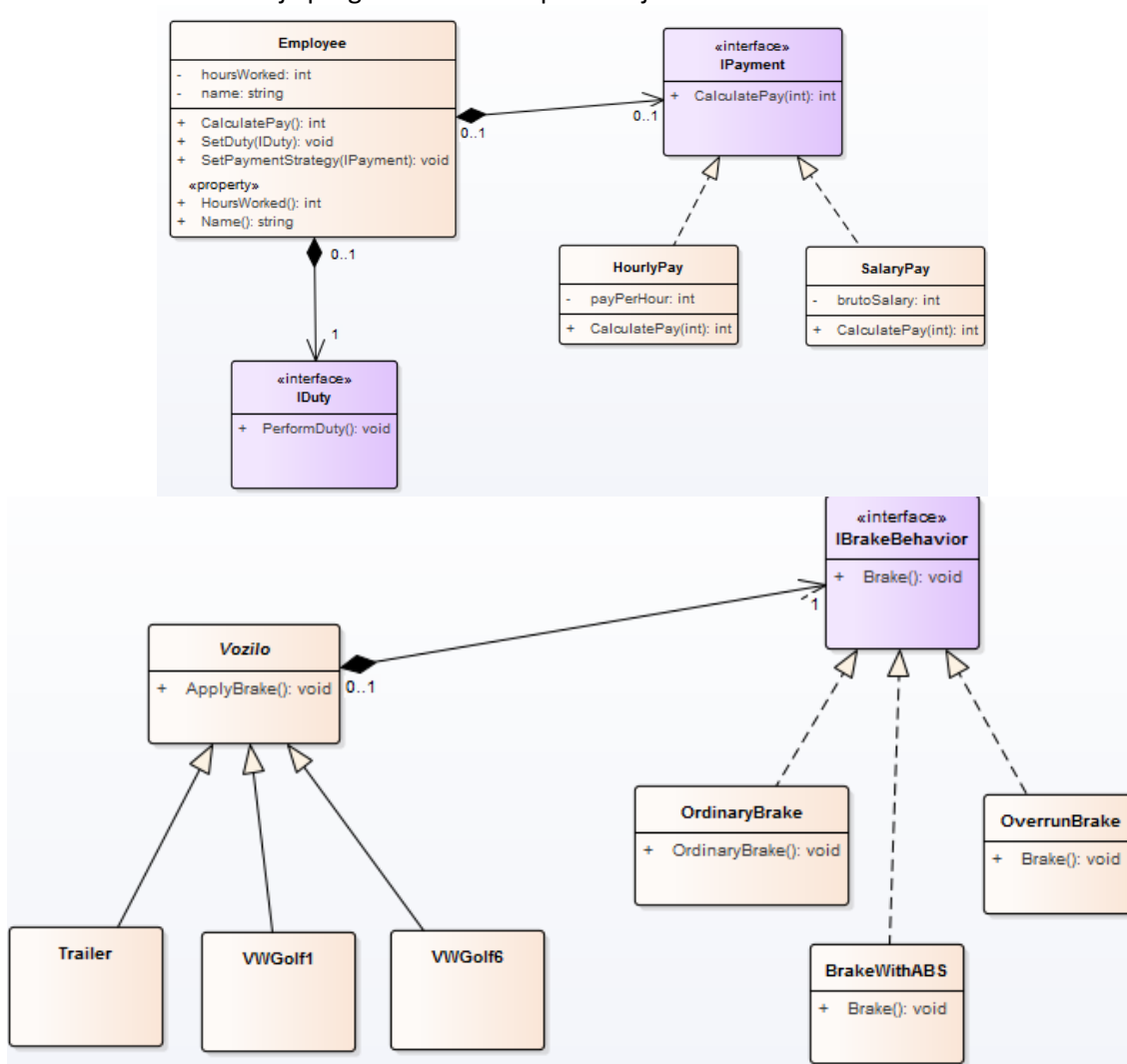
7. Abstract factory

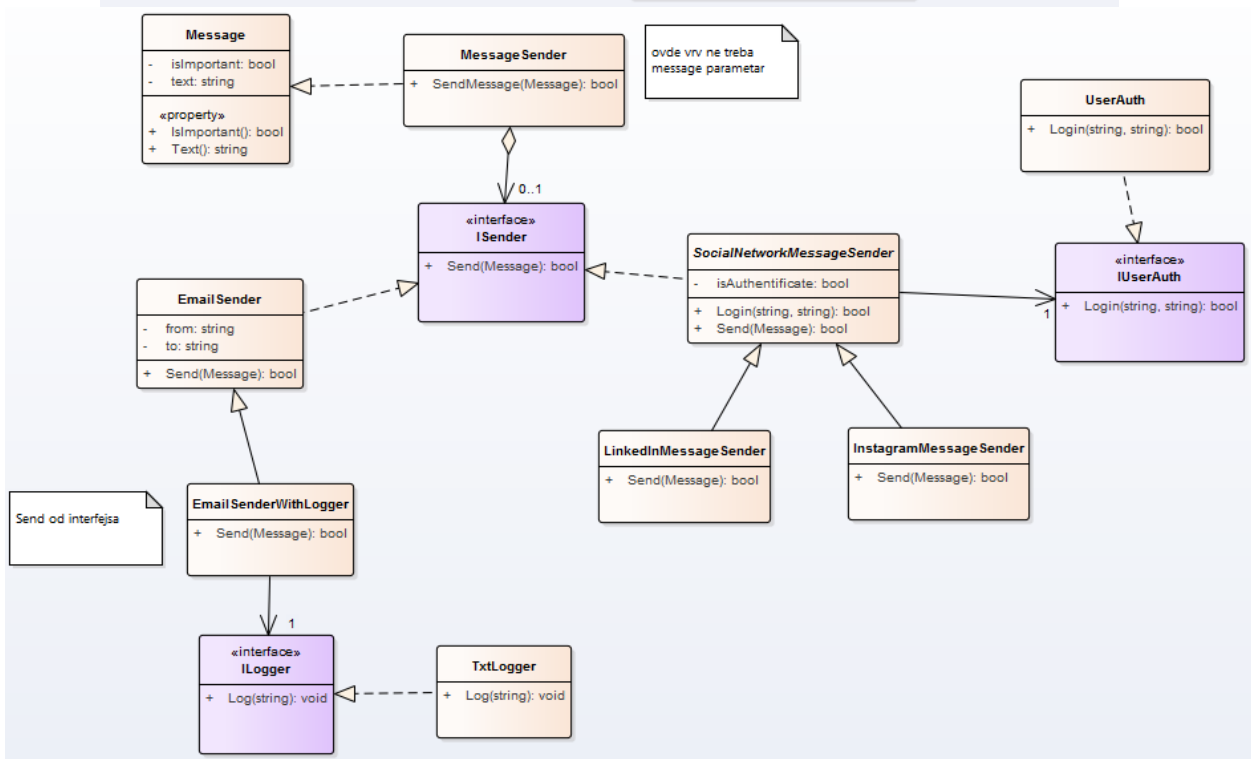
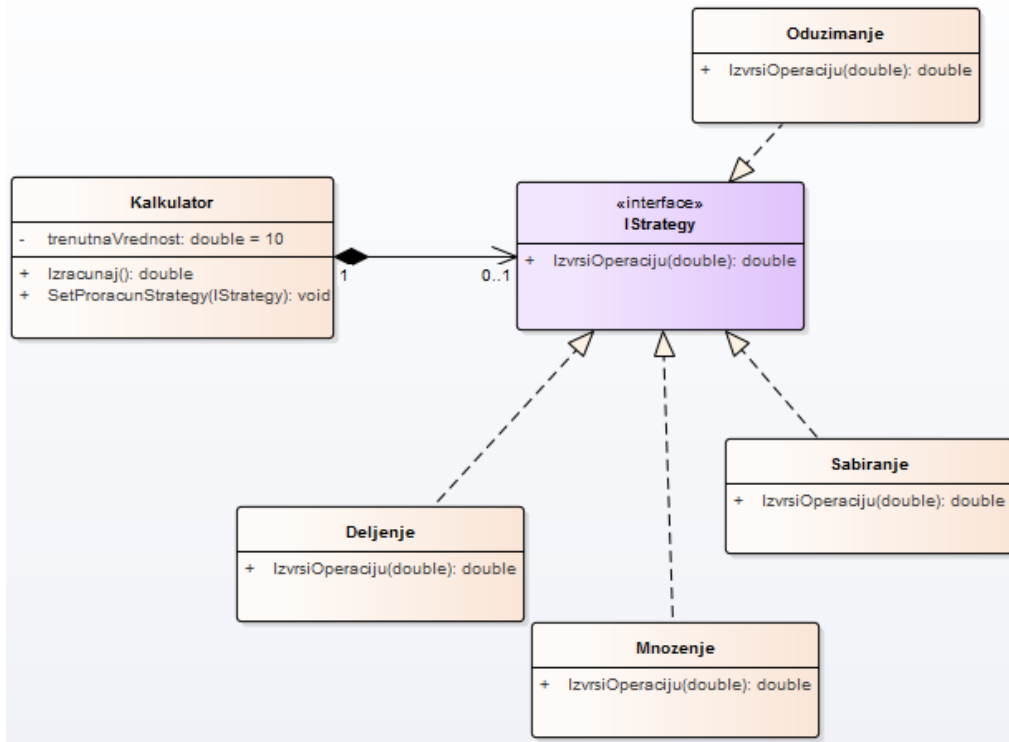
Vise nacina za proizvodnju nekih objekata, instanciranje objekata, message i vise message, privatnih, ovakvih i onakvih, sve se razlikuje kako se oni prave, konkretne implementacije fabrike, kada je akcenat na kreiranju objekta. Kada se proizvodi iz neke familije koriste zajedno.

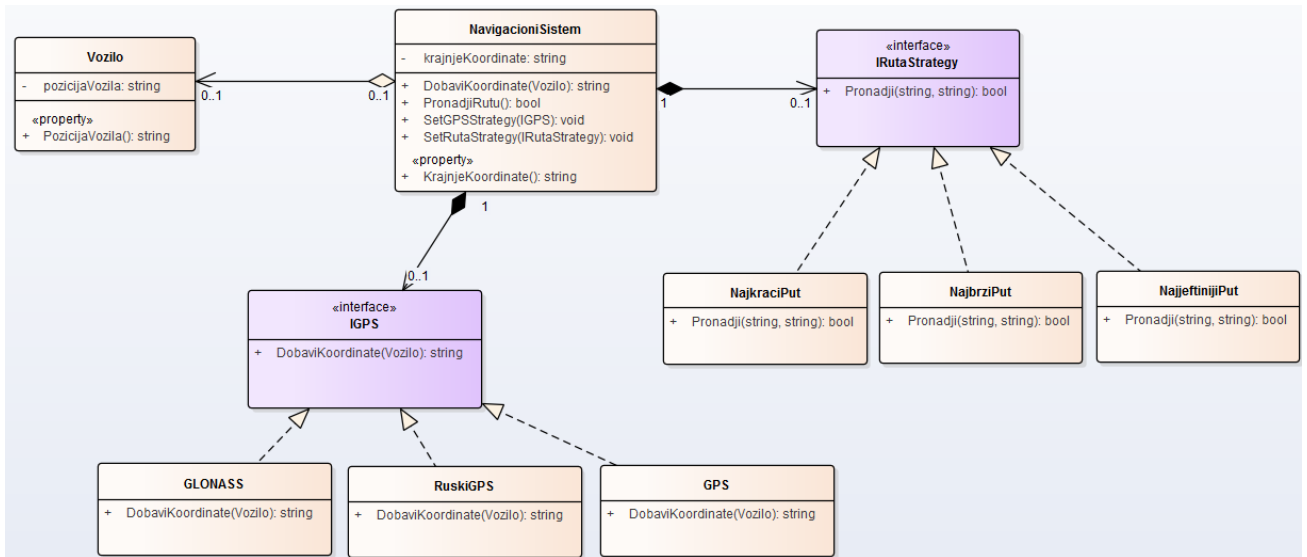


8. Strategy

Razni nacini ponasanja, razni tipovi operacija, strategije, na jedan nacin se radi pretraga teksta, salje poruka, na razlicite nacine izvrsava nesto. Definise grupu algoritama i omogucava laku zamenu algoritma tokom izvrsavanja programa. Obrazac ponasanja







SetGPSStrategy ima kao parametar interfejs(IGSP). Kada kreiram strategiju ja biram neku on onih koje implementiraju taj interfejs.

Jos jedan patern: MVC - model view controller

```

Vozilo vozilo = new Vozilo("300.002311,255.122324");
NavigacioniSistem navsistem = new NavigacioniSistem("255.092343,324.123001", vozilo);

navsistem.SetGPSStrategy(new GPS());
navsistem.SetRutaStrategy(new NajbrziPut());
if(navsistem.PronadjiRutu())
    Console.WriteLine("Uspesno pronadjena ruta.");
else
    Console.WriteLine("Ruta nije uspesno pronadjena.");

Console.WriteLine();

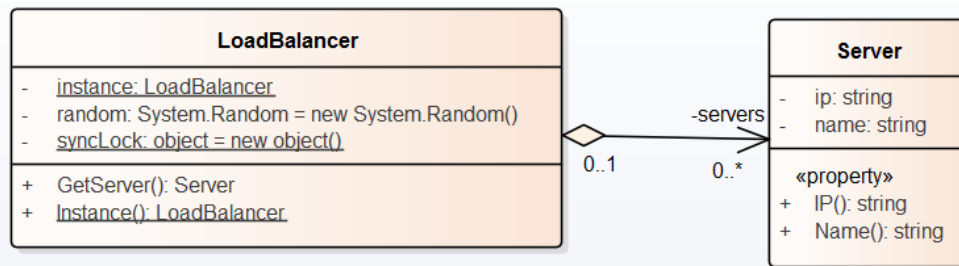
navsistem.SetGPSStrategy(new RuskiGPS());
navsistem.SetRutaStrategy(new NajkraciPut());
if (navsistem.PronadjiRutu())
    Console.WriteLine("Uspesno pronadjena ruta.");
else
    Console.WriteLine("Ruta nije uspesno pronadjena.");

Console.WriteLine();

navsistem.SetGPSStrategy(new GLONASS());
navsistem.SetRutaStrategy(new NajjeftinijiPut());
if (navsistem.PronadjiRutu())
    Console.WriteLine("Uspesno pronadjena ruta.");
else
    Console.WriteLine("Ruta nije uspesno pronadjena.");
  
```

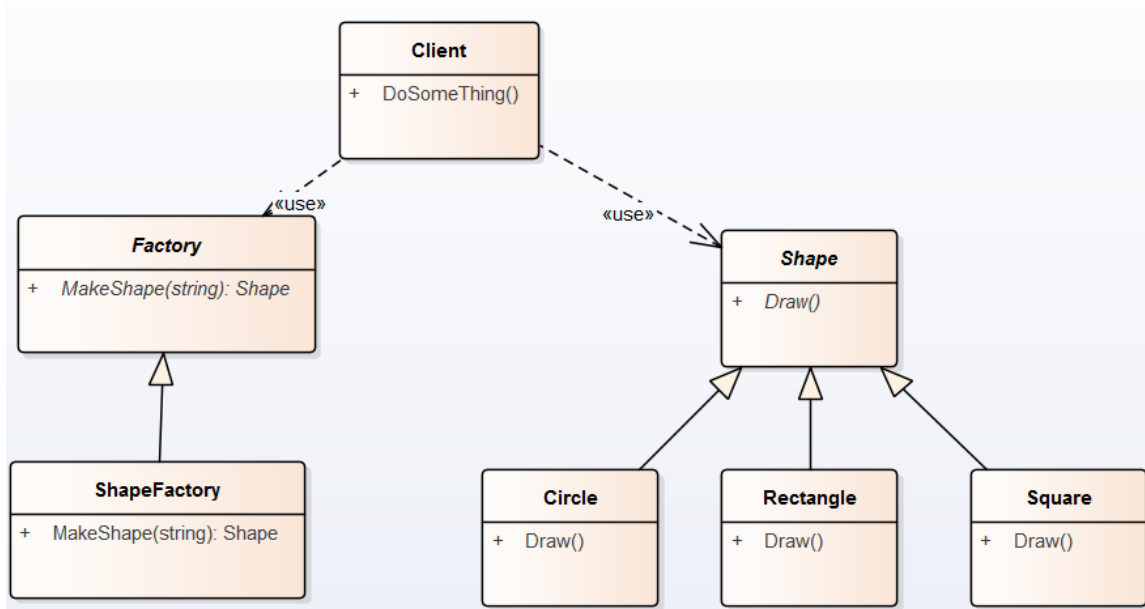
9. Singleton(unikat)

Obezbedjuje da klasa ima samo jednu instancu i daje globalni pristup toj instanci. Odgovorna za kreiranje i rad sa svojom sopstvenom jedinstvenom instancom.



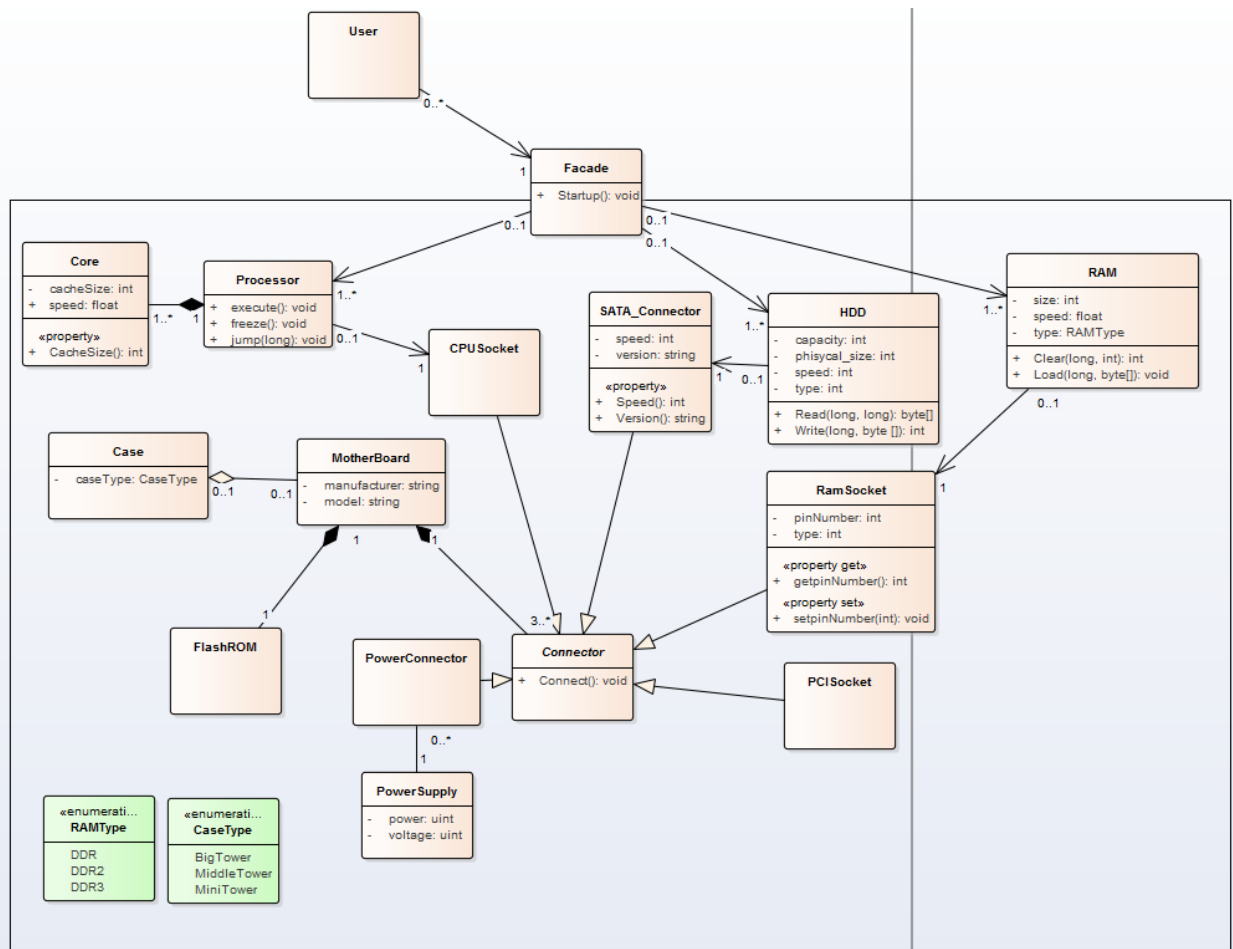
10. Factory method

Prosledjuje se zahtev za kreiranje objekata Factory metodi. Factory klasa definise interfejs za kreiranje objekata, a sama implementacija kreiranja objekta je prepustena podklasama. Zelimo da prepustimo odgovornost kreiranja objekata nekoj podklasi.



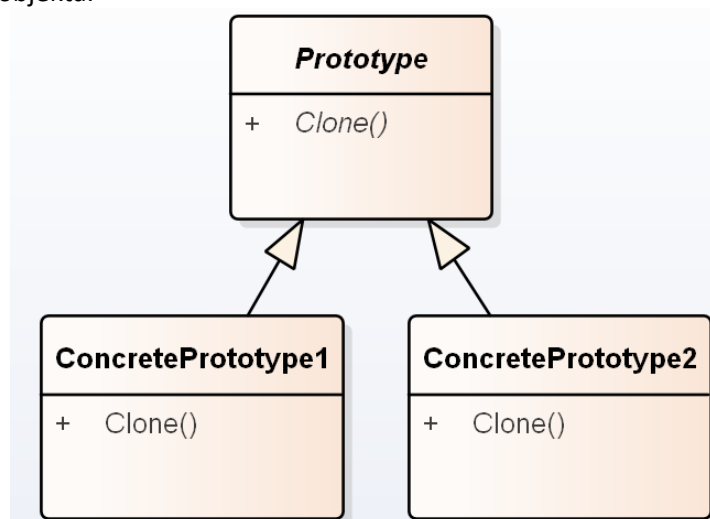
11. Facade

Pripada grupi strukturalnih obrazaca. Pruza jedinstven interfejs celog podsistema radi lakseg koriscenja. Predstavlja interfejs na visem nivou sa ciljem minimizacije komunikacije i zavisnosti izmedju klasa podsistema. Koristi se za uvođenje slojevitosti izmedju podsistema, kada postoji mnoštvo zavisnih veza izmedju korisnika i sistema.



12. Prototype

Mehanizam kako da se pravljenje objekata-duplikata određene klase poveri posebnom objektu date klase, koji predstavlja prototipski objekat te klase i koji se može klonirati. Govori kako klonirati određenu instancu objekta.



C#

Teorija:

Visestruko deljenje podataka se može spreciti **lokovanjem resursa**. Ne sme da se desi da jedna nit radi sa podacima, a druga ih brise. Primer veb sajt u kome se u jednom trenutku 2 korisnika pokušavaju da se registruju u isto vreme. Ako dozvolimo istovremenu registraciju može se desiti da 2 coveka imaju isto korisnicko ime. Koristimo lock da zaključamo resurs i u tom trenutku niko drugi ne može da izvrši registraciju.

```
private static readonly object zaključajResurs = new object();

#region Dodaj korisnika -> Registruj
public static bool DodajKorisnika(Korisnik korisnik)
{
    lock (zaključajResurs)
    {
        using (SqlConnection connection = new SqlConnection(myCon))
        {
            try
            {
                string komanda = $"INSERT INTO PUSGS.dbo.Korisnik(KorisnickoIme,Email,
```

Tuple u C# je vrsta podataka koja omogućava skladištenje više elemenata različitih tipova u jednoj varijabli. Tuples su read-only, što znači da nakon kreiranja, ne možete dodati, ukloniti ili promeniti elemente. Ovaj kod definiše Tuple sa dva elementa - string koji predstavlja ime i integer koji predstavlja godine. Nakon toga, Tuple se koristi za skladištenje imena "John" i godina 30. Memorija se oslobadja sa `person.Dispose();`

```
Tuple<string, int> person = new Tuple<string, int>("John", 30);
Console.WriteLine("Name: " + person.Item1 + ", Age: " + person.Item2);
```

Instanca varijable je atribut objekta klase. Objekti se stvaraju iz klase i imaju svoje vlastite vrednosti za sve atribute. Svaki objekat klase ima svoju kopiju atributa koji se definisu. Ovi atributi se obično nazivaju instancama varijabli.

Kontrola toka - if/then/else, while, do while, for

XMI (XML Metadata Interchange) - predstavlja standard za razmenu meta podataka. Podaci o podacima, opisuju karakteristike i kontekst podataka u informacionom modelu. Sadrže tehnicke karakteristike, sadržaj i prirodu informacija, veze između entiteta i veze sa drugim podacima.

-

`private Button print = new Button();` //kreira i inicijalizuje dugme

Kreira se pozivom konstruktora 'new Button()'. **Konstruktor** je metoda koja se poziva prilikom stvaranja novog objekta i inicijalizuje ga sa početnim vrednostima. Time što se dodeljuje instanci varijable 'print' vrši se **inicijalizacija**.

-

Event je obaveštenje da se dogodila neka akcija. Obavestava program da se nešto desilo i program mora da reaguje na odgovarajući način.

Objekti mogu da se shvate kao celine u programskom kodu koje odlikuje određeno stanje i ponašanje. **Može se deklarirati i inicijalizovati više puta.**

```
MyObject obj1 = new MyObject();  
MyObject obj2 = new MyObject();
```

Prilikom nasleđivanja u C#-u, **prvi objekat ne može direktno menjati drugi objekat**. Nasleđivanje omogućava da se uvede nova funkcionalnost ili promene u postojećoj klasi, ali **sam objekat koji nasleđuje (podklasa) ne menja izvorni objekat (nadklasu)**. Podklasa može da proširi ili predefiniše ponašanje metoda ili svojstava koje je nasledila, ali to ne utiče na izvorni objekat. Izvorni objekat ostaje nepromenjen i može se koristiti nezavisno od podklasa koje su bazirane na njemu.

Metode - funkcije koje definišu radnje koje će budući objekti moći da obavljaju nad svojim sopstvenim podacima ili sa drugim podacima u programu. Opisuje ponašanje objekta.

Dizajn klase treba da opiše: koji će podaci biti predstavljeni, kako će biti predstavljeni, kako će biti međusobno povezani, mehanizmi za obradu podataka.

Klasa je skup opstih karakteristika i mogućih stanja objekta. Objekat je konkretna realizacija klase. Kada se koristi "static" sa klasom, to znači da ta klasa neće biti instancirana, odnosno da se neće kreirati objekti koji pripadaju toj klasi. Umesto toga, metode i polja te klase će biti dostupni direktno preko klase, bez potrebe za instanciranjem. Ovde su statičke i metoda i klasa.

```
MyClass.MyMethod(); // Pristup statičkoj metodi bez instanciranja klase
```

Polje je pojedinačni imenovani podatak koji može biti: primitivni podatak koji pripada nekom od osnovnih tipova(int, float, char), kolekcija podataka bilo kog tipa(niz, lista, red, stek..), objekat klase(iste ili različite), pokazivač/referenca na podatak, objekat ili kolekciju bilo kog od navedenih tipova. To su zapisani podaci koji definišu stanje objekta.

Atribut definiše **stanje**, a ponašanje promenu. (atribut je npr [Serializable]-objekti mogu da se pretvore u niz bajtova)

Kreiranje klase, objekta, pristup poljima:

```
public class Pravougaonik  
{  
    public Double a, b, O, P;  
}
```

```
Pravougaonik P1 = new Pravougaonik();
```

```
P1.a = 12.55;  
P1.b = 10.40;  
Console.WriteLine(P1.a);  
Console.WriteLine(P1.b);
```


Refleksija u programiranju je mehanizam koji omogućava programu da analizira, prepozna i manipuliše sa informacijama o samom sebi u toku izvršavanja. Ovo uključuje pristup informacijama o klasama, metodama i poljima, kao i mogućnost izvršavanja dinamičkih operacija poput stvaranja novih objekata i pozivanja metoda. Kroz refleksiju programer **može pristupiti** raznim informacijama o objektima u kodu (polja, metode), bez obzira da li su **javne ili privatne**, i može izvršiti operacije nad njima u toku izvršavanja programa. Ovo često može biti korisno u situacijama kada je potrebno **dinamički (u runtime-u, npr. želimo instanciranje objekta iz eksternog izvora ili odabir interfejsa zavisno od potreba ili postavki) stvarati objekte, čitati ili menjati vrednosti svojstava, pozivati metode ili čitati informacije o atributima klase ili metoda. Pristup privatnom polju nije preporučljiv jer krši enkapsulaciju (korisno ili neophodno za debugging ili unit testiranje).**

Razlika: `a.Equals(b)` --- `a == b`

Equals poredi vrednosti dva objekta i ako su jednake vraća true, a ako nisu false. Dok **==** poredi reference i ako su objekti smesteni na istoj adresi u memoriji, tj. referenciraju na isti objekat tada vraća true, a inače false.

`string username; // Ovo će javiti grešku jer se string promenljiva ne može inicijalizovati na null`

```
string? username = null; // dozvoljeno je dodeliti null vrednost
```

Virtual method uvek mora da ima default implementaciju

```
public class Shape
{
    public virtual void Draw()
    {
        Console.WriteLine("Drawing a shape");
    }
}

public class Circle : Shape
{
    public override void Draw()
    {
        Console.WriteLine("Drawing a circle");
    }
}
```

Regularni izraz (Regex) - obrazac (patern) za pretragu i manipulaciju nizova karaktera

Funkcije sa podrazumevanim vrednostima

```
void PrintMessage(string message = "Hello, world!")
{
    Console.WriteLine(message);
}

PrintMessage(); // Ispisuje: "Hello, world!"
PrintMessage("Custom message"); // Ispisuje: "Custom message"
```

Funkcije sa promenljivim brojem parametara

```
void PrintNumbers(params int[] numbers)
{
    foreach (int number in numbers)
    {
        Console.WriteLine(number);
    }
}

PrintNumbers(1, 2, 3); // Ispisuje: 1 2 3
PrintNumbers(10, 20, 30, 40, 50); // Ispisuje: 10 20 30 40 50
```

Lambda izraz je anonimna funkcija koja se može koristiti za definisanje kratkog koda ili izraza koji se mogu proslediti ili dodeliti drugim funkcijama. Lambda izraz ekvivalentan f-ji sa desne strane:

```
Func<int, int> square = x => x * x;
```

```
int result = square(5); // rezultat će biti 25
```

```
int Square(int x)
{
    return x * x;
}
```

```
int result = Square(5); // rezultat će biti 25
```

Kada kreiramo objekat izvedene klase istovremeno će se kreirati instance i roditeljske i izvedene klase. Konstruktor roditeljske klase se prvo izvršava kako bi se inicijalizovala roditeljska komponenta objekta, a zatim se izvršava konstruktor izvedene klase za inicijalizaciju dodatnih komponenti specifičnih za izvedenu klasu.

```
class ParentClass
{
    string ime, prezime;

    public ParentClass(string ime, string prezime){
        this.ime = ime;
        this.prezime = prezime;
    }

    public override string ToString(){
        return $" {ime} {prezime}";
    }
}
```

```
class ChildClass : ParentClass
{
    int godine;

    public ChildClass(string ime, string prezime, int godine) : base(ime, prezime){
        this.godine=godine;
    }

    public override string ToString(){
        return base.ToString() + $" {godine}";
    }
}
```

```
static void Main(string[] args)
{
    ParentClass parent = new ParentClass("Pera", "Peric");
    ChildClass child = new ChildClass("Zika", "Zikic", 25);
    Console.WriteLine(parent.ToString());
    Console.WriteLine(child.ToString());
}
```

Stringovi su imutabilni(nepromenljivi) - vrednost stringa ne može direktno da se promeni nakon što je jednom kreiran. Njegova vrednost i dalje postoji, jedino smo možda promenom izgubili pokazivač tj. postojeće tekst 'hi' i tekst 'hello', ako ne sačuvamo originalnu vrednost ne znači da smo je promenili. Ona nastavlja da postoji, ali mi više ne pokazujemo na taj objekat.

```
string tekst = "hi";
string originalnaVrednost = tekst; // Čuvanje originalne vrednosti

tekst = "hello"; // Ažuriranje vrednosti

Console.WriteLine(originalnaVrednost); // Ispisuje "hi"
Console.WriteLine(tekst); // Ispisuje "hello"
```

Konstruktor specijalizovana metoda kojom se zadaje početno stanje objekta. Nema povratni tip i ima isti naziv kao i sama klasa.

Postoje: podrazumevani konstruktor, konstruktor kopije, konstruktor sa parametrima.

1. **Podrazumevani konstruktor** se odnosi na konstruktor koji se automatski generise od strane kompajlera ako se ne definiše drugi konstruktor. Može biti prazan konstruktor ili konstruktor sa parametrima ako se definiše. Prazan konstruktor je jedna od mogućnosti za podrazumevani konstruktor(implementira se bez parametara, a koristi za inicijalizaciju objekta na početne vrednosti).

```
class Osoba {
public:
    string ime;
    int godine;
    Osoba() {
        ime = "nepoznato";
        godine = 0;
    }
};
```

2. **Konstruktor sa parametrima** je metoda koja se poziva kada se objekat klase kreira sa prosledjenim parametrima. Ovaj konstruktor se implementira sa jednim ili više parametara i koristi se za inicijalizaciju objekta sa specifičnim vrednostima.

```
class Osoba {
public:
    string ime;
    int godine;
    Osoba(string ime, int godine) {
        this->ime = ime;
        this->godine = godine;
    }
};
```

3. **Konstruktor kopije** je metoda koja se poziva kada se objekat klase kreira kopiranjem drugog objekta. Ovaj konstruktor se implementira sa jednim parametrom koji je objekat klase i koristi se za kreiranje novog objekta sa identičnim vrednostima kao originalni objekat.

```
class Osoba {
public:
    string ime;
    int godine;
    Osoba(const Osoba &o) {
        ime = o.ime;
        godine = o.godine;
    }
};
```

This - predstavlja pokazivac na trenutni objekat koji se obradjuje. Odnosi se na objekat koji se trenutno kreira. Korisno je koristiti "**this**" kada se imenuje lokalna varijabla ili argument koji ima isto ime kao i atribut klase. Kada se koristi **this** onda se jasno ukazuje da se odnosi na **atribut** klase (vidi konstruktor sa parametrima), a ne na lokalnu varijablu.

Destruktor - metoda koja služi za ciscenje resursa memorije. U C#-u se retko koristi jer ima ugradjen garbage collector.

Destruktor - metoda (funkcija) koja se poziva kada se objekat unisti ili ide izvan opsega (scope) u kojem je stvoren. Koristi se za rucno oslobadjanje resursa ili izvodenje drugih ciscenja kada objekat vise nije potreban. Memorija se oslobadja recimo u C#, a u C mora rucno da se dealocira. Zavisi da li je ugradjen garbage collector.

Garbage collector - mehanizam koji se koristi za automatsko upravljanje memorijom. Koristi se za identifikaciju i oslobadjanje memorije koja vise nije u upotrebi, a stvorena je tokom izvorsavanja programa. Automatski prati objekte u memoriji i oslobadja resurse koje objekti vise ne koriste, cime se smanjuje opasnost od curenja memorije. Primarni cilj je ocuvati resurse i spreciti prekoracenje memorije.

Boxing - konverzija tipa vrednosti u referentni tip.

```
int number = 42;
object boxedNumber = number; // Boxing: Konverzija int u object
Console.WriteLine(boxedNumber); // Ispisuje: 42
```

Unboxing - proces izvlačenja vrednosti iz objektnog u pretvaranje natrag u tip vrednosti. Neophodno je eksplicitno pretvaranje tipova(kastovanje).

```
object boxedNumber = 42;
int number = (int)boxedNumber; // Unboxing: Konverzija object u int
Console.WriteLine(number); // Ispisuje: 42
```

Primena boxinga i unboxinga

```
List<object> razliciteVrste = new List<object>();

int broj = 42;
string tekst = "Hello, world!";
double decimalniBroj = 3.14;
int[] nizBrojeva = new int[] { 1, 2, 3 };

razliciteVrste.Add(broj);
razliciteVrste.Add(tekst);
razliciteVrste.Add(decimalniBroj);
razliciteVrste.Add(nizBrojeva);

foreach (var item in razliciteVrste)
{
    if (item is int)
    {
        int brojIzListe = (int)item;
        Console.WriteLine($"Broj: {brojIzListe}");
    }
    else if (item is string)
    {
        string tekstIzListe = (string)item;
        Console.WriteLine($"Tekst: {tekstIzListe}");
    }
    else if (item is double)
    {
        double decimalniBrojIzListe = (double)item;
        Console.WriteLine($"Decimalni broj: {decimalniBrojIzListe}");
    }
}
```

Internal state

```
public class Person {  
    public string Name { get; set; }  
    public int Age { get; set; }  
    private bool isMarried;  
  
    public Person(string name, int age, bool isMarried) {  
        Name = name;  
        Age = age;  
        this.isMarried = isMarried;  
    }  
  
    public bool IsMarried() {  
        return isMarried;  
    }  
}
```

U ovom primeru, klasa `Person` ima tri svojstva koja opisuju njeno stanje: `Name`, `Age` i `isMarried`. Prva dva svojstva su javna i mogu se čitati i pisati izvan klase, dok je `isMarried` privatno svojstvo koje se može čitati samo iz klase putem metode `IsMarried()`. Ovo čuva stanje objekta od nepoželjne izmene izvan klase.

Garbage collector(GC) - deo koji automatski upravlja memorijom koja se koristi za cuvanje podataka. Osnovna funkcija je da pronalazi i uklanja objekte iz memorije koji vise nisu u upotrebi, cime se oslobadja memorija koja moze biti ponovo upotrebljena. Kod C i C++ se mora rucno alocirati i dealocirati memorija, dok kod Jave i C# imamo GC. Kada se objekat kreira, GC prati referencu na objekat i odrzava broj referenci na objekat u zivotu. Kada objekat vise nije referenciran, GC automatski uklanja objekat iz memorije kako bi se prostor oslobodio. Korisno za greske i lakse odrzavanje koda, ali moze usporiti program jer se izvorsava u pozadini.

IDisposable - koristi se da omoguci kontrolu nad oslobadjanjem resursa kojima ne upravlja GC. To su na primer **otvorene datoteke, baze podataka, mrezne veze** i slicno. IDisposable nam omogucava da eksplicitno(sami preuzimamo odgovornost) oslobodimo te resurse pozivom `Dispose()` metode. Posebno je vazno za resurse koji imaju ogranicenja, kao sto su maksimalan broj otvorenih konekcija prema bazi podataka. Rucno oslobadjanje resursa moze pomoci pri optimizaciji upotrebe resursa. Na taj nacin mozemo odmah osloboditi resurse koji nisu potrebni umesto da cekamo GC da pokrene proces ciscenja. IDisposable se cesto koristi sa using blokom koji obezbedjuje automatsko pozivanje `Dispose()` metode. **Cak i u slucaju izuzetka ili prekida izvorsavanja vrsi zatvaranje.** IDisposable je konvencija .NET Frameworka koja se koristi da se naznaci da klasa ima resurse koje treba osloboditi. Ako imamo vise resursa koje zelimo da zatvorimo kreiramo vise razlicitih metoda za oslobadjanje u `IDisposable` i pozivamo ih ovako:

```
myObject.DisposeResource1();  
myObject.DisposeResource2();
```

Sa druge strane postoji **finalize()** metoda koja je implicitno pozvana od GC da oslobodi resurse. U Javi se isto ovo(`Disposable`) zove `Closeable` i `AutoCloseable`

Async-await omogućavaju visenitno programiranje **bez direktnog formiranja niti**. Npr. Windows Form poseduje samo jedan thread - glavnu ili UI nit. Ako u toj niti koristis I/O operaciju aplikacija ce cekati dok ti pristupis nekom fajlu. Zbog toga windows pauzira nit tako da on ne koristi ni jedan CPU resurs, ali na taj nacin zadrzava memoriju. **Asinhrono programiranje** omogućava da pokrenemo neku metodu i nastavljamo da radimo drugi posao dok se metoda ne izvrši do kraja. Kod **sinhronog programiranja** moramo cekati da se metoda izvrši do kraja. **Async** se koristi da se markira metoda koja izvršava asinhronu operaciju. Sinhronicno se pokrece trenutna nit. Na **await** kompajler kreira kod koji cemo videti bez obzira da li je nasa asinhrona operacija završena.

Nit je jedinica obrade koja se moze izvršavati istovremeno sa drugim nitima unutar istog procesa.

Sinhrona nit se izvršava u skladu sa glavnom niti, tj. ceka na odredjene dogadjaje, poput unosa podataka, pre nego sto nastavi s izvršavanjem. Sinhrona nit ima tendenciju da blokira programski kod i zauzme resurse dok se dogadjaj ne pojavi, a zatim nastavlja sa izvršavanjem. Primer: GUI app gde se ceka unos teksta, klik misa,...

Asinhrona nit nastavlja sa izvršavanjem nezavisno o glavnoj niti, ne ceka na dogadjaje pre nego sto nastavi sa izvršavanjem.

AAA(Authentication, Authorization, Accounting) - skup praksi i tehnologija koje se koriste za sigurnost i upravljanje pristupom u aplikacijama i mrežama.

Autentifikacija - proces identifikacije korisnika aplikacije ili sistema, sto znaci da se korisnik potvrđuje kao legitimni korisnik pre nego sto mu se dozvoli pristup aplikaciji. Obicno ukljucuje proveru korisnickog imena i lozinke ili upotrebu drugih biometrijskih podataka.

Autorizacija - proces odredjivanja dozvola koje se dodeljuju korisniku nakon sto se autentifikuje. Ukljucuje dodeljivanje privilegija ili uloga korisnicima kako bi se omogućilo da pristupe samo odredjenim delovima aplikacije ili sistema.

Akaunting(racunovodstvo) - pracenje aktivnosti korisnika nakon sto se autentifikuje i dobije dozvolu za pristup. Ovaj proces ukljucuje belezenje i pracenje aktivnosti korisnika, kao sto su vreme provedeno na aplikaciji, radnje koje su izvršene i slicno.

Sigurnost se odnosi na zaštitu od neovlašćenog pristupa i korišćenja informacija, resursa i funkcija sistema ili aplikacije. To ukljucuje zaštitu podataka i sistema od hakera, virusa, krađe identiteta i drugih vrsta sajber napada. Odnosi se na **zastitu od namernih napada**.

Bezbednost se odnosi na zaštitu od svih vrsta opasnosti koje mogu ugroziti aplikaciju ili sistem. Ovo ukljucuje sigurnost, ali takođe obuhvata i druge faktore kao što su stabilnost, pouzdanost, zaštitu od grešaka u kodu, zaštitu od hardverskih kvarova i prirodnih katastrofa. Odnosi se na zastitu od **stetnih uticaja bilo koje vrste, ukljucujuci i nesrece, greske i kvarove**.

Delegat je u programiranju koncept(u sustini to je klasa) koji opisuje tip podataka koji moze da referencira na jednu ili vise metoda i da ih izvršava kada se poziva. Umesto da koristimo direktan poziv na odredjenu metodu koristimo delegat da bi tu metodu pozvali. Cesto se koristi kao nacin za prenosenje metoda kao argumenta drugoj metodi ili funkciji. Koristi se za dinamicko vezivanje metoda na neku varijablu, sto znaci da se ta varijabla moze koristiti za pozivanje razlicitih metoda u zavisnosti od situacije. Delegat predstavlja referencu na metodu, a kada se pozove zapravo se poziva metoda samo se vrši preko delegata. **Time se omogućava da se metode lako menjaju, tako sto se delegat postavlja da pokazuje na drugu metodu** koja ima isti potpis kao i prvobitna.

```

delegate void ProcessDataDelegate(int[] data);

class Program
{
    static void Main()
    {
        int[] data = { 1, 2, 3, 4, 5 };

        ProcessDataDelegate processDataDelegate = new ProcessDataDelegate(DoubleData);
        processDataDelegate += new ProcessDataDelegate(SquareData);

        processDataDelegate(data);
    }

    static void DoubleData(int[] data)
    {
        for (int i = 0; i < data.Length; i++)
        {
            data[i] *= 2;
        }
    }

    static void SquareData(int[] data)
    {
        for (int i = 0; i < data.Length; i++)
        {
            data[i] *= data[i];
        }
    }
}

```

Rezultat je

```

Original data: 1 2 3 4 5
Doubled data: 2 4 6 8 10
Squared data: 4 16 36 64 100

```

- Metodu je moguće proslediti drugoj metodi kao argument bez modifikacije koriscenjem delegata.
- Ispisivanje vrednosti multidimenzionalnih nizova Niz[i,j]
- Try-finally može da se koristi bez catch.
- Using nije isto što i try-finally. Using omogućava da se resursi oslobode nakon što se završi sa radom.
- U jednom try bloku može biti definisano više catch blokova, a program će probati da odgovori na prvi exception. Korisno kada se izuzetak može obraditi na različite načine u zavisnosti od tipa izuzetka.

Apstraktna klasa ne može da ima ni jednu instancu

Interfejs može da ima beskonacno instanci

Sta je objekat, a sta klasa?

Klasa je sablon za kreiranje objekta. Objekat je instanca klase.

Klasa je zivotinja, a objekti tj. instance su: lav, macka, pas...

Sealed klasa - ne može biti nasledjena

Entity framework core je objekt-relacioni mapper za .NET Core

Mapira redove u tabelama baze podataka na objekte klasa modela i obrnuto

Omogucava interakciju sa bazom podataka bez pisanja SQL upita i parsiranja odgovora

Ako je u nekom sistemu kriticna brzina bolje je pisati SQL upite rucno

Proces rada u EF:

- Code-First: kreiranje klasa pa na osnovu njih kreiramo bazu podataka
- Database-First: iz postojece baze podataka kreiramo klase u programu
- Model-First: nekim alatom za modelovanje kreiramo model klasa i na osnovu njega se generisu klase iz baze podataka

Veza 1:N zavisni entitet je onaj koji ima strani kljuc

kada je 1:1 neophodno je reci koji entitet je zavisan jer EF ne moze sam zakljuciti

Kod veze N:N EF zna da treba da kreira trecu povezanu tabelu

DTO - data transfer objekti nose podatke izmedju procesa ili slojeva aplikacije

on se u troslojnoj arhitekturi nalazi izmedju slojeva

Kada je objekat u C# **serijalizovan** on je konvertovan u byte stream

```
val = Console.ReadLine();  
  
// convert to integer  
res = Convert.ToInt32(val);  
  
// display the line  
Console.WriteLine("Input = {0}", res); <-- Ispis
```

String interpolacije \$

```
string name = "John";  
int age = 30;  
  
string message = $"My name is {name} and I am {age} years old.";  
Console.WriteLine(message);
```

ovo su viticaste zagrade

Interfejs - grupisemo zajednicko ponasanje vise klasa(to ponasanje su metode; grupisemo metode bez implementacije). Kasnije u klasama prosledimo i tako **izbegavamo redundantnost(suvisnost) tj. ponavljanje metoda**. Da bi klasa implementirala interfejs mora da implementira sve njegove metode. Interfejs je tip. Svaka navedena clanica **implicitno ima public pristup i vidljivost nije moguće menjati**. Interfejs moze naslediti jedan ili vise interfejsa, a klase mogu samo jednu klasu i vise interfejsa. Nije moguće kreirati instancu interfejsa. Interfejs ne sadrzi polja kao klase, interfejs sadrzi isključivo apstraktne clanice koje mogu biti dogadjaji intekseri(tajmeri), metode ili svojstva.

Apstraktna klasa - je tip. Karakteristike: moze imati podrazumevanu implementaciju metoda, moze imati polja, apstraktne metode tj. metode bez implementacije, ne moze da se instancira. Koristi se kao osnova za druge klase koje nasledjuju njene attribute i metode. **Koristi se kada postoji podrazumevano ponasanje koje implementiramo**, dok se izvedenoj klasi omogucava implementacija pojedinih metoda. Koriste se i kada se ocekjuje dodavanje nove funkcionalnosti jer je to olaksano cinjenicom da je moguće dodati metodu u izvedenoj klasi, a da pri tome nije potrebno menjati klijentski kod(klase koje su vec izvedene iz osnovne). Primer apstraktne klase sa svim delovima:

```

public abstract class Shape {
    // Apstraktna metoda za izračunavanje površine oblika
    public abstract double calculateArea();

    // Metoda sa implementacijom za izračunavanje obima oblika
    public double calculatePerimeter() {
        return 0.0;
    }

    // Polje za boju oblika
    private String color;

    // Konstruktor sa parametrom za postavljanje boje
    public Shape(String color) {
        this.color = color;
    }

    // Getter za boju
    public String getColor() {
        return color;
    }

    // Setter za boju
    public void setColor(String color) {
        this.color = color;
    }
}

```

Konkretna klasa - ona koja se može instancirati (kreirati objekt tog tipa). Svaka konkretna klasa koja implementira interfejs mora implementirati sve članice.

Virtual - koristimo kada želimo da implementiramo podrazumevano ponašanje, a ipak dozvolimo izvedenim klasama da implementiraju drugacije ponašanje

Abstract metode - metode koje će se implementirati tek u izvedenim klasama se obeležavaju sa `abstract`. Kada se metode implementiraju koristi se `override`. Kada se kreira podklasa koja nasleđuje superklasu, podklasa može predefinisati (override) metode iznad nje kako bi prilagodila ponašanje klase svojim potrebama. Pri definisanju metode, metoda u podklasi ima isto ime, argumente i povratnu vrednost kao i metoda u nadklasi, ali razlicitu implementaciju koja je specifična za podklasu. Apstraktne metode **ne mogu da imaju implementaciju u trenutku deklarisanja u baznoj klasi (abstract klasa), implementacija se očekuje u bilo kojoj potklasi koja nasleđuje baznu klasu**. Potrebno je navesti njeno ime, povratni tip i parametre koje prima. **Svaka potklasa koja nasleđuje tu baznu klasu mora da metodu implementira na svoj način.**

Primer: `public abstract void DoSomething();`

Using blok nam govori da je objekt raspoloživ dok **try-catch** koristimo za exception handling (rukovanje greskama, hvatanje izuzetaka). Using je bolji kod velikih projekata jer na kraju zatvara sve, kao što bi se u finally radilo kod garbage collector-a, pa je više optimizovan using. **Finally** odradi svoj deo i ako je postojao error i ako nije.

Exceptions(izuzeci) - Dogadjaj koji ukazuje na neocekivanu situaciju ili gresku u logici programa. Koriste se za kontrolisano upravljanje greskama i omogucavaju da se program nastavi izvorsavati i izvrsi odgovarajuca logika cak i ako se javi greska. Obicno se generisu kada se otkrije greska ili se dogodi nesto sto narusava ocekivano ponasanje programa. Kada se uhvate u catch bloku omogucavaju da preduzmemo odgovarajuce akcije ili prikazemo korisniku informativnu poruku. Pruzaju fleksibilnost u obradi gresaka i omogucavaju da preduzmemo odgovarajuce korake za oporavak ili prikazivanje informacije o gresci.

Error(greska) - Predstavlja neocekivati problem ili situaciju koja se javlja tokom izvorsavanja programa. Dolazi do prekida normalnog izvorsavanja programa. Mogu biti hardverski problemi, sistemske ili programske greske. Primer su: greske u sintaksi, deljenje nulom, prekoracenje kapaciteta, nepostojanje resursa.

Za kontrolisano upravljanje greskama se koristi try-catch-finally.

Try - obuhvatanje dela koda u kojem se moze javiti izuzetak ili greska.

Catch - koristi se za hvatanje izuzetka koji se dogodi unutar try bloka i navodi se kod koji treba da se izvrsi ukoliko se dogodi odgovarajuci izuzetak

Finally - Kod koji ce biti izvrsen cak i ako try/catch imaju return ili ukoliko se izuzetak nije uhvatio

Throw - koristi se za rucno generisanje izuzetaka unutar bloka try. Njime mozemo prekinuti normalno izvorsavanje koda i generisati izuzetak sa specifcnom porukom.

```
try
{
    int num = -1;

    if (num < 0)
    {
        throw new ArgumentException("Broj ne može biti negativan.");
    }

    Console.WriteLine("Ovaj kod će se izvršiti samo ako nema izuzetka.");
}
catch (ArgumentException ex)
{
    Console.WriteLine("Uхваćen je izuzetak: " + ex.Message);
}
```

Ispis bi bio: Uhvacen je izuzetak: Broj ne moze biti negativan.

Namespace - kontejner u koji smestamo da bi sprecili koliziju tj. preplitanje; jedinstveni identifikator za odredjeni set pojmova. Primer: imamo projekat koji rade Pedja, Zika i Pera. Svako od nas napravi klasu "niz". Ako pokusamo da spojimo sve u celinu nastaje kaos. U tom slucaju se primenjuje namespace koji kaze Pedja::Niz, Pera::Niz, Zika::Niz i svaka klasa je rasporedjena tamo gde pripada.

XML - masinski citljiv format(struktuiran, ima pravila), struktura stabla(nasledjivanja). Njegove etikete nisu definisane i njegova prosirivost je u mogucnosti dodavanja novih etiketa koje daju znacenje podacima. Sve podatke smestamo u obliku stringova i to su elementi. Uvek imamo otvarajucu i zatvarajucu etiketu. Moguce je dodati atribut pod navodnicima, ali imaju mane: teska manipulacija iz programskog koda, teska prosirivost, ne opisuju strukturu, ne mogu da sadrze visestruke vrednosti.

```

public class Baza
{
    public static void DodajKorisnika()
    //kada iz jedne klase zelim da pristupim metodi druge klase, koristi se static pa znaci da pripada
    klasi, a ne objektu klase(moze se pozvati bez kreiranja instance klase)
    public static PozicijaPolja[, ] BFSPronadji(LineEntity line, char[, ] BFSlinije)

    PozicijaPolja[, ] prev = BFSPath.BFSPronadji(line, BFSprom.BFSlinije);

    drugaKlasa.Metoda

```

Static promenljiva - recimo imamo 500 studenata. Svako ima ime, prezime i fakultet. Za svako ime i prezime moramo zauzeti memoriju, a za fakultet mozemo uzeti staticku promenljivu. Ta promenljiva zauzima samo jednu memorijsku jedinicu. To polje ce dobiti memoriju samo jednom i ta osobina se deli na sve objekte. **Nestaticke promenljive pripadaju instanci klase, a staticke metode samoj klasi.**

Staticki metod - pripada klasi, a ne objektu klase. Moze biti pozvan bez potrebe da se kreira instanca klase. Moze pristupiti statickim podacima i moze menjati njihovu vrednost. Main metod je statican zato sto objekat ne mora da poziva staticki metod; ako bi main bio ne-staticki prvo bi se kreirao objekat, a zatim pozivao main() metod sto bi dovelo do problema alokacije dodatne memorije.

--Moze pristupiti samo statickim podacima(ne moze inicijalizovati nestaticke prom.), ne moze da se instancira, instance funkcija mogu da pozovu staticke funkcije i pristupe statickim podacima.

Staticka promenljiva se menja ili direktnim pozivom klase ili preko poziva staticke metode:

```

public class MojKod
{
    public static int brojac;

    public static void PovecajBrojac()
    {
        brojac++;
    }
}

MojKod.brojac = 10;
Console.WriteLine(MojKod.brojac); // Ispisuje 10

MojKod.PovecajBrojac();
Console.WriteLine(MojKod.brojac); // Ispisuje 11

```

Staticki blok - koristi se da inicijalizuje staticki podatak-clan. Izvrsava se pre main metode u vremenu učitavanja klase.

Staticka ugnjezdena klasa - untrasnja staticka klasa u klasi, služi kao mehanizam struktuiranja tipova. Untrasnje klase u klasi omogucuju poseban odnos izmedju njihovih objekata i objekata spoljasnje klase.

Konstantne varijable (const) se koriste kada se vrednost promenljive ne sme menjati tokom izvršavanja programa. One su korisne za definisanje nekih stalnih vrednosti u kodu, poput matematičkih konstanti ili fiksni vrednosti koje se koriste u aplikaciji. $\pi = 3,14$

Member variable (član promenljiva) je promenljiva koja je deklarirana unutar klase i može biti pristupljena i korišćena u okviru te klase. Member variable **pripada određenom objektu klase i svaki objekat klase ima svoju kopiju member variable. Member variable se često naziva i instance variable**, jer je svaki objekat klase instanca koja ima svoje vlastite vrednosti member variable. Member variable je definisan na nivou klase i može biti vidljiv i dostupan u svim metodama, konstruktorima i drugim delovima klase.

```

public class Person
{
    public string name; // Member variable

    public void Display()
    {
        Console.WriteLine("Name: " + name);
    }
}

```

```

namespace IndiabixConsoleApplication
{
    class Sample
    {
        static Sample()
        {
            Console.Write("Sample class ");
        }
        public static void Bix1()
        {
            Console.Write("Bix1 method ");
        }
    }
    class MyProgram
    {
        static void Main(string[] args)
        {
            Sample.Bix1();
        }
    }
}

```

Ispis ce biti: "Sample class Bix1 method". Kada se pozove klasa staticki konstruktor se automatski poziva. Ukoliko izbacimo taj konstruktor ispis ce biti "Bix1 method".

LINQ(Language-Integrated Query) - omogucava izvršavanje upita nad razlicitih izvorima podataka, kao sto su kolekcije objekata, baze podataka, XML dokumenti i druge strukture podataka. Prednosti: LINQ je integrisan u jezik(C#), dostupan u svim .NET aplikacijama bez dodatnih biblioteka, univerzalnost(moze biti koriscen za razlicite izvore podataka poput kolekcije objekata, SQL baze podataka, XML dokumente, JSON podatke...).

```

var query = from num in numbers
             where num > 3
             orderby num descending
             select num;

```

Rekurzija - funkcija poziva samu sebe sa manjim ili slicnim problemom u odnosu na trenutni problem, sve dok ne dostigne osnovni slucaj ili uslov zaustavljanja. Vrsi se rekurzivna dekompozicija - obradjuju se manji podproblemi, umesto da se pokuša resavanje kompletnog resenja odjednom. Smanjuje se ponavljanje koda jer se koristi isti za svaki slucaj. Moze se desiti prekoracenje steka(stack overflow) ili beskonacno rekurzivno izvršavanje.

Deepcopy (duboka kopija) - kopija objekta čija svojstva ne dele iste reference(ne upućuju na iste osnovne vrednosti) kao one izvornog objekta od kojeg je napravljena kopija. Kao rezultat toga, kada promenimo ili izvor ili kopiju, možemo biti sigurni da ne izazivamo promenu ni drugog objekta; to jest, nećemo nenamerno izazvati promenu u izvoru ili kopiji koje ne očekujemo. To ponašanje je u suprotnosti sa ponašanjem **Shallowcopy (plitke kopije)**, u kojoj promene izvora ili kopije također mogu prouzrokovati promenu drugog objekta(jer dva objekta dele iste reference). **Deep copy** u kome se u 'listaVodova' ubacuju svi elementi liste 'listaNeobrisanihVodova', bez .ToList() bi se prosledila samo referenca i onda bi to bio shallow copy, a ovako je deep copy:

```
listaVodova = listaNeobrisanihVodova.ToList();
```

Web servis je aplikacija smeštena na nekom serveru, koja je pored osnovne namene dizajnirana da **podrži interakciju između dve mašine** preko mreže i **omogućiti razmenu informacija** između njih. Smatra se da je svaki servis i web servis ako je:

- Dostupan preko interneta ili (interne mreže)
- Koristi standardizovan sistem poruka
- Prepoznatljiv je od strane mehanizma za pretragu
- Nije vezan za operativni sistem ili programski jezik

Web servisi u saradnji sa GUI aplikacijom su nešto između web i desktop aplikacije. Web servis na serveru daje funkcionalnost i podatke, a desktop aplikacija samo daje prilagodljivi grafički interfejs koji popunjava sa dobijenim podacima od servisa. Web servis u saradnji sa GUI aplikacijom je **fleksibilniji od web aplikacije** jer korisnik može da prilagodi izgled aplikacije na klijentu dok korisnik web aplikacije mora da koristi web browser i nema uticaja na to kako će aplikacija izgledati na ekranu. Primeri web servisa: google maps API, tiwtter API, youtube API, weather API, e-commerce API... Prednosti u odnosu na web aplikaciju:

Štedljiviji su po pitanju opterećenja mreže i resursa servera jer pri komunikaciji šalju samo odgovor dok web aplikacije pored odgovora šalju i HTML sa formom i opisom kako odgovor treba da bude prikazan. Web servisi su idealni su za "male" uređaje koji nisu PC tj. mobilne Pocket PC... jer je na takvom uređaju instalirana samo front-end aplikacija, a teži deo se odradjuje na serveru.

Lakši su za razvoj, testiranje i održavanje. Kod Web aplikacije pored neophodne funkcionalnosti je potrebno istestirati i dizajn na svim browserima i platformama dok kod web servisa autor brine samo o funkciji dok o prikazu brine klijentska aplikacija.

Kod servisa se komunikacija odvija između klijenta i servera tako što **klijent pošalje zahtev serveru** koji onda server taj zahtev primi i obradi ga pa u odnosu na njega formira odgovor koji zatim pošalje nazad klijentu. Klijent-server stil arhitekture odvaja problematiku dve strane komunikacijskog kanala, što znači da klijentsku stranu uopšte ne zanima način čuvanja informacija na serveru jer **uvijek postoji uniforman način pristupa tim resursima**. S druge strane, server je nezavisan od klijenta i ne zanima ga kako je implementiran korisnički interfejs niti u kojem je stanju pojedini klijent, čime je serverska strana znatno pojednostavljena. Ovime je postignuta njihova nezavisnost i lakše razdvojeno razvijanje obe strane, odnosno moguće je npr. unaprediti serversku stranu ili potpuno izmeniti njegovu logiku, a da klijent to uopšte ne primeti dok god je način pristupa resursima isti.

Deadlock je kada više procesa ili niti čekaju jedni druge da završe posao, a pri tome blokiraju jedni druge tako da ne mogu da nastave sa izvršavanjem svojih zadataka. Najcesce čekaju da drugi oslobodi resurs.

Static i abstract klasa ne mogu da se instanciraju. Na pitanje kako se zove klasa koja ne moze da se instancira odgovor je apstraktna. Apstraktne ne mogu da se instanciraju direktno vec se koriste kao bazne klase za druge koje ih nasledjuju. Staticki clanovi pripadaju celokupnoj klasi, a ne pojedinacnim instancama te klase. Mogu se koristiti bez potrebe za stvaranjem instance klase.

Eksplicitni poziv (engl. explicit invocation) se odnosi na situaciju kada se poziv funkcije ili metode vrši direktno, odnosno eksplicitno, od strane programera. To znači da programer namerno navodi koju funkciju ili metodu želi da pozove i prosleđuje potrebne argumente.

```
Klasa obj = new Klasa();  
obj.Metoda(); // Eksplicitni poziv metode
```

Main metoda

To je glavni tok izvršavanja programa: sva logika programa, poput poziva drugih metoda, izvršavanja operacija ili interakcije sa korisnikom. U njoj programer moze pokrenuti i kontrolisati izvršavanje drugih delova programa. Operativni sistem pronalazi i pokrece metodu Main(). Povratni tip je void sto znaci da ne vraca nikakvu vrednost. Oznacena je sa "static" sto znaci da je dostupna bez kreiranja instance klase. Prihvata argument "string[] args", koji predstavlja argumente komandne linije koje program moze da primi pri pokretanju.

Operatori:

~ -koristi se za invertovanje(1 u 0 i 0 u 1)

```
byte b1 = 0xF7;           1111 0111  
byte b2 = 0xAB;           1010 1011  
byte temp;  
temp = (byte)(b1 & b2);   1010 0011  
Console.Write (temp + " "); Ekskluzivno ili  
temp = (byte)(b1 ^ b2);   0101 1100  
Console.WriteLine(temp);
```

Ⓜ 163 92 ✓

13/2 vraca celo brojnu vrednost -> 6

& moze da se koristi da se bit stavi na OFF tj. da se moze koristiti za iskljucivanje(postavljanje na 0).
Takodje pomocu njega mozemo proveriti da li je bit OFF ili ON(0 ili 1)
| moze da se koristi da stavi bit na ON

Rezultat je **7 7**

```
int num = 1, z = 5;  
  
if (!(num <= 0))  
    Console.WriteLine( ++num + z++ + " " + ++z );  
else  
    Console.WriteLine( --num + z-- + " " + --z );
```

Zelimo da isključimo 4. bit sa desna tako da ne poremetimo ostale (nesto & 1111 0111) - ako je 0 vratice 0, a ako je 1 vratice 1. Kada dodje do 4. sa desna ne može da vrati 1 jer je 0 u drugom izrazu.

n = n & 0xF7

```
byte b1 = 0xAB;           1010 1011
byte b2 = 0x99;           1001 1001
byte temp;
temp = (byte)~b2;          Inverzijom svih bita dobijamo
                           0110 0110 = 102
Console.WriteLine(temp + " ");
temp = (byte)(b1 << b2);   bitovi b1 se pomeraju za b2 u levo pa
                           je rezultat u binarnom 0000 0000
Console.WriteLine(temp + " ");
temp = (byte)(b2 >> 2);    b2 bitovi se pomeraju za 2 mesta u desno
                           0010 0110 = 38
Console.WriteLine(temp);
```

102 0 38 ✓

```
int i, j = 1, k;
for (i = 0; i < 5; i++)
{
    k = j++ + ++j;
    Console.WriteLine(k + " ");
}
```

1.

Prvi prolaz petlje (i = 0):

- a. Vrednost j je 1.
 - b. Izraz j++ + ++j se računa sledećim koracima:
 - i. Prvo se evaluira j++, što daje vrednost 1. Nakon toga, vrednost j postaje 2.
 - ii. Zatim se evaluira ++j, što daje vrednost 3.
 - iii. Zbir j++ + ++j je 1 + 3 = 4.
 - c. Ispisuje se vrednost 4.
2. Drugi prolaz petlje (i = 1):
- a. Vrednost j je 3 (povećana u prethodnom prolazu petlje).
 - b. Izraz j++ + ++j se računa sledećim koracima:
 - i. Prvo se evaluira j++, što daje vrednost 3. Nakon toga, vrednost j postaje 4.
 - ii. Zatim se evaluira ++j, što daje vrednost 5.
 - iii. Zbir j++ + ++j je 3 + 5 = 8.
 - c. Ispisuje se vrednost 8.
3. Treći prolaz petlje (i = 2):
- a. Vrednost j je 5 (povećana u prethodnom prolazu petlje).
 - b. Izraz j++ + ++j se računa sledećim koracima:
 - i. Prvo se evaluira j++, što daje vrednost 5. Nakon toga, vrednost j postaje 6.
 - ii. Zatim se evaluira ++j, što daje vrednost 7.
 - iii. Zbir j++ + ++j je 5 + 7 = 12.
 - c. Ispisuje se vrednost 12.
4. Četvrti prolaz petlje (i = 3):
- a. Vrednost j je 7 (povećana u prethodnom prolazu petlje).
 - b. Izraz j++ + ++j se računa sledećim koracima:
 - i. Prvo se evaluira j++, što daje vrednost 7. Nakon toga, vrednost j postaje 8.
 - ii. Zatim se evaluira ++j, što daje vrednost 9.

- iii. Zbir $j++ + ++j$ je $7 + 9 = 16$.
 - c. Ispisuje se vrednost 16.
- 5. Peti prolaz petlje ($i = 4$):
 - a. Vrednost j je 9 (povećana u prethodnom prolazu petlje).
 - b. Izraz $j++ + ++j$ se računa sledećim koracima:
 - i. Prvo se evaluira $j++$, što daje vrednost 9. Nakon toga, vrednost j postaje 10.
 - ii. Zatim se evaluira $++j$, što daje vrednost 11.
 - iii. Zbir $j++ + ++j$ je $9 + 11 = 20$.
 - c. Ispisuje se vrednost 20.

```
int a = 10, b = 20, c = 30;
int res = a < b ? a < c ? c : a : b;
Console.WriteLine(res);
```

Prvo se radi ($a < b$), ako je uslov uspunjen izracunava se izraz ' $a < c ? c : a$ '. Ako uslov nije zadovoljen onda se ispisuje b . Posmatra kao da je funkcija ' $a < b ? fun : b$ ', posto je tacno radi se fun.

Uslovni operator ($?:$) vraca jednu od dve vrednosti u zavisnosti od vrednosti bulovog izraza. Operator **as** u C# .NET-u se koristi za obavljanje konverzija izmedju kompatibilnih referentnih tipova. Operator $\rightarrow$ kombinuje dereferenciranje pokazivaca i pristup clanovima.

Stringovi:

Kopiranje stringa $s1$ u string $s2$

```
String s1 = "String";
String s2;
s2 = String.Copy(s1);
```

Budući da je `String` u C# nemutabilan (immutable) tip podataka, to znači da se ne može menjati nakon kreiranja. Kada se kreira string sa određenim sadržajem, a zatim se drugoj promenljivoj dodeli referenca na taj isti string, ta druga promenljiva će takođe referisati isti objekat u memoriji. Tako da, iako se koristi dve promenljive $s1$ i $s2$, one će referisati isti objekat stringa "Hi". Zbog toga se može reći da će samo jedan objekat biti kreiran u ovom kodu.

```
String s1, s2;
s1 = "Hi";
s2 = "Hi";
```

String.Compare - vraca 0 ako su jednaki, 1 ako je prvi veci od drugog, -1 ako je drugi veci.

String.CompareTo - Da li su jednaki gleda po vrednost ("FIVE STAR" > "Five Star")

Konvertovanje u string

```
Single f = 9.8f;  
String s;  
s = Convert.ToString(f);
```

```
Single f = 9.8f;  
String s;  
s = f.ToString();
```

Konvertovanje stringa u int

```
String s = "123";  
int i;  
i = int.Parse(s);
```

```
String s = "123";  
int i;  
i = Convert.ToInt32(s);
```

```
String s = "123";  
int i;  
i = CInt(s);
```

Stringovi se kreiraju na heap-u. Heap je memorija gde se skladište objekti koji žive više od jedne funkcije. Stringovi se čuvaju na heap-u jer su nemutabilni (ne mogu se promeniti nakon što su kreirani) i često imaju promenljivu dužinu. Heap pruža fleksibilnost za alokaciju memorije za stringove promenljive dužine, dok je stek bolji za čuvanje manjih podataka sa kratkim životnim vekom.

Provera da li je sadržaj stringova jednak

```
if (s1 == s2)
```

```
int c;  
c = s1.CompareTo(s2);
```

Propertiji:

Mogu biti ili read only ili write only.

Propertiji (svojstva) su zapravo metode koje rade kao članovi podataka.

Indeksator u C# omogućava da se **klasa, struktura ili interfejs** indeksiraju kao niz. To znači da možete pristupiti elementima objekta pomoću sintakse indeksiranja, slično kao kod nizova.

Indeksator (indexer) u C# je specijalna vrsta propertija koji omogućava pristup elementima objekta putem indeksa. Radi slično kao niz (array), ali se može primeniti na bilo koji objekat koji implementira indeksator.

Pravilan nacin koriscenja indexed proprijetija

```
Student s = new Student();  
s[3] = 34;
```

```
Student s = new Student();  
Console.WriteLine(s[3]);
```

U ovom primeru, klasa MyCollection ima indeksator this[int index] koji omogućava pristup elementima objekta na osnovu indeksa.

-- Potpis indeksera sastoji se od broja i tipova njegovih formalnih parametara, indekseri su slicni svojstvima osim sto njihovi pristupnici uzimaju parametre, tip indeksera i tip njegovih parametara moraju biti dostupni barem koliko i sam indeks.

```
public class MyCollection  
{  
    private string[] items = new string[5];  
  
    public string this[int index]  
    {  
        get { return items[index]; }  
        set { items[index] = value; }  
    }  
}  
  
class Program  
{  
    static void Main(string[] args)  
    {  
        MyCollection collection = new MyCollection();  
        collection[0] = "Prvi element";  
        collection[1] = "Drugi element";  
        collection[2] = "Treci element";  
  
        Console.WriteLine(collection[1]); // Output: Drugi element  
    }  
}
```

Polimorfizam:

Overload je moguc za true, false, + operatore

Kada se koristi kao modifikator, kljucna rec 'new' eksplicitno sakriva clana nasledjenog iz osnovne klase. Menja ponasanje osnovne klase zamenom clana osnovne klase novim izvedenim clanom.

Ključna rec 'operator' se koristi za overload korisnicki definisanih tipova definisanjem funkcija statickih clanova.

```
public static sample operator + ( sample a, sample b )
```

Virtual mogu biti: metode, property i event-i

Za run-time polimorfizam je neophodno:

Metoda base klase koja je overridden mora biti virtual, abstract ili override.

I override metoda i virtual metoda moraju imati isti modifikator nivoa pristupa.

Apstraktna metoda je implicitno virtuelna metoda.

Ako nasledjena klasa ne obezbedi svoju verziju virtuelne metode koristice se metoda base klase

Overload nije moguc za sledece operatore:

1. `?:` (uslovni operator)
2. `&&` (uslovni logički operator "i")
3. `||` (uslovni logički operator "ili")
4. `.` (operator tačke)
5. `::` (operator dvostrukog dvotačka)
6. `=>` (operator lambda izraza)
7. `sizeof` (operator za dobijanje veličine tipa)
8. `typeof` (operator za dobijanje tipa)
9. `is` (operator provere tipa)
10. `as` (operator konverzije)
11. `checked` (operator provere prekoračenja)
12. `unchecked` (operator bez provere prekoračenja)
13. `new` (operator kreiranja novog objekta)

Implicitno znači da se dešava automatski, bez potrebe za eksplicitnim navedenjem ili definisanjem. U tom slučaju, programski jezik ili sistem automatski obavlja određene radnje ili donosi određene odluke na osnovu konteksta ili unapred definisanih pravila.

Eksplicitno znači da se jasno i nedvosmisleno navede ili definiše. Programer mora eksplicitno navesti ili definisati određene akcije, pravila ili konfiguracije da bi se stvari odvijale na željeni način.

```

class Baseclass
{
    public void fun()
    {
        Console.WriteLine("Base class" + " ");
    }
}
class Derived1: Baseclass
{
    public new void fun()
    {
        Console.WriteLine("Derived1 class" + " ");
    }
}

class Baseclass2
{
    public virtual void fun()
    {
        Console.WriteLine("Base2 class" + " ");
    }
}
class Derived2 : Baseclass2
{
    public override void fun()
    {
        Console.WriteLine("Derived2 class" + " ");
    }
}

```

```

public static void Main(string[] args)
{
    Console.WriteLine("Ključna rec 'new'\n");
    Derived1 d1 = new Derived1();
    d1.fun();
    Console.WriteLine("\n");
    Baseclass bc = new Baseclass();
    bc.fun();

    Console.WriteLine("\nKljučna rec 'override'\n");
    Derived2 d2 = new Derived2();
    d2.fun();
    Console.WriteLine("\n");
    Baseclass2 bc2 = new Baseclass2();
    bc2.fun();

    Console.WriteLine("\nPozivanje preko base klase\n");
    Baseclass2 bc22 = new Derived2();
    bc22.fun(); //Prepise preko base
    Console.WriteLine("\n");
    Baseclass bc11 = new Derived1();
    bc11.fun(); //Ispise base jer se koristilo new
}

```

```

Ključna rec 'new'

Derived1 class Base class
Ključna rec 'override'

Derived2 class Base2 class
Pozivanje preko base klase

Derived2 class Base class

```

Kada se koristi **new** mora da bude public ispred. Ne moram da pisem public za klase jer su **package-private**. To znaci da main program tretira klase u istom fajlu kao da su public, a ne private. Private su za sve van tog fajla.

Postavljanjem reference tipa `BaseClass` na objekat klase `ChildClass`, omogućeno je korišćenje objekta kroz interfejs koji nudi `BaseClass`, čime se pridržava principa polimorfizma. Ovo omogućava da se pristupi metodama i svojstvima definisanim u `BaseClass`, ali i da se iskoristi specifična implementacija tih metoda i svojstava u `ChildClass` ukoliko su one predefinisane.

Delegati:

Koriste se za multithreading, event handling. Predstavljaju reference na metode i koriste se za implementaciju povratnog poziva(callback) i događaja(events). Omogućavaju da se metoda prosledi kao argument ili da se dodeli promenljivoj, kako bi se izvršila kasnije.

Deklaracija klase je neophodna za implementiranje delegata. Ključni razlog zašto delegati zahtevaju deklaraciju klase je da bi se obezbedio mehanizam za pozivanje metoda kroz delegat. Pozivanje metoda preko delegata uključuje pokazivače na metode, referencu na ciljni objekat (ako je metoda nestatička) i odgovarajuću reprezentaciju potpisa metode. Sve to zahteva stvaranje odgovarajuće klase koja može izvršiti taj zadatak.

```

delegate void del(int i);

```

Deklarisanjem delegata klasa 'del' ce biti kreirana, 'del' klasa ce biti nasledjena od `MulticastDelegate` klase, 'del' klasa ce sadrzati konstruktor sa jednim argumentom i 'invoke()' metodu

Delegati su type-safe. To znaci da se vrsi provera kako bi se obezbedilo da mu se dodeljuju samo metode sa odgovarajucim potpisom.

Obezbedjuju omotac za pokazivace funkcija. To znaci da omogucavaju koriscenje funkcija kao

vrednosti i njihovo prosledjivanje kao parametara.

Deklaracija delegata mora da odgovara potpisu metode koju nameravamo da pozovemo koristeći ga.

Delegati se ne mogu koristiti za pozivanje procedura koje primaju promenljiv broj argumenata.

Mozemo koristiti delegat kada ne znamo koji objekat treba biti obavesten da je kliknut dok runtime ne bude u toku.(program je aktivan i korisnik klikne dugme)

Delegat je referencnog tipa.

Ako imamo sortiranje objekata bilo kog tipa(int, Single, byte...) i zelimo da implementiramo funkciju za poredjenje koristicemo delegat.

Kljucna rec 'ref' moze da se koristi uz staticke funkcije/subroutes(potprograme) i instance funkcija/subroutes.

Primer delegata:

```
delegate int del(int i);  
del d;  
Sample s = new Sample();  
d = new del(ref s.MyFun);  
d(10);
```

```
delegate void del(int i, Single j);  
del d;  
Sample s = new Sample();  
d = new del(ref s.MyFun);  
d(10, 1.1f);
```

Datatypes - tipovi podataka:

Ako integer literal(celobrojni literal) premasuje opseg bajtova, pojaviće se greska pri kompilaciji.

Ne mozemo implicitno pretvoriti neliteralne numericke tipove vece velicine skladista u bajt.

Mozemo kastovati integralne kodove znakova(integral character codes).

Integer(celobrojni) tipovi su byte, Integer, Short, Long... Char obicno nije svrstan jer ima velicinu od 2 bajta i moze da predstavi samo jedan znak; koristi se za rad sa znakovima i stringovima, dok ne koristimo aritmeticke operacije za njega; celobrojni tipovi podataka imaju siru primenu u matematickim operacijama, brojanju, indeksiranju i slicno.

Widening conversions(proširenje konverzija) se odnosi na automatsko konvertovanje vrednosti iz jednog tipa podatka u drugi veci tip podatka koji moze da prihvati siri raspon vrednosti. Odvija se automatski bez potrebe za eksplicitnim deklarisanjem ili koriscenjem dodatnih operacija.

```
int num = 10;
long bigNum = num; // Automatsko proširenje konverzija iz int u long

float value = 3.14f;
double biggerValue = value; // Automatsko proširenje konverzija iz float u double

char character = 'A';
int unicodeValue = character; // Automatsko proširenje konverzija iz char u int

Console.WriteLine(bigNum); // Output: 10
Console.WriteLine(biggerValue); // Output: 3.14
Console.WriteLine(unicodeValue); // Output: 65
```

Single je vrednosnog tipa. Predstavlja broj u pokretnom zarezu sa jednostrukom preciznoscu(32 bita). Byte ne može da skladišti sign u smislu negativnog broja. Predstavlja 8-bitni neoznačeni celobrojni podatak. To znači da može da čuva samo pozitivne vrednosti 0-255.

```
Console.WriteLine(sizeof(Decimal)); //16
```

Kada se odradi boxing vrednosnog tipa, mora se dodeliti i konstruisati potpuno novi objekat.

Da vrednost ne bi mogla da se menja dodelu radimo ovako:

```
const float pi = 3.14F;
```

Ne bi bilo ispravno uraditi

```
const float pi; pi = 3.14F;
```

 jer const vrednosti moraju biti inicijalizovane prilikom deklaracije. Konstante se ne mogu menjati nakon što su inicijalizovane.

Svaki tip vrednosti ima implicitni podrazumevani konstruktor koji inicijalizuje podrazumevanu vrednost tog tipa.

Svi tipovi vrednosti su izvedeni implicitno iz klase `System.ValueType`

Promenljive referentnih tipova se nazivaju objekti i čuvaju reference na stvarne podatke.

3. validno, a 4. nije

```
3. int i = 10, j = 10;
```

```
4. int i, j = 10;
```

Pretvori se u int, a potom vrati u byte.

```
byte a = 11, b = 22, c;
c = (byte) (a + b);
```

Default vrednost za Boolean je False

Svaki tip podatka je ili vrednost ili referencni tip.

Funkcije:

```
static void Main(string[] args)
{
    int num = 1;
    funcv(num);
    Console.WriteLine(num + ", ");
    funcr(ref num);
    Console.WriteLine(num + ", ");
}

static void funcv(int num)
{
    num = num + 10; Console.WriteLine(num + ", ");
}

static void funcr (ref int num)
{
    num = num + 10; Console.WriteLine(num + ", ");
}
```

Rezultat: 11, 1, 11, 11 -> Prvo ispise 11, ali je num lokalna promenljiva pa zato naredni put 1, nakon toga se prosledjuje referenca i onda je ispis 11.

- Argument koji koristi ključnu reč params mora biti poslednji argument u promenljivoj listi argumenata metode.
- Prolazak po referenci eliminiše dodatni trošak kopiranja velikih podataka.
- Argument koji se prosleđuje ref parametru mora biti prethodno inicijalizovan.
- Promenljive prosleđene kao out argumenti nisu obavezno inicijalizovane pre nego što budu prosleđene. Out argumenti se koriste za vraćanje vrednosti iz metode, pa se često koriste za promenljive koje će biti inicijalizovane unutar same metode.
- Kada se koristi ref parametar, kako pozivajuća metoda tako i metoda koja se poziva moraju eksplicitno koristiti ključnu reč ref prilikom definisanja i pozivanja parametra. To je neophodno kako bi se obeležilo da se argument prosleđuje po referenci, a ne po vrednosti.
- Funkcija vraća vrednost, dok potprogram(procedura) ne može da vrati vrednost.
- Funkcija može da vrati jednu vrednost(ta vrednost može biti niz)
- Definicije funkcije ne mogu biti ugnjezdene, ali pozivi mogu biti ugnjezdjeni
- Funkcije se mogu pozivati rekurzivno

```
static void Main(string[] args)
{
    int i = 10;
    double d = 34.340;
    fun(i);
    fun(d);
}

static void fun(double d)
{
    Console.WriteLine(d + " ");
}
```

Int se implicitno konvertuje u double, ali se ne ispisuju tacka i 0. Ispis je: 10 34.34

- C# omogućava funkciji da ima argumente sa podrazumevanim vrednostima.
- C# omogućava funkciji da ima promenljiv broj argumenata.
- U C#-u, ako se izostavi tip povratne vrednosti u definiciji metode, podrazumeva se da je povratni tip `void`, a ne rezultuje greškom pri kompilaciji.
- Ponovno definisanje parametra metode u telu metode dovodi do greške pri kompilaciji.
- Ključna reč `params` se koristi za specificiranje sintakse funkcije sa promenljivim brojem argumenata.)

Definicije potprograma ne mogu biti ugnjezdene.

Potprogram se može pozvati rekurzivno.

Pozivi potprograma mogu biti ugnjezdjeni.

Ako želimo da funkcija vrati decimal moramo koristiti primer C. Radi se o tome da funkcija ne može direktno u namespace. Mora biti unutar nekog tipa pa onda moramo dodati `static` da bi bilo na nivou klase. Bez `static` je metoda. Metode su vezane za klasu ili objekat, dok je funkcija kod koji obavlja neki zadatak i može biti pozvana iz drugih delova koda.

Ⓑ

```
decimal fun(int i, Single j, double k)
{ ... }
```

Ⓒ

```
static decimal fun(int i, Single j, double k)
{ ... }
```



Nasledjivanje:

Ispis je "Base class"

```
class Baseclass
{
    public void fun()
    {
        Console.WriteLine("Base class" + " ");
    }
}
class Derived1: Baseclass
{
    new void fun()
    {
        Console.WriteLine("Derived1 class" + " ");
    }
}
class Derived2: Derived1
{
    new void fun()
    {
        Console.WriteLine("Derived2 class" + " ");
    }
}
class Program
{
    public static void Main(string[] args)
    {
        Derived2 d = new Derived2();
        d.fun();
    }
}
```

Containership (kontejnerski odnos) je koncept u objektno orijentisanom programiranju koji se odnosi na odnos između klasa, gde jedna klasa sadrži (ima instancu) drugu klasu kao svoj član ili atribut. Na primer, možemo imati klasu "Automobil" koja sadrži objekte "Motor", "Točkovi" i "Volan" kao svoje članove. Automobil ima agregatni odnos sa ovim objektima, gde se oni smatraju delovima automobila. Takođe, objekti mogu biti drugih klasa koje predstavljaju entitete u domenu problema. Containership i inheritance(nasledjivanje) spadaju u reuse mehanizme.

Strukture:

Prostor potreban za strukturne(struct) promenljive se dodeljuje na steku.

Nije dozvoljeno kreiranje prazne strukture.

U y se prosledjuje 10, ali je y.i samo kopija instance tako da je rezultat 20 10

```
struct Sample
{
    public int i;
}
class MyProgram
{
    static void Main()
    {
        Sample x = new Sample();
        x.i = 10;
        fun(x);
        Console.Write(x.i + " ");
    }
    static void fun(Sample y)
    {
        y.i = 20;
        Console.Write(y.i + " ");
    }
}
```

- Strukture mogu da sadrze proprietije i konstruktore.
- Strukture su uvek vrednosnog tipa.
- Struktura se unistava kada je out of scope(van dosega). To znaci da je ogranicena na kod u kome je definisana ili deklarirana(kao sto lokalne promenljive ne mogu da se koriste van opsega).
- Svi vrednosni(value) tipovi u C# su nasledjeni od ValueType, koji se nasledjuje od Object.
- U strukturama clanovi ne mogu da budu protected tipa
- Struktura ne moze da ima prazan konstruktor
- Metode mogu da postoje u strukturi
- Klase su referencnog, a strukture vrednosnog tipa
- Objekti se kreiraju sa new, a promenljive strukturne mogu da se kreiraju sa ili bez new
- Strukture mogu implementirati interfejs, ali ne mogu naslediti drugu strukturu
- Clanovi strukture ne mogu biti deklarirani kao protected

- Greška je inicijalizacija polja instance u strukturi(dovela do ciklične zavisnosti u rasporedu strukture, što nije dozvoljeno).

```
struct MyStruct
{
    public int myField = 10; // Greška: Ne može
}

class Program
{
    static void Main()
    {
        MyStruct myStruct = new MyStruct();
    }
}
```

```
struct MyStruct
{
    public int myField;

    public MyStruct(int value)
    {
        myField = value;
    }
}

class Program
{
    static void Main()
    {
        MyStruct myStruct = new MyStruct(10);
    }
}
```

Namespace:

Da bi se koristili elementi potrebno je: dodati referencu namespace-a, importovati namespace, potom koristiti elemente

Primeri namespace-a: System.Security, System.Threading, System.Drawing, System.Xml, System.Web, System.Data

```
College.Lib.Book b = new College.Lib.Book();
b.Issue();
```

```
using College.Lib;
Book b = new Book();
b.Issue();
```

Namespace nam pomaze da kontrolisemo vidljivost elemenata koji su prisutni u njemu.

Pre upotrebe klase se mora dodati referenca biblioteke

Ne postoji ogracenje za broj nivoa tokom ugnjezdavanja namespace-a

Moze imati neogranicen broj using(za biblioteke) elemenata

Svaka klasa, struktura, enum, delegat i interfejs moraju pripadati nekom namespace-u

Ako nije spomenuto namespace ima ime trenutnog projekta

Namespace treba da bude importovan tako da mozemo da koristimo elemente u njemu

```
using System.Windows.Forms;
ListBox lb = new ListBox();
```

```
using LBControl = System.Windows.Forms;
LBControl lb = new LBControl();
```

```
using LBControl = System.Windows.Forms.ListBox;
LBControl lb = new LBControl();
```

Interfejsi:

Da bi funkciju jasno povezali sa određenim interfejsom definisemo je ovako(interfejs.funkcija):

```
int IMyInterface.fun2()
{ }
```

U interfejsu mozemo deklarirati: svojstva, metode, događaji

```
public interface IExampleInterface
{
    event EventHandler MyEvent;

    string MyProperty { get; set; }

    void MyMethod();
}
```

U interfejsu NE MOŽEMO deklarirati: enumeracije i strukture
Interfejs može biti nasledjen

Velicina referenci na interfejse zavisi od arhitekture(može biti 4 ili 8 bajta zavisno od 32/64-bitnog sistema). Za 6 metoda objekat klase može zauzimati 24 bajta ili 48.

Ako klasa delimično implementira interfejs to bi trebalo da bude apstraktna klasa. Može se delimično implementirati interfejs korišćenjem parcijalne klase. Parcijalni delovi se kombinuju prilikom kompilacije i koriste klasu kao implementaciju interfejsa.

```
public interface IExampleInterface
{
    void MethodA();
    void MethodB();
}

public partial class ExampleClass : IExampleInterface
{
    public void MethodA()
    {
        // Implementacija MethodA
    }
}

public partial class ExampleClass
{
    public void MethodB()
    {
        // Implementacija MethodB
    }
}
```

Klasa koja implementira interfejs može eksplicitno implementirati članove tog interfejsa. To znači da radimo: void **Interfejs**.MetodA();

Klasa može implementirati više interfejsa

Strukture ne mogu naslediti klasu, ali mogu implementirati interfejs

U C#.NET-u se : koristi da oznaci da član klase implementira određeni interfejs

Članovi interfejsa su automatski public

Atributi:

[Serializable()] atribut se pregleda u run-time

Koristi se za konvertovanje objekta u strim bajtova kako bismo objekat transportovali kroz mrežu

Prilagodjeni(**custom**) **atribut** su mehanizam koji omogućava dodavanje dodatnih metapodataka i informacija o programskim elementima, kao što su klase, metode, polja ili svojstva. Omogućavaju programerima dodatne informacije koje su korisne za razne svrhe poput: refleksije, serijalizacije, generisanja koda... Primenjuju se iznad svake klase, metode, polja ili drugih programskih elemenata na koje želite da dodate dodatne metapodatke. Primer prilagođenog atributa -

AttributeUsage(atribut):

```
[AttributeUsage(AttributeTargets.Class | AttributeTargets.Method,  
AllowMultiple = true, Inherited = false)]
```

Meta custom atributa ne može da bude namespace.

Parametar se atributu prosledjuje preko pozicije ili preko imena.

Kada se atribut primenjuje koristeći poziciju parametra, vrednost se prosledjuje atributu redosledom kako je definisano u konstruktoru atributa:

```
[CustomAttribute("Value1", 123)]
```

Koriscenjem imena parametra da bismo prosledili vrednost atributu:

```
[CustomAttribute(Parameter1 = "Value1", Parameter2 = 123)]
```

Atribut može biti pregledan u compile-time i link-time.

Link-time je vreme linkovanja. Faza između compile-time i runtime (kada se program pokrene). To je faza u procesu kompilacije softverskog projekta gde se objektni kod i biblioteke povezuju zajedno kako bi se stvorio izvršni program ili biblioteka. To je faza nakon kompilacije gde se generise objektni kod iz izvornog koda.

Tokom te faze linker uzima objektna fajlove koji su generisani u prethodnoj fazi kompilacije i povezuje ih kako bi se formirao izvršni program ili biblioteka. Može uključivati optimizaciju, gde linker poboljšava performanse ili smanjuje veličinu izvršnog programa.

Linker je programska komponenta za povezivanje objektnog koda i biblioteka kako bi se firmirao izvršni program ili biblioteka. Povezuje više modula objektnog koda ili biblioteka kako bi se program izvršio pravilno. Odradjuje: rezoluciju simbola (usklađuje tipove i adrese), resavanje referenci (ako kod sadrži reference na neke objekte, funkcije, biblioteke), resavanje eksternih veza, resavanje višestrukog definisanja (ako postoji više definicija simbola sa istim imenom), relokaciju (prilagođava adrese instrukcija i podataka kako bi se uklopile i adresnu mapu programa).

Objektni kod - rezultat procesa kompilacije izvornog koda koji se sastoji od niskog nivoa instrukcija. To je prevedena verzija izvornog koda koja može biti izvršena direktno na odgovarajućem računarskom hardveru ili virtuelnoj masini. Objektni moduli su način za organizovanje, to su samostalne jedinice koje se mogu povezati da zajedno formiraju izvršni program.

Compile-time - proces prevodjenja izvornog u izvršni ili objektni kod.

CLR(Common Language Runtime) može promeniti ponašanje koda u zavisnosti od atributa koji se na njega primenjuju. Predstavlja virtuelnu masinu koja omogućava pokretanje .NET aplikacija.

Atribut može imati parametre.

Prilikom kompajliranja C#.NET programa primenjeni atributi se beleže u metapodacima assembly-ja. Primenjeni atributi se mogu pročitati iz assembly-ja pomoću klase Reflection.

Assembly - osnovna jedinica distribucije softvera koja sadrži izvršni kod, meta podatke i druge resurse potrebne za izvršavanje ili upotrebu aplikacije. Obično je predstavljen kao fizički fajl sa odgovarajućom ekstenzijom(.dll za biblioteke, .exe za izvršne aplikacije). Objedinjuje: izvršni kod, metapodatke, resurse i druge informacije za pravilno funkcionisanje aplikacije.

Linker ne može da inspektuje(detaljno analizira ili pruži informacije o atributu prilikom procesa povezivanja) primenjeni atribut. On se uglavnom fokusira na proces vezivanja modula i generisanje izvršnog koda, a ne na analizu ili interpretaciju atributa. Linker vrši inspekciju tokom izvršavanja programa. Detaljnu analizu atributa ili pružanja informacija o njima vrši runtime okruženje i mehanizmi poput refleksije.

Refleksija - omogućava programu analiziranje, otkrivanje i manipulisanje strukturom, tipovima i atributima tokom izvršavanja. U vreme izvršavanja program može da istražuje strukturu i karakteristike objekata, umesto da se samo oslanja na informacije dostupne tokom faze kompilacije. Korisna je kada program mora da radi sa nepoznatim tipovima ili kada je potrebno postići fleksibilnost i dinamičnost u izvršavanju koda. Često se koristi u razvoju alata za testiranje, integrisanje različitih komponenti ili rešenja za različite tipove podataka.

Pravilan način primene atributa na assembly je sledeći:

```
[ assembly : AssemblyDescription("DCube Component Library") ]
```

Ispravna upotreba:

```
[assembly: Tested("Sachin", testgrade.Good)]  
[Tested("Virat", testgrade.Excellent)]  
class customer { /* .... */ }
```

Atribut ne može da se primeni na enum i namespace.

Atribut može da se primeni na metode, klase, assembly-je.

Kontrola instrukcija:

if iskaz biraz koji se iskaz izvrsava na osnovu Boolean izraza

Ako imamo switch-case, break izlazi samo iz tog bloka

ispis je: blue

```
public enum color
{ red, green, blue };

class SampleProgram
{
    static void Main (string[ ] args)
    {
        color c = color.blue;
        switch (c)
        {
            case color.red:
                Console.WriteLine(color.red);
                break;

            case color.green:
                Console.WriteLine(color.green);
                break;

            case color.blue:
                Console.WriteLine(color.blue);
                break;
        }
    }
}
```

```
char ch = Convert.ToChar ('a' | 'b' | 'c');
switch (ch)
{
    case 'A':
    case 'a':
        Console.WriteLine ("case A | case a");
        break;

    case 'B':
    case 'b':
        Console.WriteLine ("case B | case b");
        break;

    case 'C':
    case 'c':
    case 'D':
    case 'd':
        Console.WriteLine ("case D | case d");
        break;
}
```

Vrsi se bitska operacija | (OR). Znaci 97,98,99 i nad njima se primeni OR. Rezultat bude 99 i kastuje se u tip char sto je 'c'. Tako da je ispis "case D | case d".

```
int a;
String res;
if (a % 2 == 0)
    res = "Even";
else
    res = "Odd";
```

```
int a;
String res;
a % 2 == 0 ? res = "Even" : res = "Odd";
Console.WriteLine(res);
```

Prvi primer je ok, a drugi nije jer res mora imati neku pocetnu vrednost. Ovako ce biti null i desice se NullReferenceException.

```
if (age > 18 && no < 11)
    a = 25;
```

Uslov no<11 ce biti proveren samo ako je age>18 true.
&& je poznat kao logicki operator kratkog spoja.

```
switch (id)
{
    case 6:
        grp = "Grp B";
        break;

    case 13:
        grp = "Grp D";
        break;

    case 1:
        grp = "Grp A";
        break;

    case ls > 20:
        grp = "Grp E";
        break ;

    case Else:
        grp = "Grp F";
        break;
}
```

Kompajler ce pricaviti gresku za 'case ls>20' i za 'case Else'

Grananje(branching) se vrsi pomocu izraza za skok koji izazivaju trenutni prenos kontrole programa.

Pravilno ispisuje:

```
char[ ] arr = new char[ ] { 'k', 'i', 'c', 'i', 't' } ;
```

```
foreach (int i in arr)
{
    Console.WriteLine((char) i);
}
```

```
int i, j, id = 0; switch (id)
{
    case i:
        Console.WriteLine("I am in Case i");
        break;

    case j:
        Console.WriteLine("I am in Case j");
        break;
}
```

Kompajler prijavljuje gresku za 'case i' i 'case j' jer varijable ne mogu da se koriste u case-ovima. Mora imati neku vrednost. Ne moze da se koristi promenljiva, izraz ili dinamicki odredjene vrednosti. U konstrukciji moraju biti konstante: case 1: , case 'a': , case "hello": , case MyEnum.Value1: , const int x = 5; i potom case x:

Klase i objekti:

Clanovima instance klase moze se pristupiti preko objekta te klase.

Svi objekti kreirani iz klase ce zauzimati jednak broj bajtova u memoriji.

Paradigma OOP daje jednak znacaj podacima i procedurama koje rade na podacima.

```
class Sample
{
    int i;
    Single j;
    public void SetData(int i, Single j)
    {
        i = i;
        j = j;
    }
    public void Display()
    {
        Console.WriteLine(i + " " + j);
    }
}
class MyProgram
{
    static void Main(string[] args)
    {
        Sample s1 = new Sample();
        s1.SetData(10, 5.4f);
        s1.Display();
    }
}
```

Ispis je 0 0 jer nema kljucne reci this

Data members klase su podrazumevano private.

Funkcije klase su podrazumevano public.

Privatna funkcija moze pristupiti javnoj funkciji unutar iste klase.

```
sample c;  
c = new sample();
```

Kreira objekat bez imena tipa sample.

Kreira referencu c na steku i objekat tipa sample na hipu.

1. Stek (Stack):

- Stak je struktura podataka koja se koristi za organizovanje lokalnih promenljivih i povratnih adresa funkcija.
- Stak je pravougaonik, LIFO (last in, first out) struktura podataka.
- Podaci se smeštaju na stek prilikom izvršavanja funkcije i uklanjaju se sa steka kada funkcija završi sa izvršavanjem.
- Promenljive na steku imaju kratkotrajno trajanje, a čim funkcija završi, memorija se oslobađa.
- Stak je efikasan za upravljanje malim, lokalnim promenljivim i jednostavnim podacima.

2. Hip (Heap):

- Hip je oblast memorije koja se koristi za dinamičku alokaciju podataka tokom izvršavanja programa.
- Hip je veći i fleksibilniji od steka, ali i sporiji.
- Podaci na hipu imaju dugotrajno trajanje i mogu se koristiti između različitih funkcija.
- Programer je odgovoran za ručno upravljanje memorijom na hipu, uključujući alociranje i dealociranje memorije.
- Podaci na hipu se zadržavaju sve dok ih programer eksplicitno ne oslobodi ili dok program završi.

```
int i;  
int j = new int();  
i = 10;  
j = 20;  
String str;  
str = i.ToString();  
str = j.ToString();
```

Radi skroz ispravno

Staticke funkcije klase nikada ne primaju referencu this.

Funkcije clanice instance uvek dobijaju referencu this.

Dok pozivamo clana funkcije, od nas se ne trazi da eksplicitno prenesemo referencu this.

Konceptualno, svaki objekat klase ce imati podatke o instanci (stanje objekta) i funkcije instance (metode objekta) koje definise ta klasa.

Objekti su uvek nameless. Bezimeni objekti se nazivaju i anonymous objektima. Kada je objekat inicijalizovan, ali nije dodeljen nijednoj referentnoj promenljivoj, naziva se anonimnim.

Primer "new Employee();"

Ako dodamo referencu "Employee emp = new Employee();" objekat vise nije anoniman.

Nizovi:

`intMyArr.GetUpperBound(0)` vraća najveći indeks u prvoj dimenziji

`intMyArr.GetUpperBound(1)` vraća najveći indeks u drugoj dimenziji

Za niz: `int[, ] intMyArr = {{7, 1, 3}, {2, 9, 6}, {7, 1, 3}, {2, 9, 6}};`

Dakle, `intMyArr.GetUpperBound(0)` će vratiti najveći indeks u prvoj dimenziji(0,1,2,3), koji je 3 dok će `intMyArr.GetUpperBound(1)` vratiti najveći indeks u drugoj dimenziji(0,1,2), koji je 2.

```
int[] a = {11, 3, 5, 9, 4};
```

Referenca 'a' je kreirana na steku.

Elementi niza su kreirani na heap-u.

Prilikom deklarisanja niza kreira se nova klasa niza koja je izvedena iz `System.Array`

Default(podrazumevana) vrednost elemenata numerickog niza je nula.

Ispis svih elemenata

```
int[][] a = new int[2][];
a[0] = new int[4]{6, 1, 4, 3};
a[1] = new int[3]{9, 2, 7};
foreach (int[] i in a)
{
    /* Add loop here */
    Console.WriteLine(j + " ");
    Console.WriteLine();
}
```

`foreach (int j in i)` ✓

'j' predstavlja vrednost svakog pojedinacnog elementa u redu 'i'

```
int[ , , ] a = new int[ 3, 2, 3 ];
Console.WriteLine(a.Length);
```

Prva dimenzija je 3, druga dimenzija 2, treća dimenzija 3. Kada se vraća ukupna dužina to je $3*3*2=18$

1-D array je u prevodu 1-dimenzioni

```
int[][] intMyArr = new int[2][];
intMyArr[0] = new int[4]{6, 1, 4, 3};
intMyArr[1] = new int[3]{9, 2, 7};
```

Ovo je jagged niz gde se `intMyArr[0]` odnosi na nulti 1-D niz, a `intMyArr[1]` na prvi 1-D niz

```
int[] intMyArr = {11, 3, 5, 9, 4};
```

`intMyArr` je referenca na objekat klase koju kompajler izvodi iz `System.Array` klase

Enumeracije:

Promenljiva se ne može dodeliti elementu enum-a.

Enum vrednosti se ne mogu popuniti (populate) iz baze podataka.

Enum varijable mogu biti definisane unutar klase, namespace-a, strukture, interfejsa i kada su tamo deklarirani tretiraju se kao public

Enum je ključna rec deklarirana u System namespace-u

Enum ne može da bude float tipa

Svaki enum je nasleđen iz Object klase

Svaki enum je vrednosnog tipa

default tip podatka za enum je int

Enum može biti deklarisan unutar ili van klase, namespace-a.

```
enum color : int
{
    red = -3,
    green,
    blue
}
Console.Write( (int) color.red + ", " );
Console.Write( (int) color.green + ", " );
Console.Write( (int) color.blue );
```

Ispis je -3, -2, -1

Enum element ne može da dobije vrednost van enum deklaracije tj. bloka. Program će prijaviti gresku

```
enum color: int
{
    red,
    green,
    blue = 5,
    cyan,
    magenta = 10,
    yellow
}
Console.Write( (int) color.green + ", " );
Console.Write( (int) color.yellow );
```

Ispis: 1, 11

```
enum color : byte
{
    red = 500,
    green = 1000,
    blue = 1500
}
```

Kompajler će prijaviti gresku jer su vrednosti van range-a za byte tip

```
enum color
{
    red,
    green,
    blue
}
color c = color.red;
Type t;
t = c.GetType();
string[] str;
str = Enum.GetNames(t);
Console.WriteLine(str[ 0 ]);
```

c.GetType(); vraca tip 'color'

Enum.GetNames(t) vraca niz stringova koji sadrzena imena svih vrednosti enumeracije, ispis je 'red'

```
class Sample
{
    private enum color : int
    {
        red,
        green,
        blue
    }
    public void fun()
    {
        Console.WriteLine(color.red);
    }
}
class Program
{
    static void Main(string[] args)
    {
        // Use enum color here
    }
}
```

Enum color je private i ne moze se koristiti u main(), moramo je proglasiti za public da bismo mogli da koristimo van Sample klase.

Da bismo definisali promenljivu tipa 'enum color' u main(), trebalo bi da koristimo naredbu Sample.color c;

Rukovanje izuzecima:

Primeri exception-a: Exception, DivideByZeroException, OutOfMemoryException, InvalidOperationException

Izuzeci se desavaju u toku run-time

Ako ne uhvatimo exception u runtime onda ce ga uhvatiti CLR

finally blok se koristi za clean up operacije poput zatvaranja konekcije ka bazi ili mrezi

```
Unhandled Exception: System.IndexOutOfRangeException: Index was outside the bounds of the array: at
IndiabixConsoleApplication.MyProgram.SetVal(Int32 index, Int32 val) in D:\Sample\IndiabixConsoleApplication
\MyProgram.cs:line 26 at IndiabixConsoleApplication.MyProgram.Main(String[] args) in D:\Sample
\IndiabixConsoleApplication\MyProgram.cs:line 20
```

SetlVal() je pozvana iz Main() u liniji 20. Exception se desio u liniji 26 u funkciji SetVal(). Runtime exception se pojavio u projektu IndiabixConsoleApplication.

```

static void Main(string[] args)
{
    int index = 6;
    int val = 44;
    int[] a = new int[5];
    try
    {
        a[index] = val ;
    }
    catch(IndexOutOfRangeException e)
    {
        Console.Write("Index out of bounds ");
    }
    Console.Write("Remaining program");
}

```

Ispis: Index out of bounds Remaining program

Unhandled Exception: System.IndexOutOfRangeException:
 Index was outside the bounds of the array.
 at IndiabixConsoleApplication.Program.Main(String[] args) in
 D:\ConsoleApplication\Program.cs:line 14

Exception nije hendlovan, desio na liniji 14 jer je program pokusao da pristupi elementu niza koji je izvan granica

Sve vrednosti podesene u exception objektu dostupne su u catch bloku

Dok se izbacuje korisnicki definisani izuzetak, višestruke vrednosti se mogu postaviti u izuzetak, objekat

Izuzeci mogu da se izbaci i iz konstruktora, ali kod izuzetka ne može da se pribavi

Nije obavezno da sve klase čiji objekti mogu biti izbaceni naredbom throw budu izvedene iz klase System.Exception. Možemo napraviti svoju klasu za određenu vrstu exception-a.

Collection classes(klase kolekcija):

Klasa ArrayList sadrži unutrašnju klasu koja implementira interfejs IEnumerator

Unutrašnja klasa ArrayList-e može pristupiti članovima klase ArrayList

Da biste pristupili članovima ArrayList iz unutrašnje klase, potrebno je da joj prosledite referencu klase ArrayList

Ako imamo 4 treda onda imamo 4 enumeratora koji rade na ArrayList objektu

Input/Output je index-based kod BitArray i ArrayList

Input/Output je key based kod HashTable i SortedList (ključ ne može biti null, a vrednost može)

Stack kolekcija:

Može se koristiti za evaluaciju izraza(redosled kojim se izvršava, npr. množenje ima prednost).

Svim elementima u kolekciji stek može se pristupiti pomoću pokazivaca

Najvišem elementu kolekcije stek može se pristupiti pomoću metode Peek()

HashTable se proverava pomoću ključa: t.ContainsKey("nekiKljuč");

Pristup svim elementima reda:

```
Queue q = new Queue();
q.Enqueue("Sachin");
q.Enqueue('A');
q.Enqueue(false);
q.Enqueue(38);
q.Enqueue(5.4);

IEnumerator e;
e = q.GetEnumerator();
while (e.MoveNext())
    Console.WriteLine(e.Current);
```

Uredjene(ordered) kolekcije su Stack i Queue. To znaci da se prilikom dodavanja ili uklanjanja elemenata pridrzavaju odredenog redosleda.

Ako je Capacity properti od ArrayList kolekcije podesen na 4, a mi dodamo 5. element kolekcija se automatski prosiruje za svoju velicinu tako da joj je sada Capacity 8.

Broj elemenata u nizu ArrayList vidimo pomocu 'arr.Count'

HashTable implementira IDictionaryEnumerator interfejs u unutrašnjoj klasi. Parovima ključ-vrednost prisutnim u HashTable može se pristupiti koriscenjem svojstava kljuceva i vrednosti unutrašnje klase koja implementira interfejs IDictionaryEnumerator

Pristup elementima steka:

```
Stack st = new Stack();
st.Push(11);
st.Push(22);
st.Push(-53);
st.Push(33);
st.Push(66);

IEnumerator e;
e = st.GetEnumerator();
while (e.MoveNext())
    Console.WriteLine(e.Current);
```

Collection klase u Framework Class Library koriste efikasne algoritme za upravljanje kolekcijama cime poboljsavaju performanse programa.

Generics:

Omogucava definisanje i koriscenje generickih tipova ili metoda. Genericki tipovi omogucavaju da se definise kod koji radi sa bilo kojim tipom podatka, pruzacuci fleksibilnost i bezbednost tipova.

Genericki tipovi se deklarisu pomocu tipicne sintakse sa uglastim zagradama <> nakon imena tipa.

List<T> - može da sadrzi elemente bilo kog tipa T. Ne moramo pisati kod za svaki pojedinačni tip.

Mogu se koristiti u definiciji klasa, struktura, interfejsa i metoda.

Klase koje se nalaze u System.Collections.Generic su: Stack, SortedDictionary

Generics je koristan za kolekcije klasa u .NET Framework-u

Generics je language feature. To je osobina jezika koja omogucava definisanje i koriscenje generickih tipova i metoda.

Generic procedure moraju imati bar jedan tip parametra.

Generics obezbeduju sigurnost tipa bez dodatnih troskova višestrukih implementacija

```

public class Generic<T>
{
    public T Field;
    public void TestSub()
    {
        T i = Field + 1;
    }
}
class MyProgram
{
    static void Main(string[] args)
    {
        Generic<int> gen = new Generic<int>();
        gen.TestSub();
    }
}

```

Kompajler prijavljuje gresku: Operator '+' is not defined for types *T* and *int*.

Izvršava kod normalno ispisom: *IndiaBIX 4.2*

```

public class TestIndiaBix
{
    public void TestSub<M> (M arg)
    {
        Console.Write(arg);
    }
}
class MyProgram
{
    static void Main(string[] args)
    {
        TestIndiaBix bix = new TestIndiaBix();
        bix.TestSub("IndiaBIX ");
        bix.TestSub(4.2f);
    }
}

```

```

public class MyContainer<T> where T: IComparable
{
    // Insert code here
}

```

Klasa *MyContainer* zahteva da njegov tip argumenta mora da implementira interfejs *IComparable*
Ovaj zahtev za tipom argumenta se naziva *constraint*(ogranicenje)

```
public class MyContainer<T> where T: class, IComparable
{
    //Insert code here
}
```

Postoji vise ogranicenja za tip argumenta klase MyContainer

Klasa MyContainer zahteva da njegov tip argumenta mora biti referentni tip(klasa, interfejs ili delegat) i mora da implementira IComparable interfejs. To se postize kljucnom recju 'where'. Uslovljava da moras dodati instancu klase koaj u sebi sadrzi 'CompareTo' metodu i koja je implementirala interfejs(mozda je poenta interfejsa bas ta metoda i jos neka npr. ToString).

Primer koda za nastavak rada sa ovim:

```
public class Student : IComparable<Student>
{
    public string Name { get; set; }
    public int Age { get; set; }

    public int CompareTo(Student other)
    {
        return Age.CompareTo(other.Age);
    }

    public override string ToString()
    {
        return $"Ime: {Name}, Godine: {Age}";
    }
}

public class Program
{
    public static void Main()
    {
        MyContainer<Student> container = new MyContainer<Student>(5);

        container.Add(new Student { Name = "Marko", Age = 20 });
        container.Add(new Student { Name = "Jelena", Age = 22 });
    }
}
```

API/REST API/SOAP:

Da bi se ostvarila komunikacija između dva sistema potrebno je da imaju "tacku" za kontakt, tzv. interfejs. Interfejs je posrednik(veza) pri komunikaciji između dva odvojena sistema koji zajedno rade i može biti:

Hardverski interfejs (npr. volan, gas i menjač su *"tačke pristupa vozilu"* i predstavljaju posrednika između automobila i osobe koja upravlja sa njim)

Programski interfejs tzv. **"API"** koje predstavlja **"tačke pristupa"** pri komunikaciju između dva programa

Rad sa servisima podrazumeva da se resursima pristupa na udaljenom serveru (pristup je putem internet mreže), ovakva mrežna komunikacija sa sobom donosi problem pristupa i prenosa složenih struktura podataka kao i same organizacije resursa na serveru. Programeri su tokom vremena kreirali komunikaciju između dva programa na razne načine tj. pravili različite tipove API-ja, ali su u jednom trenutku standardizovali način komunikacije tj. napravili su skup pravila koja mrežne aplikacije trebaju pratiti pri međusobnoj komunikaciji (npr. jedan od takvih standardizovanih api-ja je REST API).

Ono na šta moramo obratiti pažnju je da kada se jednom definiše API i pusti u "promet", u slučaju da je nakon nekog vremena potrebno napraviti izmene, nije pametno menjati do sada postavljene "end point-e" jer bi bilo u najmanju ruku neljubazno prisiljavati potrošače API-ja da menjaju već izradjene programe da bi se prilagodili na promenu. Kada se napravi promena uvek treba težiti da nastavite s podrškom za postojeća svojstva (end point-e), a da za promene dodate nova svojstva umesto menjanja postojećih.

API(application programming interface) - predstavlja set definicija i protokola. Potreban je za razvoj aplikacija i integraciju(njihovo dopunjavanje) jer služi za razmenu podataka između dva različita softvera, kao izvor/posiljalac informacija(server) i kao korisnik. Programi koriste API da komuniciraju, pribave informacije ili pokrenu funkciju. API se ponasa kao posrednik između korisnika(klijenta) i resursa(servera). Neki od razloga koriscenja API-ja: racionalizacija deljenja resursa i informacija, kontrolisanje ko kome ima pristup uz pomoc autentifikacije i definisanja prava, bezbednost i kontrola, nema potrebe za razumevanjem specifičnosti softvera, dosledna komunikacija između servisa(cak iako koriste različite tehnologije)

API omogućuje programerima da pristupe odredjenim funkcionalnostima ili podacima iz sistema.

Primeri API-ja:

1. Wikipedia pruži API za pristup svojim podacima. Omogućuje programerima da pristupe informacijama iz wikipedije, kao što su clanci, stranice s kategorijama, poveznice, mediji i drugi podaci. Moguće je koristiti Wikipedijin API za razvoj različitih aplikacija koje se temelje na njenim podacima, kao što su aplikacija za pretraživanje, prikazivanje sadržaja, analizu podataka itd.
2. GitHub API: Omogućuje programerima da pristupe GitHub repozitorijima, korisnicima i drugim informacijama o projektima.
3. Google Maps API: Omogućuje programerima da uključe Google mape i lokacijske podatke u svoje aplikacije.
4. Spotify API: Omogućuje programerima da pristupe informacijama o muzici, poput pesama, albuma, izvođača i još mnogo toga.
5. REST Countries API: Omogućuje programerima da pristupe informacijama o svim državama u svijetu, uključujući geografske, demografske i druge informacije.
6. NASA API: Omogućuje programerima da pristupe velikoj količini podataka i informacija vezanih za svemir i NASA misije.

7. YouTube API: Omogućuje programerima da pristupe informacijama o videima, kanalima i drugim informacijama vezanim za YouTube.

Paginacija - proces deljenja rezultata zahteva na više stranica kako bi se omogućilo efikasno dohvaćanje podataka. Kada API vraća veliki skup podataka, paginacija omogućava korisniku da dobije samo određeni broj rezultata po zahtevu, umesto da dobije sve rezultate odjednom (na primer umesto da dobije 500 stranica on dobije 5). Obično se postize kroz upotrebu parametara zahteva koji definišu koji broj stranice (page) i broj rezultata po stranici (limit). Na ovaj način API vraća samo odgovarajući segment rezultata zahteva. Korisna je kada je kompletan skup podataka prevelik da bi se dohvatio u jednom zahtevu ili kada je efikasno dobavljanje samo dela podataka prioritarno. Omogućava bolju kontrolu nad podacima koji se prikazuju i smanjuje opterećenje na serveru i mreži.

REST nije protokol kao SOAP već je koncept koji se bazira na promeni stanja klijenta i sve akcije se obavljaju u trenutku promene tog stanja, sa mogućnošću vraćanja na neko od prethodnih stanja.

REST API - RESTful se odnosi na softversku arhitekturu koja je skraćena za "transfer reprezentativnog stanja". Jednostavan način slanja i primanja podataka između klijenta i servera. Ove web usluge koriste protokol bez stanja da bi tekstualne reprezentacije svojih online resursa učinili dostupnim za čitanje i obradu. Klijent obavlja dobro poznate aktivnosti zasnovane na HTTP protokolu kao što su preuzimanje, azuriranje i brisanje. Ne postoji posebna tehnologija na strani klijenta za REST jer odgovara različitim projektima (web development, iOS aplikacije, IoT uređaji). Postoji ne mora da se pridržavate tehnologije na strani klijenta, možete izgraditi bilo koju infrastrukturu. Idealan je za aplikacije u cloud-u zbog nezavisnog statusa.

Nepostojanje stanja ("**stateless**") podrazumeva da server **ne pamti nikakve podatke o klijentu**, odnosno da klijent **sa svakim novim zahtevom mora slati sve potrebne informacije za razumevanje i obradu tog zahteva**, i ne može se osloniti na to da će ga server "prepoznati", odnosno iskoristiti informacije iz prethodnih zahteva istog klijenta.



Podaci (kao što su slike, video snimci i tekst) sadrže resurse u REST-u. Klijent posećuje određeni URL i šalje zahtev serveru da dobije odgovor.

Zahtev (URL koje pristupate) sadrži četiri komponente:

1. Endpoint koji je zapravo URL sa strukturom root-endpoint/?
2. Metod sa jednim od pet mogućih tipova (GET, POST, PUT, PATCH, DELETE)
3. Header-i koji posluju različite funkcije, uključujući autentifikaciju i obezbeđujući informacije o sadržaju body sekcije
4. Podaci (ili body) koje **saljete** (znači bez GET) ka serveru sa POST, PUT, PATCH ili DELETE zahtevima.

HTTP zahtevi omogucuju rad sa bazama podataka:

POST zahtev za kreiranje zapisa(rekorda)

GET zahtev da procitate ili dobijete resurse(dokument ili sliku, kao i kolekciju drugih resursa) od servera

PUT i PATCH zahtevi da azurirate zapise

DELETE zahtevi da obrisete resurse sa servera

Ove operacije se odnose na cetiri moguće akcije, poznatije kao CRUD: Create, Read, Update, Delete.

Server salje podatke klijentu u jednom od sledecih formata:

1. HTML

2. JSON(koji je najcesci zahvaljujuci njegovoj nezavisnosti od nekog konkretnog programskog jezika i pristupacnosti od strane ljudi ili masina)

3. XLT

4. PHP

5. Python

6. plain text

Prednost REST-a je u skalabilnosti, fleksibilnosti formata vracenih podataka(nije unapred definisan), prenosivosti i nezavisnosti, jednostavnost.

Jednostavnost - odgovor se ne mora parsirati kako bi se izvukao rezultat

Nezavisnost - odvojeni rad klijenta i servera znaci da programeri nisu vezani ni za jedan deo projekta.

Prenosivost i prilagodljivost(skabilnost) - REST API-ji rade samo kada su podaci iz jednog zahteva uspesno isporuceni. Omogucavaju vam da migrirate sa jednog servera na drugi i azurirate bazu podataka u bilo kom trenutku.

Prosirivost - s obzirom da server i klijent deluju odvojeno, lako i brzo se moze prosiriti projekat.

Nedostatak: PUT i DELETE se ne mogu koristiti kroz firewall ili u nekim pretrazivacima

6 arhitektonskih ograničenja su principi za projektovanje resenja:

1. Jedinstveni interfejs(dosledan korisnicki interfejs)

Koncept nalaze da svi API upiti za isti resurs, bez obzira na njihovo poreklo, treba da budu identicni, odnosno na jednom specificnom jeziku. Jedna uniformna identifikacija resursa(URI) je povezana sa istim podacima, kao sto su ime korisnika ili adresa e-poste. Drugi princip uniformnog interfejsa kaze da poruke treba da budu samoopisne. Oni moraju biti razumljivi da bi server mogao da odredi kako da se nosi sa tim(na primer tip zahteva, tipovi mime-a)

2. Klijent-server odvajanje

Rade nezavisno i ne moraju da znaju za druge. Primer: klijent ima samo jedinstvenu identifikaciju resursa(URI) zahtevanog resursa i ne moze da komunicira sa serverskim programom na bilo koji drugi nacin. S druge strane, server ne bi trebalo da utice na klijentski softver. Dakle, salje osnovne podatke preko HTTP-a. To znaci da mozete izmeniti klijentski kod bez uticanja na server, mozete zadržati klijentske i serverske programe i modularne i nezavisne sve dok svaka strana zna koji format poruke da dostavi drugoj. **Odvajanjem korisnickog interfejsa od ograničenja skladištenja podataka** poboljšavamo fleksibilnost interfejsa na razlicitim platformama i povecavamo skalabilnost.

3. Stateless komunikacija izmedju klijenta i servera

Sistemi zasnovani na REST-u su bez stanja (stateless), što znači da stanje klijenta ostaje nepoznato serveru i obrnuto. Ovo ograničenje omogućava serveru i klijentu da razumeju bilo koju poslatu poruku, čak i ako nisu videli prethodne. Svaki zahtev treba da sadrži sve informacije potrebne za njegovo dovršavanje. **Klijentske aplikacije moraju da sačuvaju stanje sesije**(API ne mora-benefit) jer serverske aplikacije ne bi trebalo da čuvaju nikakve podatke povezane sa zahtevom klijenta.

4. Keširanje podataka

REST zahteva keširanje resursa na strani klijenta ili servera gde god je to moguće. Keširanje podataka i odgovora je ključno u današnjim aplikacijama jer rezultira boljim performansama na strani klijenta. Dobro vođeno keširanje može smanjiti ili eliminisati neke interakcije na relaciji klijent-server. Takođe daje serveru više opcija za skalabilnost zbog manjeg opterećenja servera. Keširanje povećava brzinu učitavanja stranice i omogućava vam pristup prethodno pregledanom sadržaju bez internet veze.

5. Slojevita sistemska arhitektura(na strani servera)

REST API slojevi imaju svoje odgovornosti i dolaze u hijerarhijskom redosledu. Na primer, jedan sloj može biti odgovoran za skladištenje podataka na serveru, drugi za primenu API-ja na drugom serveru, a treći za autentifikaciju zahteva na drugom serveru. Ovi slojevi deluju kao posrednici i sprečavaju direktnu interakciju između klijentskih i serverskih aplikacija. Kao rezultat toga, klijent ne zna kom serveru ili komponenti se obraća. Kada svaki sloj obavlja svoju funkciju pre nego što prenese podatke na sledeći, to poboljšava ukupnu sigurnost i fleksibilnost API-ja jer dodavanje, izmena ili uklanjanje API-ja ne utiče na druge komponente interfejsa.

6. Kodiranje na zahtev (on-demand)

Najčešći scenario korišćenja REST API-ja je isporuka statičkih reprezentacija resursa u XML-u ili JSON-u. Međutim, ovaj model arhitekture omogućava korisnicima da preuzimaju i pokreću kod u obliku Java apleta ili skripti (kao što je JavaScript). Na primer, klijenti mogu da preuzmu kod za prikazivanje za UI vidžete pozivanjem vašeg API-ja. U principu server posalje izvršni kod klijentu koji se može pokrenuti direktno tokom vremena rada.

Izazovi:

Sporazum o REST endpoint-ima

API-ji treba da ostanu dosledni bez obzira na konstrukciju URL-a. Ali sa porastom mogućih kombinacija metoda, teže je održati uniformnost u velikim kodnim bazama.

Verzioniranje kao karakteristika REST API-ja

API-ji zahtevaju redovno ažuriranje ili verzioniranje da bi se sprečili problemi sa kompatibilnošću. Međutim, stari endpoint-i ostaju operativni, što povećava opterećenje. Verzioniranje je upravljanje izvornim kodom tako da se promene skladište u bazi podataka.

Veliki broj metoda autentifikacije

Možete odrediti koji su resursi dostupni za koje tipove korisnika. Na primer, možete da odredite koje usluge treće strane mogu da pristupe adresama e-pošte klijenata ili drugim osetljivim informacijama i šta mogu da urade sa ovim promenljivima. Ipak, 20 različitih metoda autorizacije koje postoje mogu otežati vaš početni API poziv. Zbog toga programeri ne nastavljaju sa projektom zbog početnih poteškoća.

Bezbednosni propusti REST API-ja

Iako RESTful API-ji imaju slojevit strukturu, i dalje mogu postojati neki bezbednosni problemi. Na primer, ako aplikacija nije dovoljno bezbedna zbog nedostatka enkripcije, može da otkrije osetljive podatke, ili haker može da pošalje hiljade API zahteva u sekundi, uzrokujući DDoS napad ili druge zloupotrebe API usluge koje mogu da sruše vaš server.

Prekomerno prikupljanje podataka i zahteva

Server može da vrati zahtev sa svim podacima, koji mogu biti nepotrebni, ili ćete možda morati da pokrenete više upita da biste dobili potrebne informacije.

RESTfull servisi se koriste:

1. Kod ograničenog propusnog opsega i resursa(povratna informacija može biti u bilo kom obliku)
2. Kod operacija koje ne koriste stanja(ukoliko neka operacija treba da bude nastavljena onda REST nije pravi pristup i SOAP verovatno predstavlja bolje rešenje)
3. Kod situacija gde je moguće kesiranje(ukoliko informacija može biti kesirana zbog operacija koje ne koriste stanja onda je ovaj pristup odlican)

Big Web Services (tzv. SOAP bazirani servisi) - slede SOAP standarde, a podaci se prenose u XML formatu, a opisani su WSDL. Koriscenje SOAP protokola omogućava komunikaciju izmedju aplikacija na razlicitim operativnim sistemima i razlicitim tehnologijama tako sto aplikacije razmenjuju poruke "dogovorenog" formata.

Oblasti u kojima se SOAP bazirani servisi dobro "snalaze" su:

kod **formalnih ugovora** kada obe strane trebaju da se slože oko formata za razmenu informacija
kod **operacija koje koriste stanja**

Mana SOAP baziranih servisa je kompleksnost i prevelika „opširnost“ pri korišćenju XML-a, mada ova mana ne sprečava Microsoft i IBM da ga veoma često koriste. U mane se takodje smatra potrošnja resursa da bi se parsirao XML.

SOAP poruka je običan XML dokument koji se sastoji iz sledećih elemenata:

Envelope element (identifikuje XML dokumenat kao SOAP poruku) je obavezan deo

Zaglavlje (header) koji je opcioni deo

Telo (body) je obavezni deo i nosi informacije (parametre) zahteva i vraća rezultate tj.odgovor Web servisa.

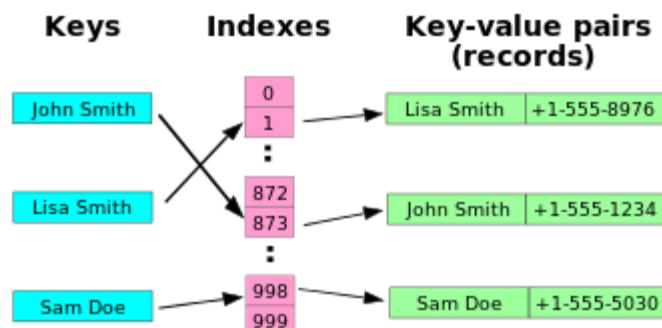
Fault element koji je opcioni i pruža informacije o tome šta treba uraditi ukoliko dodje do greške prilikom procesuiranja poruke

Strukture podataka:

1. **Hes tabela ili hes mapa** je struktura podataka koja koristi hes funkciju za preslikavanje određenih ključeva. Hes funkcija se koristi za transformisanje ključa u indeks, to jest mesto u nizu elemenata gde treba traziti odgovarajucu vrednost. Hes kolizija je kada parovi razlicitih kljucева imaju iste hes vrednosti. Prednost hes tabela je u brzini. HashMap ne sme da ima kljuc koji je duplikat, ali moze da ima iste vrednosti. Imamo parove kljuc-vrednost. Hes funkcija hesira vrednost i onda je taj rezultat pozicija u memoriji i pristupa se direktno tome umesto da se prelazi neki određen deo/blok i gubi vreme.

Hash vrednost predstavlja indeks u internoj tablici hash mape koja sadrži parove ključ-vrednost. Ova tablica se često naziva i bucket array, a svaki element u njoj predstavlja jedan bucket. Kada se pristupa elementu u hash mapi, hash funkcija se prvo primenjuje na ključ kako bi se dobila hash vrednost. Zatim se hash vrednost koristi da bi se pronašao indeks bucket-a u kojem se može nalaziti traženi ključ. Ovaj proces može da bude spor ako se bucket-i popune previše, što dovodi do kolizija, odnosno situacija kada dva različita ključa imaju istu hash vrednost.

Hash funkcije - jednosmerne funkcije. Dobijanje poruke fiksne duzine od poruke proizvoljne velicine, ne postoji inverzna funkcija i nije moguće naci originalnu poruku na osnovu izlaza. Za isti ulaz se generise isti izlaz, slaba otpornost na koliziju.



Kolekcije: liste, recnik

Kolekcije su klase za cuvanje podataka

2. **Lista** - ima indekse

3. **Recnik** - vrednostima dodeljuje kljuc. Asocijativni niz/mapa/recnik predstavlja apstraktni tip podataka skup parova kljuc-vrednost, tako da je kljuc jedinstven odnosno pojavljuje se samo jednom u skupu. Operacije: dodavanje(umetanje), zamena, uklanjanje(brisanje), pretraga. Dodavanje dodaje novi kljuc-vrednost par u kolekciju, vezujuci novi kljuc i dodeljenu vrednost. Argumenti ove operacije su kljuc i vrednost. Zamena menja vrednost jednog kljuc-vrednost para koji se vec nalazi u kolekciji, vezujuci stari kljuc i novu vrednost. Kao i kod umetanja, argumenti ove operacije su kljuc i vrednost. Uklanjanje uklanja kljuc-vrednost par iz kolekcije, brisuci vezu izmedju datog kljuca i njegove vrednosti. Argument ove operacije je kljuc. Pretraga pronalazi vrednost(ukoliko postoji) koja je vezana za dati kljuc. Argument ove operacije je kljuc, a vraca se vrednost. Ukoliko vrednost nije pronadjena, neke implementacije asocijativnih nizova kreiraju izuzetak. Dictionary u C# omogucuje brzi pristup podacima, sto ga cini korisnim za mnoge primene, poput pretrazivanja, sortiranja i filtriranja podataka.

asocijativni niz ili hash je onaj kome se pristupa preko imenovanih indeksa

Automobili["auto1"]="BMW"

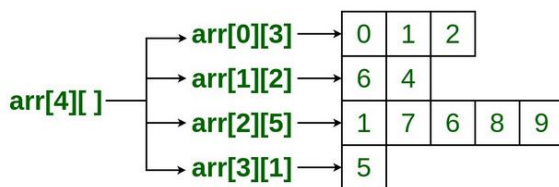
ne moze svuda tako npr js moze iskljucivo Automobili[1]="BMW";

4. **Hashset** - vrednosti se hesuju tj. kriptuju i kasnije se pronalaze po toj vrednosti. Nema veze koliko je dugacak; ne moze da se skladišti isti objekat više puta(uopšte ne sme da ima duplikat elemenata).

ArrayList je dinamički niz. Mozes dodavati i uklanjati elemente iz njega u toku runtime. Elementi nisu automatski sortirani.

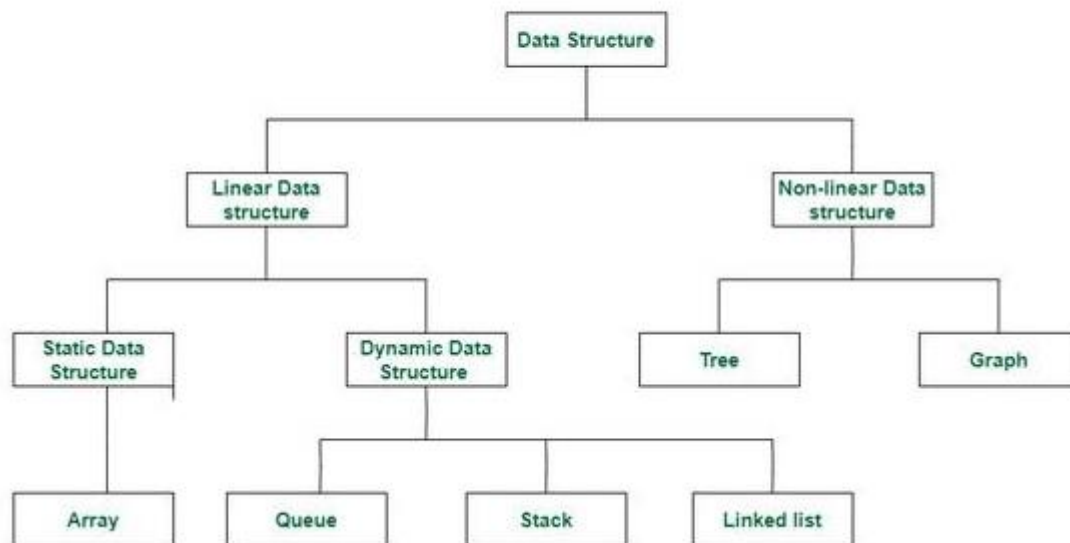
Nizovi mogu biti: jednodimenzionalni, visedimenzionalni, nazubljeni(jagged) - neravnomerno rasporedjeni

Jagged Array



Strukture podataka:

Classification of Data Structure



Strukture podataka su skladišta koja se koriste za organizovanje podataka. Predstavljaju način uređivanja podataka na računaru tako da se može menjati i ažurirati efikasno. Spram toga kakvi su zahtevi projekta biramo odgovarajuću strukturu podataka. Ako želimo da podatke smestamo sekvencijalno(jedne do drugih) u memoriji koristimo nizove. Strukture podataka predstavljaju kolekciju tipova podataka uređenu specifičnim redom.

Strukture podataka se dele na: linearne(cvorovi tj. podaci su poredjani jedni do drugih), razgranate(stabla, postoji koreni cvor i svaki cvor može biti povezan sa samo jednim ili više svojih direktnih potomaka, ali ne može pokazivati na svoje pretke), umrežene(grafovi, bilo koji cvor može biti povezan sa bilo kojim).

Prema mogućnosti dodavanja i uklanjanja pojedinačnih elemenata se dele na: statičke i dinamičke. **Statičke** su one kod kojih uklanjanje i dodavanje novih elemenata nije moguće.

Dinamičke dozvoljavaju dodavanje i uklanjanje elemenata.

Kod **statičkih struktura**, povezanost elemenata ostvaruje se implicitno(posredno), time što susedni elementi zauzimaju susedne lokacije u memoriji, dok se kod **dinamičkih struktura** povezivanje vrši preko pokazivaca(ilij referenci). **Pokazivaci** pokazuju na bilo koju memorijsku lokaciju(mogu da čuvaju bilo koju adresu). **Reference** su specijalizovani pokazivaci koji mogu pokazivati samo na adrese drugih promenljivih.

Cvor je kolekcija podataka, tipično implementirana kao slog(struct) ili objekat klase, koja sadrži jedan ili više podataka opsteg tipa(osnovni podaci, nizovi, liste, slogovi ili objekti klase i slično) i jedan ili više pokazivaca na srodne cvorove.

Strukture(struct) u C# su ograničene i ne mogu da imaju nasleđivanje, konstruktore bez argumenata, destruktore ili inicijalizatorske blokove. Koriste se kada se prosleđuju kao argumenti funkcija, dok se klase prosleđuju kao reference. Sastoje se od više različitih tipova podataka.

```

public struct Point {
    public int X;
    public int Y;
    public Point(int x, int y) {
        X = x;
        Y = y;
    }
    public override string ToString() {
        return string.Format("{0}, {1}", X, Y);
    }
}

```

Linearne strukture podataka - elementi su uredjeni u redosledu jedan za drugim. Lako se implementiraju, ali se sa povećanjem kompleksnosti programa povećava i kompleksnost operacija. Tu spadaju: nizovi, stek, red, linked list(spregruta lista)...

Niz elemente smesta jedne do drugih rasporedjene u jedan memorijski blok. **Pristup preko indeksa.**

double[] niz = new double[10];

Operacije: niz[2], niz.length

Stek smesta podatke po LIFO(last in first out) principu. Princip su palacinke, poslednja pecena je na vrhu, a prva se uzima.

Operacije: Push(dodaj), Pop(izvadi), IsEmpty(da li je prazan), IsFull(da li je pun), Peek(get value of top element without removing it)

Kada se stek inicijalizuje TOP == -1, pokazivac top prati element na vrhu

Red koristi FIFO(first in first out) princip. Kao red u supermarketu, ko je prvi dosao prvi dolazi do kase.

Operacije: enqueue(dodavanje), dequeue(skidanje), IsEmpty, IsFull, Peek(get the value of the front of the queue without removing it)

Razliciti tipovi reda:

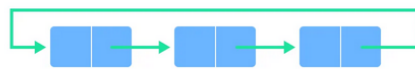
Simple Queue

In a simple queue, insertion takes place at the rear and removal occurs at the front. It strictly follows the FIFO (First in First out) rule.



Circular Queue

In a circular queue, the last element points to the first element making a circular link.

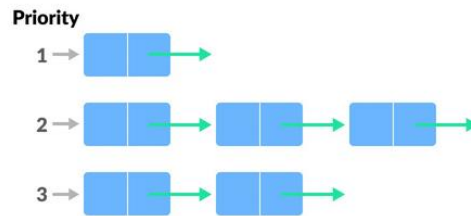


Circular Queue Representation

The main advantage of a circular queue over a simple queue is better memory utilization. If the last position is full and the first position is empty, we can insert an element in the first position. This action is not possible in a simple queue.

Priority Queue

A priority queue is a special type of queue in which each element is associated with a priority and is served according to its priority. If elements with the same priority occur, they are served according to their order in the queue.

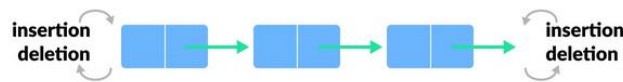


Priority Queue Representation

Insertion occurs based on the arrival of the values and removal occurs based on priority.

Deque (Double Ended Queue)

In a double ended queue, insertion and removal of elements can be performed from either from the front or rear. Thus, it does not follow the FIFO (First In First Out) rule.



Ulancane liste(linked list) -- Spregnuta(povezana) lista elemente povezuje kroz niz cvorova. Svaki cvor sadrzi podatak i adresu sledeceg cvora. Ulancana lista je zapravo dinamicki niz i omogucuje proizvoljno("dinamicko") dodavanje i uklanjanje.

Vrste listi su: jednostruko spregnuta, dvostruko spregnuta, cirkularna(kruzna).

Singly Linked List

It is the most common. Each node has data and a pointer to the next node.

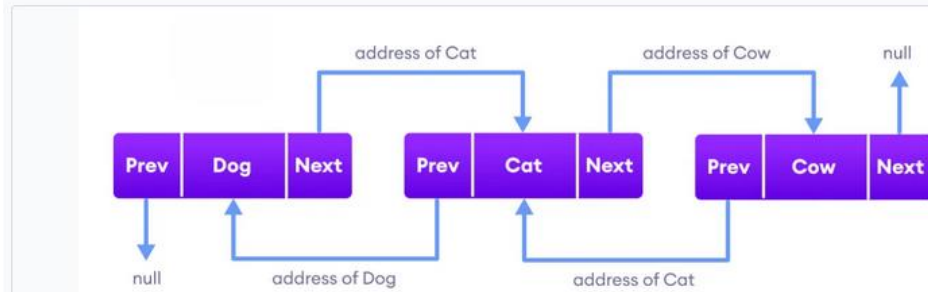


Doubly Linked List

We add a pointer to the previous node in a doubly-linked list. Thus, we can go in either direction: forward or backward.

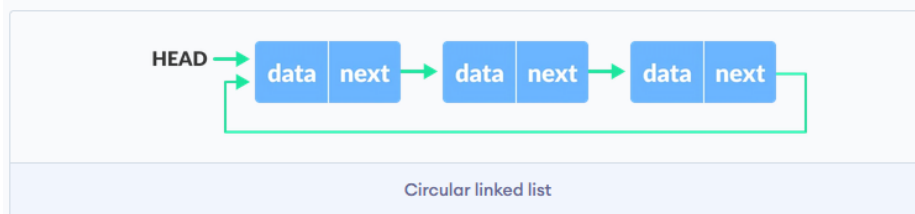


connected through links (**Prev** and **Next**).



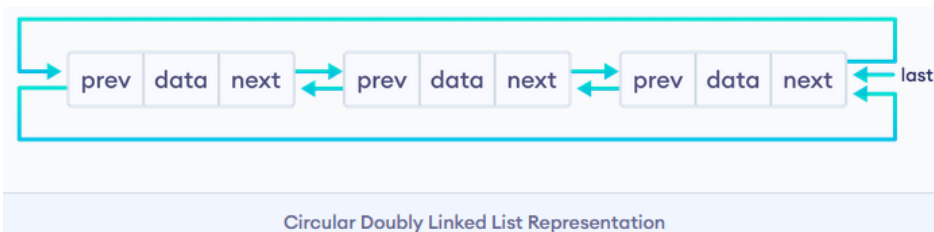
Circular Linked List

A circular linked list is a variation of a linked list in which the last element is linked to the first element. This forms a circular loop.



A circular linked list can be either singly linked or doubly linked.

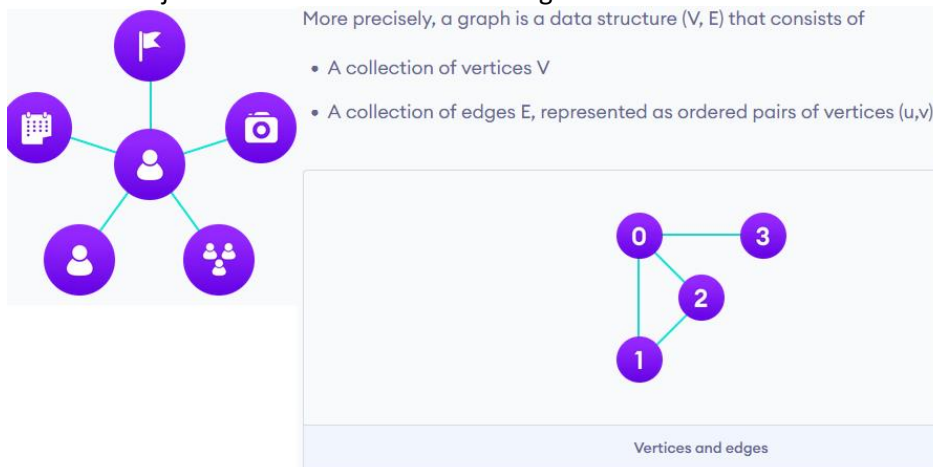
- for singly linked list, next pointer of last item points to the first item
- In the doubly linked list, `prev` pointer of the first item points to the last item as well.



Nelinearne strukture podataka nemaju redosled kojim smestaju podatke. Uredjene su hijerarhijskim manirom gde ce element biti povezan sa jednim ili vise elemenata. Dele se na stabla i grafove. Tu jos spadaju i mape. Ukoliko krenemo od odredenog elementa mozda necemo moci da dodjemo do svih, dok kod linearnih struktura polazenjem od pocetnog elementa sigurno dolazimo do krajnjeg. Nema potrebe da imamo slobodan veliki blok memorije. Kompleksnost uvek ostaje ista.

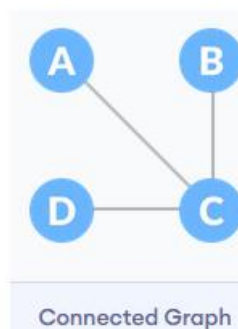
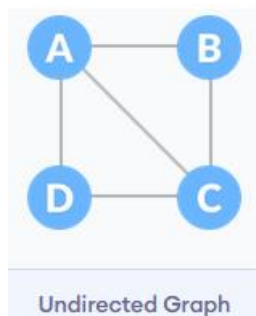
Primer primene binarnog stabla je telefonski imenik. Svaki unos ima svoj kljuc, a binarno stablo mozemo koristiti da organizujemo podatke na temelju kljuca(imena osoba/broj telefona).

Graf - ima cvorove od kojih svaki ima vezu ka nekom drugom cvoru. Cvor se zove vertex.



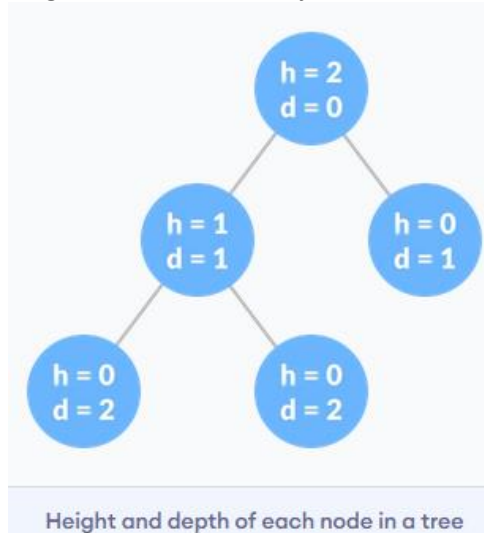
In the graph,

```
V = {0, 1, 2, 3}
E = {(0,1), (0,2), (0,3), (1,2)}
G = {V, E}
```

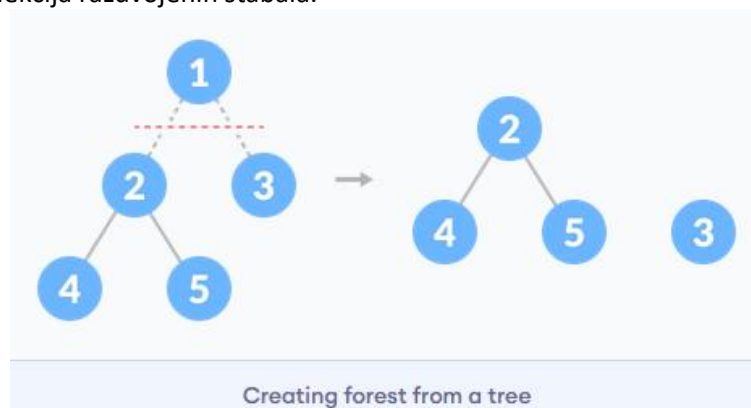


Neusmeren graf je onaj gde nema pravca. **Povezan graf** je onaj koji je uvek putanja od jednog vertexa ka drugom.

Stablo - takodje predstavlja kolekciju cvorova i njihovih veza, ali u slucaju stabla samo jedan cvor moze da se nalazi izmedju 2 veze i na taj nacin povezuje cvorove razlicitih nivoa. Ima koren(pocetni cvor), grane(veze izmedju cvorova), listove(cvorove koji su poslednji u nizu). Visina cvora predstavlja broj grana koje mora preci da bi dosao do najdubljeg lista. Dubina cvora je broj grana koje je potrebno preci od korena do datog cvora. Visina stabla je visina korena ili dubina najdubljeg lista.



Suma(forest) je kolekcija razdvojenih stabala.



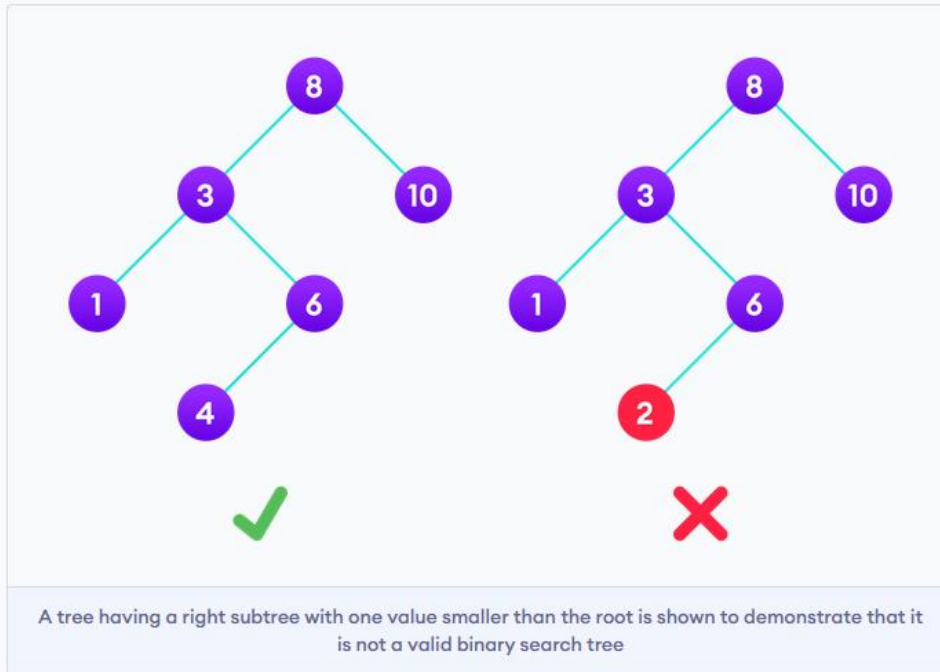
Razliciti tipovi stabla: binarno, binary search tree(BST),AVL stablo...

Za binary search je potrebno da se lista prvo sortira pa da se tek onda radi pretraga. Probaj da napravis console app i da odradis sve sort i search algoritme.

Binary Search Tree(BST)

The properties that separate a binary search tree from a regular **binary tree** is

1. All nodes of left subtree are less than the root node
2. All nodes of right subtree are more than the root node
3. Both subtrees of each node are also BSTs i.e. they have the above two properties



The binary tree on the right isn't a binary search tree because the right subtree of the node "3" contains a value smaller than it.

Binarno stablo je ono u kome svaki roditelj moze imati najvise 2 deteta. Sastoji se od 3 stavke: podatak, adresa levog deteta, adresa desnog deteta.
Postoje razlicite vrste binarnog stabla:

1. Full Binary Tree

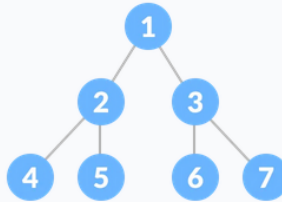
A full Binary tree is a special type of binary tree in which every parent node/internal node has either two or no children.



Full Binary Tree

2. Perfect Binary Tree

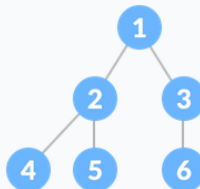
A perfect binary tree is a type of binary tree in which every internal node has exactly two child nodes and all the leaf nodes are at the same level.



3. Complete Binary Tree

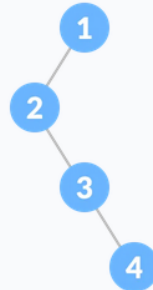
A complete binary tree is just like a full binary tree, but with two major differences

1. Every level must be completely filled
2. All the leaf elements must lean towards the left.
3. The last leaf element might not have a right sibling i.e. a complete binary tree doesn't have to be a full binary tree.



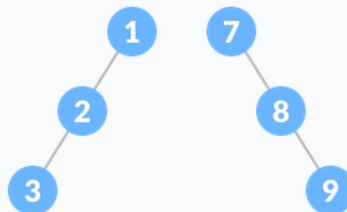
4. Degenerate or Pathological Tree

A degenerate or pathological tree is the tree having a single child either left or right.



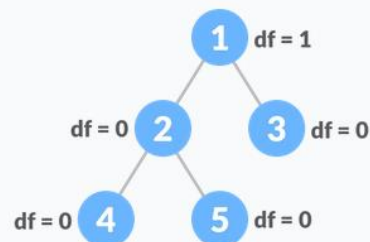
5. Skewed Binary Tree (iskrivljeno)

A skewed binary tree is a pathological/degenerate tree in which the tree is either dominated by the left nodes or the right nodes. Thus, there are two types of skewed binary tree: **left-skewed binary tree** and **right-skewed binary tree**.

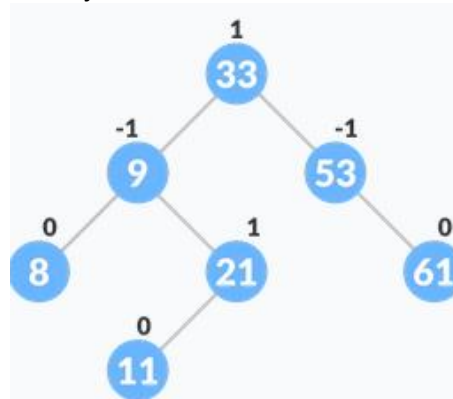


6. Balanced Binary Tree

It is a type of binary tree in which the difference between the height of the left and the right subtree for each node is either 0 or 1.



AVL stablo je self-balancing binary search tree. Svaki cvor ima informaciju koja se zove balance factor cije su vrednost -1, 0 ili +1. Faktor ravnoteze cvora je razlika izmedju visine levog podstabla i visine desnog postabla tog cvora. 1 znaci da je levo element vise, -1 znaci da je desno element vise. Ako premasi ove brojeve radi se AVL rotacija.



HashMap je ekvivalentna recniku u C#.

Dictionary(recnik) kolekcija koja ima kljuc-vrednost parove. Svaka vrednost ima kljuc. Operacije: Add, Clear, ContainsKey, ContainsValue, Equals, GetType, Remove.

HashSet kreira kolekciju koja koristi hes tabelu za skladistenje. Razlika izmedju liste i **Set**-a je u tome sto set sadrzi samo **unikatne elemente bez ponavljanja**.

Hes funkcija je svaki algoritam koji podacima proizvoljne duzine dodeljuje podatke fiksne duzine. Korisno kada su originalni podaci suvise glomazni da bi se koristili u celosti. Prakticna primena je struktura podataka koja se zove **hes tabela** u kojoj su podaci smesteni asocijativno. Linearna pretraga imena osobe u listi traje duze kako se duzina liste povecava, ali hesirana vrednost moze skladistiti referencu na originalni podatak, pa tako dobijamo konstantno vreme za pretragu. Koriste se u kriptografiji za ubrzano trazenje stavki u bazama podataka. Hes funkcije se koriste za izgradnju keseva za velike skupove podataka uskladenih u sporim medijima. Kes je jednostavniji od hes tabele za pretragu, posto svaki sudar moze da se resi odbacivanjem starije od dve sudarajuce stavke. Hes funkcije se prvenstveno koriste u hes tabelama da se brzo pronadje podatak(u recniku npr.) ako imamo dat kljuc, odnosno rec cije znacenje trazimo.

Svakom slovu stringa se dodeljuje 2 bajta, a za brojeve imamo Int32 i Int64(4 i 8 bajta, a veliki broj).

Heap je struktura podataka organizovana po principu stabla koja mora da zadovoljava uslove: **kljuc svakog cvora je veci ili jednak od kljuceva njihovih sinova**. Koristi se za pronalazenje najvećih ili najmanjih elemenata. Kada je maksimalan broj dece jednog cvora 2, to je binarni hip.

Sortiranje:

Merge sort - rastavlja niz na 2 dela i svaki deo rastavlja tako sve dok ne dodje do 1 broja u delu.

Nakon sto se to desi upoređuje 1 broj sa brojem do njega , potom 1 podniz sa drugim i tako sve dok se na kraju ne dobije ceo niz.

28539417

28 35 49 17

2358 1479

Bubble sort - porede se susedni elementi. Veci ide desno.

Quick sort - dodelis jednom broju da bude pivot i onda svaki clan niza uporedjujes sa njim. Ukoliko broj poredis sa nekih elementom koji je veci od pivota i jedan je manji onda njihova mesta menjas. Sa leve strane ti bude sortirano, a desnom delu dodelis novog pivota.

7 2 1 6 8 5 3 4

izaberemo 4 kao pivota -> 2 1 3 4 8 5 7 6

podelimo u dva niza

2 1 3 4 8 5 7 6

2 1

2

Kada se dodje do jednog elementa zaustavlja se

Selection sort - proglasi indeks za minimalan i pocni od narednog for petlju. Ukoliko je manji broj sa vecim indeksom zameni im indekse, a na kraju kada prodjes sve elemente najmanji postavi na prvi indeks i u tom trenutku povecaj indeks i rekurzivno ponavljas.

Ide ispočetka i bira current minimum, current item i poredi i tako do kraja.

Sortiranje u opadajućem redosledu bez sort funkcija

```
static void Main()
{
    int[] numbers = { 5, 2, 9, 1, 3 };

    SortDescending(numbers);

    Console.WriteLine("Sortirani niz u opadajućem redosledu:");
    foreach (int number in numbers)
    {
        Console.Write(number + " ");
    }
}

static void SortDescending(int[] array)
{
    int n = array.Length;

    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - i - 1; j++)
        {
            if (array[j] < array[j + 1])
            {
                int temp = array[j];
                array[j] = array[j + 1];
                array[j + 1] = temp;
            }
        }
    }
}
```

Algoritmi:

Prebrojavanje slova, brojeva i specijalnih znakova

```
string input = "Hello! 123#";

int specialChars = 0;
int letters = 0;
int numbers = 0;

foreach (char c in input)
{
    if (Char.IsLetter(c))
    {
        letters++;
    }
    else if (Char.IsDigit(c))
    {
        numbers++;
    }
    else
    {
        specialChars++;
    }
}

Console.WriteLine("Specijalni znakovi: " + specialChars);
Console.WriteLine("Slova: " + letters);
Console.WriteLine("Brojevi: " + numbers);
```

Za Niz.Contains(element); Array.Sort(Niz) -- koristi using System.Linq
Array.IndexOf(niz, number); Niz.Distinct().ToArray()

Task 1

Programming Language
C#

This is a demo task.

Write a function:

```
class Solution { public int solution(int[] A); }
```

that, given an array A of N integers, returns the smallest positive integer (greater than 0) that does not occur in A.

For example, given A = [1, 3, 6, 4, 1, 2], the function should return 5.

Given A = [1, 2, 3], the function should return 4.

Given A = [-1, -3], the function should return 1.

Write an **efficient** algorithm for the following assumptions:

- N is an integer within the range [1..100,000];
- each element of array A is an integer within the range [-1,000,000..1,000,000].

Copyright 2009–2023 by Codility Limited. All Rights Reserved. Unauthorized copying, publication or disclosure prohibited.

Performance describes if the program behaves correctly for large inputs. Points are deducted if the program exceeds the time limit on large data sets, which indicates a sub-optimal approach.

Tasks summary

MissingInteger		Correctness	Performance	Task score
C#		100%	50%	77%
Example test cases	Correctness test cases	Performance test cases		
Passed 3 out of 3	Passed 5 out of 5	Passed 2 out of 4		

Files

solution.cs x

test-input.txt x

task1

solution.cs

test-input.txt

```
1 using System;
2 // you can also use other imports, for example:
3 // using System.Collections.Generic;
4
5 // you can write to stdout for debugging purposes, e.g.
6 // Console.WriteLine("this is a debug message");
7
8 class Solution {
9     public int solution(int[] A) {
10         // Implement your solution here
11         if(A.Length == 0)
12             return 1;
13
14         bool poz=false;
15
16         foreach(int br in A)
17         {
18             if(br<=0)continue;
19             poz=true;
20             break;
21         }
22
23         if(poz==false)return 1;
24
25         int poc=1;
26         int prom=poc;
27         for(int i=0; i<A.Length; i++)
28         {
29             for(int j=0;j<A.Length;j++)
30             {
31                 if(poc == A[j])
32                 {
33                     poc++;
34                 }
35                 if(prom==poc)
36                 {
37                     return poc;
38                 }
39                 else
40                 {
41                     prom=poc;
42                 }
43             }
44             return prom;
45         }
46     }
```

Correctness 100%

Performance 75%

Task score 88%

Sta ako je niz 1,1,1,1,1,1,1,1,1,1,1,

```
using System.Linq;
class Solution {
    public int solution(int[] A) {
        // Implement your solution here
        if(A.Length == 0)return 1;

        bool poz=false;

        foreach(int br in A)
        {
            if(br<=0)continue;
            poz=true;
            break;
        }
        if(poz==false)return 1;

        int poc=1;
        Array.Sort(A);
        if(!A.Contains(poc))return 1;

        int index = Array.IndexOf(A, poc);

        for(int i = index; i<A.Length-index; i++)
        {
            if(poc==A[i])
            {
                poc++;
            }
        }
        return poc;
    }
}
```

```
using System.Linq;
class Solution {
    public int solution(int[] A) {
        // Implement your solution here
        if(A.Length == 0)return 1;
        int[] niz = A.Distinct().ToArray();

        int poc=1;
        if(!niz.Contains(poc))return 1;
        Array.Sort(niz);

        int index = Array.IndexOf(niz, poc);

        for(int i = index; i<niz.Length-index; i++)
        {
            if(poc==niz[i])
            {
                poc++;
            }
            else
            {
                break;
            }
        }
        return poc;
    }
}
```

Provera da li je broj Armstrongov

```
static void Main()
{
    Console.Write("Unesite broj: ");
    int number = int.Parse(Console.ReadLine());

    if (IsArmstrong(number))
    {
        Console.WriteLine("Broj je Armstrongov.");
    }
    else
    {
        Console.WriteLine("Broj nije Armstrongov.");
    }
}
```

```
static bool IsArmstrong(int number)
{
    int originalNumber = number;
    int result = 0;
    int power = GetNumberOfDigits(number);

    while (number != 0)
    {
        int digit = number % 10;
        result += (int)Math.Pow(digit, power);
        number /= 10;
    }

    return result == originalNumber;
}

static int GetNumberOfDigits(int number)
{
    int count = 0;

    while (number != 0)
    {
        count++;
        number /= 10;
    }

    return count;
}
```


Da li je string palindrom

```
static void Main(string[] args)
{
    Console.WriteLine("Unesite string:");
    string str = Console.ReadLine();

    if (IsPalindrome(str))
    {
        Console.WriteLine("String je palindrom.");
    }
    else
    {
        Console.WriteLine("String nije palindrom.");
    }
}
```

```
static bool IsPalindrome(string str)
{
    // Uklanjam sve razmake i pretvaramo sve karaktere u mala slova
    str = str.Replace(" ", "").ToLower();

    int left = 0;
    int right = str.Length - 1;

    while (left < right)
    {
        if (str[left] != str[right])
        {
            return false;
        }

        left++;
        right--;
    }

    return true;
}
```

Da li je broj zagrada odgovarajući

```
class Program
{
    static void Main()
    {
        string unos = "Ovo je (primer) sa (zagrada).";

        if (ProveriZagrada(unos))
        {
            Console.WriteLine("Broj zagrada je odgovarajući.");
        }
        else
        {
            Console.WriteLine("Broj zagrada nije odgovarajući.");
        }
    }
}

static bool ProveriZagrada(string tekst)
{
    int brojOtvorenihZagrada = 0;

    foreach (char znak in tekst)
    {
        if (znak == '(')
        {
            // Ako je otvorena zagrada, povećavamo broj otvorenih zagrada
            brojOtvorenihZagrada++;
        }
        else if (znak == ')')
        {
            // Ako je zatvorena zagrada, smanjujemo broj otvorenih zagrada
            brojOtvorenihZagrada--;

            // Ako broj otvorenih zagrada postane negativan, to znači da ima više zatvorenih zagrada nego otvorenih
            if (brojOtvorenihZagrada < 0)
            {
                return false;
            }
        }
    }

    // Na kraju, ako je broj otvorenih zagrada jednak nuli, to znači da su svi parovi zatvorenih i otvorenih zagrada pravilno upareni
    return brojOtvorenihZagrada == 0;
}
```

Proceduralno programiranje:

Primenjuje princip u kome se problem sagledava "odozgo-nadole"(top-down), koji podrazumeva da se problem prvo sagledava u celosti, a zatim po potrebi pravilno razbija na potprobleme koji se pretoce u funkcije. U velikim projektima je problem sto je slozenost velika i nije moguće sagledati ga odjednom nego se deli na manje celine i tada se primenjuje OOP.

OOP:

Primenjuje pristup "odozdo-nagore"(bottom-up) koji definise osnovne delove sistema(objekte) i pojedinačni način funkcionisanja svakog od delova, kao i načine njihovog povezivanja. Celokupan sistem nastaje spajanjem i interakcijom pojedinačnih elemenata.

Primer: simulacija voznje. Znamo da se javljaju automobili i staze, ali prvo definisemo pravila za svaki objekat i potom medjusobne odnose koji odgovaraju zakonitostima fizike iz spoljnog sveta(gravitacija, inercija, detekcija sudara i slicno), cime se na kraju kreira funkcionalan sistem.

Paradigme pored oop:

Imperativna - KAKO izvršiti logiku i definisati tok iskaza za menjanje stanja programa. Tu spadaju: proceduralna paradigma, OOP, paralelno programiranje.

Deklarativna - STA izvršiti i definise programsku logiku, ali ne daje detalje kontrole toka. Tu spadaju: logicko programska paradigma(cinjenice i pravila), funkcionalna, paradigma baze podataka.

Koncepti OOP:

1. **Nasledjivanje** - mogućnost da se na osnovu postojećih klasa izvedu nove klase koje prosiruju, koriste ili menjaju ponašanje definisano u baznoj klasi. Bazna klasa - ona koja je nasledjena. Izvedena - ona koja nasledjuje baznu klasu. Postoje jednostruko i višestruko nasledjivanje. Jednostruko je kada klasa A nasledjuje samo klasu B. Višestruko nasledjivanje ima C++. Izvedena klasa je potklasa(potomak), a opstija je nadklasa(predak). Kreira se hijerarhijska nasledna linija. Prklasa u C# iz koje su izvedene sve ostale je klasa Object.

Primer: imamo klasu "Osoba" i treba nam jos da definisemo radnika. Posto bi imao ista polja mi cemo naslediti klasu osoba. Time imamo ista polja i mozemo da koristimo iste metode i jos imamo mogućnost dodavanja novih polja i metoda.

```
public Radnik(String Ime, String Prezime, String DatumRodjenja,
               DateTime PocetakRada) : base(Ime, Prezime, DatumRodjenja)
{
    this._RadnoMesto    = RadnoMesto;
    this._DatumRodjenja = DatumRodjenja;
    this._PocetakRada   = PocetakRada;

    RacunanjeStaza();
}
```

Inicijalizuju se sva polja objekta klase Radnik, uključujući polja koja pripadaju klasi Osoba, kao i zajednička polja koja su definisana u obe klase.

Bazna klasa(base) je roditeljska klasa. Ako imamo K1, K2, K3 i K2:K1, K3:K2 onda je za K3 base K2.

```

class Osoba {
    public string ime;
    public string prezime;
    public int godine;

    public Osoba(string ime, string prezime, int godine) {
        this.ime = ime;
        this.prezime = prezime;
        this.godine = godine;
    }

    public override string ToString() {
        return $"{ime} {prezime} ({godine} godina)";
    }
}

class Radnik : Osoba {
    public double plata;

    public Radnik(string ime, string prezime, int godine, double plata) : base(ime, prezime, godine) {
        this.plata = plata;
    }

    public override string ToString() {
        return base.ToString() + $", plata: {plata}";
    }
}

class Program {
    static void Main(string[] args) {
        Osoba osoba1 = new Osoba("Marko", "Markovic", 30);
        Console.WriteLine(osoba1);

        Radnik radnik1 = new Radnik("Petar", "Petrovic", 25, 50000);
        Console.WriteLine(radnik1);
    }
}

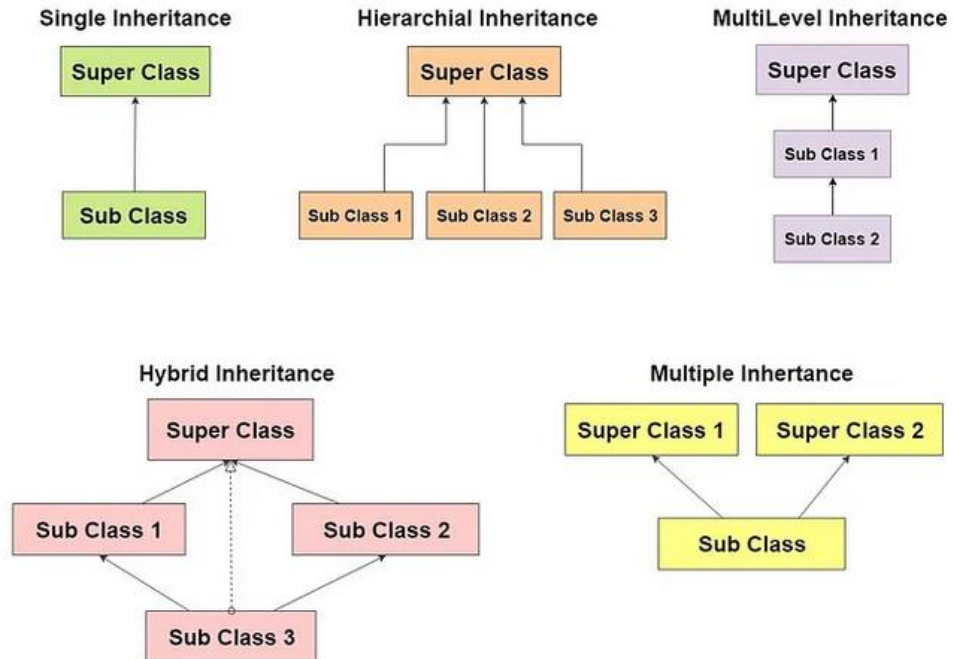
```

Ispis:

```

Marko Markovic (30 godina)
Petar Petrovic (25 godina), plata: 50000

```



Problem dijamanta - Javlja se u C++, u C# ne jer nema visestrukog nasledjivanja. Javlja se kada vise klasa koje se nasledjuju imaju istoimeni metod, a klasa koja nasledjuje ne implementira taj metod. Ako pokusamo da pozovemo metodu javlja se ambiguitet jer nije jasno koju implementaciju metode treba koristiti. Ako klasa implementira taj metod ili nasledjuje samo jednu klasu sa istoimenim metodom, problem dijamanta se ne javlja.

2. **Polimorfizam** (viseoblicje, poly - mnogo, morph - oblik) - sposobnost da se metode ponasaju na vise razlicitih nacina, ima mogucnost da metodu deklarises da je virtual(moze da se override). U baznoj klasi ima odredjeno ponasanje, a u izvedenoj imamo override i drugacije ponasanje zbog tela metode, a sve ostalo je isto(najcesce override ToString metode zbog ispisa). Omogucava da nad baznom klasom/interfejsom pozoves metodu i da on nadje bas onaj objekat koji je potreban npr. `interfejs.ToString()` on nadje bas tacan tip koji treba. Tipa: ako je `int` onda njega pretvara, ako je `float` onda njega. To je osobina da ista metoda deluje razlicito, u zavisnosti od tipa ili broja parametara koji joj se prosledjuju. Npr. `MessageBox` se razlikuje po argumentima. Npr. moze i metode `Ukljuci/Iskljuci`, mozes da ih koristis i za brisace i za svetla automobila, ali radi na drugaciji nacin. Klasa `'Pravougaonik'` nasledjuje klasu `'Oblik'`. Radi override metode za obim i povrstinu(prilagodi ih).

```
public override double Obim() {  
    return 2 * (A + B);  
}  
  
public override double Povrsina() {  
    return A * B;  
}
```

Compile-time polimorfizam se odnosi na odluku o tome koji će se metod pozvati tokom faze kompajliranja. Ova odluka se zasniva na tipu objekta ili promenljive koja poziva metod. Na primer, u jeziku C++, funkcija koja uzima dva cela broja može se koristiti i za sabiranje i za množenje. Ova funkcija će se pozvati u zavisnosti od operatora koji se koristi (+ ili *), a ova odluka se donosi tokom kompajliranja. **Kompajler** cita izvorni kod i prevodi ga u niz instrukcija koje racunar moze da izvrši. To se zove **overloading**, dok je kod runtime polimorfizma overriding.

Runtime polimorfizam se odnosi na odluku o tome koji će se metod pozvati tokom izvršavanja programa, na osnovu tipa objekta koji poziva metod. U ovoj vrsti polimorfizma, odluka o tome koji će se metod pozvati se ne donosi tokom faze kompajliranja, već tokom faze izvršavanja. Ovo omogućava da se program prilagodi promenljivim uslovima tokom izvršavanja. Primer runtime polimorfizma je virtuelna funkcija u jeziku C++. Kada se poziva virtuelna funkcija nad objektom, odluka o tome koji će se metod pozvati se donosi u zavisnosti od tipa objekta u trenutku izvršavanja. **Izvršavanje** je proces pokretanja izvrsnog koda koji je generisan kompajliranjem. Racunar cita instrukcije iz izvrsnog koda i izvršava ih.

3. **Enkapsulacija(ucaurivanje)** - grupacija, zastita, pokrivanje podataka koji su slicni(najcesce u okviru klase, mozes u interface da enkapsuliras bitne metode). Ovim konceptom je informacija zasticena od direktnog pristupa i jedini nacin da se promeni je kroz definisane metode. Poznati su ulaz i izlaz, ali ne i ono sto se odvija unutar funkcije. Vrsi se preko **modifikatora pristupa(access modifiers): public, private, protected**. Javni clanovi mogu slobodno da se koriste u bilo kom delu programa(i van same klase), dok su privatni dostupni samo u okviru klase u kojoj su definisani. **Polja su najcesce privatna, dok su metode javne. Time su pristup i promena tih podataka ograniceni na sam objekat. Unutrasnja implementacija objekta se skriva, a pristup objektu se obezbedjuje preko javnih metoda. Druge klase ne mogu da promene vrednost polja, omogucava nam da vrsimo validaciju prilikom postavljanja vrednosti polja.** Kod `private` - clanovi se ne mogu naslediti. `Protected` - clanovima se moze pristupiti u okviru izvorne ili izvedene klase, tj. mogu se nasledjivati.

Primer: vozac automobila na raspolaganju ima volan, menjac, gas, kvacilo i kocnicu, cime ima mogucnost da upravlja autom, ali nema uticaja na dizajn karburatora, prenosnog mehanizma, klipova... To je posao autoinzenjera. Isto tako programer dizajnira i odrzava klase, a korisnik koristi aplikaciju.

Accessors property(accessors) - mehanizam za pristup i manipulaciju privatnim poljima. Sastoje se od get i set blokova. Get za citanje, a set za postavljanje nove vrednosti svojstva. Mozemo pristupiti vrednosti svojstva kao sto bismo pristupili polju:

```
public class Person
{
    private string name;

    public string Name
    {
        get { return name; }
        set { name = value; }
    }
}
```

```
Person person = new Person();
person.Name = "John"; // Postavljanje vrednosti svojstva
Console.WriteLine(person.Name); // Čitanje vrednosti svojstva
```

Ne bi smeo da unosis obim i površinu, vec samo da ih citas. Potrebno je da se oni azuriraju pri promeni stranica a i b. U slucaju da su obim i površina tipa **public** mogli bi ih promeniti i staviti negativnu vrednost sto je matematski nemoguće. Zato koristimo **private**. Razumno je pretpostaviti da ce bar povremeno postojati potreba za azuriranjem stranica pravougaonika i citanjem vrednosti pojedinačnih polja.

Primer citanja vrednosti privatnog polja:

```
public Double Citanje_a()
{
    return this.a;
}
```

Primer za azuriranje:

```
public void Upis_a(Double a)
{
    // Radi preglednosti nećemo ovde
    // pisati deo koda sa porukom o grešci,
    // izuzecima i sl.

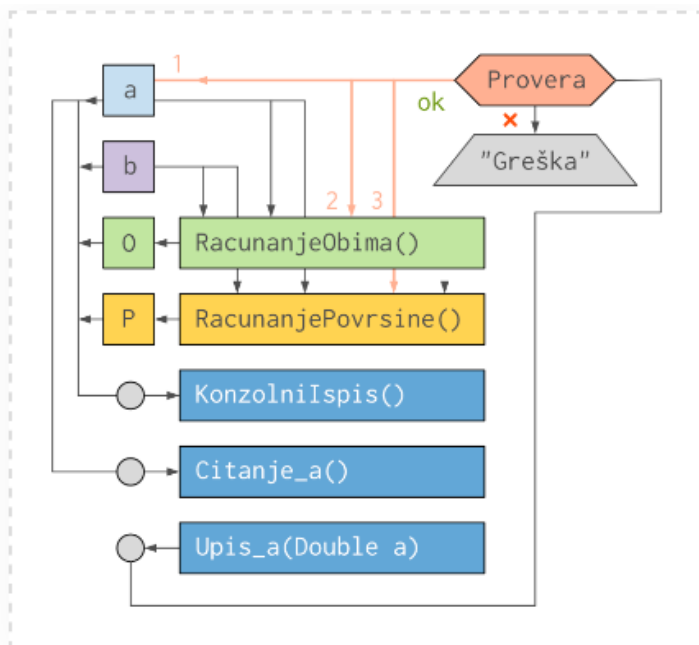
    if (a <= 0) return;

    // Ako je uneta vrednost pozitivan broj,
    // ažuriraćemo obim i površinu

    this.a = a;
    RacunanjeObima();
    RacunanjePovrsine();
}
```

Na ovaj nacin se radi prover a i ukoliko je rezultat validan radi se racunanje i promena **polja a**.

Prikaz desavanja "ispod haube".



Šematski prikaz klase pravougaonik (konstruktor je izostavljen zarad preglednosti)



Šematski prikaz objekta klase Pravougaonik: spolja gledano, objekat nudi samo nekoliko metoda i ne prikazuje unutrašnje mehanizme obrade podataka

Moguće je koristiti i svojstva(samo u C#), to je konvencija. Koristi get i/ili set metodu. Primer: vracamo vrednost polja.

```

public UInt32 RadniStaz
{
    get
    {
        return _RadniStaz;
    }
}

```



```

public class Pravougaonik
{
    private Double _a, _b, _O, _P;

    public Pravougaonik(Double a, Double b)
    {
        this.a = a;
        this.b = b;
        RacunanjeObima();
        RacunanjePovrsine();
    }

    public Double a // Ovo je "svojstvo" a (nije isto što i polje a,
    { // koje smo koristili u prethodnom opisu klase)
        get
        {
            return _a;
        }
        set
        {
            if (a > 0)
            {
                this.a = value;
                RacunanjeObima();
                RacunanjePovrsine();
            }
        }
    }
}

```

Citanje

```

Pravougaonik P = new Pravougaonik(5.5, 10.5);
MessageBox.Show("Stranica a: " + P1.a.ToString());

```

dobićemo ispis stranice a (pozivom P1.a dobijamo vrednost polja P1._a, a zatim se poziva opšta funkcija ToString() kojom se ispisuje vrednost).

4. **Apstrakcija** - pojednostavljivanje karakteristika stvarnog objekta, gde se zanemaruju detalji i uzimaju samo zajednicke karakteristike klase u zavisnosti od potrebe programa. To je odabir svojstava-polja koja ce opisivati dati objekat. **Npr.** za pravougaonik definises visinu i sirinu ako program treba da racuna, a ako je graficka predstava trebaju ti i koordinate tacaka, boja, debljina linija.

Na primer, možda ćete morati da koristite ime osobe u drugoj klasi, ali ne morate znati da se ime skladišti u **ime** polju.

Nivoi pristupa klasama:

Public - vidljive u celom projektu i mogu biti nasledjene bilo gde u projektu

Private - vidljive samo u okviru iste klase i **ne mogu biti nasledjene** u bilo kojoj drugoj klasi

Protected - vidljive samo u okviru iste klase ili u okviru nasledjene klase koja se nalazi u drugom fajlu

Internal - vidljive samo u okviru istog projekta i mogu biti nasledjene samo u klasi koja se nalazi u istom projektu. Dostupna za nasledjivanje samo u okviru istog **assembly**-ja (fizicka jedinica izgradjena kao jedan ili vise fizickih fajlova, a moze biti koriscena kao jedna jedinica).

Protected internal - vidljive u okviru istog projekta i vidljive samo u okviru iste klase ili u okviru nasledjene klase koja se nalazi u drugom fajlu

-- Nivoi pristupa se odnose samo na klase i ne odnose na instance klase

Sealed se ne moze naslediti i prosiriti, ali moze instancirati - prevencija bilo kakve promene

Abstract se moze naslediti, ali ne moze instancirati direktno.

Static se ne moze ni instancirati ni naslediti

Partial se sastoji od vise fizicki odvojenih delova koji formiraju celinu

Relacione baze podataka:

Baza podataka predstavlja bilo kakvu organizovanu kolekciju podataka koja se cuva u elektronskom obliku. Poslednjih godina se odnosi na relacionu bazu podataka jer su relacioni sistemi za skladištenje i obradu podataka najrasprostranjeniji i najcesce korisceni. Tu spadaju: SQL Server, Oracle, Db2, MySQL, PostgreSQL... Za poslove administracije relacionih baza se koristi SQL (Structured Query Language), specificno namenjen radu sa bazama podataka. Nerelacione baze podataka su NoSql.

Relacionu bazu najprakticnije mozemo shvatiti kao sistem medjusobno povezanih tabela, u kome se tezi sto racionalnijem i sto ekonomicnijem zapisu podataka.

Primarni kljuc - jedinstveni podatak u okviru jednog sloga (reda) po kome je moguće prepoznati ceo slog. U praksi se najcesce oznacava sa "id" i koriste se redni brojevi.

Veza = relacija (ono po cemu je relacioni model dobio naziv). Kada primarni kljuc iz jedne tabele upotrebimo u drugoj (radi prepoznavanja podataka iz prve), takav podatak nosi naziv **sekundarni kljuc**. Primer: primarni kljuc "id" u jednoj tabeli, sekundarni kljuc "prodavac_id" u drugoj tabeli.

Celobrojna vrednost u polju "id" je vezana za jedan red, ali sam po sebi nema nikakve sustinske veze sa prodavcem "Petrom Petrovicem". Takav podatak predstavlja **surogatni (vestacki) primarni kljuc**.

Prirodni kljuc moze da se odabere, ali taj podatak ne sme da se ponavlja, a ime prodavca nije jedinstven podatak i moze se ponoviti. Prirodni kljuc bi bio email, ali se tekstualni podaci pretrazuju na manje efikasan nacin od brojcanih. Prirodni primarni kljuc se uglavnom koristi da spreči dupliranje slogova po kolonama.

Entitet je objekat koji u sistemu ima znacaj i koji se moze nedvosmisleno odrediti i zapisati. Zapisuju se u okviru jedne tabele, vezan za jedinstven primarni kljuc, entiteti iz jedne tabele se mogu navoditi u drugim tabelama preko sekundarnih/spoljnih kljuceva. Primer: entitet prodavac je odredjen skupom pojedinacnih podataka (ime, prezime, datum rođenja), to jest nedvosmisleno je odredjen i zapisan.

Atribut oznacava pojedinacni podatak koji se koristi pri definisanju odredjenog entiteta. Atributi prodavca su: ime, prezime, datum rođenja...

Tipovi relacija izmedju entiteta:

1:1 - spoljni kljuc odredjenog entiteta iz jedne tabele, moze se u drugoj tabeli pojaviti samo u jednom slogu

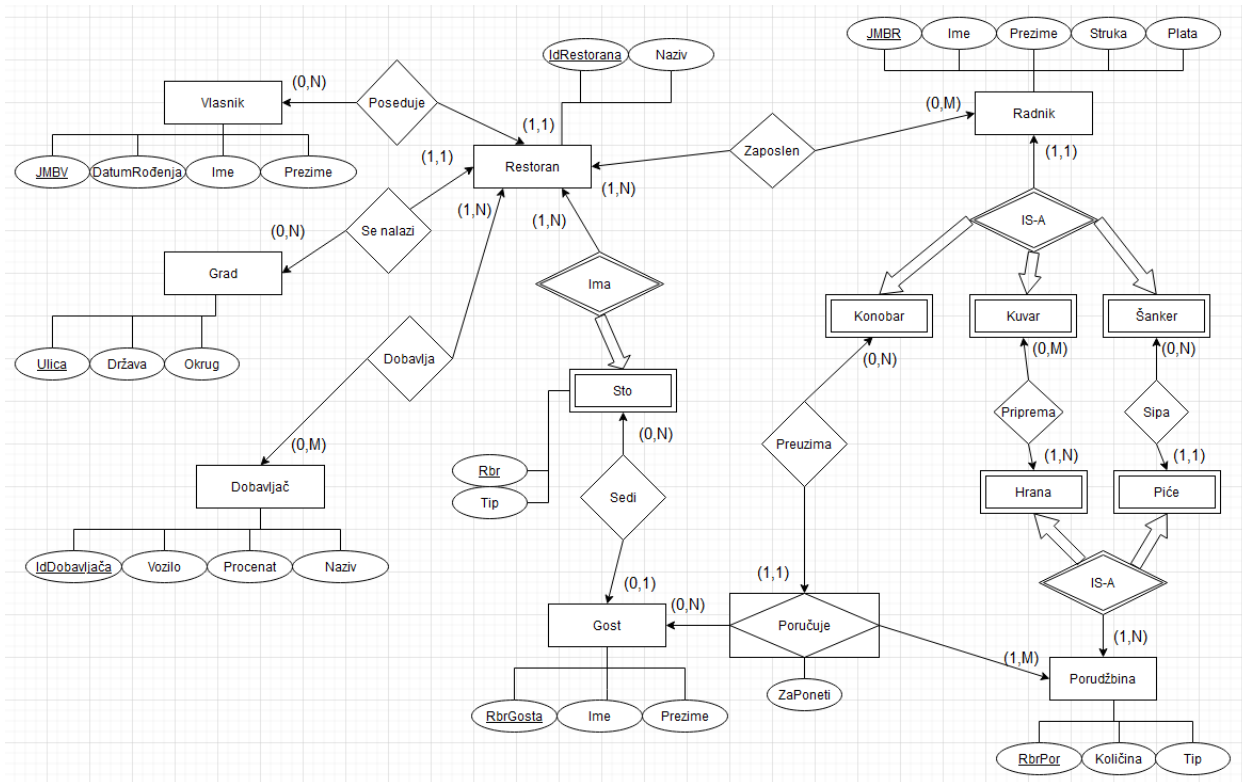
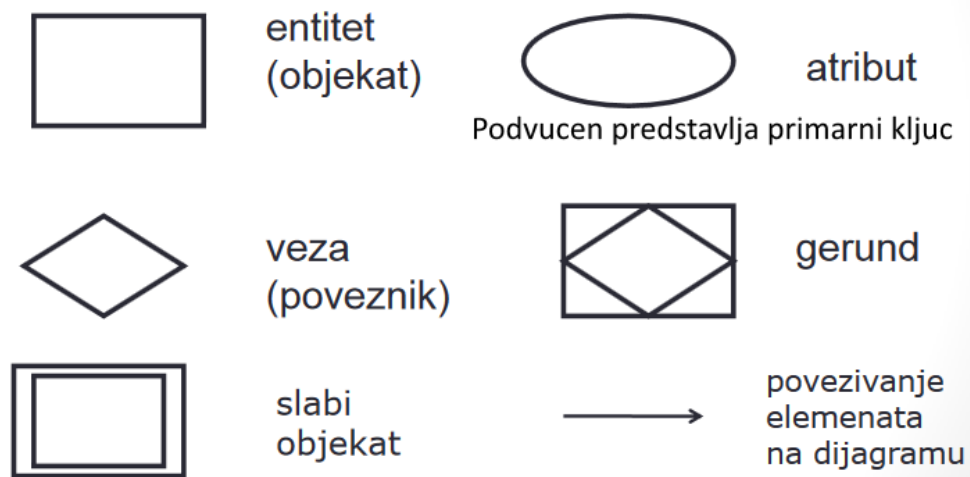
1:n - spoljni kljuc odredjenog entiteta iz jedne tabele, moze se u drugoj tabeli pojaviti u vise slogova

n:n - vise entiteta iz jedne tabele moze se povezati sa vise entiteta iz druge tabele

Struktura relacione baze i organizacija podataka:

Tabela je sistem medjusobno povezanih celija (rasporedjenih u redove i kolone), u koje se upisuju podaci (o odredjenoj kategoriji entiteta). Pojedinacna celija se naziva **polje**. Ceo red se naziva **slog**.

ER DIJAGRAM:



Relaciona sema za restoran

Vlasnik({JMBV,DatumRođenja,Ime,Prezime},{JMBV})
Grad({Ulica,Država,Okrug},{Ulica})
Dobavljač({IdDobavljača,Vozilo,Procenat,Naziv},{IdDobavljača})

Restoran({IdRestorana,Naziv,JMBV,Ulica},{IdRestorana})
null(Restoran,JMBV) = $\perp$
null(Restoran,Ulica) = $\perp$

Dobavlja({IdRestorana,IdDobavljača},{IdRestorana+IdDobavljača})
Dobavlja[IdDobavljača] $\subseteq$ Dobavlja[IdDobavljača]
Dobavlja[IdRestorana] $\subseteq$ Restoran[IdRestorana]
Restoran[IdRestorana] $\subseteq$ Dobavlja[IdRestorana]

Zaposlen({IdRestorana,JMBR},{IdRestorana,JMBR})
Zaposlen[IdRestorana] $\subseteq$ Restoran[IdRestorana]
Zaposlen[JMBR] $\subseteq$ Radnik[JMBR]
Restoran[IdRestorana] $\subseteq$ Zaposlen[IdRestorana]

Sto({Rbr,Tip,IdRestorana},{Rbr+IdRestorana})
Sto[IdRestorana] $\subseteq$ Restoran[IdRestorana]
Restoran[IdRestorana] $\subseteq$ Sto[IdRestorana]

Gost({RbrGosta,Ime,Prezime,Rbr,IdRestorana},{RbrGosta})
Gost[Rbr+IdRestorana] $\subseteq$ Sto[Rbr+IdRestorana]
null(Gost,Rbr) = $\top$
null(Gost,IdRestorana) = $\top$

Radnik({JMBR,Ime,Prezime,Struka,Plata},{JMBR})

Konobar({JMBR},{JMBR})
Konobar[JMBR] $\subseteq$ Radnik[JMBR]

Kovar({JMBR},{JMBR})
Kovar[JMBR] $\subseteq$ Radnik[JMBR]

Šanker({JMBR},{JMBR})
Šanker[JMBR] $\subseteq$ Radnik[JMBR]

Poručuje({ZaPoneti,RbrGosta,RbrPor,JMBR},{RbrGosta,RbrPor,JMBR})
Poručuje[JMBR] $\subseteq$ Radnik[JMBR]
Poručuje[RbrGosta] $\subseteq$ Gost[RbrGosta]
Poručuje[RbrPor] $\subseteq$ Porudžbina[RbrPor]
Porudžbina[RbrPor] $\subseteq$ Poručuje[RbrPor]
null(Poručuje,JMBR) = $\perp$

Porudžbina({RbrPor,Količina,Tip},{RbrPor})

Hrana({RbrPor},{RbrPor})
Hrana[RbrPor] $\subseteq$ Porudžbina[RbrPor]

Piće({RbrPor,JMBR},{RbrPor})
Piće[JMBR] $\subseteq$ Radnik[JMBR]
null(Piće,JMBR) = $\perp$

Priprema({RbrPor,JMBR},{RbrPor+JMBR})
Priprema[RbrPor] $\subseteq$ Porudžbina[RbrPor]
Priprema[JMBR] $\subseteq$ Radnik[JMBR]
Porudžbina[RbrPor] $\subseteq$ Priprema[RbrPor]

Porudžbina[RbrPor] $\subseteq$ Hrana[RbrPor] $\cup$ Piće[RbrPor]

Radnik[JMBR] $\subseteq$ Konobar[JMBR] $\cup$ Kuvar[JMBR] $\cup$ Šanker[JMBR]

Konobar[JMBR] $\cap$ Kuvar[JMBR] $\cap$ Šanker[JMBR] = $\emptyset$

SQL:

Upit je niz tekstualnih komandi(prakticno omanja skripta), preko kojih se od baze zahteva odredjeni vid obrade podataka.

SQL sistemu za upravljanje bazom podataka(skracenicna je DBMS-database management system) prosledjuje upit. Spada u deskriptivne jezike.

Tabela - struktura koja predstavlja organizovanu kolekciju podataka. Red je zapis, a kolona atribut.

Relacija - povezanost ili veza izmedju tabela. Koriste se za uspostavljanje veze izmedju tabela na osnovu zajednickih vrednosti atributa.

Kljuc - atribut ili kombinacija atributa koji jedinstveno identifikuju svaki zapis(red) u tabeli. Primary key je poseban kljuc koji jedinstveno identifikuje svaki red u tabeli. Foreign key se koristi za uspostavljanje veza izmedju tabela putem zajednickih kljuceva.

Indeks - struktura podataka koja se koristi za ubrzavanje pretrage i pristupa podacima u tabeli. Indeks se kreira na jednom ili vise atributa i omogucava pretrazivanje i sortiranje, poboljsava performanse nad velikim kolicinama podataka. Najcesce B-stablo i Hash index

Komande: kreiranje baze podataka(CREATE DATABASE), kreiranje tabele(CREATE TABLE), upis podataka u tabele(INSERT), citanje podataka(SELECT), upit za azuriranje(UPDATE), upit za uklanjanje slogova(DELETE), izmena strukture tabele(ALTER TABLE, DROP)

UNIQUE - jedinstvena vrednost

EXISTS - postoji

DISTINCT - da prikaze bez duplikata

SUBSTRING(Student, 1,5) - prikaz prvih 5 karaktera stringa

% - poklapanje 0 ili vise karaktera

_ - poklapanje tacno 1 karaktera

Select * from Student where studentIme LIKE 'a%' AND studentPrezime 'Markovi_'

SELECT kod koga prebrojavamo nesto nece prikazati podatke gde je vrednost NULL.

```
CREATE DATABASE prodaja;
```

```
USE prodaja;
```

```
CREATE TABLE prodavci
```

```
(
    id int NOT NULL AUTO_INCREMENT PRIMARY KEY,
    ime varchar(255) NOT NULL,
    prezime varchar(255) NOT NULL,
    datum_rodjenja datetime NOT NULL,
    adresa varchar(255) NOT NULL,
    broj int NOT NULL,
    email varchar(255) NOT NULL
);
```

```
CREATE TABLE prodaja
```

```
(
    id int NOT NULL AUTO_INCREMENT PRIMARY KEY,
    datum datetime NOT NULL,
    artikl varchar(255) NOT NULL,
    cena double NOT NULL,
    kolicina double NOT NULL,
    prodavac_id int NOT NULL,
    FOREIGN KEY (prodavac_id) REFERENCES prodavci(id)
)
```

```
INSERT INTO prodavci
```

```
(ime, prezime, datum_rodjenja, email)
```

```
VALUES
```

```
("Petar", "Jovanović", "1979-06-15", "petar_jovanovic@str.co.rs"),
("Ivan", "Kovačević", "1993-04-27", "ivan_kovacevic@str.co.rs"),
("Jelena", "Marković", "1984-09-09", "jelena_markovic@str.co.rs"),
("Ana", "Stanković", "1991-08-17", "ana_stankovic@str.co.rs");
```

```
INSERT INTO prodaja
```

```
(datum, artikl, cena, kolicina, prodavac_id)
```

```
VALUES
```

```
("2020-06-15", "Sveska", 80.00, 4, 1),
("2020-06-15", "Olovka", 56.57, 2, 1),
("2020-06-15", "Gumica", 34.96, 2, 2),
("2020-06-15", "Selotejp", 134.57, 1, 2);
```

```
SELECT * FROM prodaja
```

```
SELECT
    datum,
    artikl,
    cena AS 'Cena po artiklu',
    kolicina AS 'Količina',
    cena * kolicina AS 'Ukupna cena po artiklu',
FROM
    prodaja
```

```
SELECT
    datum,
    artikl,
    cena AS 'Cena po artiklu',
    kolicina AS 'Količina',
    cena * kolicina AS 'Ukupna cena po artiklu'
FROM
    prodaja
WHERE
    cena * kolicina > 300
ORDER BY
    cena * kolicina DESC
```

```
UPDATE
    prodavci
SET
    email='petar_m_jovanovic@str.co.rs'
WHERE
    id=1
```

```
DELETE FROM
    prodavci
WHERE
    id=3
```

```
ALTER TABLE
    prodavci
ADD
    kucna_adresa varchar(255) NOT NULL
AFTER
    email;
```

```
ALTER TABLE
    prodavci
DROP
    kucna_adresa
```

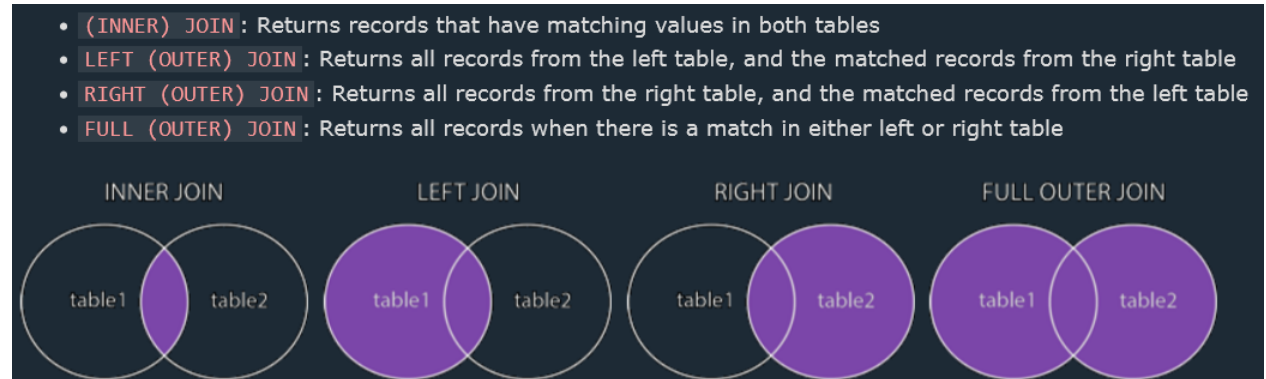
Ugnjezdeni upiti:

```
SELECT
    ime, prezime, email
FROM
    prodavci
WHERE
    prodavci.id = (SELECT prodaja.prodavac_id FROM prodaja WHERE prodaja.id=3)
```

Pridruživanje podataka

Preko komande JOIN moguće je spojiti i prikazati podatke dve ili više tabela, pri čemu se kao kriterijum za spajanje koriste podaci koji se pojavljuju u svim tabelama koje učestvuju u spajanju.

LEFT - Skup A, presek sa skupom B; INNER - presek ; FULL OUTER - unija skupa A i B



```
SELECT
    prodaja.datum,
    prodaja.artikl,
    prodaja.cena,
    prodaja.kolicina,
    prodavci.ime,
    prodavci.prezime
FROM
    prodavci INNER JOIN prodaja
ON
    prodavci.id = prodaja.prodavac_id
```

INNER JOIN

Ako za spajanje pozivamo komandu `INNER JOIN`, upit će izdvojiti samo one slogove iz tabele `studenti` koji su se pojavili u tabeli `upis` i samo one slogove iz tabele `kursevi` koji su se pojavili u tabeli `upis`.

```
SELECT
    CONCAT(studenti.ime, " ", studenti.prezime) AS "Student",
    studenti.br_indeksa,
    kursevi.naziv AS "Kurs"
FROM
    (studenti INNER JOIN upis ON studenti.id=upis.student_id)
    INNER JOIN kursevi ON kursevi.id=upis.kurs_id
ORDER BY
    studenti.id, kursevi.id
```

LEFT JOIN

Ako za spajanje pozivamo komandu `LEFT JOIN`, upit će izdvojiti sve slogove iz tabele `studenti` i samo one slogove iz tabele `kursevi` koji su se pojavili u tabeli `upis`.

```
SELECT
    CONCAT(studenti.ime, " ", studenti.prezime) AS "Student",
    studenti.br_indeksa,
    kursevi.naziv AS "Kurs"
FROM
    (studenti LEFT JOIN upis ON studenti.id=upis.student_id)
    LEFT JOIN kursevi ON kursevi.id=upis.kurs_id
ORDER BY
    studenti.id, kursevi.id
```

OUTER JOIN

Ako za spajanje koristimo komandu `OUTER JOIN`, upit će prikazati sve studente i sve kurseve.

```
SELECT
    CONCAT(studenti.ime, " ", studenti.prezime) AS "Student",
    studenti.br_indeksa,
    kursevi.naziv AS "Kurs"
FROM
    (studenti FULL OUTER JOIN upis ON studenti.id=upis.student_id)
    FULL OUTER JOIN kursevi ON kursevi.id=upis.kurs_id
ORDER BY
    studenti.id, kursevi.id
```

Za sortiranje podataka koristimo `ORDER BY` prodaja.datum. Sortira ih po datumu. Koristimo alias za cenu, a podatke o prodavcu spajamo preko funkcije `CONCAT` **`CONCAT(ime,prezime) AS "Prodavac"` prikazuje rezultat kao novu kolonu Prodavac, a medju rezultatima se nalaze pravi rezultati npr. Pera Peric**

```
SELECT
    prodaja.artikl,
    CONCAT(prodaja.cena * prodaja.kolicina, "din.")
    AS "Cena po artiklu (bez PDV-a)",
    CONCAT(prodavci.ime, " ", prodavci.prezime)
    AS "Prodavac"
FROM
    prodavci INNER JOIN prodaja
ON
    prodavci.id = prodaja.prodavac_id
ORDER BY
    prodaja.datum
```


artikl	Cena po artiklu (bez PDV-a)	Prodavac
Sveska	320	Petar Jovanović
Olovka	113.14	Petar Jovanović
Gumica	69.92	Ivan Kovačević

Profesor ima: id, ime. Predmet ima id, id_profesora i naziv_predmeta. Nadji predmet koji predaje profesor sa imenom Marko.

(radi se inner join preko id_profesora = id za tabelu profesor)

```
SELECT naziv_predmeta FROM predmet INNER JOIN profesor ON predmet.id_profesora=profesor.id
WHERE ime='Marko'
```

--U WHERE uslovu mozes pristupiti bilo kojoj koloni obe tabele

Wildcard(dzoker) - su karakteri "_" i "%".

Koristi se uz WHERE kada tacno podudaranje nije moguće u naredbi SELECT

WHERE limitira redove koji se vracaju

Brisanje tabele iz BP - DROP TABLE CUSTOMER;

Imamo dve tabele, "Customers" i "Orders". Tabela "Orders" ima spoljni ključ "customer_id" koji referencira ključ "id" u tabeli "Customers". Kada se promeni vrednost polja "id" u tabeli "Customers", želimo da se automatski ažuriraju sve povezane redove u tabeli "Orders".

```
CREATE TABLE Customers (
  id INT PRIMARY KEY,
  name VARCHAR(50) );
```

```
CREATE TABLE Orders (
  order_id INT PRIMARY KEY,
  order_date DATE,
  customer_id INT,
  FOREIGN KEY (customer_id) REFERENCES Customers(id) ON UPDATE CASCADE
);
```

U ovom primeru, kada se vrednost polja "id" u tabeli "Customers" promeni, svi povezani redovi u tabeli "Orders" će takođe biti automatski ažurirani da odražavaju novu vrednost.

Na taj način, upotrebom ON UPDATE CASCADE klauzule, obezbeđujemo da se veze između tabela održavaju ispravno i da se promene propagiraju na povezane redove kako bi se očuvao **integritet baze podataka**.

DDL je skup komandi u SQL-u koji se koristi za definisanje, modifikovanje i brisanje strukturalnih elemenata baze podataka. SQL data definition commands make up a DDL (Data Definition Language). Komande su: create, alter, drop, truncate(brisanje svih redova iz tabele), rename(imena objekta).

Kada pravis tabelu trebas da vodi racuna o tipovima podataka, primarnim kljucevima i default vrednostima

Provera duzine **LEN(naziv)<3**

CREATE INDEX ID;

Index u SQL-u je struktura koja se koristi za poboljšanje performansi upita nad bazom podataka. On omogućava brže pretraživanje i filtriranje podataka tako što organizuje podatke u određenom redosledu i pruža efikasan pristup podacima.

Indexi se kreiraju na jednoj ili više kolona u tabeli. Kada se izvršava upit koji uključuje kolone na kojima su kreirani indexi, baza podataka koristi indexe kako bi pronašla relevantne podatke mnogo brže nego kroz potpuni sken tabele.

Postoje različite vrste indexa u SQL-u, kao što su klasterisani index, nefragmentirani index, jedinstveni index, kompozitni index itd.

HAVING se koristi kao WHERE, ali za grupe, a ne za redove. **Moze da se koristi samo uz GROUP BY**

Benefit standardnog relacionog jezika je u tome sto smanjuje troskove obuke

Kada se 3 ili vise AND ili OR uslova kombinuju lakse je koristiti IN ili NOT IN

Ekvivalentno:

```
SELECT NAME FROM CUSTOMER WHERE STATE = 'VA';
```

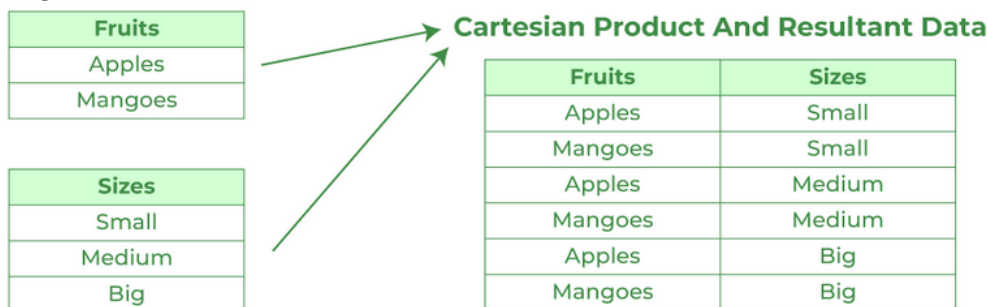
```
SELECT NAME FROM CUSTOMER WHERE STATE IN ('VA');
```

DML(Data Manipulation Language) - komande za manipulaciju podacima u BP(select, insert, update, delete)

Podupit u SELECT naredbi ima poseban oblik koji se ne moze duplirati spajanjem

Correlated subquery - upucuje na kolonu u spoljasnjem upitu i izvrsava podupit jednom za svaki red u spoljasnjem, dok **uncorrelated subquery** prvo izvrsava podupit i prosledjuje vrednost spoljasnjem upitu

Cartesian join (cross join) - kreira se kombinacija svakog reda iz jedne tabele sa svakim redom iz druge tabele.



Equi-join je vrsta operacije spajanja tabela u SQL-u koja se vrši na osnovu uslova jednakosti između kolona iz različitih tabela. Ova vrsta join-a omogućava spajanje redova iz tabela na osnovu vrednosti kolona koje se podudaraju.

```
SELECT *  
FROM tabela1  
JOIN tabela2 ON tabela1.kolona = tabela2.kolona;
```

Natural join je vrsta operacije spajanja tabela u SQL-u koja se automatski vrši na osnovu zajedničkih kolona sa istim imenom između tabela. Ova vrsta join-a eliminiše potrebu za eksplicitnim navođenjem kolona za spajanje, već se kolone automatski uparuju na osnovu imena.

```
SELECT *  
FROM tabela1  
NATURAL JOIN tabela2;
```

Procedure su skupovi SQL instrukcija koje se mogu izvršavati više puta. Koriste se za grupisanje logički povezanih SQL instrukcija i omogućavaju izvršavanje kompleksnih operacija. Mogu prihvatiti parametre, izvršavati upite, azurirati podatke, obradjivati logiku i vratiti rezultate. Kada se procedura kreira, ona postaje objekat u bazi podataka koji se može izvršavati po potrebi. To je program koji izvršava neku određenu radnju nad podacima BP i koji se čuvaju u BP. Mana procedure je povećanje mreznog saobraćaja.

```
CREATE PROCEDURE GetEmployees  
AS  
BEGIN  
    SELECT * FROM Employees;  
END;
```

Procedure su objekti koji se koriste za grupisanje logičkih operacija koje se izvršavaju na BP. koriste se za organizovanje ponovljivog koda, izvršavanje kompleksnih operacija, omogućavanje transakcija i obezbeđivanje složenih sigurnosnih mehanizama.

```
CREATE PROCEDURE GetEmployeeDetails  
    @EmployeeId INT  
AS  
BEGIN  
    SELECT * FROM Employees WHERE EmployeeId = @EmployeeId;  
END
```

Ako zelimo da koristimo SQL procedure u kombinaciji sa C#, mozemo koristiti ADO.NET biblioteku za komunikaciju sa BP.

```
using (SqlConnection connection = new SqlConnection(connectionString))
{
    connection.Open();

    using (SqlCommand command = new SqlCommand("GetEmployeeDetails", connection))
    {
        command.CommandType = CommandType.StoredProcedure;
        command.Parameters.AddWithValue("@EmployeeId", 123);

        using (SqlDataReader reader = command.ExecuteReader())
        {
            while (reader.Read())
            {
                // Obrada rezultata
            }
        }
    }
}
```

Trigeri su posebni objekti koji se aktiviraju ili "okidaju" kada se odredjeni dogadjaj desi u bazi podataka. To mogu biti dogadjaji kao sto su umetanje, azuriranje ili brisanje podataka u odredjenoj tabeli. Kada se dogadjaj aktivita trigger se pokrece i izvrsava zadate SQL instrukcije ili logiku. To je sacuvani program koji je vezan za tabelu ili view.

```
CREATE TRIGGER UpdateSalary
ON Employees
AFTER UPDATE
AS
BEGIN
    -- Logika za ažuriranje plate nakon promene podataka
END;
```

Triger se aktivira nakon izvrsavanja UPDATE operacije nad tabelom Employees.

View je virtuelna tabela kojoj se moze pristupiti pomocu SQL komandi. Moze sakriti kolone, redove i komplikovanu SQL sintaksu. Krijemo poverljive podatke i prikazujemo samo rezultat.

Alternativni ključ u SQL-u se odnosi na jedan ili više atributa u tabeli koji su jedinstveni za svaki red, ali nisu primarni ključ. Drugim rečima, alternativni ključ je kandidat za primarni ključ, ali nije odabran kao primarni ključ za tabelu.

ALTER menja strukturu tabele(dodaje novu kolonu, menjamo definiciju postojece kolone)

```
ALTER TABLE Customers
ADD PhoneNumber VARCHAR(20);
ALTER TABLE Customers
ALTER COLUMN Email VARCHAR(255);
```

CHECK se koristi za ogranicavanje vrednosti kolona(koje se mogu uneti ili izmeniti).

```
CREATE TABLE Zaposleni (
    IDZaposlenog INT PRIMARY KEY,
    Godine INT CHECK (Godine >= 18),
```

SUBSTRING(Naziv, 1, 3) vraća **prva tri karaktera**, dok SUBSTRING(Naziv, LEN(Naziv) - 2, 3) vraća poslednja tri karaktera stringa.

SUBSTRING - koristi se za izdvajanje dela stringa na osnovu određenih parametara. Prihvata tri argumenta nad kojim se primenjuje, pocetni indeks i opciono broj karaktera koje zelite izdvojiti. SUBSTRING('Ovo je primer', 5, 2) izdvaja deo stringa počevši od 5. karaktera i uzima 2 karaktera, što rezultira stringom 'je'.

TRIM - uklanjanje vodećih ili pratećih razmaka(ili drugih znakova) sa stringa.

LTRIM - uklanja vodeće(leve) razmake sa stringa

RTRIM - uklanja prateće(desne) razmake sa tringa

Ako je primarni ključ u jednoj tabeli 001 002 003 A sekundarni u drugoj 1 2 3 ... Može se odraditi JOIN na ovaj način. Pretvara 001 u 1, a "1" u celobrojnu vrednost.

```
SELECT *
FROM tabela1 t1
INNER JOIN tabela2 t2 ON CAST(t1.primarni_kljuc AS INT) = CAST(t2.sekundarni_kljuc AS INT)
```

```
SELECT *
FROM tabela1
INNER JOIN tabela2 ON tabela1.NAZIV_ID = CAST(tabela2.NAZIV_ID_REF AS CHAR(3))
```

Na primer, ako vrednost kolone "Ime" u tabeli "t1" za određeni red bude "JohnDoe", CAST(t1.Ime AS CHAR(3)) će rezultirati vrednost "Joh" jer će biti zadržana samo prvih 3 karaktera.

CAST - sece visak ili dopunjava razmacima preostala mesta

SQL upiti su brzi od radnje "Code First" pristupa u .NET Core aplikacijama zbog nekoliko faktora:

1. Optimizacija baze podataka: Kada koristite SQL upite direktno, imate veću kontrolu nad optimizacijom upita. Možete napisati efikasne upite koji koriste indekse, optimizovane planove izvršavanja ili specifične tehnike za poboljšanje performansi. U "Code First" pristupu, ORM (Object-Relational Mapping) alati generišu upite na osnovu vašeg koda, što može rezultirati manjom optimizacijom upita.
2. Migracije i složenost modela: "Code First" pristup uključuje migracije koje se primenjuju na bazu podataka kako bi se održao model podudaran sa kodom. Ove migracije mogu biti složene i uključivati dodavanje, brisanje ili izmenu tabela, indeksa, veza i drugih elemenata baze podataka. Ove migracije mogu usporiti izvršavanje "Code First" pristupa u odnosu na direktno izvršavanje optimizovanih SQL upita.
3. Nepotrebni zahtevi ka bazi podataka: U "Code First" pristupu, ORM alati često izvršavaju dodatne upite ka bazi podataka kako bi dohvatili podatke o strukturi tabele, veza između entiteta i slično. Ovi dodatni zahtevi mogu uticati na performanse aplikacije.
4. Caching: Kada koristite SQL upite direktno, imate veću kontrolu nad keširanjem rezultata upita. Možete implementirati keširanje rezultata upita na nivou baze podataka ili na aplikativnom nivou kako biste ubrzali pristup podacima. U "Code First" pristupu, ORM alati obično imaju svoje mehanizme keširanja, ali oni možda neće biti prilagođeni specifičnim potrebama vaše aplikacije.

Napomena: performanse znacajno variraju u zavisnosti od implementacije, kolicine opdataka, konfiguracije baze podataka i drugih faktora. Code First može biti brzi od direktnih SQL upita.

Funkcije u SQL-u su objekti koji se koriste za izvršavanje određenih operacija nad podacima u bazi. One uzimaju ulazne parametre, izvršavaju određene radnje i vraćaju rezultat. Funkcije se koriste za izračunavanje vrednosti, transformaciju podataka, filtriranje rezultata upita i obavljanje različitih manipulacija nad podacima.

Primer f-je(nema ulazne parametre) koja vraća ukupan broj zaposlenih iz tabele Employees:

```
CREATE FUNCTION GetEmployeeCount()
RETURNS INT
AS
BEGIN
    DECLARE @Count INT;
    SELECT @Count = COUNT(*) FROM Employees;
    RETURN @Count;
END
```

Za korišćenje SQL funkcija u kombinaciji sa C# koristi se ADO.NET.

```
using (SqlConnection connection = new SqlConnection(connectionString))
{
    connection.Open();

    using (SqlCommand command = new SqlCommand("SELECT dbo.GetEmployeeCount()", connection))
    {
        int employeeCount = (int)command.ExecuteScalar();
        Console.WriteLine("Total employees: " + employeeCount);
    }
}
```

SELECT Ime, Prezime, COUNT(obaveze)

FROM Radnik

GROUP BY Ime

--Upit neće proći jer kada se koristi GROUP BY mora da se navede sve što se selektuje(Ime, Prezime)

Kaskadno brisanje(CASCADE) - kada obrise nešto u jednoj tabeli briše se i u drugoj

Indeks u SQL-u počinje sa 1. Ne može početi sa 0.

NoSQL(Not only SQL) se koristi za projekte koji su bazirani na mikroservisima. NoSQL baze su fleksibilnije. Ne zahteva rigidnu strukturu, ne ispadas zbog odluke vezane za dizajn. Nema veze da li si ranije uneo numeric, varchar... Usput mozes da adaptiras model. Problem je sto bez rigidnih pravila i strukture podaci mogu da budu nedosledni i mozes završiti sa nepovezanim podacima bez referenci ukoliko nisi pazljiv. Primeri NoSQL baza su: Apache Cassandra(bazirana na kolonama), MongoDB(bazirana na dokumentima), Neo4j(bazirana na grafovima). MongoDB se bazira na dokumente pohranjene u JSON formatu. Neo4j podatke pohranjuje kao cvorove i veze izmedju njih. Cvorovi su entiteti, a veze odnosi. Upiti se izvorsavaju pomocu trazilica grafa. Pogodan je za softver za analizu i pretrazivanje, sisteme za preporucivanje, drustvene mreze. Apache Cassandra je baza za Big Data, a za ostalo nije dobar izbor jer je potrebno posedovati vrhunski hardver. Postavljena je grupisano, dok se podaci distribuiraju preko cvorova u kruznom maniru(ring). **Big Data** su podaci koji su toliko obimni i slozeni da se tradicionalni alati i tehnike ne mogu iskoristiti za njihovu analizu i obradu. Tu spadaju podaci koji su previse veliki da bi se obradili na jednom racunaru ili bazi podataka, **podaci koji se brzo menjaju i azuriraju**, podaci koji su u razlicitim formatima i izvorima i podaci koji sadrze **velike kolicine šuma(nebitnih podataka)** u odnosu na korisne informacije. Primer: podaci prikupljeni iz drustvenih mreza, medicinske datoteke, finansijske transakcije... Neki kazu da su to podaci veci od 5 terabajta, a drugi da se zbog njih prelaze kapaciteti tradicionalni baza podataka i koriste posebni alati za obradu.

WEB:

Tagovi: prosti - ne sadrže zatvarajući tag, složeni - sadrže i otvarajući i zatvarajući.

Osnovni tagovi su html, head - informacije o dokumentu, body - sve što vidimo, sadržaj.

Jos neki tagovi: <h1>...<h6>, <p>,
, <hr/>, <!-- -->

Tagovi za formatiranje teksta: , <i>, <u>, <sub>, <sup>

WEB verzije

WEB/1.0, uglavnom statičke stranice, korisnici su puki citaoци(wikipedia)

WEB/2.0, uvođenje dinamičkog sadržaja, korisnici sami kreiraju sadržaj, sadržaj je personalizovan prema korisniku

WEB/3.0, AI interpretiranje, rankiranje i preporuka (SEO)

WEB/4.0/5.0, integracija sa AR(Augmented Reality) i VR(Virtual Reality)

Stilovi za CSS se ugrađuju:

inline style - unutar samih HTML elemenata(atribut style)

internal style sheet - upotrebom taga <style> unutar dokumenta(unutar <head> taga)

external style sheet - kreiranjem spoljašnje datoteke stilova(.css datoteka)

Prioritet je:

1. unutar HTML elementa -- <h1 style="color:blue">Tekst</h1>
2. <style> tag -- selektor {svojstvo: vrednost; ...}
3. spoljašnja datoteka stilova -- <link rel="stylesheet" type="text/css" href="stilovi.css">
4. kako citac prikazuje

Struktura HTTP **zahteva**:

Pocinje redom: **METOD/putanja HTTP/verzija**

METHOD je: GET, POST...

dodatni redovi sadrže attribute oblika -> ime: vrednost

prazan red na kraju - Ako je POST zahtev posle praznog reda idu parametri forme

Napisati prvi i poslednji red HTTP zahteva (HTTP verzija 1.1) koji se dobija kada se klikne na dugme "Posalji" (a nista se ne unese u polja) u sledecoj formi:

```
<form action=http://localhost/Proba method="POST">
```

```
<input type="text" name="polje1" value="tekst1">
```

```
<input type="text" name="polje2" value="tekst2">
```

```
<input type="submit">
```

```
</form>
```

```
POST /FormServlet HTTP/1.1
```

```
...
```

```
Content length: 27
```

```
\r\n //prazan red
```

```
polje1=tekst1&polje2=tekst2
```

Odgovor pocinje redom: **HTTP/verzija kod tekstualni_opis**

Postavljanje i pracenje sesije u ASP.NET --- ASP.NET definise id koji se cuva u cookie-ju na

klijentu(to je njegov ugradjen mehanizam za pracenje sesije):

```
MyUser u = (MyUser)Session["User"];
```



```
if (u == null)
{
    u = new MyUser();
    Session["User"] = u;
}
```

Session inactivity – obično 20 minuta, podešava se u web.config fajlu ili u kodu
Session.Timeout = 60; -- broj min. za koji sesija može biti nekorisćena pre nego što je napuštena

Href funkcija - "raspakuje" prosledjen URL na pun naziv. Puna putanja je na primer
/folder/folder1/test.cshtml

Request - objekat klase Request povezan sa zahtevom i tipa je HttpRequestBase. Omogućava pristup parametrima prosledjenim sa zahtevom (putem operatora[]), utvrđivanje tipa zahteva (GET, POST, HEAD... - svojstvo HttpMethod), i HTTP zaglavlju (cookies, User-Agent)

Response - objekat klase Response povezan sa odgovorom klijentu i tipa je HttpResponseBase. Dozvoljeno postavljanje HTTP statusnih kodova i zaglavlja odgovora (response headers). Redirekcija na drugu stranicu prilikom ocitanja tekuće.

Session objekat je povezan sa zahtevom i tipa je HttpSessionStateBase. Sadrži asocijativnu mapu objekata koji će pratiti korisnike, na osnovu kukija.

Cache je promenljiva koja se koristi za smestanje globalnih vrednosti na nivou aplikacije koje imaju privremeni karakter. Prilikom dodavanja entiteta u Cache omogućeno je postavljanje životnog veka za entitet. Metode koje to omogućuju su add i insert prilikom čega je moguće postaviti apsolutni životni vek entiteta (npr. 60 minuta) ili životni vek koji se vezuje za nekoriscenje entiteta (npr. 10 minuta).

Application je promenljiva koja se koristi za smestanje globalnih vrednosti na nivou aplikacije koje se mogu inicijalizovati u Global.asax.cs datoteci. Objekti postavljeni u Application nestaju samo ako se web aplikacija ugasi. Preporučuje se za podesavanje globalnih promenljivih koje sadrže parametre podesavanja aplikacije. Ne treba je koristiti za skladištenje kolekcija koje sadrže veliki broj objekata.

Page sadrži asocijativnu mapu objekata zakacenih i vidljivih samo na tekućoj stranici. Mora se prvo definisati da bi se koristio.

WebApi - Web aplikacija je JavaScript aplikacija, koja se izvršava na klijentu (browser). Web server služi da primi i pošalje podatke od/ka web aplikaciji. Metode C# klase na serveru se "gadjaju" iz klijenta preko URL-a

ASP.NET API (isto potpuno) - omogućava kreiranje i izlaganje web servisa i RESTful API-ja za komunikaciju između različitih aplikacija.

Server šalje **cookie** klijentu u okviru http response. Klijent čuva primljeni cookie i šalje ga uz svaki http request.

GET zahteva resurs od servera. Podaci iz forme se nalaze u url-u
HEAD samo odgovor, bez resursa

POST podaci iz forme se nalaze posle zaglavlja zahteva

GET /FormServlet?tekst=abcd HTTP/1.1

...

\r\n //prazan red

POST/FormServlet HTTP/1.1

...

Content-length: 10

\r\n //prazan red

tekst=abcd

Ako su **cookies** isključeni u browseru da bi se sesija pratila koristi se **URL Rewriting mehanizam**

Application atribut je parametar forme, a ako stoji multipart to znaci da je datoteka

Primer HTTP odgovora koji redirektuje klijenta na pera.jsp

HTTP/1.1 302 Object moved

Location:pera.jsp

U JSP tehnologiji se sesija prati pomocu cookies. U zahtevu Cookie: ime=vrednost,
u odgovoru Set-Cookie: ime=vrednost

Permanent connection u HTTP protokolu verzije 1.1 je kada klijent od servera zatrazi da se konekcija ne zatvara odmah po slanju odgovora, vec da se zadrzi jos malo. U zaglavlju http zahteva i odgovora se stavi atribut **Connection = Keep-Alive**

Minimalan broj atributa u HTTP zahtevu je 0.

Cookie se salje preko http response.

Razor je serverska strana. Omogucuje da ne moramo eksplicitno da postavimo layout za svaku stranicu vec moze da se definise layout logika za sve stranice, sto nam omogucava da cshtml fajlovi budu citljiviji i da se odrzavaju lakse. Potrebno je u okviru views foldera dodati datoteku _ViewStart.cshtml i u njoj definisati zeljeni layout, a uklonimo definisanje layout-a iz ostalih cshtml stranica. Razor(MVC 5 standard) - cshtml (svaki dinamicki generisan kod zapocinje oznakom @)

Thread Pool - efikasnije od manualnog kreiranja i rada "obicnih niti". Nit se uzima iz bazena, vraca u bazen zavrsetkom rada.

Thread - putanja izvršavanja u okviru procesa operativnog sistema. Jedan proces moze imati vise niti, a **svaki thread ima svoj niz instrukcija** koje se izvršavaju nezavisno od drugih niti unutar procesa. To znaci da se istovremeno mogu izvršavati razlicite niti unutar jednog procesa, sto omogucava da se poslovi izvršavaju paralelno. Posедује svoju memoriju, treba puno vremena za njegovo kreiranje i ubijanje, mnogo je jednostavnije uzeti iz thread pool. Koristimo da bi izvršili vise asinhronih operacija

u paraleli međutim niko ne garantuje da su threadovi dobro raspoređeni nad našim procesorom/jezgrima

Task - jedna specifična operaciju ili zaduženje koje se mora izvršiti u okviru nekog procesa ili aplikacije. Svaki zadatak može biti sastavljen od više niti u okviru procesa. To su opirnije definisani skupovi operacija koje se moraju izvršiti. Na primer: sortiranje podataka, azuriranje baze podataka ili slanje e-poste. Može se predstaviti kao asinhrona operacija (ono što thread treba da izvrši). Raspoređuje se ravnomerno po "jezgrima" čime maksimalno optimizujemo brzinu izvršavanja programa.

Proces - jedna ili više povezanih operacija koje se izvršavaju u okviru operativnog sistema. Svaki proces ima svoje resurse (memorija, datoteke) i okolinu u kojoj se izvršava. Procesi se koriste za organizovanje i upravljanje aplikacijama i operacijama u računarskom sistemu. Svaki proces ima svoj identifikator (ID) koji se koristi za praćenje i upravljanje procesom

Semafori održavaju skup dozvola. Ako nema više dozvola, nit koja je pozvana se blokira. Release može da deblokira pozivajuću nit.

7. ASP

Osnovno:

- Tehnologija za pravljenje web sajtova i web aplikacija.
- Active Server Pages
- Web aplikacije nisu C# programi koji počinju od **public static void Main()** ili **public static init Main()**.
- Web aplikacije se sastoje iz web servera, HTML-a i server side koda (C#, VB,...) koji omogućuje dinamičko kreiranje web sadržaja.
- ASP poseduje 2 tehnologije za generisanje dinamičkog sadržaja u okviru HTML stranica:
 - ASPX (starije verzije MVC, WebForms)
 - Razor (MVC 5 standard)** - **cshtml** stranica sadrži HTML obogaćen C# kodom .Svaki dinamički generisan kod započinje sa znakom @

ASP.NET struktura projekta:

- **Models** folder
 - sadrži C# klase koje ćemo koristiti
- **Scripts** folder
 - sadrži JavaScript datoteke
- **App_Data** folder
 - sadrži podatke
- Datoteke:
 - **Web.config** datoteka
 - podešavanja web projekta (kao web.xml za Java Servlet)
 - **ApplicationInsights.config** datoteka
 - telemetrija, odn. merenje performansi (web app analytics)
 - **packages.config** datoteka
 - konfiguracija instaliranih paketa
 - bazira se na NuGet package manager-u

jQuery je JavaScript biblioteka. Osmisljena da pojednostavi programiranje: jednostavan pristup elementima, izmena izgleda web stranice, sadržaja, animacije, interakcija sa korisnikom

AJAX - asynchronous JavaScript and Xml - tehnika za kreiranje brzih dinamičkih web stranica. Umesto da pristupamo serveru i osvežavamo citavu stranicu, pristupimo serveru samo radi osvežavanja delova stranice. Omogućava i da JavaScript ne mora da čeka na odgovor od servera već može da izvršava druge skriptove dok čeka odgovor. Kombinacija već postojećih tehnologija: JavaScript/Dom, XMLHttpRequest, CSS, XML (često JSON)

MVC - bazira se na sablonu (Model View Controller) - cilj sablona je da razdvoji prikaz informacija od obrade korisnickih interakcija. M - model koji koristimo u radu aplikacije, V - prikaz UI, C - obrada ulaza

Model - folder Models, opisuje biznis logiku, a sadrzi: sve potrebne podatke koji se koriste, moze da sadrzi dobijene podatke iz baze podataka, podaci su nezavisni od prezentacije.

View - vizualizuje podatke iz modela dobijene od kontrolera. Nije svestan porekla podataka (za to je zasluzan model).

Controller - obezbedjuje vezu izmedju korisnika i podataka, upravlja prezentacionim slojem i slojem podataka (modelom), upravlja tokom izvršenja aplikacije, svi zahtevi (akcije korisnika) idu preko kontrolera, a on donosi odluku koji segment aplikacije ce biti prikazan.

JSON - JavaScript Object Notation - jednostavan format za razmenu podataka predstavljenih pomocu teksta. Nezavistan od konkretnih tehnologija koji se koristi. Lako se razume. Sintakticki identican kodu za kreiranje objekata u JS-u.

```
{"name":"John","today":"2023-03-30T12:32:15.675Z","city":"New York"}
```

```
<script>
const obj = {name: "John", today: new Date(), city: "New York"};
const myJSON = JSON.stringify(obj);
document.getElementById("demo").innerHTML = myJSON;
</script>
```

```
<script>
const txt = '{"name":"John", "age":30, "city":"New York"}'
const obj = JSON.parse(txt);
document.getElementById("demo").innerHTML = obj.name + ", " + obj.age;
</script>
```

John, 30

Kako se prati sesija u ASP.NET okruženju? Dati primer u kodu kako se ona postavlja i proverava.

OVO JE ZA PRACENJE SESIJE uz pomoc HTTP request-a i HTTP response-a

Koristi se cookie mehanizam. Server šalje cookie klijentu u okviru http response, klijent čuva primljeni cookie i šalje ga uz svaki http request.

GET / HTTP/1.1

Host: localhost

User-Agent: Mozilla/5.0

Connection: Keep-Alive

HTTP/1.0 200 OK

Date: Sat, 18 July 2020

Status: 200

Set-Cookie: ASPSESSIONIDGGQQAQAEK=JKKPPFKCMFNSKJSDJ

<html> <head></head> <body></body> </html>

GET /Test HTTP/1.1

Host: localhost

User-Agent: Mozilla/5.0

Connection: Keep-Alive

Cookie: ASPSESSIONIDGGQQAQAEK=JKKPPFKCMFNSKJSDJ

Za pracenje sesije u ASP.NET okruzenju

Praćenje sesije se svodi na kreiranje objekata koji se vezuju na sesiju.

Kada korisnik pozove neki resurs na serveru (stranica, kontroler) u okviru njega može da se koristi objekat vezan za sesiju.

```
User u = (User)Session[„user“];
```

```
if ( u == null) {
```

```
    u = new User();
```

```
    Session[„user“] = u;
```

```
}
```

Session inactivity – podešava se u Web.config fajlu ili u kodu Session.Timeout = 60;

Prenos podataka iz kontrolera u pogled:

- ViewData
 - o C: ViewData[“Name”] = “Aleksandar”;
 - o V: <p>@ViewData[“Name”]</p>
- ViewBag
 - o C: ViewBag.Name = “Aleksandar”;
 - o V: <p>@ViewBag.Name</p>
- TempData
 - o C: TempData[“Name”] = “Aleksandar”;
 - o V: <p>@TempData[“Name”]</p>
 - o Koristi se kod redirekcije između više akcija, kako se podaci ne bi izgubili
 - o return RedirectToAction(“Action2”);
- Model
 - o C: return View(user);
 - o V: @model MyApp.Models.User ; <p>@Model.Name</p>

Pretraga DOM stabla i pristup elemenata forme u JavaScript-u

- `document.getElementById('field_id');`
- `document.getElementsByName('name')[whole_number];`
- `document.getElementsByClassName('class_name')[whole_number];`
- `document.getElementsByTagName('tag_name;')[whole_number];`

JSON primer stringa za objekat klase student i adresa

```
{
  "id": 15,
  "ime": "pera",
  "prezime": "peric",
  "adresaStanovanja": {
    "grad": "Novi Sad",
    "ulica": "Fruskogorska",
    "broj": "11A"
  }
}
```

Postman - alat za kontrolu ispravnosti HTTP servera, moguć pregled HTTP zaglavlja kao i generisanja. Takođe je moguće slanje zahteva i analiza header-a i HTTP response-a.

HTML(Hyper Text Markup Language)

Oznacavaju elemente. Sastoje se od otvarajućeg i zatvarajućeg taga(<>), Nisu osetljivi na mala i velika slova.

HTML5

Audio i video sadržaj, crtanje, novi atributi, poboljšana semantika...

Promise - obećanje je proxy(proxy su posrednički objekti koji omogućavaju presretanje objekata) za vrednost koju ne moramo znati u vreme kreiranja. Može biti u 3 stanja:

1. Pending: inicijalno stanje, niti ispunjeno niti odbijeno
2. Fulfilled: operacija je uspesno završena
3. Rejected: operacija nije uspesno završena

settled: fulfilled ili rejected

Promise može biti uspešan ili neuspešan jednom

Callback - funkcija kao parametar funkcije

Promise u JavaScript-u se često koristi za obradu asinkronih operacija. Na primer, kod AJAX poziva, možete koristiti Promise za dobijanje odgovora od servera. U JS ne vraćaju svi HTTP Promise objekte, možemo koristiti callback funkcije umesto Promise objekta.

Web storage je baza podataka u web citacu, čuva parove ključ-vrednost, do 10MB u domenu, podrška u savremenim citacima. Postoje:

1. **Local storage**: trajno čuva podatke
2. **Session storage**: čuva podatke u toku sesije, gubi se po zatvaranju citaca

Event ima sledeće atribute:

key: osobina koja je promenljiva

newValue: nova vrednost

oldValue: stara vrednost

url: pun URL gde se dogadjaj desio

storageArea: localStorage ili sessionStorage objekat

Kako veb citaci učitavaju sadržaj:

Parsira HTML dokument redom. Elementi se dodaju u DOM kako se parsiraju. Kako se dodaju u DOM tako se učitavaju(slike, skriptovi, stilovi...). Citaci mogu citati više resursa istovremeno. Download eksternog JavaScript fajla će blokirati druga dobavljanja jer bi skript mogao da promeni DOM ili preusmeri citac na drugu adresu. Citac neće početi druga paralelna dobavljanja pre nego što se skript preuzme, parsira i izvrši. Skript je zgodno staviti na sam kraj HTML fajla ispred </body>. To garantuje da će citac parsirati ceo fajl i da je DOM spreman pre nego što mu skript pristupi. Ako stavimo script na početak <body> počeo da se izvršava pre nego što je dokument u celosti parsiran(može dovesti do greske).

JavaScript

var - promenljiva vidljiva svuda

let i const - vidljive samo u bloku

let - deklaracija promenljive

const - deklaracija konstante(ne možeš joj dodeliti drugu vrednost)

Kontrola scope-ova, funkcija i memorije:

```
var x = 100;
var f = function(){
  var x = 200;
  return function(){
    x += x;
  }
}
```

```

    return x;
  }
}
var g = f();
console.log(g());
console.log(x+x);
console.log(g());
Resenje:
400
200
800

```

Repna rekurzija nece povecavati stek neograniceno, rekurzija je bezbedna. Ne dolazi do stack overflow. Kada se dodje do kraja ponovo se ide na pocetak. Repni poziv mora da bude poslednji poziv u rekurziji da bi se ona primenila, inace bi bila obicna rekurzija.

Cascade - mogucnost da pozovemo vise metoda prosledjujuci im isti objekat

Koncept modula - smestanje srodnih f-ja u poseban fajl, sprecavanje kolizije imena

Objekat je skup osobina(property). **Osobina** je imenovani kontejner za vrednost

Closure je par koji cine funkcija i okruzenje za razresavanje slobodnih promenljivih

Globalni objekat - window. Svi objekti koji se naprave se nalaze u tom globalu.

HTML dokument je DOM(document object model) stablo. Web citac predstavlja stranicu kao dokument, objekat koji predstavlja koren stavlja dokument. Preko DOM korena mozemo:

1. Promeniti stablo dokumenta
2. Napraviti novi HTML dokument iz pocetka
3. Pristupiti i zameniti cvorove stabla(HTML elemente u DOM stablu)

Dogadjaj je objekat koji se salje kada se dogode akcije na stranici, najcesce kao posledica interakcije sa korisnikom. Npr: click, mouseover, mouseout, keyup, focus, load, blur...

Rutiranje - omogucava navigaciju izmedju razlicitih prikaza, back i forward u web citacu, razmenu parametara izmedju komponenti

Dependency injection(zavisnost injekcija) - **DIP**(Dependency injection pattern)

Mehanizam za dodelu zavisnosti klasama koje ih koriste. Ako UserController komunicira sa UserService servisom ne moze to ciniti direktno vec samo preko interfejsa. Sakrivanjem implementacija iza interfejsa postizemo slabiju povezanost izmedju implementacije i apstrakcije i lakse testiranje metoda. Ciklusi: scoped, transient i singleton.

Injekcija - mehanizam prosledjivanja reference na drugi objekat(npr. prosledjivanje servisa komponenti)

Dependency - objekat treba da koristi drugi objekat, npr. komponenta poziva servis - komponenta je zavisna od servisa. Zavisnost je objekat koji moze da se koristi iz drugog objekta.

DIP - umesto da se objekti stvaraju i konfigurisu u kodu aplikacije, DI ih injektira u objekte koji ih koriste. Cilj je odvojiti logiku kreiranja objekata od logike aplikacije.

Dependancy injection omogućava fleksibilnost i olaksava zamenu implementacije interfejsa bez potrebe za promenom koda u klasi koja koristi taj interfejs. Pravim klasu koja implementira interfejs i kasnije pravim instancu interfejsa koriscenjem te klase. Ako zelim nesto da menjam menjam ConsoleLogger, a ne glavnu klasu.

```

// Primer interfejsa
public interface ILogger
{
    void Log(string message);
}

// Implementacija interfejsa
public class ConsoleLogger : ILogger
{
    public void Log(string message)
    {
        Console.WriteLine(message);
    }
}

// Klasa koja koristi interfejs kroz Dependency Injection
public class MyClass
{
    private readonly ILogger _logger;

    public MyClass(ILogger logger)
    {
        _logger = logger;
    }

    public void DoSomething()
    {
        _logger.Log("Doing something...");
    }
}

// Glavni program
public class Program
{
    public static void Main()
    {
        // Kreiranje instance loggera
        ILogger logger = new ConsoleLogger();

        // Kreiranje instance klase MyClass sa injektovanim loggerom
        MyClass myClass = new MyClass(logger);

        // Pozivanje metode na instanci klase MyClass
        myClass.DoSomething();
    }
}

```

Za Dependency Injection (DI) u C#, prvo trebate definisati interfejs i implementaciju tog interfejsa, a zatim koristiti neki DI kontejner ili ručno konfigurisati injekciju zavisnosti.

1. Definišite interfejs:

Prvo trebate definisati interfejs koji će predstavljati apstrakciju vašeg servisa ili komponente. Na primer:

```
csharp Copy code

public interface IMyService
{
    void DoSomething();
}
```

2. Implementirajte interfejs:

Zatim implementirajte interfejs u odgovarajućoj klasi:

```
csharp Copy code

public class MyService : IMyService
{
    public void DoSomething()
    {
        // Implementacija funkcionalnosti
    }
}
```

3. Konfigurirajte Dependency Injection kontejner (npr. ASP.NET Core Dependency Injection):

U okviru ASP.NET Core, možete koristiti ugrađeni Dependency Injection kontejner. U Startup klasi dodajte konfiguraciju u metodu `ConfigureServices`:

```
csharp Copy code

public void ConfigureServices(IServiceCollection services)
{
    services.AddTransient<IMyService, MyService>();
}
```

Ovaj kod registruje interfejs `IMyService` i njegovu implementaciju `MyService` za Dependency Injection.

4. Koristite injekciju zavisnosti:

Sada možete injektovati interfejs u druge klase, na primer u kontrolere, servise ili komponente. Evo primera u ASP.NET Core kontroleru:

```
csharp Copy code

public class MyController : Controller
{
    private readonly IMyService _myService;

    public MyController(IMyService myService)
    {
        _myService = myService;
    }

    public IActionResult Index()
    {
        _myService.DoSomething();
        return View();
    }
}
```

Prototype - atribut koji pokazuje na osnovne funkcionalnosti, ima ga svaki objekat u js. Ne postoji nasledjivanje nego se to radi preko prototype-a. New kreira objekat i poveze ga sa prototype-om.

JSON web token - novi vid autentifikacije, proverava ima li fajlova vezanih za neki domen npr. gmail.
SPA - single page application

Tokeni - ako zelimo da sacuvamo tokene dobijene od nekih eksternih servisa i koristimo ih za autentifikaciju to mozemo uciniti tako sto ih stavimo u localStorage. Kako bismo se autentifikovali prilikom slanja zahteva serverima moramo u zahtev umetnuti token koji nam je server izdao. Jedan od nacina je da napravimo HTTP presretac koji presretne svaki zahtev i u njegov header stavi token. Drugi nacin za umetanje tokena je **JWT modul** koji presretanje radi automatski

JWT (JSON Web Token) je standard za sigurnu razmenu informacija između stranica putem JSON objekata. JWT se sastoji od tri dijela:

1. Header - sadrži informacije o tome kako bi JWT trebao biti obradjen. Obično sadrži informacije o tome koje se algoritme koristi za potpisivanje i provjeravanje JWT-a.

2. Payload - sadrži informacije koje se prenose kao JWT, npr. korisničko ime ili uloga korisnika. Payload može sadržavati i neke dodatne informacije kao što su vrijeme isteka tokena ili nadzorne liste.

3. Potpis - potpisuje JWT da bi se osiguralo da nijedan podatak nije promijenjen u tranzitu.

JWT se obično koristi za proveru autentičnosti korisnika. Kada korisnik pokuša pristupiti zaštićenoj stranici, server mu šalje JWT koji sadrži informacije o korisniku i koji je potpisan. Klijent zatim šalje JWT natrag na server za svaki zahtjev koji šalje, a server ga provjerava kako bi se osiguralo da je JWT valjan i da korisnik ima prava pristupa traženoj stranici. **JWT se može vratiti u okviru HTTP cookie-ja.** JWT se može koristiti i za razmjenu podataka između aplikacija ili servisa u distribuiranom okruženju. JWT može biti šifriran kako bi se osigurala privatnost informacija.

Razlika između JWT i kolačića je u tome što je JWT nezavisan od servera, dok su kolačići vezani za specifičan server i automatski se uključuju u svaki zahtev upućen tom serveru. JWT se koristi za prenos informacija između strana, dok se kolačići koriste za čuvanje podataka na klijentskoj strani. JWT je samodovoljan i ne zahteva stanje na serveru, dok kolačići zahtevaju čuvanje podataka na serveru u vezi sa sesijom. JWT se koristi za autentifikaciju i autorizaciju, dok se cookie koristi za čuvanje informacija o sesiji.

1. Bez stanja (stateless): JWT je bezstanovni mehanizam, što znači da server ne čuva informacije o korisniku ili sesiji. Sve potrebne informacije su uključene u sam JWT token. Ovo olakšava horizontalno skaliranje i smanjuje opterećenje servera.
2. Skalabilnost: Kako JWT čuva sve potrebne informacije u samom tokenu, nema potrebe za skladištenjem podataka na serveru ili bazi podataka. To omogućava lako skaliranje aplikacije jer nema potrebe za deljenim skladištima podataka.
3. Nezavisnost od sesije: Dok se kod kolačića sesija održava na serveru, JWT omogućava da se svi podaci o sesiji čuvaju na klijentskoj strani. To znači da se server ne mora brinuti o praćenju stanja sesije, već se sve potrebne informacije nalaze u samom JWT tokenu.
4. Fleksibilnost: JWT može sadržavati proizvoljne podatke u obliku JSON objekta. Ovo omogućava prilagodljivost i dodavanje dodatnih informacija u JWT token, kao što su korisnički podaci ili dozvole.
5. Sigurnost: JWT tokeni mogu biti potpisani digitalnim potpisom kako bi se osigurala integritet i autentičnost podataka. Ovo omogućava proveru da li je JWT token promenjen ili manipulisan od strane neovlašćenih strana.

Kolačići, s druge strane, imaju svoje prednosti i specifične primene u web razvoju. Oni mogu biti pogodni za čuvanje manjih količina podataka na klijentskoj strani i često se koriste za održavanje sesija, praćenje korisničkih preferencija i praćenje korisnika.

Važno je napomenuti da JWT i kolačići mogu biti kombinovani u rešenjima za autentifikaciju i autorizaciju, u zavisnosti od potreba aplikacije i bezbednosnih zahteva.

JWT se uglavnom koristi za autentifikaciju korisnika, a provera prava se obavlja u interakciji sa BP.

REST principi - REST je servis koji stimulise upotrebu standarda: HTTP, URI, XML/HTML/JSON/GIF/JPEG

Pojam **resursa**: svaki entitet na webu je resurs. Npr: web sajt, HTML strana, XML fajl, web servis, fizicki uređaj. Svaki resurs je identifikovan svojim URI-jem.

RESTful servisi su stateless: stanje je isključivo na klijentu. Transakcije u nadležnosti klijenta

Collection URI - Element URI

RESTful servisi: klijent server, bez stanja(stateless), racunati na kesiranje(cacheable), jedinstveni interfejs, slojeviti sistem, kod na zahtev

Prednost REST servisa:

1. Raznovrsnost klijenata
2. Prilagodljivost
3. Pristup informacijama koje ne mozemo posedovati
4. REST je odlican za monetizaciju podataka

GET - izlista URI-je i eventualno druge podatke o elementima kolekcije. Dobija reprezentaciju kolekcije u obliku odgovarajuceg MIME tipa. **MIME tip** za HTML dokument je "text/html", a za sliku "image/jpeg". Sastoji se od type/subtype(tip/podtip). To je nacin za identifikaciju vrste datoteke na osnovu njenog sadrzaja.

POST - kreira novi element kolekcije; URL novog elementa se vraća u odgovoru. Tretira dati element kao kolekciju i kreira novi element u njoj.

PUT - zameni celu kolekciju novom. Zameni element novim, a ako ne postoji kreira ga.

DELETE - uklanja celu kolekciju. Uklanja dati element kolekcije.

Web1

Korisnici mogu samo da konzumiraju podatke(dobiju pregled informacija). Nije omogućeno editovanje podataka od strane korisnika. Vlasnik web sajta je ujedno i vlasnik sadržaja. Osnovni primer su statičke HTML stranice koje se dobiju od strane web servera.

Web2

Internet postaje interaktivniji - dvosmerna razmena informacija. Korisniku je omogućeno da menja i učestvuje u izgradnji web sadržaja: socijalne mreže(menjanje svoje slike i profila, objave), blogovi, wikipedia, izrada use case dijagrama. Kompanije skupljaju podatke o nama kako bi na osnovu toga odredili kakav sadržaj da nam prikazu i zadrže nas duže na svojim stranicama. Vlasnik web sajta i dalje ostaje vlasnik sadržaja. Targeted advertising - kompanije na osnovu skupljenih podataka znaju koje oglase da nam prikazu. Interoperabilnost između sajtova postaje važna: logovanje preko google-a, facebook-a, povlačenje drugih sadržaja sa drugih web sajtova. Pojavila se značajnija količina informacija za obradu - SPA.

Web3

Tezi da oslobodi internet centralizacije, on tezi da ga decentralizuje. Decentralizacija podrazumeva uklanjanje centralnih tačaka kontrole i prebacivanje kontrole i moći na korisnika korišćenjem blockchain tehnologije. Vecina u okviru mreže odlučuje koji je podatak tačan. Ideja je da internetska struktura nije pod kontrolom velikih kompanija i centralnih autoriteta, već je distribuirana i daje korisnicima veću kontrolu nad svojim podacima i identitetom.

Blockchain - podaci organizovani u blokove, podaci u narednom bloku zavise od prethodnih podataka, izražena potreba u kriptovalutama. Distribuirano skladište podataka - kopija se nalazi kod svih učesnika u mreži. Implementacija se radi u: GO, RUST. P2P komunikacija. Korisnici u okviru mreže odlučuju koji podatak može da se sačuva u skladištu.

Web server

Hardware: hostuje web server software, povezan je na internet(statički IP), skladišti web sadržaj(HTML i js fajlove, css, slike...)

Software: obezbeđuje klijentu(npr. browser-Chrome) da pristupi web sadržaju, komunikacija na nivou zahtev/odgovor, IIS, Apache...

Osnovni zadaci softvera: koristi odgovarajući protokol za komunikaciju(najčešće HTTP/HTTPS), odlučuje zahteve na odgovarajućem portu(HTTP: port 80, HTTPS: port 443), šalje zahtevani resurs kao odgovor

Nakon što posaljemo zahtev serveru:

Na serveru se obrađuju svi zahtevi koji stižu, uključujući i GET zahtev, i server generiše HTML koji šalje nazad korisniku. Tek kada korisnik dobije HTML od servera, browser kreira DOM i prikazuje korisniku sadržaj stranice.

Server može poslati korisniku cookie i HTML istovremeno u odgovoru na HTTP zahtev. Kada korisnik primi ovaj odgovor, pregledač će interpretirati HTML dokument i prikazati ga na ekranu, dok će cookie biti sačuvan na strani korisnika i koristiće se za održavanje sesije na serveru.

Troslojna arhitektura

Kod treba da ima neku logicku strukturu, delimo ga u slojeve gde ce svaki sloj biti zaduzen za odredjeni deo posla. Slojevi mogu komunicirati, ali samo preko apstrakcija(interfejsa) i to tako da se medjusobno ne preskacu i da nizi sloj ne vidi visi sloj

HTTP zahtev -> sloj kontrolera -> sloj servisa -> sloj podataka

Sloj kontrolera - ulazna tacka za spoljne zahteve ka aplikaciji, svaka metoda kontrolera se mapira na jedinstvenu URL putanju, uz to da mora biti definisan i HTTP glagol nad metodom(GET, POST, PUT...). Sve operacije nad korisnicima staviti u UserController

Sloj servisa - biznis logika: sortiranje nizova, slanje mejlova ili bilo sta za namenu aplikacije. Ne sme direktno komunicirati sa bazom i ne sme poznavati HTTP zahteve, niti odgovore. Ne sme da poziva metode kontrolera direktno. Moze izvesti validaciju podataka i javiti kontrolerskom sloju ako je neka greska.

Sloj podataka - jedini sme direktno da komunicira sa bazom, ali ne sme pozivati metodu nijednog sloja iznad. Trebao bi da enkapsulira upite ka bazi i da vraca modele kao odgovor.

Arhitektura sa 5 slojeva - prosirenje troslojne arhitekture. Dodati su sloj korisnickog interfejsa(presentation layer) i sloj aplikacionog servera(application server layer)

1. **Sloj korisnickog interfejsa(presentation layer)** - odnosi se na ono sto korisnik vidi i interaktuje sa njim. To moze biti web stranica, mobilna aplikacija, desktop aplikacija... Ovaj sloj se fokusira na prezentaciju podataka i omogucava korisnicima da komuniciraju sa aplikacijom.

2. **Sloj kontrolera(controller layer)** - odgovoran za obradu korisnickih zahteva i upravljanje komunikacijom izmedju sloja korisnickog interfejsa i sloja servisa. Kontroler prima zahtev od korisnika, validira ulazne podatke i poziva odgovarajucu funkciju u sloju servisa

3. **Sloj servisa(service layer)** - fokusira se na poslovnu logiku aplikacije. Sadrzi funkcije i servise koji obradjuju zahtev korisnika iz sloja kontrolera. Ovaj sloj je nezavisan od sloja korisnickog interfejsa i sloja podataka

4. **Sloj aplikacionog servera(application server layer)** - omogucava komunikaciju izmedju sloja servisa i sloja podataka. Sadrzi poslovnu logiku koja se izvsava na serveru, sto omogucava bolju skalabilnost i bezbednost aplikacije. Takodje, obuhvata infrastrukturu koja podrzava rad aplikacije, poput baze podataka, servera za obradu poruka...

5. **Sloj podataka(data layer)** - fokusira se na skladistenje i manipulaciju podacima aplikacije. Sadrzi bazu podataka i skripte koje omogucavaju pristup i manipulaciju podacima. Nezavistan od drugih slojeva, sto omogucava da se podaci mogu koristiti u razlicitim aplikacijama i na razlicitim platformama.

Funkcionisanje 5 slojeva arhitekture bazira se na razdvajanju odgovornosti i nezavisnosti slojeva, sto omogucava bolju organizaciju i odrzavanje aplikacije. Svaki sloj ima jasno definisanu ulogu i odgovornost, sto olaksava razvoj aplikacije i resavanje problema. Omogucava skalabilnost.

Code-First : prvo kreiramo klase, pa na osnovu njih kreiramo bazu podataka

Database-First : iz postojece baze kreiramo klase

Model-First : nekim alatom za modelovanje kreiramo model klasa i na osnovu njega generise klase i bazu podataka

Migracije - C# kod koje ce Entity Framework pretvoriti u SQL skriptu

DTO(data transfer object) - objekti koji nose podatke izmedju procesa ili slojeva aplikacije

Autentifikacija sa tokenom

Podatke o korisnicima mozemo cuvati u bazi podataka, a po unosu validnih kredencijala korisniku izdajemo json web token na osnovu kog se dalje autentifikuje pri pristupu nasim servisima. U token stavljamo sve podatke koje nam je bitno da znamo o korisniku, a koji **nisu poverljivi jer token sam po sebi nije zasticen**. Izdati token se potpisuje nasim privatnim kljucem ili lozinkom kako bi se sprecilo podmetanje vestackog tokena. Privatnim kljucem potpisuje server, dok javni kljuc koristi klijent za proveru ispravnosti potpisa. U token mozemo staviti **claimove(prava) koja korisnik ima**. Mozemo staviti podatke o izdavaocu i primaocu tokena i njegovo trajanje. Dobijeni token korisnik stavlja u HTTP zahteve prema nasem serveru i tako se autentifikuje. Nas server medjutim mora verifikovati tokene iz svakog zahteva kako bi se uverili da se neko nije lazno predstavio ili podmetnuo token. Dodatno, kako bi validacija bila izvrшена mora se dodati autentifikacija u middleware. Authorize atribut je za odredjivanje prava spram uloge, inace ce svi koji imaju token moci da rade isto sto i admin.

Claim - podatak u tokenu

CORS

Iz bezbednosnih razloga skriptovi su ograniceni samo na pristup podacima koji imaju isto poreklo.

Poreklo je kombinacija: **URI sema**(http ili https), **hostname**, **port**(default je 80, ali zavisi od browsera)

Primer: **http://www.example.com/dir/page.html**

URL	status	razlog
http://www.example.com/dir/page2.html	OK	isti protokol, host i port
http://www.example.com/dir2/other.html	OK	isti protokol, host i port
http://username:password@www.example.com/dir2/other.html	OK	isti protokol, host i port
http://www.example.com:81/dir/other.html	NOK	različit port
https://www.example.com/dir/other.html	NOK	različit protokol
http://en.example.com/dir/other.html	NOK	različit host
http://example.com/dir/other.html	NOK	različit host
http://v2.www.example.com/dir/other.html	NOK	različit host
http://www.example.com:80/dir/other.html	???	zavisi od browsera

Cross-Origin Resource Sharing pruza mogucnost da sajt A dopusti pristup svojim podacima skriptovima sa sajta B. Implementira se na serveru kako bi omogucio klijentu pristup resursima sa drugih domena. CORS je sigurnosni mehanizam koji se koristi u web aplikacijama kako bi sprecio izvodjenje skripti sa neovlascenih izvora. Klijent moze da navede dodatne informacije o CORS-u u zahtevima prema serveru, ali server donosi odluku da li je zahtev dopusten.

Access-Control-Request-Method: metoda pravog zahteva

Access-Control-Request-Headers: lista posebnih zaglavlja u zahtevu, razdvojenih zarezom

Access-Control-Allow-Origin: kao kod prostog zahteva

Access-Control-Allow-Methods: lista dozvoljenih HTTP metoda

Access-Control-Allow-Headers: lista dozvoljenih zaglavlja u zahtevu

Access-Control-Allow-Credentials: kao kod prostog zahteva

Access-Control-Max-Age: omogucava kesiranje ovog odgovora dati broj sekundi

CORS - mehanizam kojim server specificira kojim domenima dozvoljava da ga kontaktiraju. Ovde recimo mozemo podesiti da iskljucimo da frontend nase aplikacije moze slati zahteve zadnjoj strani.

CORS (Cross-Origin Resource Sharing) se implementira na strani servera. U većini slučajeva se dodaje na serveru kao middleware u server-side frameworku, ili se ručno dodaje u kodu.

```
public void Configure(IApplicationBuilder app, IWebHostEnvironment env)
{
    // Dodavanje CORS middleware-a u pipeline
    app.UseCors(builder => builder
        .AllowAnyOrigin() // Dozvoljava sve origin-e
        .AllowAnyMethod() // Dozvoljava sve HTTP metode
        .AllowAnyHeader() // Dozvoljava sve HTTP zaglavlja
    );

    // Ostatak koda aplikacije
    // ...
}
```

CORS sa ogranicenjima origin-a:

"origin" predstavlja skup informacija o poreklu resursa koji pokušava da pristupi nekom drugom resursu preko HTTP zahteva. On obično predstavlja URL odakle je poslat HTTP zahtev (ili skup URL-ova ako se radi o iframe-u ili sličnom elementu).

Na primer, ako stranica sa URL-om "http://www.example.com" pokuša da pristupi resursu na URL-u "http://api.example.com", "origin" bi bio "http://www.example.com".

U ovom primeru, dozvoljavamo pristup resursima samo sa "http://example.com" i "https://example.com" originima. Sve druge origin-e će biti odbijene.

```
app.UseCors(builder => builder
    .WithOrigins("http://example.com", "https://example.com")
    .AllowAnyMethod()
    .AllowAnyHeader()
);
```

CORS, ko ga proverava? Ako imas kod i pokrenes izbaciti ti CORS gresku, ako isti ubacis u postman radice, to je jer ga klijent proverava

Web sadrzaj

Staticki - prethodno pripremljen web sadrzaj se salje kao odgovor klijentu(recimo HTML strna). Na osnovu zahteva pronalazi se odgovarajuci fajl koji se vraca korisniku. Nepostojanje fajla je error 404.

Dinamicki - Sadrzaj se generise nakon prijema zahteva. Proverava se da li postoji vec predefinisani sadrzaj; ukoliko postoji vraca se kao odgovor; ukoliko ne postoji server pokusava da generise sadrzaj; ako je uspesno generisan vraca se klijentu; ako nije generisan vraca se error 404.

Generisanje sadrzaja

Aplikativni server = web aplikacija -- zaduzen za logiku

Programski jezici: C#, Java, PHP, JS

Skladiste podataka - najcesce baza podataka

npm install - instalacija npm paketa postujuci redosled prioriteta ukoliko on postoji. Podrazumevano instalira module iz package.json. Instalira sve JavaScript pakete.

HTTP se nalazi na aplikativnom sloju u TCP/IP steku, to je protokol za komunikaciju na webu, connectionless - ne cuva konekciju

1. Web citac inicira komunikaciju otvarajuci TCP vezu prema serveru
2. Web citac salje zahtev serveru
3. Server obradjuje zahtev i salje odgovor
4. Server zatvara vezu. Naredni zahtevi moraju ici kroz nove konekcije

Media independent - bilo koji tip podataka(MIME-type) se moze prenositi sve dok klijent i server mogu da ih obrade

Connectionless - klijent i server su u vezi samo tokom jednog ciklusa zahtev/odgovor; ne cuva se stanje u komunikaciji

HTTP je klijent/server model

Klijent salje zahtev: metoda, URI, verzija protokola, modifikatori zahteva, informacije o klijentu, opciono i sadržaj

Server salje odgovor: verzija protokola, kod rezultata, informacije o serveru, sadržaj

HTTP metode - moguće metode u zahtevu

GET - za citanje podataka sa web servera, ne bi trebalo da menja podatke i stanje servera

HEAD - isto kao GET, ali vraća se samo prva linija zaglavlja bez tela odgovora

POST - za slanje(novih) podataka na web server. Podaci se salju u telu zahteva, a server ih obradjuje i vraća odgovor

PUT - azurira resurs novim vrednostima(salje ceo resurs). Ako ne postoji stvara se novi resurs.

PATCH - azurira resurs novim vrednostima(salje deo resursa)

DELETE - uklanja resurs sa servera

CONNECT - uspostavlja tunel prema serveru(proxy) identifikovanim URI adresom

OPTIONS - zahteva opis mogućih operacija(metoda) za dati resurs

TRACE - loopback test sa slanjem poruke. Za testiranje veze. Vraća kopiju poslatog zahteva kako bi se moglo pratiti šta se događa sa zahtevom u mreži

Status operacije

1xx - informativne poruke, od HTTP/1.1

2xx - uspešno obradjen zahtev

- 200 OK
- 201 Created
- 204 No Content

3xx - redirekcija: usluga dostupna na drugom URI

- 302 Moved Temporarily
- 304 Not Modified

4xx - greska na strani klijenta

- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found

5xx - greska na strani servera

- 500 Internal Server Error

- 200 OK: Zahtev je uspešno obradjen, i odgovor sadrži tražene podatke.
- 201 Created: Zahtev je uspešno obradjen i rezultirao je kreiranjem novih resursa.
- 400 Bad Request: Zahtev nije uspešno obradjen zbog neispravnog formata ili strukture zahteva.
- 401 Unauthorized: Zahtev zahteva autentifikaciju ili validne kredencijale za pristup resursima.
- 403 Forbidden: Zahtev je validan, ali server odbija pristup resursima zbog nedozvoljenih privilegija.
- 404 Not Found: Traženi resurs nije pronađen na serveru.
- 500 Internal Server Error: Došlo je do interne greške na serveru prilikom obrade zahteva.

Novosti u HTTP/2.0

Multiplexing: više asinhronih HTTP zahteva kroz jednu TCP konekciju, i odgovori se mogu primati asinhroni. Svaki zahtev i odgovor imaju svoj streamId, podeljeni u delove(binarne) i svaki deo ima svoj frameld. Stream - kolekcija frejmova. **Asinhron** - ne ceka odgovor vec se obradjuje u pozadini.

Server push: vise odgovora na jedan zahtev

Header Compression: kompresija HTTP zaglavlja kao i sadrzaja

Request prioritization: prioritet medju zahtevima poslatim na isti domen

Binary protocol: HTTP/2.0 je binarni protokol za razliku od HTTP/1.1

Primer: klijent salje zahtev za index.html. Server proceni da ce klijent traziti i styles.css i script.js i salje push promise da predupredi zahtev. Kaze u kojim strimovima ce biti odgovor za styles.css i script.js

HTTP/1.0 koristi novu TCP konekciju za svaki ciklus zahtev/odgovor

HTTP/1.1 može da koristi istu TCP konekciju za više ciklusa zahtev/odgovor, ali je protokol i dalje connectionless i stateless

- uvedeno zbog unapređenja performansi
- jedna stranica sa 20 slika bi zahtevala 21 TCP konekciju po HTTP/1.0
- jedna stranica sa 20 slika, 5 CSS fajlova i 10 JavaScript fajlova...

HTTP/2.0 novine radi unapređenja performansi

- Smanjenje kasnjenja
- Smanjenje broja otvorenih TCP konekcija
- Bolja bezbednost na webu
- Kompatibilnost sa HTTP/1.1 klijentima i serverima
- Moze se koristiti gde i HTTP/1.1

Koje metode je moguće pozvati nad objektima: userOne, userTwo, adminOne, adminTwo:

```
function User(email, name){
  this.email = email;
  this.name = name;
  this.online = false;
}

User.prototype.login = function(){
  this.online = true;
  console.log(this.email, 'has logged in');
};

var userOne = new User('ryu@xxx.com', 'Ryu');

function Admin(email, name){
  User.call(this, email, name);
}

Admin.prototype = Object.create(User.prototype);

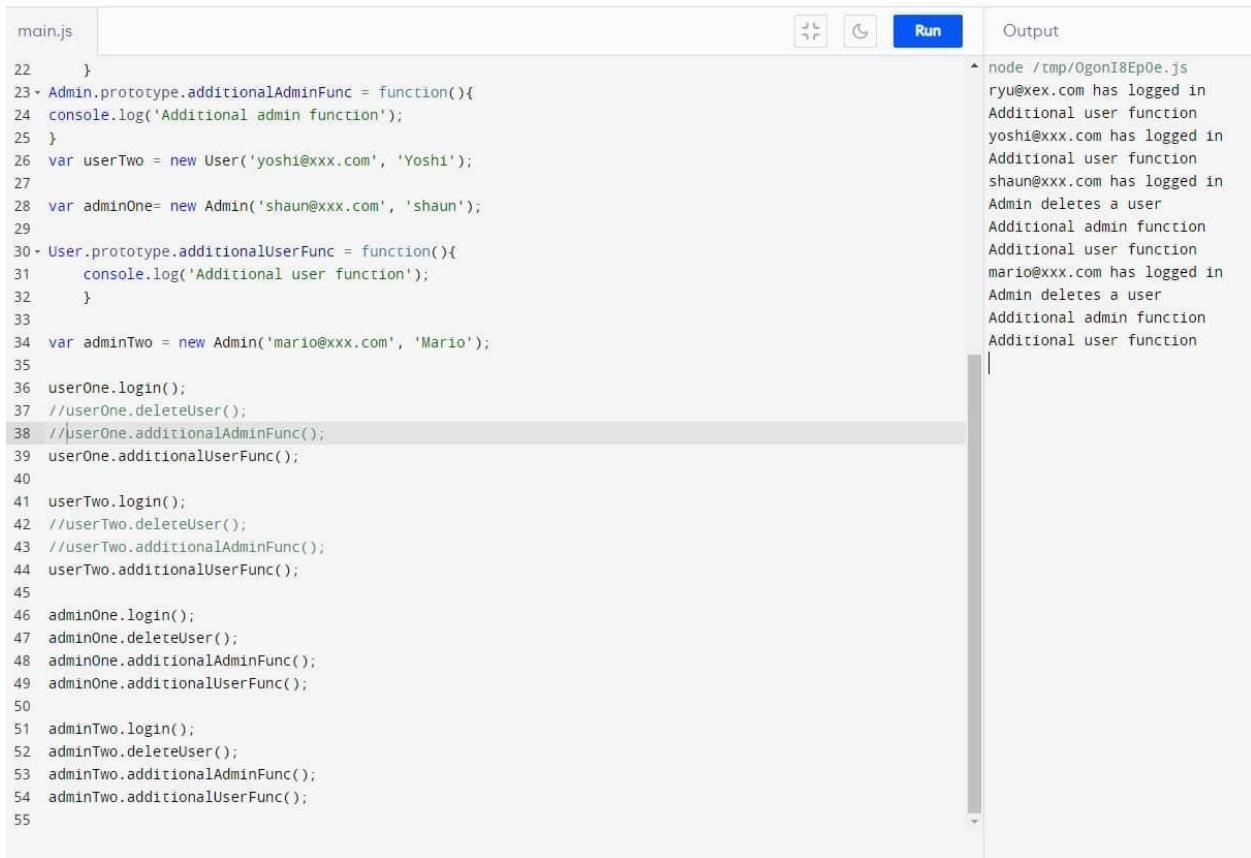
Admin.prototype.deleteUser = function(){
  console.log('Admin deletes a user');
};

User.prototype.additionalUserFunc = function(){
  console.log('Additional user function');
};

var userTwo = new User('yoshi@xxx.com', 'Yoshi');
var adminOne = new Admin('shaun@xxx.com', 'Shaun');

Admin.prototype.additionalAdminFunc = function(){
  console.log('Additional admin function');
};

var adminTwo = new Admin('mario@xxx.com', 'Mario');
```



```
main.js
22 }
23 Admin.prototype.additionalAdminFunc = function(){
24   console.log('Additional admin function');
25 }
26 var userTwo = new User('yoshi@xxx.com', 'Yoshi');
27
28 var adminOne= new Admin('shaun@xxx.com', 'shaun');
29
30 User.prototype.additionalUserFunc = function(){
31   console.log('Additional user function');
32 }
33
34 var adminTwo = new Admin('mario@xxx.com', 'Mario');
35
36 userOne.login();
37 //userOne.deleteUser();
38 //userOne.additionalAdminFunc();
39 userOne.additionalUserFunc();
40
41 userTwo.login();
42 //userTwo.deleteUser();
43 //userTwo.additionalAdminFunc();
44 userTwo.additionalUserFunc();
45
46 adminOne.login();
47 adminOne.deleteUser();
48 adminOne.additionalAdminFunc();
49 adminOne.additionalUserFunc();
50
51 adminTwo.login();
52 adminTwo.deleteUser();
53 adminTwo.additionalAdminFunc();
54 adminTwo.additionalUserFunc();
55
```

node /tmp/OgonI8Ep0e.js
ryu@xex.com has logged in
Additional user function
yoshi@xxx.com has logged in
Additional user function
shaun@xxx.com has logged in
Admin deletes a user
Additional admin function
Additional user function
mario@xxx.com has logged in
Admin deletes a user
Additional admin function
Additional user function

Routing omogućuje navigaciju izmedju razlicitih prikaza, razmenu parametara izmedju komponenti koje upravljaju prikazima.

Guards - interfejsi koji govore router-u da li da dozvoli prelazak na odredjenu rutu. Rute zabranjujemo kada: korisnik nije autorizovan, nije autentifikovan(potreban login), podaci jos uvek nisu dostupni.

Servisi su klase koje su dostupne razlicitim komponentama. Njihova uloga je komunikacija sa serverskim delom aplikacije, omogucavaju komunikaciju izmedju komponenti. Komponente se bave view-related funkcionalnostima. Servisi su zaduzeni za: dobavljanje podataka sa servera, validiranje korisnickih unosa, logovanje podataka na konzolu.

Smart grid - potrebe za: nadziranjem i upravljanjem distributivnom mrežom, ocekivan rad u realnom vremenu ili blizu realnog vremena, detekcija i otklanjanje kvarova, planiranje razvoja distributivne mreze i pomoc pri izvodjenju radova, prikupljanje dodatnih podataka vezanih za distributivnu mrežu, komunikacija sa ostalim IT sistemima

RBAC - sistem od vise razlicitih uloga korisnika

Sta je cors(web2), kako se implementira(u konfiguraciji nekako)
Detaljno ispricaj kada ukucas u browser link kako to funkcioniše
Razlika izmedju POST I GET metode, kako saljes podatke
Sesije, cookies, kako u browseru funkcioniše

Na koji nacin moze da se poboljsa hesovanje funkcije, tj. dodatno pored hesovanja kakva enkripcija?
Solt je odgovor - sa random vrednoscu pomesas i onda na to hesujes

Razlika izmedju apstraktne i interfejsa, apstraktna metoda mora biti implementirana samo u apstraktnoj klasi

Dependancy injection - solid princip sa interfejsom(kao strategy) da bi mogla da se radi nadogradnja

Od cega se sastoji HTTP zahtev?

DOM

HTML

Liste:

nabrojive(odreder) - `<ol start="3"> <li>voce</li> </ol>`

neuredjene(unordered) - `<ul type="square"> <li>voce</li> </ul>`

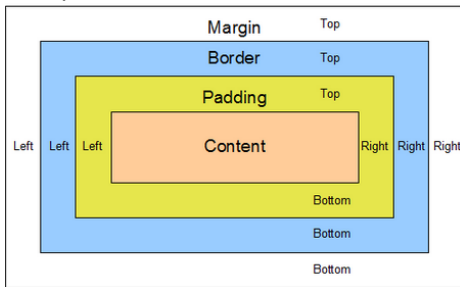
definicione(definition) - `<dl> <dt>crna kafa</dt><dd><vrucice pice></dd> </dl>`

Tabele:

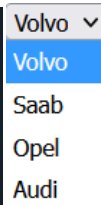
`<table>` , `<tr>` , `<td>` , `colspan`, `rowspan`

Forme:

text, password, email, radio, checkbox, submit, reset, image, file, button, hidden,



```
<select name="cars" id="cars">
  <option value="volvo">Volvo</option>
  <option value="saab">Saab</option>
  <option value="mercedes">Mercedes</option>
  <option value="audi">Audi</option>
</select>
```



Razlika <div> i

Div se koristi za grupisanje i organizaciju vecih delova sadrzaja, dok se **span** koristi za stilizaciju ili manipulaciju manjih delova teksta ili sadrzaja. Div je block element(zauzima prostor do kraja), a span je inline element(zauzima samo taj deo gde je tekst).

Bootstrap - front-end framework koji kombinuje HTML,CSS, JS. Pruza gotove stilove, komponente i skripte koje se mogu koristiti.

Flexbox - omogucava fleksibilno pozicioniranje i rasporedjivanje elemenata u jednodimenzionalnim redovima ili kolonama. Svi item elementi ce se prikazati jedan pored drugog sa jednakim prostorom izmedju njih:

```
.item {
  flex-basis: 30%;
```

Grid - kreiranje dvodimenzionalnih rasporeda elemenata. Omogucava precizno pozicioniranje elemenata u redovima i kolonama. Omogucava izradu mreza clanaka, galerije i slicno.

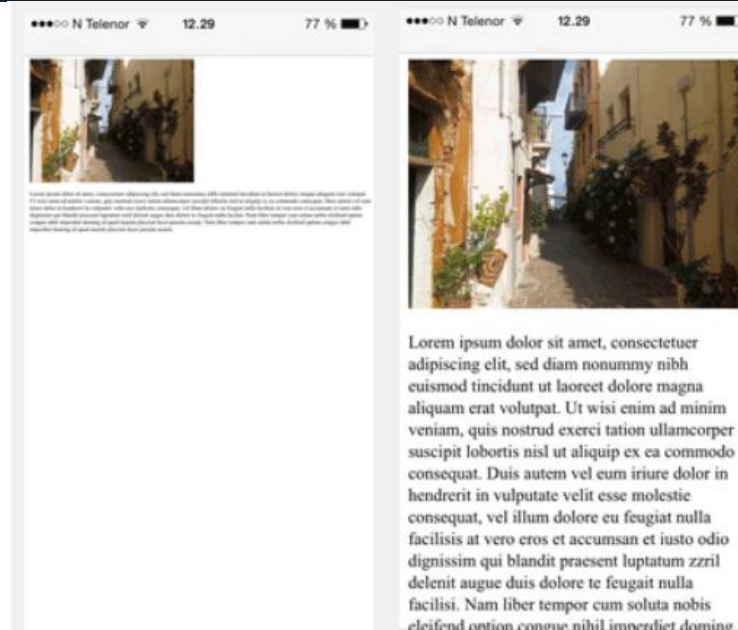
```
display: grid;
grid: 150px / auto auto;
grid-gap: 10px;
```

alt - alternativni tekst ukoliko se slika ne prikaze

Meta podaci - informacije o dokumentu ili veb stranici koji se koriste za opisivanje njenog sadržaja, strukture, karakteristika... Ne prikazuju se korisniku direktno već se koriste za: pretragu, optimizaciju za pretraživače, prikazivanje ikona...

Primer pregleda sajta pomocu telefona sa i bez meta taga viewport:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```



<sub> - dole

<sup> - gore

<mark> - highlighted text

 - precrtan tekst

<ins> - podvuceno

<link> -- za eksternu listu stilova. Dodaje se u <head> tag.

```
<head>  
  <link rel="stylesheet" href="styles.css">  
</head>
```

Otvaranje prozora u novom tabu (target="_blank")

```
<a href="https://www.w3schools.com/" target="_blank">Visit W3Schools!</a>
```

Dugme kao link:

```
<button onclick="document.location='default.asp'">HTML Tutorial</button>
```

Klik na sliku koja nas pomocu linka prebacuje negde. Moze biti: rect, poly, circle, default(ceo region)

```


<map name="workmap">
  <area shape="rect" coords="34,44,270,350" alt="Computer" href="computer.htm">
  <area shape="rect" coords="290,172,333,250" alt="Phone" href="phone.htm">
  <area shape="circle" coords="337,300,44" alt="Cup of coffee" href="coffee.htm">
</map>
```



Ukoliko zelimo da pruzimo razlicite verzije slike za razlicite uredjaje ili situacije koristi se <picture> tag. Moze se koristiti za prilagodjavanje slika na osnovu velicine ekrana, rezolucije ili drugih faktora.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
<body>

<h2>The picture Element</h2>

<picture>
  <source media="(min-width: 650px)" srcset="img_food.jpg">
  <source media="(min-width: 465px)" srcset="img_car.jpg">
  
</picture>
```

Favicon - ikonica koja se pojavljuje na tabu tvog sajta. S obzirom da je mala treba da bude jednostavna slika sa velikim kontrastom

```
<link rel="icon" type="image/x-icon" href="/images/favicon.ico">
```

```
<caption>Naslov tabele</caption>
<colgroup>
  <col span="2" style="background-color:red">
  <col style="background-color:yellow">
</colgroup>
```

Naslov tabele

ISBN	Title	Price
3476896	My first HTML	\$53
5869207	My first CSS	\$49

CSS:

Stilovi za CSS se ugradjuju:

inline style - unutar samih HTML elemenata(atribut style)

internal style sheet - upotrebom taga <style> unutar dokumenta(**unutar <head> taga**)

external style sheet - kreiranjem spoljasnje datoteke stilova(.css datoteka)

Prioritet je:

1. unutar HTML elementa -- <h1 style="color:blue">Tekst</h1>
2. <style> tag -- selektor {svojstvo: vrednost; ...}
3. spoljasnja datoteka stilova -- <link rel="stylesheet" type="text/css" href="stilovi.css">
4. kako citac prikazuje

Stilizovanje HTML elemenata:

Tag selektor - kada koristimo samo tag selektor(div, p, h1...). Stilovi ce se primeniti na sve elemente tog odredenog HTML taga na stranici.

```
p {  
  /* Stilovi za p tagove */  
}
```

Id selektor(#) - za HTML elemente koji imaju odredjeni ID atribut <div id="myDiv">

```
#myDiv {  
  /* Stilovi za element sa ID-om myDiv */  
}
```

Klasni selektor(.) - kada stilizujemo grupu HTML elemenata koji dele isti klasni atribut <p class="highlight">.

```
.highlight {  
  /* Stilovi za elemente sa klasom highlight */  
}
```

Stil za linkove:

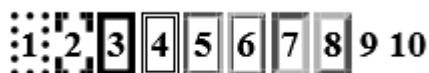
a:link, a:visited, a:hover, a:active

float - postavljanje elemenata levo ili desno u kontejneru.

flex - fleksibilniji raspored(nov CSS)

<pre>table, th, td { border: 1px solid black; border-collapse: collapse; border-color: blue; }</pre>	<pre>table, th, td { border: 1px solid black; border-collapse: collapse; <!-- borderi se spajaju u 1 --> }</pre>
--	--

```
<tr>  
  <th style="border-style:dotted;">1</th>  
  <th style="border-style:dashed;">2</th>  
  <th style="border-style:solid;">3</th>  
  <th style="border-style:double;">4</th>  
  <th style="border-style:groove;">5</th>  
  <th style="border-style:ridge;">6</th>  
  <th style="border-style:inset;">7</th>  
  <th style="border-style:outset;">8</th>  
  <th style="border-style:none;">9</th>  
  <th style="border-style:hidden;">10</th>  
</tr>
```



Width:50% znaci 50% ekrana... Isto ovako moze rowspan

```
<table style="width:50%;text-align:center;">
  <tr>
    <th colspan="2">Name</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>Jill</td>
    <td>Smith</td>
    <td>50</td>
  </tr>
</table>
```

Name		Age
Jill	Smith	50

Na ovaj nacin moze da se boji i colgroup

```
th:nth-child(even),td:nth-child(even) {
  background-color: #D6EEEE;
}
tr:nth-child(even) {
  background-color: rgba(150, 212, 212, 0.4);
}
th:nth-child(even),td:nth-child(even) {
  background-color: rgba(150, 212, 212, 0.4);
}
```

```
tr:hover {background-color: #D6EEEE;}
```

```
th, td {
  padding-top: 10px;
  padding-bottom: 20px;
  padding-left: 30px;
  padding-right: 40px;
}
```

```
table {
  border-spacing: 20px;
}
```

Firstname
Jill

Stilizovanje unordered listi pomocu **list-style-type**: none, square, circle, disc(crni circle umesto praznog)

Stilizovanje ordered listi pomocu **type**: 1, A, a,i,l pored toga **start**: 50(neki broj)

Primer: <ol type="I" start="50"> ili : <ol type="1" start="50">

L. Coffee 50. Coffee

LI. Tea 51. Tea

Tipovi dijagrama definisanih UML-om:

Strukturni dijagrami (prikazuju statički aspekt sistema):

1. Class - opisuje klase koje cine sistem i njihove relacije
2. Object - slicni kao klasni, samo prikazuju objekte, bas instance u nekom momentu
3. Component - opisuje softverske komponente koje cine sistem, biblioteke, paketi, fajlovi...
4. Deployment - opisuje hardverske komponente nad kojima se vrse softverske, usko je povezan sa component dijagramom

Dijagrami ponasanja (prikazuje dinamički aspekt sistema):

1. Use case - ponasanje sistema pri interakciji izmedju sistema i korisnika
2. Sequence - opis toka poruka izmedju objekata pri realizaciji neke operacije
3. Collaboration - opis komunikacije izmedju objekata sistema
4. Statechart - opisuje razlicita stanja objekta tokom zivotnog veka te promena iz stanja u stanje dinamički instancirane
5. Activity - prikaz dinamičkog ponasanja sistema od jedne akcije do druge

Use case - opis ponasanja sistema ili njegove funkcije pod razlicitim okolnostima u skladu sa odgovorima sistema na zahteve korisnika sistema u cilju postizanja nekog poslovnog rezultata. Koristi se u ranoj fazi razvoja softvera kao scenario, prikaz onoga sto se desava pri interakciji korisnika i sistema. Ima 4 osnovna elementa: sistem(koji se modeluje), ucesnik(koji inicira interakciju), slucaj koriscenja(opis interakcije izmedju korisnika i sistema - skup scenarija-prikupljanje funkcionalnih zahteva sistema), veze(izmedju ovih elemenata).

Potrebno je definisati potrebne funkcionalnosti, identifikovati ciljeve, revidirati razumevanje pre dizajna i implementacije. Imamo:

1. slucaj koriscenja(use case) - sta funkcionalnost treba da podrzi. Postize cilj za ulogu. Koje funkcionalnosti, sta svaka uloga mora da uradi, da li je u stanju to da uradi, sta su ulazi i izlazi iz sistema.

2. Uloge u sistemu(actors) - definise koje funkcije su kome dostupne. Neko ko ima interakciju sa sistemom predstavlja ulogu, a ne korisnika. Moze biti: uredjaj, softver, osoba. Komunikacija sa sistemom preko poruka(ulazno-izlazne).

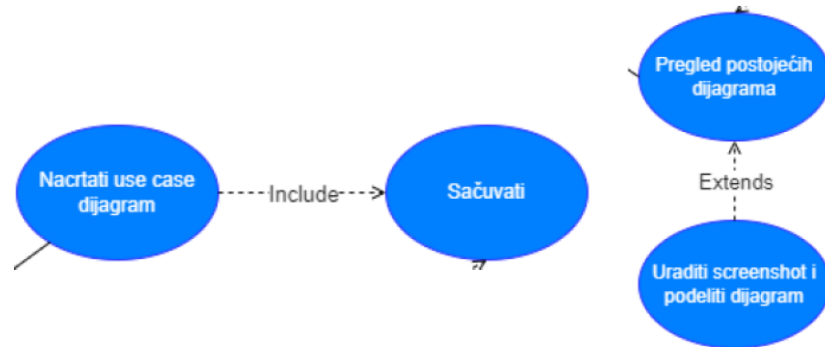
Veze na dijagramu:

1. Includes - izdvaja zajednicke delove razlicitih slucajeva koriscenja

2. Extends - opis slucaja koriscenja u posebnim slucajevima. Varijacija toka informacija(dodatna akcija koja moze, a ne mora da se izvrši)

Modelovanje:

1. Identifikovati sve ucesnike i uveriti se da su obuhvacene sve funkcije koje koriste
2. Ako slucaj koriscenja zahteva ostale ucesnike, povezati i njih na dijagramu
3. Koristi extends da se opisu opciona ponasanja. Koristi include da se izdvoje zajednicki delovi



user story - agilne price

Oba sadrže: ko radi, šta radi, i kriterijum ispunjenosti

Misuse case - koristi se za testiranja kako bi dodatno obezbedili sistem i prepoznali malverzacije

(da li administrator sistema može da obrise zalbu potrosaca na nestanak struje; da li zaposleni u distribuciji može da se zali na nestanak struje)

User story primer: Kao novi korisnik, želim stvoriti novi korisnički račun u aplikaciji kako bih mogao pristupiti svim njenim funkcijama

Class diagram - prikazuje skup klasa, interfejsa, saradnji i njihovih relacija. On opisuje strukturu sistema i specificira staticke i logicke aspekte modela. Imenuje i modelira koncepte u sistemu i specificira te logicke seme baze podataka(koji elementi ce se upisivati u bazu podataka). Sadrzi skup klasa, saradnji i njihovih relacija. Specificira staticke i logicke aspekte modela.

Klasa - skup objekata sa zajednickim atributima, operacijama, metodama, vezama i semantikom. Ne zauzima memoriju dok se ne inicijalizuje i nparavi objekat. Do tog trenutka je samo nacrt. Ne moramo inicijalizovati klasu i kreirati njen objekat ukoliko su metode staticke. Tada mozemo samo pozvati metodu.

Relacije izmedju klasa definisanih UML-om

Dependency(zavisnost) - kada jedan element zavisi od drugog.

Shopping cart --> product, da bi kupili moramo imati proizvod

Association(asocijacija) - bilo kakva logicka veza

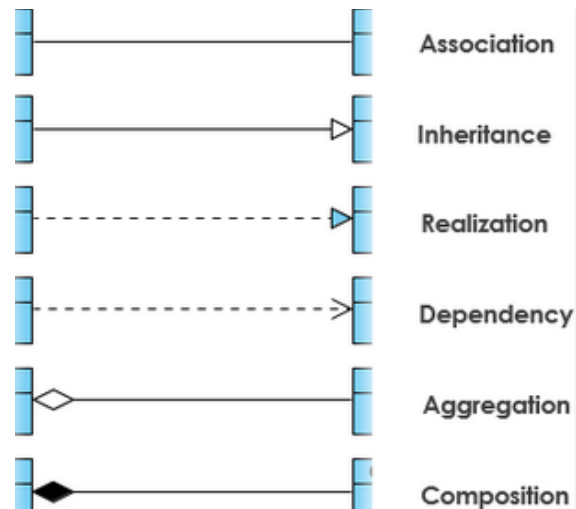
Passenger --> airplane, avion ima 0 ili vise putnika

Generalization/Inheritance(nasledjivanje) - jedan element modela(child) baziran je na drugom elementu modela(parent)

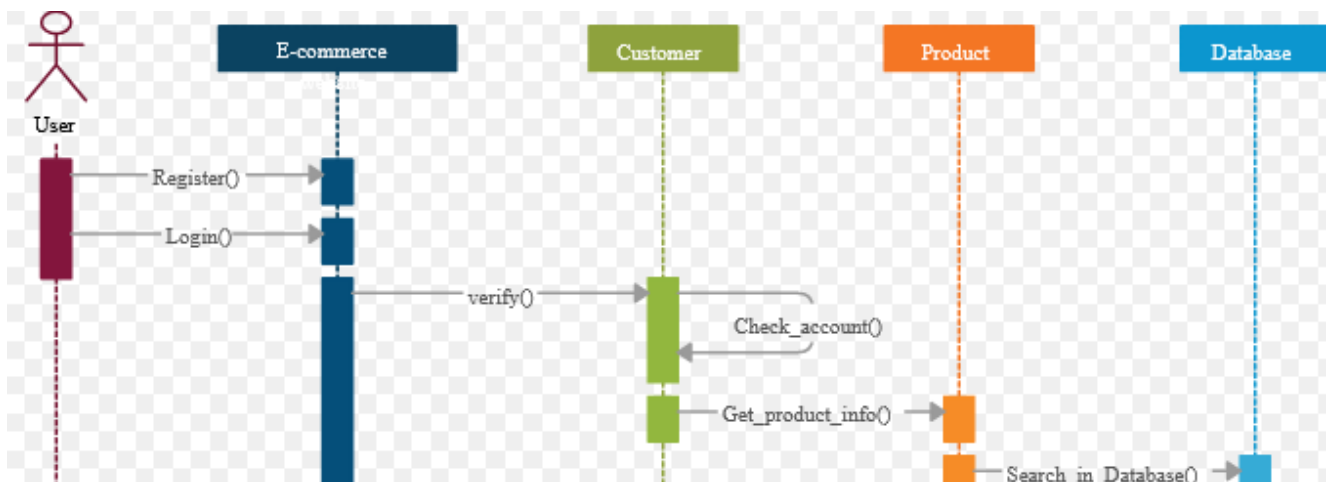
Parent <-- Child

Realization(realizacija) - element modela realizuje ponasanje drugog elementa(interfejs)

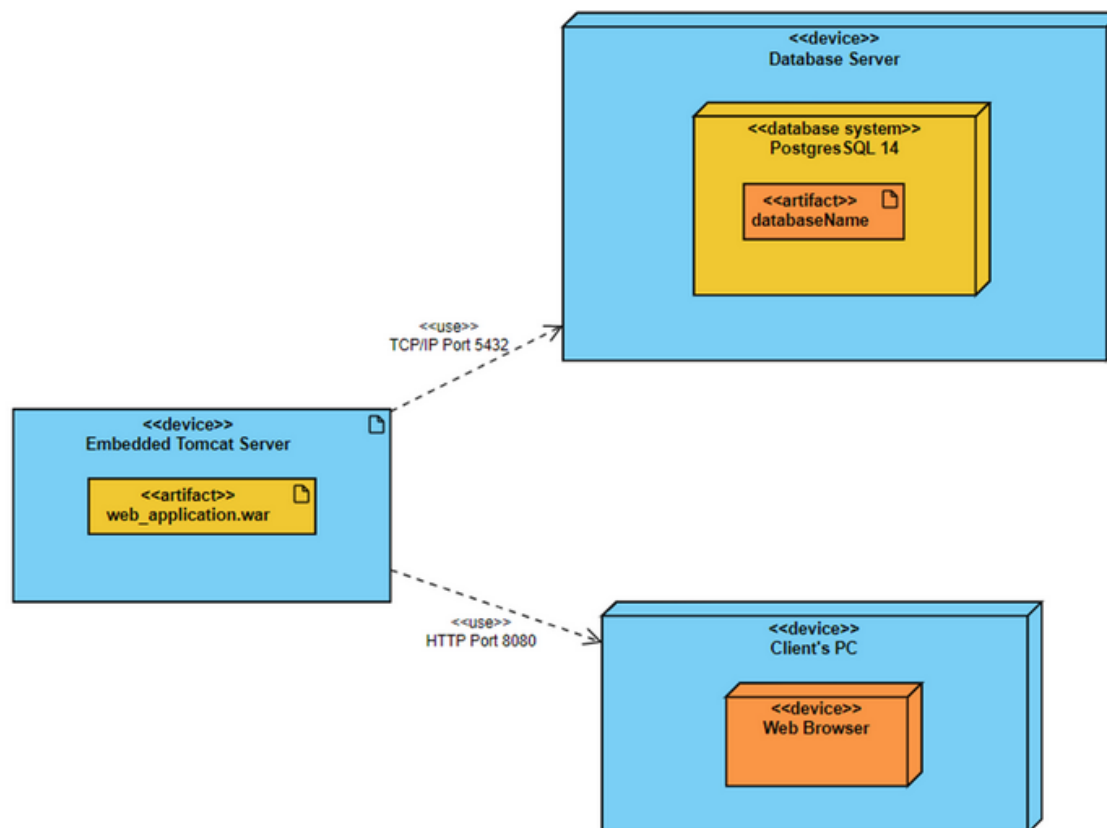
Client --> Supplier, vise klijenata moze da realizuje jednog supliera



Sequence diagram - služe za opis toka poruka između objekata kojima se realizuje odgovarajuća operacija u sistemu. Koristi se za modelovanje dinamičkih aspekata sistema. Dijagram interakcije, jedna od realizacija use case dijagrama. Prikazuje tok poruka između objekata za dati slučaj koriscenja. Ima dve dimenzije: vreme (pokazuje po vertikalnoj dimenziji), kolekcija objekata (prikazuje po horizontalnoj liniji). Pravougaonikom se predstavljaju objekti koji sadrže ime klase. Vertikalna linija označava vek trajanja objekta, a horizontalne linije označavaju slanje poruka - poziv metoda.



Dijagrami komponenti (component diagrams) - definiše odgovarajuću fizicku strukturu vezanu za implementaciju i predstavlja deo arhitekturne specifikacije. Svrha mu je organizacija izvornog koda, konstruisanje izvršne verzije i specificiranje fizicke baze podataka. Koristimo ih za opis strukture kompleksnog sistema. Nisu funkcionalnosti nego delovi koje prave funkcionalnosti. Potrebno je da imaju dobru komunikaciju sa ostalim komponentama i da zajedno sa njima formiraju kompletan sistem. One treba da su zamenljive ili po potrebi upotrebljive u drugim sistemima.



URI (Uniform Resource Identifier) je skup karaktera koji identifikuje odgovarajuće ime ili resurs u mreži. Može identifikovati resurs preko lokacije ili imena (ili oba). On je nadskup URL-a i URN-a.

Predstavlja ime, lokaciju ili obe stvari. URI je superset URL-a. Pruža tehniku kojoj se omogućava samo definisanje/opisivanje nečega. Primer:

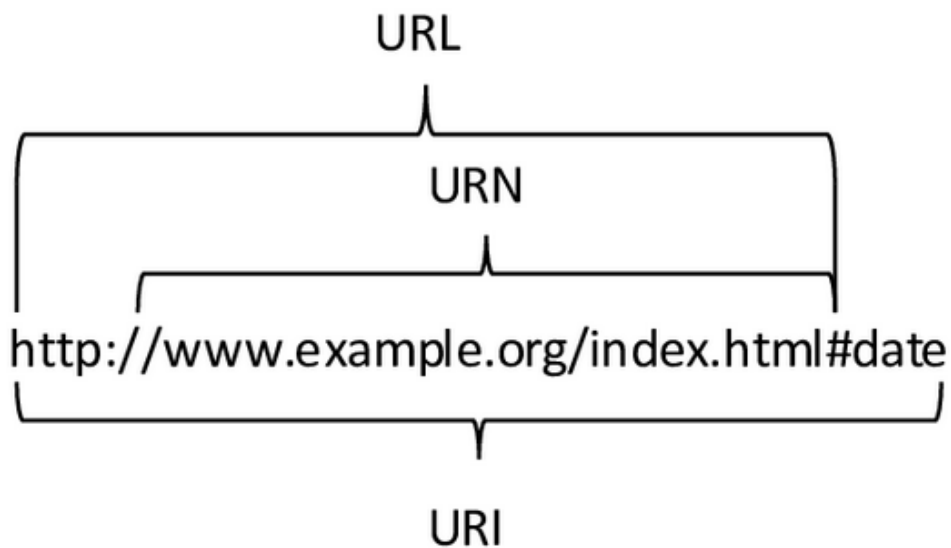
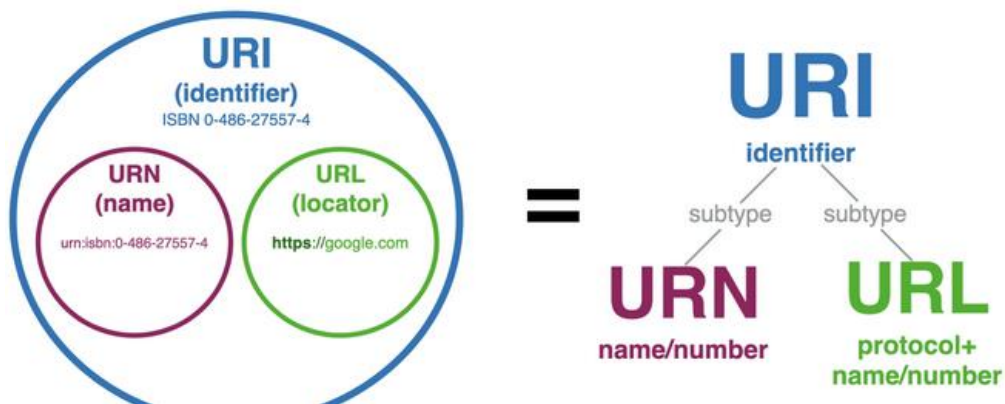
```

foo://example.com:8042/over/there?name=ferret#nose
  \  /      \      \      \      \      \
  |          |      |      |      |
scheme authority path query fragment
  
```

URL (Uniform Resource Locator) je podskup URI-ja koji specificira gde se resurs nalazi, ali pored toga što ukazuje na lokaciju obezbeđuje i mehanizme za pristup resursu. Npr: (http://), (ftp://) ... URL predstavlja samo **lokaciju**. Koristi se kako bi opisao nešto.

URN - resurs sa imenom, ali ne podrazumeva lokaciju ili kako da se pristupi imenu

IRI - ekstenzija za URI koja nam omogućava korišćenje internacionalnih znakova npr. cirilicu, kinski...



U URL adresi <http://stack.com/first/index.html?submit=yes&action=go#second:>

- `http:` predstavlja protokol koji se koristi za komunikaciju između klijenta i servera. U ovom slučaju, to je HTTP protokol.
- `stack.com:` predstavlja domenu ili ime servera na kojem se nalazi resurs koji se zahtijeva.
- `/first/index.html:` predstavlja putanju ili lokaciju resursa na serveru. `/first/` je naziv direktorija (foldera) na serveru, a `index.html` je naziv datoteke koja se nalazi u tom direktoriju.
- `?submit=yes&action=go:` predstavlja upit (query) koji se šalje serveru i sadrži informacije koje su potrebne za obradu zahtjeva. U ovom slučaju, upit sadrži dva parametra, `submit` i `action`, i njihove vrijednosti.
- `#second:` predstavlja fragment ili anchor, a označava određeni dio resursa na koji se može skočiti unutar dokumenta. Ovdje se odnosi na naziv oznake koja se nalazi unutar HTML koda dokumenta.

Mreže:

Zasto koristimo **ORM**?

Ciljevi upravljanja mrežom su: osigurati efikasnost, stabilnost i dostupnost mreže, te osigurati da se podaci prenose sa sto manje kašnjenja i gubitka. Glavni elementi za upravljanje mrežom su monitoring, konfiguracija, dijagnostika i upravljanje performansama.

Stek memorija koristi za skladištenje kratkoročnih podataka koji se brzo uklanjaju nakon završetka funkcije, dok se **heap memorija** koristi za skladištenje dugoročnih podataka koji se ne mogu alocirati na steku. Takođe, **stek** memorija se brže pristupa jer je organizovana kao LIFO struktura podataka, dok se **hip** memorija pristupa sporije jer zahteva dinamičku alokaciju.

Load balancer - mrežna komponenta ili sistem koji se koristi za ravnomerno raspoređivanje opterećenja između više resursa (kao što su serveri, mrežni linkovi ili aplikacije) kako bi se postigla bolja skalabilnost, performanse i visoka dostupnost sistema.

Cesto se koristi u različitim kontekstima:

1. Web serveri: u kontekstu web aplikacija, load balancer se koristi za ravnomerno raspoređivanje HTTP zahteva koji dolaze od korisnika na više web servera. To pomaže u balansiranju opterećenja između servera, sprečava preopterećenje jednog servera, omogućava horizontalno skaliranje i obezbeđuje visoku dostupnost
2. Aplikacijski serveri: Za složenije aplikacije koje koriste aplikacijske servere, load balancer se može koristiti za distribuciju zahteva na različite servere u klasterskom okruženju. Ovo omogućava bolje iskorišćenje resursa, poboljšava odziv aplikacija i omogućava dodavanje ili uklanjanje servera bez prekida u radu.
3. Baza podataka: U nekim scenarijima, load balancer se može koristiti za ravnomerno raspoređivanje zahteva prema bazama podataka u distribuiranom okruženju. Ovo može pomoći u balansiranju opterećenja između više baza podataka, poboljšava skalabilnost i omogućava visoku dostupnost podataka.
4. Mrežni linkovi: U mrežnim infrastrukturama, load balancer se može koristiti za distribuciju mrežnog saobraćaja na više mrežnih linkova. Ovo pomaže u optimizaciji protoka podataka, sprečava preopterećenje jednog linka, poboljšava pouzdanost mreže i omogućava balansiranje opterećenja na različitim mrežnim putanjama.

Protokoli:

ICMP (Internet Control Message Protocol) - nadzorno upravljački deo IP-a, koristi se za prenos informacija o gresci, primeri **poruka o greskama su nemogućnost daljeg usmeravanja datagrama i zagusenje mreže**. Najvažnije ICMP poruke su: zahtev za odjekom (echo request/replay, to je pingovanje), poruka preusmeravanja (redirect), zahtev za smanjenje brzine slanja (source quench)

IGMP (Internet Group Management Protocol) - krajnji racunari koriste za prenos informacija o pripadnosti grupama, krajnji racunar radi u dve faze... Na pocetku on salje IGMP poruku za prijavljivanje u datu grupu, tu poruku prima IGMP usmerivac. Posto je pripadnost grupi privremena, lokalni IGMP usmerivac periodicno poziva (verovatno pinguje) krajnje racunare radi provere pripadnosti. IGMP ima podrsku za multicast funkcionalnost.

Nacin dodeljivanja IP adresa - statick i dinamicki

Staticko dodeljivanje znaci da se IP adresa rucno podesava za svaki racunar, dok dinamicko dodeljivanje koristi protokol kao sto je DHCP(Dynamic Host Configuration Protokol) za automatsku dodelu IP adresa.

DHCP protokol

Koristi se za automatsku dodelu IP adresa racunarima na mrezi. Uloga DHCP protokola je da osigura da svaki racunar na mrezi ima jedinstvenu i validnu IP adresu. Karakteristike DHCP protokola su: automatizacija, fleksibilnost i podrška za podesavanje drugih parametara mreze, kao sto su DNS i gateway adresa

DHCP - dobijanje novih informacija

DHCP moze da vrati ne samo dodeljenu IP adresu u podmrezi, vec i: adresu podrazumevanog rutera(default gateway), ime i IP adresu DNS servera, podmreznu masku(koja je indikator mreznog prefiksa).

Opis koraka dobijanja IP adrese putem DHCP protokola

1. Racunar salje zahtev za IP adresu(DHCP Discover)
2. DHCP server odgovara sa potvrdom i ponudom IP adrese(DHCP offer)
3. Racunar potvrđuje primljenu ponudu (DHCP Request)
4. DHCP server potvrđuje da je IP adresa dodeljena (DHCP Ack)

Standardni aplikacioni protokoli:

TELNET i RLOGIN - podrška za udaljeni pristup i rad na nekom racunaru

SMTP(Simple Message Transfer Protocol) - protokol pomocu kog klijent salje elektronsku poruku nekom serveru. Gmail na primer u okviru njihovog sistema mozda koristi neke drugacije, ne standardne, protokole medjutim SMTP je standard za slanje elektronskih poruka van sistema. SMTP serveri uglavnom koriste TCP i port 25. Postoje tri glavne komponente u okviru elektronske poste, a to su: sam citac, server i SMTP. Citac unosi postu koju salje na server, ali takodje je i cita sa servera ili sa svoje masine. Mail se sastoji od zaglavlja i samog tela. U zaglavlju se potpisuje posiljaoc, navodi primaoc i tema same poruke sto je opciono. Telo predstavlja sam text.

Protokoli za pristup e-posti

POP3 (Post Office Protocol, version 3)- protokol pomocu kojeg se prihvata posta sa SMTP servera i cuva na klijentskoj strani, STATELESS je i Bob ukoliko promeni PC nece moci da procita ponovo mail.

IMAP (Internet Message Access Protocol) - takodje se koristi za prihvatanje poruka sa SMTP servera. Glavna razlika izmedju IMAP i POP3 protokola je to sto IMAP ne cuva same poruke na lokalnoj masini dok POP3 radi upravo to. IMAP jednostavno sinhronizuje (i STATEFULL je) nase prijemno sanduce sa SMTP serverom dok POP3 skida poruke na nas racunar.

HTTP (HyperText Transfer Protocol) - protokol za pretragu WEB stranica (W3). Protokol definise na koji nacin su poruke formatirane i na koji nacin se one odasilju, takodje govori kako odredjeni pretrazivaci i web serveri treba da reaguju na odredjene komande. Na primer sam URL(Uniform Resource Locator) predstavlja HTTP komadnu koja govori serveru koju stranicu da vrati pretrazivacu. Postoje dve verzije ovog protokola, HTTP i HTTPS(HyperText Transfer Protocol Secure) koji jednostavno komunikaciju izmedju klijenta i servera enkriptuje koristeći SSL (Secure Sockets Layers) ili

TLS (Transport Layer Security). HTTP je Stateless Protocol što znaci da se svaka komanda izvršava nezavisno bez ikakvog znanja o komandi koja je bila izvršavana pre nje, zbog toga je dosta tesko implementirati neki vid "inteligencije" radi lakse komunikacije sajta sa korisnikom. Zbog toga se sam HTTP standard proširuje raznim tehnologijama kao što su kolačići (čuvanje nekih određenih podataka o korisniku radi bolje personalizacije sadržaja i brzih odgovora, ograničena veličina kolačića (4KB) kao i broj po domenu), Java skriptice itd... Kod HTTP-a postoje dva tipa poruka (komande GET i POST), zahtevi (requests) ono što šalje korisnik i response (odgovor servera na zahtev). HTTP podržava rad sa greskama, odnosno omogućava laku identifikaciju problema uvođenjem kodova gresaka, na primer 404 greska znaci da server nije uspeo da pronađe fajl koji je korisnik zahtevao. Dva tipa konekcije, Non-Persistent i Persistent. Verzija 1.0 (Get, Post, Head), 1.1 (dodaje se PUT-upload i DELETE), 2.0 neka kompresija.

FTP (File Transfer Protocol) - Uspostavlja dva TCP kanala (default portovi su 20 za prenos i 21 za kontrolu veze) između korisnika i servera, omogućava prenos podataka (**datoteka**) na server. Samo autorizovani korisnici imaju pravo pristupa određenom serveru. Koristi se DNS kako bi se razresilo ime servera na koji se povezujemo. FTP je STATEFULL protokol (pamti stanje svog klijenta, direktorijum gde se nalazi i prethodnu autentifikaciju, podrška za rad sa više korisnika), po završetku slanja datoteke konekcija se zatvara i za novu ponovo otvara. Takođe podržava povratne/statusne kodove, can't connect. Koristi se **u kombinaciji sa TCP**.

Adrese (Fizicka i Logicka) - Postoje dva tipa adresa u mrezi, IP (logicka adresa) i MAC (Media Access Control) fizicka adresa. Logicka adresa se koristi u mrežnom sloju dok se MAC adresa koristi u link sloju. MAC adresa predstavlja svaki fizicki uređaj u mrezi. Hello protokol šalje svima Hello poruku, svi mu odgovaraju tako dobija MAC adresu, takođe se uređaju periodičnu javljaju kako bi znao da li su živi).

ARP (Address Resolution Protocol) - Prvobitno se proveriti u listi da li postoji uređaj sa određenom IP adresom, ukoliko da paket se prosledjuje na odgovarajuću MAC adresu uređaja. Ukoliko ne upućuje se ARP (broadcast-svima) zahtev kako bi se identifikovao uređaj kome je potrebno poslati neki paket. Lista sadrži parove IP-MAC adresa uređaja.

Služe za poboljšanje kvaliteta i sigurnosti prenosa podataka u komunikacionim mrežama. Koriste se za automatsko ponavljanje zahteva za prenos podataka kada se pojave greske u prenosu.

Funkcionalnosti: detekcija gresaka, korekcija gresaka, potvrda primitka, automatsko ponavljanje zahteva, kontrola propusnosti. ARP protokoli su korisni u mrežama gde se očekuje da će biti gresaka u prenosu podataka, kao što su bezicne mreze i mreze sa slaboscu signala.

WEB-Cash (Kesiranje) - Uvođenje posrednika, kopije između klijenta i samog servera. Omogućava smanjenje direktne komunikacije sa serverom i brzi pristup samom materijalu (stranicama). Smanjuje se saobraćaj ka globalnoj mrezi, **instalira ga ISP (internet service provider). Kes server (proxy) se ne ponasa samo kao server nego i kao klijent**. On ukoliko ne poseduje potreban sadržaj komunicira sa serverom dobavlja ga i potom čuva ukoliko je to moguće. Takođe komunicira sa serverom kako bi proverio samu verziju materijala koju poseduje jer je web sadržaj promenljiv (datum se navodi u HTTP zahtevu).

Veb kesiranje(proxy server) - web kesiranje je proces koji pohranjuje kopiju web stranice na lokalnom računaru (hard disk) ili serveru (proxy server) kako bi se ubrzalo učitavanje stranice prilikom

narednih poseta. Uslovni GET je metoda koja se koristi da se proveri da li je lokalna kopija web stranice azurirana.

Proxy server je resenje kojim se tezi rasterecenju originalnih servera, smanjenju saobracaja ka globalnoj mrezi i unapredjenju performansi interneta na nacin da uredjaji kada traze neki dokument prvo se obracaju lokalnom proxy serveru(kojeg instalira ISP, univerzitet i sl.) koji u sebi ima kesirane dokumente iz ranijih potrazivanja pa ako je trazeni objekat vec u cache-u proxy ga odmah isporucuje(ne izlazi se dalje u mrezu), a ako ga nema obraca se originalnom serveru koji mu ga isporucuje, on taj objekat skladišti kod sebe ako ga neko bude opet trazio i naravno klijentu dostavlja dokument koji je od proxy-ja u pocetku i trazene. Proxy se ponasa i **kao server(kada se desi cache hit) i kao klijent(cache miss pa onda on od originalnog servera trazi dokument)**. Komanda za uslovni pristup (Conditional Get) je nastala kao posledica toga sto je sadrzaj web stranica promenljiv pa se ne zna da li se na proxy-ju nalazi vremenski zadovoljavajuca ili zastarela kopija dokumenta pa proxy server u svome zahtevu na neki dokument navodi da mu se posalje nova verzija dokumenta ako je modifikovan nakon datuma navedenog u zahtevu, ako odgovor izostane znaci da je kopija dokumenta na proxy-ju bila validna, a ako originalni server otkrije da on ima noviju verziju od navedenog salje je proxy serveru, a on je usvaja.

--

U vecini slucajeva, moderne web pregledace imaju automatsko upravljanje keširanjem koje optimizuje memoriju i disk prostor koji se koristi za keširanje, cesto se oslanjaju na algoritme poput "stranica zamene" (stranice koje se rede koriste se zamjenjuju novim podacima). Ako se podaci kesiraju na hard disku PC-ja oni se brisu nakon gasenja racunara.

Ethernet - Dominantna LAN tehnologija, jeftino, jednostavno i robusno resenje (naravno uz NIC - Network Interface Controller ili mreznu karticu). **Ethernet je pristupna mreza koja se koristi za povezivanje racunara ili uredjaja preko kabela**. Ethernet je uspesniji od rivala Token Ring-a (MAU- Media Access Unit, topologija prstena najcesce, tokeni). Ethernetovo ubrzanje prenosa je 10Mbps do 100Gbps. Najcesce koristen u LAN (Local Area Network), WAN(Wide Area Network) i MAN(Metropolitan Area Network) izvedbi. Standardizovani 80ih. Na pocetku ethernet je koristio koaksialne kable kao prenosni medijum, dok danas se koristi varijanta sa paricama (twisted pair) kao i opticki medijumi. Sistemi koji komuniciraju koriste Ethernet dele podatke na manje delove koji se zovu frejmovi, svaki frejm sadrzi source i destination adresu kao i proveru da li je podatak uspesno i celokupno prenesen (checksum metod). Struktura Ethernet okvira je sledeca: adresa source i dest kao sto sam spomenuo gore, tip protokola koji ocekujemo da je sledeci, crc provera po prijemu.

CSMA/CD algoritam

Prihvata datagram sa mreznog nivoa i formira okvir za slanje. Oslusne kanal i ako je miran zapocne slanje, u suprotnom ceka da se oslobodi i pokusa ponovo kasnije. Ako u toku slanja celog okvira ne utvrdi koliziju slanje je završeno.

Logicka segmentacija(VLAN - Virtuelni LAN)

Kod mreza koje koriste mostove, tj. komunikatore kada se paket broadcastuje, salje svima moze doci do zagusenja posto svi dele isti broadcast domen. Zbog toga se mreza deli, segmentira na manje delove kako bi se zagusenje izbeglo. Broadcast paket se ne prelijeva van svog segmenta. VLAN je skup cvorova koji su zajedno grupisani u jedan broadcast domen. 1 VLAN switch = N VLAN-ova. Razlog uvodjenja: brzina, specificne potrebe kompanija, laska kontrola saobracaja mreznog administratora(lista za pristup).

Napadi na internetu/mrezama:

Packet sniffing, IP spoofing, record and playback, DOS(Denial of Service), Crv, trojanac, malweri, spyware, wireshart prisluškivanje, presretanje

Malware - neprijateljski program koji ometa normalan rad na racunaru. Cesto se samostalno sire i repliciraju preko zarazene mreze zombi racunara(botnet).

Virus - zahteva aktivnu radnju korisnika npr. da otvori fajl u prilogu mail-a, replicira se zapisom na lokalnom racunaru i dalje se siri preko mreze

Worm - pasivna infekcija, aktivira se sam po prijemu na napadnuti racunar i sam se replicira dalje

Trojanski konj - skriveni deo nekog legitimnog softvera

Spyware - spijunira i snima razlicite akcije korisnika

Adware(Spam) - slanje brojnih nezelenih poruka, pop-up prozori, oglasi, reklame

DoS - onemogucava uslugu na nacin da napadaci preuzimaju mrezne resurse(zlonamerni racunari konstantno spamuju server sa zahtevima za uslugu) i time ih cine nedostupnim za legitimne korisnike. Ne moze se spreciti jer je ranjivost sama koncepcija pruzanja usluga u klijent-server modelu.

DDoS je specifičan oblik DoS napada koji koristi distribuirane resurse cesto zarazene bot mrezom kako bi stvorio opterecenje i onemogucio rad sistema, dok se DoS odnosi na napade koji imaju za cilj uskratiti pruzanje usluge ili njihovo sporije izvršavanje zbog preopterecenja.

Packet sniffing - prisluškivanje saobraćaja unutar deljenog ethernet-a i wireless mreza gde racunar u mrezi moze primiti sve pakete i snimiti sadrzaj(sto mu koristi ako sadrzaj nije enkriptovan)

IP spoofing - slanje anketa sa laznom source adresom(predstavljjanje kao neko drugi)

Ransomware - sifruje podatke i trazim otkup(ransom)

Rootkit - instalira se na računaru sa ciljem da se prikrije od antivirusnih programa i drugih sigurnosnih alata. On pruža napadaču privilegije "root" (administratora) nad sistemom i omogućava im neovlašćen pristup i kontrolu nad računarom.

Zastita od napada na internetu:

Autentifikacija, posiljalac i primaoc medjusobno treba da potvrde identitet jedno drugom

Poverljivost, posiljaoc kriptuje, a primalac dekriptuje(privatnim kljucem, asimetricna kriptografija)

Integritet poruke, nemoguca izmena u prenosu, neki vid verifikacije po prijemu mozda

Metode kriptografije:

1. Private/Simetricni kljuc(obe strane poseduju kljuc koji samo oni vide, a ne Trudi)

2. Javni kljuc(kljuc koji se zamenjuje)

3. Monoalfabetski kod, zamena jednog slova drugim, cezarova sifra

4. Polialfabetski kod, n pravila zamene u n koraka, napredni cezar

5. Blokovo zastitno kodiranje, neki blok se deli na manje blokce, koriste se funkcije za skremblovanje, slucajan broj puta, blokova kako brute force metodom ne bi moglo lako da se pronadje resenje. Primeri DES i AES. Ideja da svaki ulazni bit utice na broj izlaznih bitova u bloku.

Osnovni zahtevi za ukljucenje u/na mrezu su: krajnji racunar, mrezna kartica, OS, TCP/IP protokol stek

Kombinovana kriptografija(simetrichna i asimetrichna) - koristi asimetrichnu za razmenu javnog ključa i šifrovanje simetrichnog ključa, a simetrichnu za dalju komunikaciju:

1. Klijent šalje zahtev serveru za uspostavljanje sigurne komunikacije. TLS/SSL
2. Server šalje javni ključ klijentu.
3. Klijent koristi javni ključ servera da šifrue simetrični ključ koji će se koristiti tokom sesije.
4. Klijent šalje šifrovan simetrični ključ serveru.
5. Server dešifruje simetrični ključ koristeći svoj privatni ključ.
6. Klijent i server koriste simetrični ključ za šifrovanje i dešifrovanje podataka tokom dalje komunikacije.

Simetrichna kriptografija(DES i AES spadaju ovde):

1. Klijent i server dogovore se oko simetričnog ključa koji će se koristiti za šifrovanje i dešifrovanje podataka tokom sesije.
2. Klijent šalje zahtev serveru za uspostavljanje sigurne komunikacije.
3. Server šalje odgovor klijentu sa dogovorenim simetričnim ključem.
4. Klijent i server koriste dogovoreni simetrični ključ za šifrovanje i dešifrovanje podataka tokom dalje komunikacije.

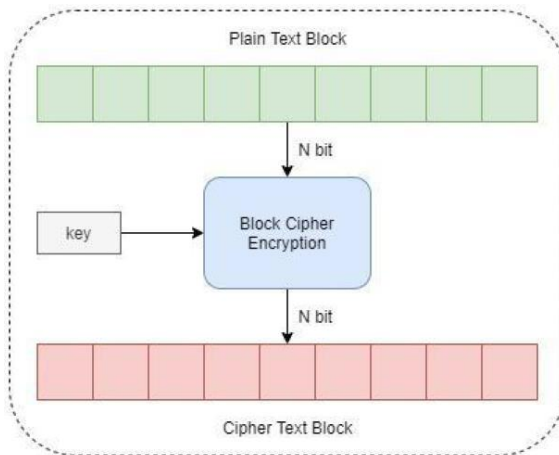
ECB(electronic codebook) i **CBC**(cipher block chaining) šifrovanje blokova podataka u kriptografiji

- **Stream cipher**

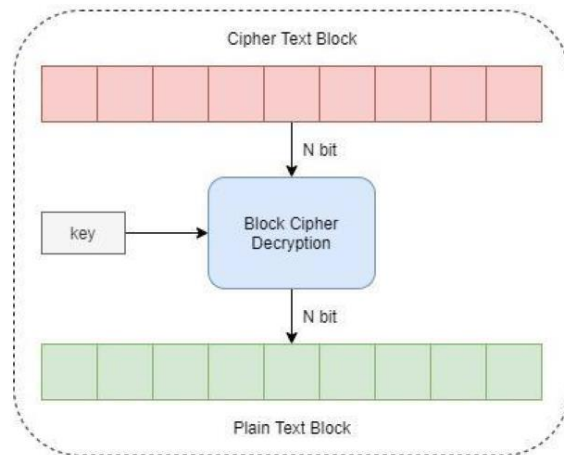
- Algoritam koji radi nad bitovima podataka. Šifrue se svaki bit poruke posebno, čime je moguće šifrovati poruke proizvoljne dužine.
- Šifrovanje se vrši bit po bit primenom operacije XOR ("ekskluzivno ili") sa odgovarajućim bitom *keystream*, a dobijena šifrovana poruka će biti iste dužine kao i *plaintext*.
- Postupak dekriptovanja se vrši na isti način - primenom XOR operacije između bitova šifrovanog teksta i keystream vrednosti.
- Najpoznatiji algoritam ovog tipa je RC4

- **Blok cipher**

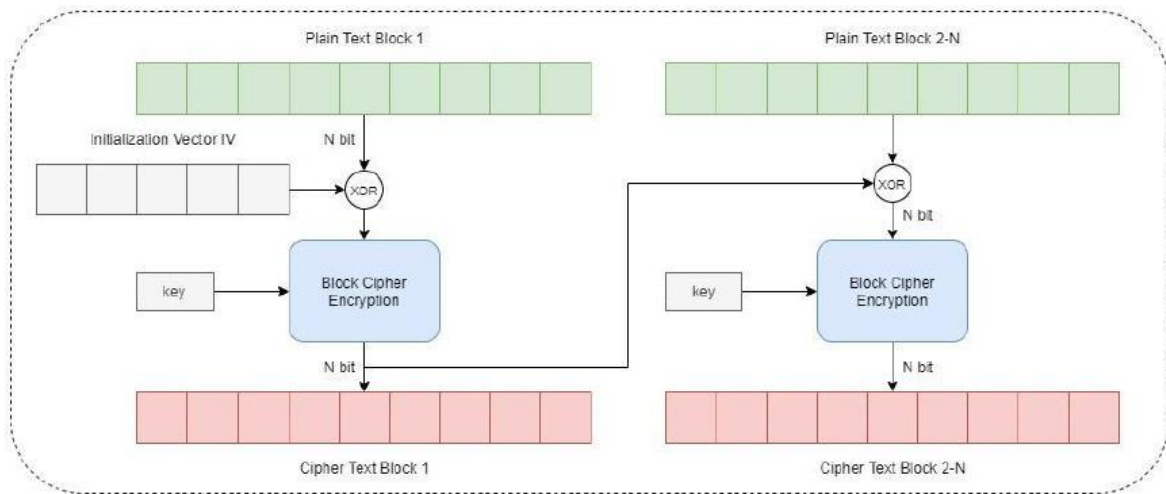
- Rade nad blokovima podataka fiksne dužine (poruka se deli na n-bitne blokove, a ukoliko je poslednji blok manji dopunjava se tako do n bita).
- Nad svakim blokom podataka se primenjuje algoritam koji je matematički dosta kompleksniji od stream cipher algoritama (kombinovane operacije zamene i permutacije).
- Algoritam se primenjuje u iterativnim postupcima, odnosno rundama.
- Rezultat je takođe blok podataka iste dužine kao i ulazni blok.
- Dva tipična moda block cipher algoritama su:
 - ECB - Electronic Codebook mod (slika 11)
 - CBC - Cipher Block Chaining mod (slika 12)
- Neki od poznatijih algoritama su AES, DES, 3DES



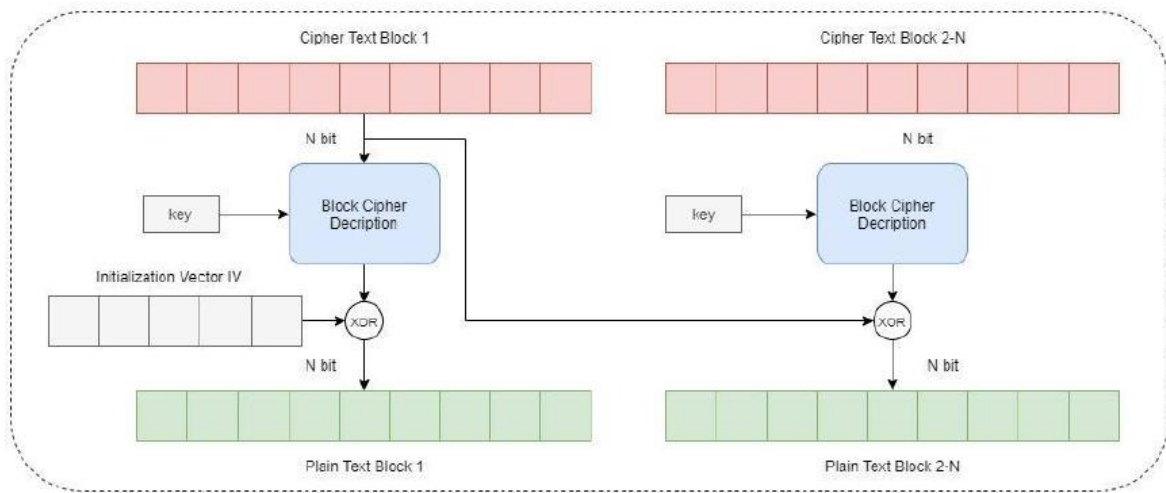
Postupak Kriptovanja ECB
Block Cipher Algoritma



Postupak Dekriptovanja ECB
Block Cipher Algoritma



Postupak Kriptovanja CBC Block Cipher Algoritma



Postupak Dekriptovanja CBC Block Cipher Algoritma

DES algoritam uzima blok podataka od 64 bita i koristi ključ od 56 bita za generisanje šifrovane poruke koja je takođe dužine 64 bita. Postupak se ponavlja 16 puta sa različitim ključevima kako bi se generisao kriptografski jaka šifra. Dekripcija se vrši na isti način samo obrnuto - šifrovana poruka se dešifrira uz pomoć istog ključa i postupka.

AES, s druge strane, koristi blokove podataka od 128 bita i može koristiti ključeve dužine 128, 192 ili 256 bita. Algoritam radi u nekoliko faza. U prvoj fazi se ključ koristi za generisanje tzv. Round Keys, koje se kasnije koriste u procesu enkripcije. Zatim se blok podataka kombinuje sa Round Keys korišćenjem posebnih matematičkih funkcija. Postupak se ponavlja u nekoliko rundi u zavisnosti od dužine ključa. Dekripcija se vrši na sličan način samo obrnuto.

Sigurnost se postiže korišćenjem ključeva koji su teški za probijanje i ponavljanjem procesa enkripcije/dekripcije u više rundi. AES je zastupljeniji.

Simetrična kriptografija se zasniva na korišćenju istog ključa za enkripciju i dekripciju podataka. Ako se ključ nekako otkrije, onda napadač može lako da dešifrira sve podatke koji su enkriptovani tim ključem. Zato je bitno da se ključevi čuvaju na sigurnom mestu i da se koriste samo kada je to neophodno. Kada se koristi na pravi način, simetrična kriptografija je vrlo sigurna i efikasna za zaštitu podataka. Brza je efikasnija za velike količine podataka.

Asimetrična kriptografija koristi dva različita ključa - javni i privatni. Javni ključ se koristi za enkripciju podataka, dok se privatni ključ koristi za dekripciju. Javni ključ se može javno objaviti, dok se privatni ključ čuva u tajnosti. Asimetrična kriptografija je vrlo sigurna jer napadač ne može da dešifrira podatke bez privatnog ključa. Međutim, ovaj pristup je mnogo sporiji od simetrične kriptografije, posebno kada su u pitanju veliki blokovi podataka. Korisna kada postoji potreba za sigurnom razmenom ključeva, potpisivanjem digitalnih dokumenata i sličnim aktivnostima.

Klijenti enkriptuje poruke javnim ključem servera, a server ih dekriptuje svojim privatnim. Server enkriptuje poruke za klijenta klijentskim javnim ključem, a klijent ih dekriptuje svojim privatnim.

JWT:

Prilikom korišćenja asimetričnih ključeva, server koristi svoj privatni ključ za potpisivanje JWT tokena, dok klijent koristi javni ključ servera za verifikaciju ispravnosti potpisa. Kada klijent dobije JWT token, on koristi javni ključ servera (koji je prethodno dobio na siguran način) za verifikaciju ispravnosti potpisa tokena. Na taj način klijent može biti siguran da je token validan i da je izdat od strane servera koji poseduje odgovarajući privatni ključ. Jedan od uobičajenih načina za klijenta da dobije javni ključ servera jeste putem SSL/TLS (Secure Sockets Layer/Transport Layer Security) protokola. SSL/TLS obezbeđuje sigurnu komunikaciju između klijenta i servera putem enkripcije i autentifikacije. Kada se klijent poveže sa serverom **preko SSL/TLS, server šalje svoj digitalni sertifikat klijentu. Sertifikat sadrži javni ključ servera, zajedno sa drugim informacijama kao što su identitet servera i potpis koji garantuje verodostojnost sertifikata.** Klijent zatim proverava ispravnost sertifikata koristeći lanac poverenja (certificate chain) i proverava da li je izdat od strane pouzdanog sertifikacionog tela (Certificate Authority - CA). Ako se sertifikat smatra validnim, klijent ekstrahuje javni ključ servera iz sertifikata i koristi ga za enkripciju podataka koji su namenjeni serveru ili za verifikaciju potpisa JWT tokena koje je dobio od servera.

TCP/IP arhitektura, komunikacija i stek

Slojevit protokol(network, internet, transport, application). Sloj je podsistem unutar steka sa zadatim odgovornostima. Aplikacija ne mora da zna nista o nacinu usmeravanja, usmeravanje se obavlja u donja 3 nivoa, TCP/IP je modularan protokol. TCP/IP stek se sastoji od mnogo protokola, najvazniji su TCP, UDP i IP.

DNS(Domain Name System) predstavlja jednu od osnovnih komponenti interneta i u osnovi je sistem koji pretvara imena racunara(hostnames) u IP adrese. Kada se ukuca www.google.com nas racunar ce poslati zahtev DNS serveru trazeci podatke o IP adresi koja je povezana sa domenom google.com. DNS server potom prevodi URL(Uniform Resource Locator) u IP adresu, odnosno, povezace se sa trazanim domenom i uputiti nas na trazenu adresu. Name serveri su ti koje svi korisnici na internetu pitaju "gde je taj sajt". Da bi domen bio aktivan neophodno je da bude vidljiv na javnom DNS serveru, a da bi funkcionisali svi servisi pod tim domenom(sajt, email i drugo) neophodno je da nadlezni name serveri tacno znaju gde se ti servisi nalaze, tj. na kojim IP adresama. Top level domain je drzava, second level domain je ime pod kojim se neka firma moze registrovati.

Primer: neki.sajt.rs -> [neki.sajt](http://neki.sajt.rs)(second level domen), [.rs](http://neki.sajt.rs)(top level domen)

DNS server se menja podesavanjima rutera.

Kada se korisnik poveze na internet njegov racunar je dobio IP adresu od provajdera. Kada korisnik zatrazi pristup veb-sajtu. DNS server ISP-a ce se koristiti da bi se prevodilo ime domena u IP adresu koja je neophodna za pristup sajtu. ISP-i takodje odrzavaju svoje DNS servere koji sadrze informacije o razlicitim sajtovima, kako bi se omogucilo brzo i efikasno pretrazivanje i pristupanje tim sajtovima. Ovi serveri su povezani sa drugim DNS serverima sirom sveta kako bi se obezbedilo da se informacije o imenima domena brzo i efikasno pronalaze i prevode u IP adrese.

DNS je distribuirana baza podataka implementirana hijerarhijskim skupom name server-a. Upotreba ovog sistema se odvija po istoimenom DNS aplikativnom protokolu koji definise komunikaciju izmedju uredjaja i name servera zarad razresenja referenci oko imena. DNS za unesen hostname(znakovni oblik) prevede u IP adresu(radi i obrnuto). Pruza mogucnost reimenovanja host racunara(ima primarno kanonicko, ali i alias ime), reimenovanje mail servera kao i mogucnost raspodele opterecenja(umnozavanje web servera i vezivanje skupa IP adresa za jedno kanonicko ime - jedan od primera bi bila logika da preusmerava na najblizi web server iz tog skupa IP adresa). DNS serveri se organizuju hijerarhijski: najvisi nivo su DNS Root serveri, nize se nalaze com, org i edu DNS serveri koji se opet granaju na nize DNS servere. Postuju pravilo ako visi DNS server ne poznaje mapiranje, pristupa se nizem DNS serveru.

Npr. kada klijent trazi www.facebook.com obraca se root serveru da nadje com DNS server na kome ce traziti dalje facebook.com DNS server na kome ce pak traziti IP adresu od www.facebook.com.

Lokalni DNS server - racunar koji pohranjuje informacije o domenima u lokalnoj mrezi i omogucuje lokalnim korisnicima da pristupaju internetu. Uloga mu je da olaksa rad korisnicima tako sto skracuje vreme potrebno za dobijanje odgovora od DNS servera i smanjuje opterecenje na internet.

DNS (Domain Name System) - Predstavlja hijerarhijski decentralizovan sistem za imenovanje racunara, servisa i generalno raznih resursa povezanih na internet ili neku privatnu mrezu. Asocira razne informacije sa DOMENOM (nama citljivom adresom na internetu). Omogucava prevodjenje nama citljivim adresama u racunaru razumljive adrese (numericke IP adrese). Postoje TLD (Top-Level Domain) serveri koji su odgovorni za com,org,edu kao i nacionalne domene, AUTH serveri, svaka organizacija sa javno dostupnim mail serverima mora obezbediti DNS servis sa podacima o njima, njega odrzava sama organizacija ili ISP. Svaki ISP ima default name server sto je prvi lokalni DNS kome racunar salje neki upit, ako on nema pojma salje taj upit sledecem DNS serveru! Takodje i ovde postoji kesiranje kako pristupanje root serverima ne bi bilo cesto. DNS koristi query i reply porukice.

Provajder ili internet provajder(ISP) je kompanija koja pruza usluge povezivanja sa internetom. U te usluge spadaju e-mail, web-hosting, virtuelni privatni serveri...

IKP

Internet je globalna mreza racunara koja omogucava razmenu informacija i usluga putem razlicitih tehnologija i protokola. Gradivni elementi interneta su: racunari, mrezni uredjaji, protokoli i infrastruktura(kablovi, sateliti...)

Operativni sistem resava problem brzine na racunaru (konkretno za RAM)

Mrezni protokoli su pravila i standardi koji regulisu komunikaciju izmedju racunara na mrezi.

Tipovi prenosa(adresiranja) u racunarskoj mrezi:

Unicast 1:1 -> 1 posiljalac i 1 primalac
Broadcast 1:ALL -> 1 posiljalac ka svima u mrezi
Multicast 1:N -> 1 posiljalac ka vise primalaca
Anycast 1:1 of N -> 1 posiljalac ka 1 od vise primalaca

Klijent-server model - jedan racunar(server) sluzi drugim racunarima(klijentima) koji traze usluge ili informacije.

Peer-to-peer - svi racunari su jednaki i mogu sluziti i kao klijenti i kao serveri.

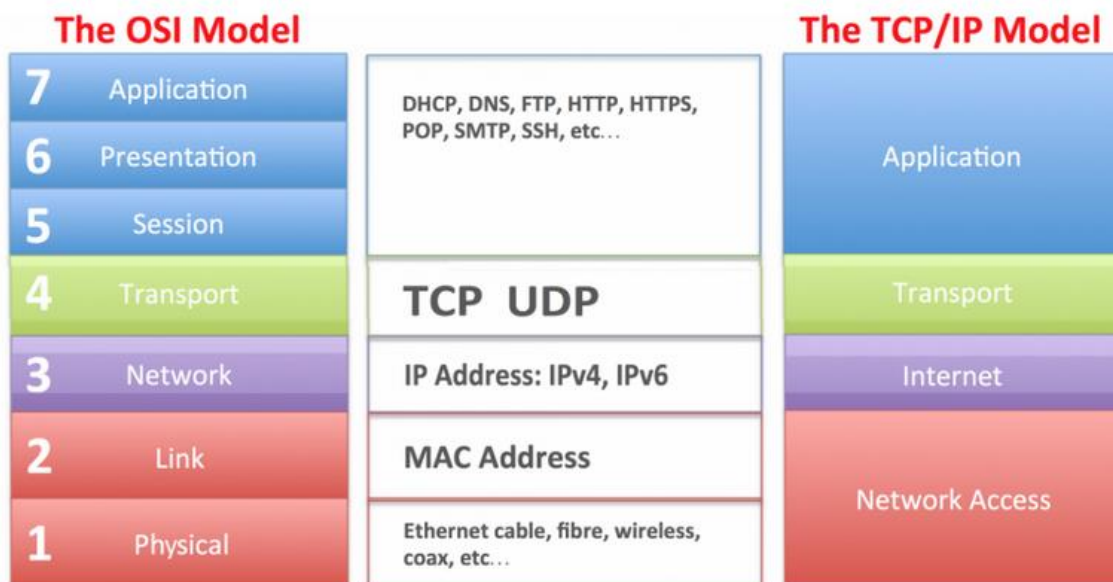
Internet protokol stek - razlika njega i ISO/OSI

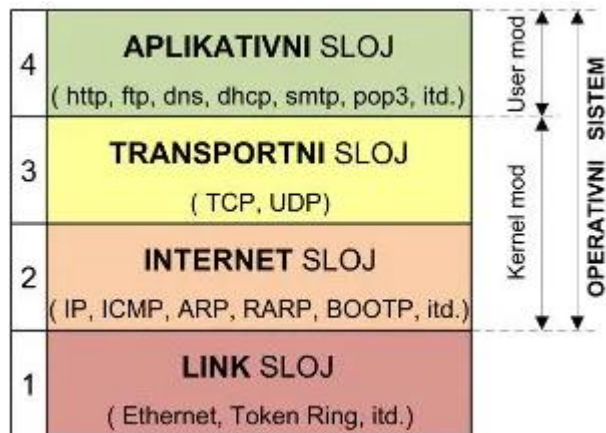
Internet protokol stek je skup protokola koji se koriste za komunikaciju na internetu.

Nivoi su:

4. Aplikacijski nivo(HTTP, FTP,...)
3. Transportni nivo(TCP, UDP)
2. Internet(IP)
1. Sloj mreznog pristupa(bluetooth, wifi, ethernet)

Razlika je u tome sto internet protokol stek ima manje nivoa i fokusira se na prakticna resenja za komunikaciju na internetu, dok ISO/OSI sluzi kao teorijski okvir za razumevanje mreznih komunikacija





Enkapsulacija paketa je proces u kojem se podaci iz jednog nivoa protokola steka prekriva(enkapsulira) u podatke drugog nivoa. Princip enkapsulacije je da svaki nivo protokola dodaje svoj nivo podataka(npr. adresu kontrolne informacije) na podatke prethodnog nivoa pre prenosnja preko mreze.

ISO/OSI model

Nedostatak prvih mreza sa komutacijom paketa je bio to sto su samo cvorovi istog proizvođača mogli međusobno da komuniciraju. Resenje problema ponudjeno je u obliku ISO/OSI referentnog modela. Model je napravljen u obliku slojevite arhitekture, svaki sloj ima sopstvene obaveze. OSI je hijerarhijska i modularna arhitektura, gde su slojevi slabo povezani.

Physical , data link, network slojevi => media layers

Transport, session, presentation, application => host layers.

Application, presentation, session - data

Transport - segment

Network - packet

Data link - frames

Physical - bits

Nivoi protokola predviđeni **ISO OSI** referentnim modelom:

1. Fizicki - prilagodjenje karakteristikama fizickog medijuma, prenos bita ili reci
2. Nivo veze(Data link) - pouzdan prenos podataka, uspostava veze, kontrola toka, reasembliranje poruka. Sluzi za direktno komuniciranje sa mreznim hardverom/karticom.
3. Mrezn - logicki model mreze, usmeravanje paketa kroz mrezu. Zaduzen za usmeravanje paketa kroz mrezu, takodje moze da upravlja grupama paketa(multicasting). Kljucne funkcije su adresiranje, rutiranje i fragmentacija i reassembly paketa, kontrola protoka i prevodjenje mreznih adresa(NAT). Najpoznatiji protokoli mreznog sloja interneta su: IP i ICMP. **Rutiranje** je proces usmeravanja izmedju mreznih cvorova. Vrsi se na ruteru, a odnosi na mrezne pakete.
4. Transportni - pouzdane transportne usluge sa-kraja-na-kraj i bavi se fragmentacijom paketa. Odgovoran je za kontrolu toka podataka/datagrama izmedju dva krajnja racunara(host, terminal, edge). Informaciju koju primi od same aplikacije on oblikuje na odgovarajuci nacin i prosledjuje je mreznom sloju, ali takodje na primaocu on poruku otpakuje i prosledjuje aplikaciji. Usluge protokola transportnog sloja su komunikacije izmedju aplikacije i mreznog interfejsa na racunaru. Mehanizmi za: uspostavljanje veze, upravljanje tokom, kontrolu gresaka, kontrolu protoka i prenos podataka izmedju aplikacija na razlicitim racunarima.
5. Nivo sesije - uspostava, rukovanje i prekid veze

6. Prezentacioni - oblik predstavljanja informacije

7. Aplikativni - podrška komunikaciji aplikaciji preko poruka. Predstavlja zajednicku spregu preko koje korisnicke aplikacije komuniciraju sa donjim slojevima, predstavlja interface izmedju aplikacija i mreze

Sloj veze

Pravljenje okvira - enkapsulira datagram u okvir, dodajuci mu zaglavlje sloja veze. Struktura zaglavlja odredjuje protokol sloja veze

Pristupanje linku - protokol za kontrolu pristupa medijumu(media access control, MAC) ako je kanal deljen. Fizicke, MAC adrese se koriste u zaglavlju okvira da bi se identifikovao posiljalac i primalac.

Pouzdana isporuka izmedju susednih cvorova - retko se koristi na linkovima sa malom verovatnocom greske(opticki kabel, upredene parice). Obicno se koristi za bezicne linkove, ali je tu veca verovatnoca gresaka.

Detekcija gresaka - greske izazvane slabljenjem signala, sumom pri prenosu. Prijemna strana detektuje postojanje gresaka. Moze da javi posiljaocu radi retransmisije ili da odbaci okvir sa greskom

Korekcija gresaka - prijemnik prepoznaje i ispravlja bitske greske bez potrebe zahteva za retransmisijom

Mrezni sloj

Transport segmenta od primaoca do posiljaoca. Na strani posiljaoca enkapsulira transportne segmente u IP datagrame i zatim ih salje ka najblizem ruteru. Na strani primaoca, mreznii sloj prima datagrame od najblizeg rutera, izdvaja segment mreznog sloja i isporucuje ga transportnom sloju.

Protokoli mreznog sloja postoje u svakom hostu i ruteru. Ruteri ispituju polja zaglavlja svih IP datagrama koji prodju kroz njih.

Transportni sloj

Usluge su:

1. Multipleksiranje i demultipleksiranje - usluga isporuke poruka izmedju dva procesa.

Multipleksiranje kod pošiljaoca: preuzima podatke sa više soketa, dodaje transportno zaglavlje koje se kasnije koristi za demultipleksiranje.

Multipleksiranje je tehnika prenosa vise podataka preko jednog komunikacionog kanala.

Demultipleksiranje je razdvajanje razlicitih signala koji su poslali istim kanalom i usmeravanje na odgovarajuće primaocce.

Multipleksiranje kod primaoca: koristi oznaku porta u zaglavlju primljenog segmenta da bi ga usmerio ka ispravnom sokuu.

- host prima IP datagram
 - Svaki datagram ima IP adresu pošiljaoca, IP adresu primaoca
 - Svaki datagram prenosi jedan segment transportnog sloja
 - Svaki segment ima broj izvornog porta (source port) i broj odredišnog porta (dest port)
- host koristi IP adrese i brojeve portova da bi usmerio segment ka odgovarajućem sokuu

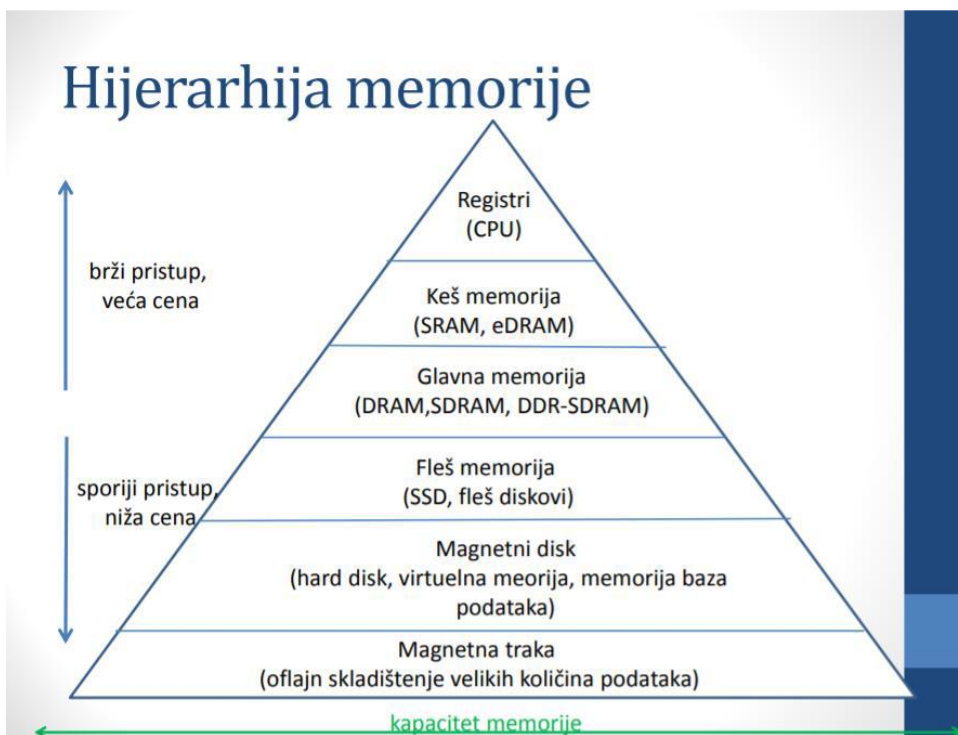
Važni parametri demultipleksiranja bez uspostave veze su: IP adresa odredišta I broj porta odredišta

Važni parametri demultipleksiranja sa uspostavom veze su: IP adresa izvornog računara, broj izvornog porta, odredišna IP adresa, broj odredišnog porta

2. Privera integriteta poruka - otkrivanje gresaka u prenosu

Metode pristupanja podacima memorije

1. Sekvencijalno - pristupanje mora da se obavlja odredjenim linearnim redom; memorija je organizovana u jedinice podataka tzv. zapise; informacije o adresama se koriste za razdvajanje za razdvajanje zapisa; zajednicki mehanizam za citanje i upisivanje pri cemu se vrši pomeranje sa trenutnog na zeljeni položaj preskakanjem svih zapisa između; vreme pristupa je promenljivo; magnetne trake koriste sekvencijalno pristupanje
2. Direktno - zajednicki mehanizam za citanje i upisivanje; pojedinačni blokovi ili zapisi imaju jedinstvenu adresu na osnovu fizicke lokacije; pristupanje se obavlja direktnim pristupom blizu traženih podataka, a zatim se pretražuje kako bi se stiglo do konacne lokacije; vreme pristupa je promenljivo; koriste ga magnetni diskovi
3. Nasumicno pristupanje - svaka lokacija u memoriji koja može da se adresira ima zaseban fizicki oziceni mehanizam za adresiranje; vreme pristupa za odredjenu lokaciju zavisi od prethodnog pristupa i nepromenljivo je; bilo koja lokacija može da se direktno adresira i da joj se direktno pristupi; koriste ga glavna memorija i neki sistemi keš memorija
4. Asocijativno pristupanje - tip memorije sa nasumicnim pristupom, poredjenjem željenih lokacija bita unutar reci pronalazi se podudarnost(ovo se radi za sve reci istovremeno); rec se preuzima na osnovu dela njenog sadržaja, a ne na osnovu njene adrese; vreme pristupa odredjenoj lokaciji ne zavisi od lokacije i prethodnik pristupa tj. nepromenljivo je; koriste ga keš memorije.



Vremenska lokalnost - sklonost programa da u bliskoj budućnosti poziva one jedinice memorije koje je pozivao u neposrednoj prošlosti

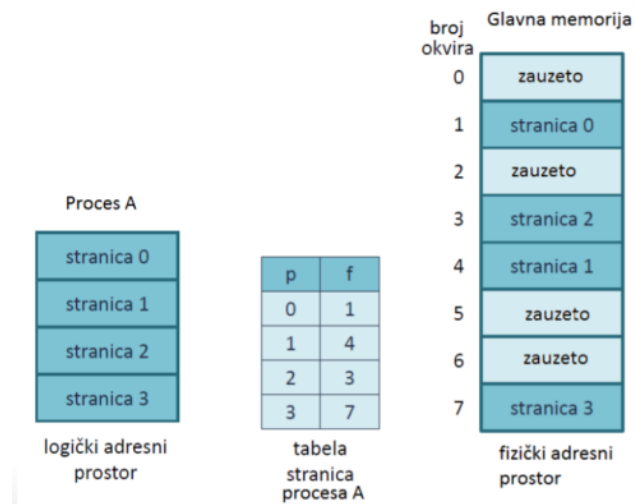
Prostorna lokalnost - sklonost programa da pristupa jedinicama memorije čije su adrese jedna blizu druge tj. održava sklonost procesora da instrukcijama pristupa redom kao i da lokacijama podataka pristupa redom(npr. obrada tabele podataka)

Logicka adresa - izrazava se kao relativni položaj u odnosu na pocetak programa; instrukcije u programu sadrže samo logicke adrese; logicke adrese su generisane od strane procesora dok se program izvršava; naziva se jos i virtuelna adresa jer ona fizicki ne postoji; logicki(virtualni) adresni prostor je skup svih logickih adresa koje generise jedan program

Fizicka adresa - stvarna adresa u memoriji; identifikuje fizicku lokaciju u memoriji; procesor pri izvršavanju procesa automatski pretvara logicke adrese u fizicke adrese tako sto na svaku logicku adresu dodaje trenutni pocetni položaj procesa koji se naziva osnovna adresa(base address)

Jedinica za upravljanje memorijom(Memory Management Unit, **MMU**) - preslikava logicke adrese u fizicke prilikom izvršavanja programa

Stranice - memorija se deli na jednake parvice nepromenljive velicine koji su relativno mali(okviri ili okviri stranice(page frames)); svaki proces se deli na parvice iste nepromenljive velicine(to su stranice(pages)); tada te parvice programa tj. stranice smestamo u raspolozive okvire; OS drži spisak slobodnih okvira



Logicka adresa obuhvata: broj stranice i relativnu adresu unutar te stranice. Procesor obavlja prevodjenje. Prvo pristupa tabeli stranica datog procesa da bi za dati broj stranice saznao broj okvira u koji je stranica smestena. Zatim pristupa podatku u glavnoj memoriji koristeći dati broj okvira memorije i relativnu adresu unutar stranice(okvira).

Segmentacija memorije - vidljiva za programere i nudi se kao pogodnost za organizovanje programa i podataka; svakom segmentu mogu da se dodele prava za pristupanje i koriscenje; program se moze posmatrati kao skup razlicitih logickih celina - segmenata

Heap manager i dinamicka alokacija memorije

`void* malloc(byteSize)` - preuzima slobodan blok

`void free();` - oslobadja blok

Greske: pristup oslobodjenoj memoriji, curenje memorije, oslobadjanje istog bloka vise puta, oslobadjanje neispravnog pokazivaca, prekoracenje bafera

Garbage collector(GC - sakupljac otpadaka) - sve celije koje nisu aktivne pretvara u slobodne, aktivni su svi objekti do kojih se može stići polazeći od svih postojećih korenih pokazivača

Oslobađanje memorije i promovisanje objekata u .NET Framework-u
(koristi strukturu podataka -> graf objekata)

- GC oslobađa memoriju postupno po generacijama umesto oslobađanja u čitavom heap-u
 - Skupljanje otpadaka u generaciji označava da se objekti skupljaju u toj generaciji i svim mlađim generacijama
 - Ako aplikacija pokuša da kreira novi objekat, a generacija 0 na heapu je popunjena -> tada GC pokrene skupljanje otpadaka u generaciji 0 da bi oslobodio memorijski prostor
 - Prelazi se preko grafa objekata i markiraju se živi objekti, a oslobađaju objekti koji nisu markirani
 - objekti koji su markirani promovišu se u sledeću generaciju -> u generaciju 1
 - kolektovanje otpadaka u generaciji 0 često oslobodi dovoljno memorije da aplikacija može da nastavi sa radom i novim alokacijama
 - Ali ako nakon kolektovanja u generaciji 0 i dalje nema dovoljno slobodne memorije -> vrši se ciklus kolekcije nad generacijom 1
 - Objekti koji prežive kolekciju u generaciji 1 prebacuju se u sledeću generaciju tj. generaciju 2
 - U slučaju kolektovanja generacije 2, preživeli objekti ostaju u toj generaciji, brišu se samo mrtvi objekti
- ➔ Promovisanje objekata
- Objekti koji prežive kolekciju nad jednom generacijom objekata promovišu se u sledeću višu generaciju
 - Kada sakupljač smeća otkrije da je stopa preživljavanja visoka u generaciji, povećava prag izdvajanja memorije za tu generaciju
 - CLR stalno balansira između dva prioriteta: ne dozvoliti da heap aplikacije postane prevelik tako da se sakupljanje otpadaka odlaže, ali i ne dozvoliti da se sakupljanje smeća izvodi prečesto zbog premalog heap-a aplikacije

IP protokol (Internet Protokol)

32-bitni, reči podeljene na podmrežni i host deo. Primer adresiranja: 192.169.0.0/24, što označava da su sve adrese od 192.168.0.0 do 192.168.0.255 deo te mreže.

IP je izuzetno nepouzdan, dok sa druge strane TCP pruža bas pouzdanost. IP se koristi kao centralna tačka mnogih protokola: TCP, UDP, ICMP, IGMP. Posедуje opcije kao što su: ograničavanje života paketa, zapis izabranih deonica na putanji paketa, usmeravanje po zadatoj putanji.

Zaduzen je za prenos i usmeravanje paketa. IP paketi(datagrami) se prosledjuju preko fizickog nivoa(mreznog steka). IP je mrežni protokol: internetworking - povezivanje heterogenih udaljenih mreža, slanje i rutiranje datagrama. Rukuje slanjem paketa kroz mreže: koristi adresu iz zaglavlja paketa i dodaje adresu posiljaoca; IP datagram je osnovna jedinica prenosa u internetu; maksimalna dužina 64 heksadecimalno, ipak 1500 kao velicina IP paketa u Ethernetu zbog ograničenja kasnjenja i opterećenja mreže(cvorova); best-effort service, tj. ne zadržava izgubljene pakete.

Zaglavlje IPv4 protokola: najvažnije su IP adrese izvora i odredišta, ostalo su dodatni elementi neophodni za prenos i rukovanje IP paketima, 20 bajta+opcije

IP fragmentacija i prikupljanje:

MTU(max. transfer unit) je najduži okvir u prenosu: karakteristika svake deonice, menja se u toku prenosa

Veliki IP datagrami se dele(fragmentiraju) unutar mreže: podela na manje pakete, prikupljanje na odredištu, IP zaglavlje identifikuje ove pakete i njihov redosled

IPv6 ne dozvoljava fragmentaciju na mrežnom nivou

Ciljevi v4 -> v6 tranzicije:

1. Prosirenje IP adrese
2. Zaglavlje fiksne duzine radi brze obrade
3. Opcije su prebacene u podatke(next header ukazuje na tip i poziciju)
4. Inace ukazuje na gornji protokol
5. nema checksum-a
6. Protokol vise ne podrzava fragmentaciju na mrežnom nivou
7. Podrska za QoS(pri, flow) - quality of service

IPSec - zastita IP sloja koja pruza tajnost na mrežnom nivou na nacin da predajni racunar sifruje podatke u IP datagramu, pruza mogucnost autentifikacije izvorsne IP adrese.

TCP/UDP adresiranje

Socket = IP adresa + broj porta(prolaza)

- Adresa (npr. 192.52.1.33) identifikuje racunar
- Port je broj n[1 - 65535] dodeljen korisničkoj aplikaciji
- Prvih 1023 "portova" su rezervisani
- Poruke se pre slanja dele na pakete
 - Dodaje im se TCP/UDP zaglavlje
- Na obe strane se formiraju Tx/Rx baferi
 - Kontrola toka sprečava prepisivanje/prepunjavanje na prijemu

Socket (socket, uticnica)

- Preko soketa procesi šalju poruke u mrežu
- Standardna API sprega (Application Programming Interface)
- Multipleksiranje / demultipleksiranje poruka

TCP

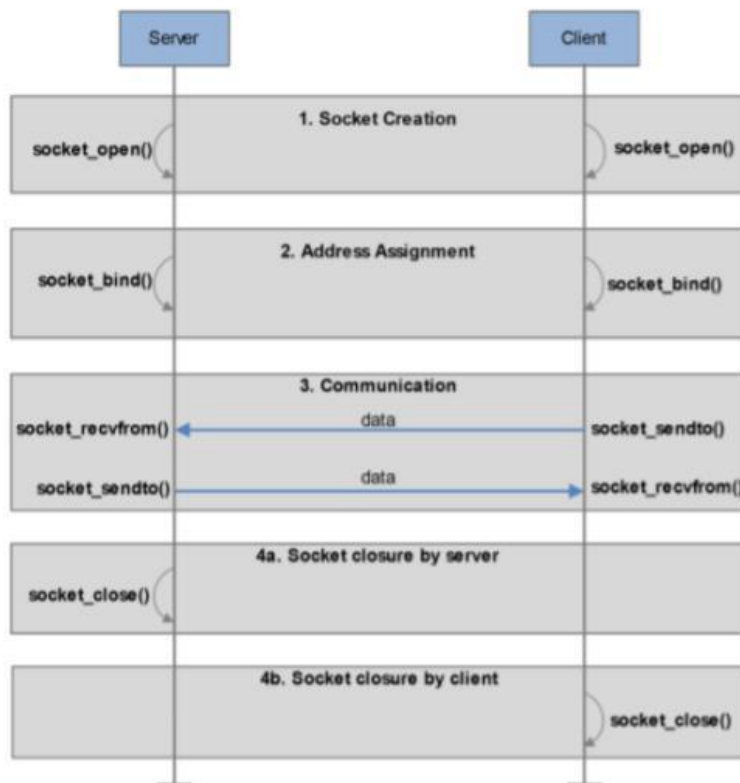
- Formira pouzdan komunikacioni kanal (preko koga se ponovo salju neuspeli paketi)
- Paketi su oznaceni brojem sekvence, koji se ponavlja, ali u duzem periodu
- Isporka se organizuje po tom broju
- U slucaju greske, TCP radi retransmisiju paketa, protokol pouzdanog prenosa koji se nalazi u transportnom sloju
- Transport Control Protocol radi na vrhu IP protokola koji je na vrhu mreznog sloja
- Kako je IP nepouzdan protokol, TCP realizuje mehanizme kontrole toka i prenosa koji obezbedjuju pouzdan prenos informacije
- TCP potvrđuje svaki paket koji primi, tako da posiljalac zna da je paket stigao na svoje odrediste
- U slucaju da TCP sa predajne strane ne primi potvrdu, on ponovo salje nepotvrđeni paket(retransmission)
- Na taj nacin aplikacija moze racunati da ce mreza obaviti pouzdanu isporuku informacije zeljenom odredistu
- TCP logicki tok: TCP je stream-oriented, jer sve podatke u prenosu tretira prosto kao niz okteta; Isporka na odredistu jeste identicna po broju i redosledu okteta, ali ne i po porukama na ulazu u socket. To znaci da ne pravi razliku između pojedinačnih poruka koje se prenose preko TCP konekcije. To znači da aplikacija mora samostalno implementirati mehanizam za razdvajanje poruka koje stižu preko TCP konekcije na odredištu.

Stop and wait

Protokol za kontrolu protoka koji zahteva da posiljalac ceka potvrdu primanja pre nego sto posalje sledeci podatak

UDP

- Ne uspostavlja komunikacioni kanal/vezu kao TCP(connectionless), jednostavniji je od TCP jer ne koristi potvrde i samim tim je mnogo brzi.
- Prosiruje IP uvođenjem prolaza(port-a). UDP = IP + adresiranje aplikacije
- Nema uspostave sesije i ne garantuje isporuku paketa
- Podrazumeva kontrolu isporuke na aplikacionom nivou
- Jednostavniji i efikasniji, "best effort" usluga
- Brzi od TCP
- UDP server je definisan parom [IP adresa, port]; svaki datagram sa tim odredistem bice prihvacen bez obzira ko ga salje
- Manja kasnjenja pogoduju multimedia aplikacijama
- Izgubljeni paket se ne znavlja
- Internet UDP korisnici: DHCP, DNS, SNMP
- Aplikacije koje koriste UDP moraju same obezbediti kontrolu i oporavak u slucaju gresaka
- Tok aplikacije u slucaju UDP uticnica: Open - evidentiranje kod UDP drivera; Bind - omogucuje prijem, definise servera; SendTo/RecvFrom - nezavisna razmena paketa podataka, druga strana se specificira parom Adresa/Port; Close - obe strane mogu ga inicirati; App poziva funkcije UDP rukovaoca, koji je deo OS



Stateless - ne pamti informacije o prethodnim transakcijama ili komunikaciji izmedju dva uredjaja. Svaki paket se tretira zasebno i ne postoji nikakva veza ili kontekst izmedju paketa. Primer je UDP.

Connectionless - ne uspostavlja prethodnu vezu ili konekciju izmedju dva uredjaja pre slanja podataka. Svaki paket se salje nezavisno. Primer je UDP.

Multithread program se deli na blokove niti koji se izvrsavaju nezavisno jedni od drugih

Nedeljive operacije(atomic)

Nedeljiva(atomic) operacija garantuje da samo jedna nit ima pristup delu memorije dok se operacija ne zavrshi(slicno kao lock).

Sinhronizacija - sinhronizacija na barijeri, bravi i semaforu

Implementira se kao tacka u programu koju zadatak ne sme da predje pre nego sto drugi zadaci dostignu logicki ekvivalentnu tacku(barijera). Obicno ukljucuje stanje cekanja bar jednog zadatka i zato produzava vreme izvrsavanja paralelne aplikacije.

Sinhronizacija na barijeri - obicno ukljucuje sve zadatke, svaki zadatak radi posao dok ne dodje do barijere, kada prekida posao i blokira se; kada poslednji zadatak dosegne barijeru svi zadaci su sinhronizovani(barijeru obicno sledi kriticka sekcija koja se mora sekvencijalno izvorsiti, zadaci se deblokiraju i nastavljaju svoj posao)

Brava(lock) - moze ucestvovati bilo koji broj zadataka(samo jedan zadatak moze istovremeno da koristi bravu i udje u kriticku sekciju, prvi zadatak koji naidje na slobodnu bravu je zkaljucava i obavlja posao, a ostali moraju da cekaju)

Semafori(semaphores) - moze ucestvovati bilo koji broj zadataka; odredjeni broj zadataka moze istovremeno da koristi semafor i koristi resurs ili udje u kriticku sekciju; postoje blokirajuci, binarni(brave) i brojacki semafori

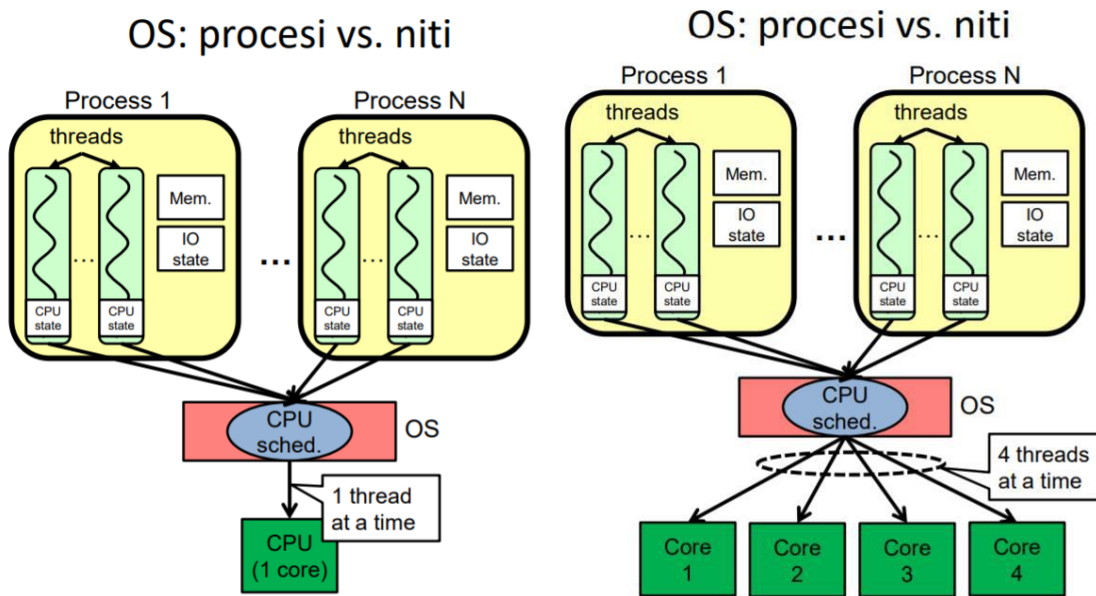
Kernel se izvrsava u paraleli preko skupa paralelnih niti. Sve niti koje su generisane startovanjem kernela se kolektivno zovu **resetka**. Niti su organizovane u blokove, a blokovi su organizovani u resetke. Blok niti je skup konkurentno izvrsavanih niti koje mogu da saradjuju izmedju sebe putem barrier - sinhronizacije i deljene memorije. Resetka je niz blokova niti koje izvrsavaju isti kernel: citaju ulaze i upisuju rezultate u globalnu memoriju, sinhronizuju se izmedju zavisnih kernel poziva

GPU i CPU procesori

CPU(centralna procesorska jedinica) - optimizovana za pristup kesiranom skupu podataka sa malim kasnjenjem, kontrolna logika za neispravno izvrsavanje, izvrsavanje desetak niti, efikasan za svaki problem bio paralelan ili ne, task paralelizam

GPU(graficka procesorska jedinica) - optimizovana za paralelan rad sa podacima i povecanje propusnosti, arhitektura tolerantna na memorijska kasnjenja, izvrsavanje desetak hiljada niti, pretpostavlja da je radno opterecenje visoko paralelno, data paralelizam, efekti u video igrima, racunarske naucne simulacije, procesiranje slike i masinsko ucenje, linearna algebra

Hardverska nit je fizicki CPU ili jezgro. Jedna hardverska nit moze da startuje-izvrsava mnogo softverskih niti. Svako fizicko jezgro moze da ponudi vise od jedne hardverske niti. Svaki program koji se izvrsava u Windows OS je **proces**. **Svaki proces kreira i startuje jednu ili vise softverskih niti**.



Paralelno racunarstvo - istovremeno koriscenje vise racunarskih resursa u cilju resavanja nekog problema. Primeri: softver se izvsava na vise procesora, problem se razdvaja na vise delova koji se mogu resavati uporedno(u paraleli), instrukcije iz svakog od delova se izvsavaju istovremeno na razlicitim procesorima, svaki deo problema se dalje razdvaja na serije diskretnih funkcija. Zahteva se podela problema, definisanje komunikacije i sinhronizacije. Moze biti: na nivou bita, na nivou instrukcija, na nivou podataka, na nivou niti.

Konkurentan rad na racunaru: 1 ili vise CPU, kvazi-simultano izvsavanje.

Paralelan rad na racunaru: vise CPU, simultano izvsavanje mnogo/svih niti iste aplikacije

Zajednicke osobine: niti/procesi se preklapano(isprepletano izvsavaju), sinhronizacija, komunikacije, sukob oko resursa, uslovi trke, potpuni zastoji.

Cilj konkurentnog: povecanje CPU iskoriscenja, povecanje brzine odziva sistema, podrška rada sa vise korisnika. Pitanja su rasporedjivanje i prioriteti. Cilj paralelnog izvsavanja je ubrzanje jedne aplikacije(veliki problem). Pitanja su: algoritmi, strukture podataka, mapiranje, balans opterecenja.

Konkurentno - izvsavanje zadataka ili procesa se deli na manje delove. Deli se procesorsko vreme i niti se izvsavaju naizmenicno. Koristimo kod dugotrajnih operacija ili kada aplikacija mora odgovoriti na vise dogadjaja istovremeno, pa se radi prekljucivanje.

Paralelno - zadaci se izvsavaju paralelno koriscenjem vise procesora, jezgara, cvorova u mrezi.

Firewall filtrira paket za paketom primenjuci fiksni skup pravila iz ACL (access control list) tabele(kod stateless-a), pravila baziranih na osnovu IP, UDP i TCP zaglavlja tj. vrednosti polja zaglavlja.

Statefull firewall pored fiksnih kriterijuma baziranim na IP, UDP, TCP zaglavlja takodje koristi ta zaglavlja da prati "smislenost" paketa koje filtrira. Aplikativne barijere prosiruju taj skup i pored pomenutih zaglavlja vrse filtraciju i **na osnovu aplikativnih podataka**(npr. dozvoli nekim internim korisnicima da koriste Telnet van lokalne mreze).

Mrežno programiranje:

Na 3 OSI nivou(mrežni sloj) oslanja se na IP mrežni protokol

Na 4 OSI nivou(transportni sloj) se mogu koristiti TCP, UDP i drugi protokoli

Portovi su dostupni tek na 4 OSI nivou

Statefull firewall može da pamti informacije o prethodnim mrežnim aktivnostima i da na osnovu tih informacija odlučuje koje konekcije treba da dozvoli, a koje ne. U kontrastu sa "**stateless**" firewall-om koji ne pamti informacije (može koristiti ACL kako bi određivao dozvoljene i zabranjene mrežne konekcije kao što su: IP adrese, portovi, protokoli...).

Primeri rada statefull firewall-a: kod TCP konekcije beleži informacije o izvoru i odredištu. Može da odredi da li dolazni paketi pripadaju postojećoj konekciji i da ih dozvoli. Kod FTP može da zabeleži informacije o prethodnom uspostavljanju kontrolne konekcije, omogućava da otvori određene portove za prenos podataka, a da ne ugrozi bezbednost sistema.

Socket - par IP adresa, port. Uspostavljena veza između dva programa određena je parom socket-a. TCP konekcija između dva čvora na mreži. Istu klasu koriste i klijent i server.

Connect(removeEndPoint), koristi se da spoji klijenta na udaljen server.

Bind(localEndPoint), prebacuje socket u režim slušanja, potrebno da se uradi na serverskoj strani)/Listen/Accept, koristi se da serverski socket sačeka i primi konekciju klijenta. Accept blokira izvršenje sve dok neki klijent ne uspostavi vezu (tada vraća konstruisan Socket objekat preko koga se komunicira sa klijentom).

IPEndPoint - klasa predstavlja jedan kraj veze (adresa, port), koristi se kao parametar za metode Connect i Bind klase Socket.

Identifikacija čvorova mreže:

- Pomoću simboličke adrese, npr. **www.yahoo.com**
- Pomoću numeričke adrese, podeljene u oktete, npr. **147.91.177.196**
- Klasa **Dns** ima statičku metodu **GetHostEntry(adr)**, koja od simboličke ili numeričke adrese (u obliku stringa) vraća objekat klase **IPHostEntry**, koja predstavlja adresu nekog računara u C#:
- Klasa **IPHostEntry** predstavlja sve adrese nekog čvora u mreži.
- Klasa **IPAddress** predstavlja jednu adresu čvora u mreži:

```
IPHostEntry ipHostEntry = Dns.GetHostEntry(Dns.GetHostName());  
IPAddress ipAddress = ipHostEntry.AddressList[0];
```
- Ako nam treba adresa lokalnog računara, koristimo: **IPAddress.Loopback**

Za prenos podataka se koriste metode:

.Send i .Receive za primanje i slanje niza bajtova

Klasa **Socket** sa klijentske strane:

Send(nizBajtova), SendFile(fileName), Receive(nizBajtova)

Klasa **Socket** sa serverske strane:

Bind(localEndPoint) - obezbeđuje da napravljeni socket može da radi kao serverski socket, na njega će se vezivati klijentski socketi metodom Connect()

Listen(maxQueueSize) - počinje da čeka klijente da se spoje, parametar definiše koliko maksimalno klijenata može da čeka

Accept() - blokira izvršenje sve dok neki klijent ne uspostavi vezu; tada vraća konstruisan Socket objekat preko koga se komunicira sa klijentom

Pracenje korisnicke sesije:

Statefull protokoli - čuvaju stanje tokom komunikacije: POP3, SMTP

Stateless protokoli - ne čuvaju stanje komunikacije između dva klijentska zahteva: HTTP (osnovni)

Primer registracije korisnika:

Klijent: uspostavlja vezu sa serverom, salje korisnicko ime za registraciju, ceka odgovor, završava komunikaciju

Server: ceka klijente u beskonacnoj petlji, za svakog klijenta pokrece poseban thread za obradu: cita korisnicko ime koje je klijent poslao, dodaje to ime u listu, izlista spisak prijavljenih korisnika, salje odgovor - spisak registrovanih korisnika

Sadržaj **www** sajta:

HTML stranice, multimedijalni elementi(slike, animacije...), drugi tipovi podataka, **www** server i klijent komuniciraju preko HTTP sadržaja.

Staticke: login/greska stranice bez dinamicke strukture, unapred napravljene i skladištene na serveru. Ukoliko se zahteva više sadržaja(slika) salje se jedan broj zahteva serveru

Dinamicke: generisu se na osnovu zahteva klijenta, ne skladište se na serveru nego se na njemu generisu i vraćaju klijentu

Mozete li reci sta se desava ako neko u browser unese naziv nase kompanije

Radi se DNS koji gđa SBB-ov ruter koji za simboličku adresu dobija numeričku IP(fizičku adresu) adresu, uspostavlja komunikaciju na 7. nivou novim http protokolom i gđam i uspostavljam konekciju

na 4. nivou preko TCP i na portu se uspostavlja konekcija. Ako unosimo logičku umesto simboličke to je u obliku 192.0.0.134.

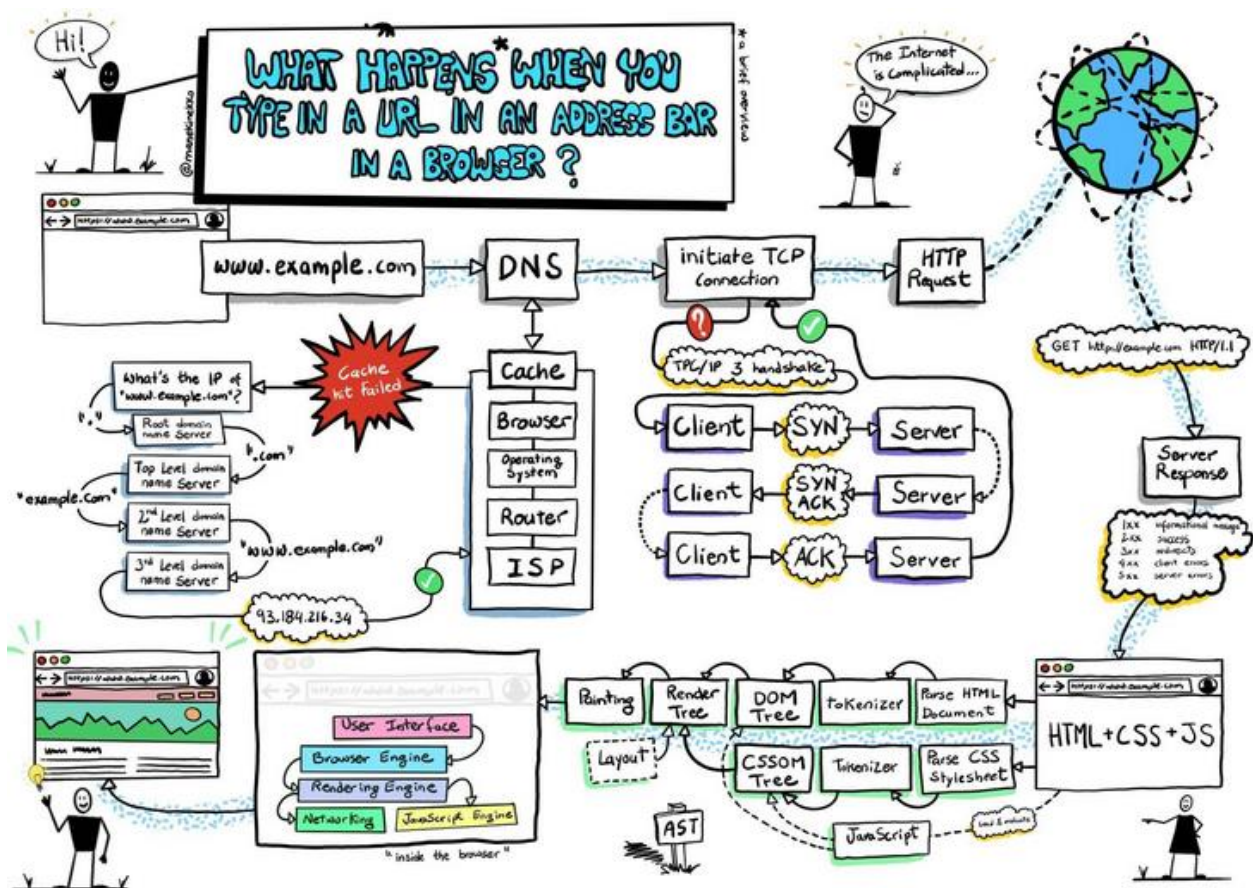
1. Korisnik u web pregledaču unosi simboličku adresu, npr. www.google.com.
2. Web pregledač šalje DNS zahtev DNS serveru koji je naveden u konfiguraciji mreže.
3. DNS server traži numeričku IP adresu odgovarajuću simboličkoj adresi www.google.com.
4. DNS server šalje odgovor sa numeričkom IP adresom web servera na koji se korisnik povezuje.
5. Web pregledač koristi dobijenu numeričku IP adresu za uspostavljanje konekcije sa web serverom.

Dakle, DNS serveri koriste simboličke adrese da bi pronašli odgovarajuće numeričke IP adrese i tako omogućili uspostavljanje komunikacije između klijenta i servera.

Ako unesete **ID 2 u URL-u**, obično se očekuje da taj ID predstavlja određeni resurs na serveru. Na primer, ako imate veb aplikaciju koja upravlja korisnicima, možete imati URL poput

<https://www.mojavebaplikacija.com/korisnici/2>, gde je "2" ID određenog korisnika.

Kada pregledač pošalje zahtev sa takvim URL-om serveru, server može da izvrši odgovarajuće akcije kako bi pronašao resurs sa datim ID-om i vratio odgovor. Na primer, server može da pretraži svoju bazu podataka korisnika, pronađe korisnika sa ID-jem 2 i generiše odgovarajuću HTML stranicu koja se zatim prikazuje u pregledaču.



1. Pregledac prvo proverava svoju lokalnu kes memoriju, kes memoriju operativnog sistema, rutera
2. Ako se u kes memoriji pregledaca ne nalazi trazena adresa salje se zahtev DNS serveru. DNS vrši razlaganje adrese na delove i obavlja pretragu u hijerarhijskoj strukturi. DNS server prvo identifikuje top level domen i povezuje sa DNS serverom koji upravlja tim domenima (npr. top level domen ".com"). Zatim proverava sledeći deo adrese ("example") on se povezuje sa odgovarajućim DNS serverom za taj drugi nivo domena. Proces se nastavlja dok DNS ne pronadje traženi DNS unos koji odgovara celokupnoj adresi. Na taj način DNS razlaze adresu i pretražuje odgovarajuće DNS servere.
3. DNS server šalje odgovor sa IP adresom koja je povezana sa unetom adresom.
4. Ta adresa se skladišti u kes.
5. Pregledac tu adresu koristi da uspostavi TCP ili UDP konekciju sa serverom koji je povezan sa tom adresom.
6. Kada se konekcija uspostavi pregledac šalje HTTP zahtev ka serveru u kojem traži specifičnu veb stranicu ili resurse
7. Server odgovara slanjem statusnog koda (npr. 404) ili slanjem sadržaja
8. Ako server pošalje HTML, CSS, JSON podatke kao odgovor na HTTP zahtev pregledac ih parsira i prikazuje korisniku. Pregledac analizira strukturu HTML dokumenta, identifikuje elemente, atribute, stilove, skripte i formira DOM (Document Object Model). Potom CSS parser obrađuje CSS kod koji je dobijen kao deo HTTP odgovora i pridružuje stilove elementima u DOM strukturi. Ako postoje slike, JavaScript fajlovi ili drugi resursi, pregledac ih učitava u pozadini. To uključuje preuzimanje slika sa servera ili kes memorije, izvršavanje JS koda i obradu drugih eksternih resursa. Pregledac prikazuje formiranu DOM strukturu u okviru prozora pregledaca. Ako postoji JS kod ili eksterni JS fajl pregledac ga izvršava nakon što je prikazan osnovni sadržaj.

HTTP

Klijent i server moraju da komuniciraju na neki nacin. **Http predstavlja protokol za komunikaciju izmedju wec citaca i servera, definise pravila po kojima se informacije razmenjuju izmedju klijenta i servera. HTTP definise da browser ne moze da skine vise od dve komponente paralelno zato se JS kod stavlja na kraju.** Browser generise poruku koja se sastoji od zaglavlja http protokola i tela http protokola. Zaglavlje sadrzi neke obavezne, ali i neke opcione delove. U prvom redu se definise **naziv metode**(obavezno je definisati metodu) za komunikaciju(get, post), posle toga navodimo **resurs koji se trazi** i na kraju ide **protokol, verzija** za komunikaciju. Ostali redovi su opcioni i sluze za parametrizovanje generisane poruke. Ukoliko server zna da protumaci dobijenu poruku od klijenta vraca odgovor. U prvom redu navodi ime protokola i verziju, zatim kod 200 koji oznacava da je sve u redu, tekst. U drugom (opcionom) redu se navodi tip sadrzaja koji se vraca klijentu, na kraju se nalazi sam resurs(index.html) koji potom citaci(browser-i) interpretiraju. Predefinisani port za komunikaciju je 80. Klijent trazi neki resurs, a server isporucuje resurs(HTML datoteka, slika, film).

HTTP komunikacija:

- zasnovana na zahtev/odgovor principu
- svaki par zahtev/odgovor se smatra nezavisnim od ostalih
- ne omogucava pracenje korisnicke sesije, tj. niza zahteva upucenih od strane istog klijenta
- U verziji 1.0 po zavrsetku isporuke odgovora klijentu konekcija se zatvara(za novu komunikaciju klijenta sa serverom opet treba da se uspostavi konekcija)
- U verziji 1.1 konekcija se ne zatvara tj. konekcija ostaje otvorena(**Keep-Alive**). Klijent ce istu konekciju da koristi pri slanju novog zahteva ka serveru. Konekcija ostaje otvorena sve dok neka od strana u komunikaciji(klijent ili server) ne odluci da je neophodno da završi komunikaciju sa drugom stranom, sto ce uraditi tako sto ce zatvoriti konekciju
- Prednost verzije 1.1:
 1. Smanjeno zauzece CPU jer je smanjen broj poruka koje se kreiraju, obradjuju i salju mrezom
 2. Smanjeno zagusenje mreze(manje poruka za kreiranje TCP konekcija)
- Mane verzije 1.1:

Situacija u kojoj je klijent preuzeo sve podatke od servera, ali nije zatvorio konekciju je problem. U takvoj situaciji server nepotrebno trosi resurse za otvorenu vezu, umesto da te resurse mogu da koriste drugi klijenti. Prethodno moze da utice na dostupnost servera da prima nove zahteve klijenta, ako je na serveru ogranicen broj klijenta koje istovremeno server opsluzuje. Server ce izvorsiti zatvaranje konekcije koja je idle u zavisnosti od konfiguracije
- HTTP je stateless protokol koji ne zahteva od servera cuvanje statusa klijenta ili korisnicke sesije klijenta tj. niza zahteva upucenih od strane istog klijenta.
- HTTP serveri prevazilaze prethodno tako sto implementiraju razlicite metode za odrzavanje i upravljanje sesijom, tipicno se oslanjajuci na jedinstveni identifikator, cookie ili neki drugi parametar koji omogucava pracenje zahteva koji originiraju od istog klijenta(npr. URL Rewriting mehanizam), kreirajuci stateful protokol iznad HTTP protokola.

Pracenje sesije se vrši preko mehanizama:

Cookies - server šalje kolačić klijentu u okviru HTTP odgovora. Klijent čuva primljeni kolačić i šalje ga uz svaki HTTP request. To su tekstualni fajlovi koji se koriste za pamćenje informacija o korisniku ili njegovim postavkama. Na primer prikaz relevantnih reklama ili prikaz stranice na jeziku koji korisnik preferira. Cookie je bolje slati u okviru HTTPS jer on sve vreme koristi SSL/TSL tj. enkriptovanu vezu. TSL je novija verzija SSL protokola za enkripciju podataka. Druga varijanta (ukoliko se koristi HTTP, a želimo sigurnost) je da server potpiše cookie privatnim ključem, a klijent ga proveri javnim ključem servera (ne postoji u samom HTTP nego mi dodatno razvijamo konfiguraciju na serveru i klijentu).

URL Rewriting - svakom URL-u se dodaje neki session-id koji označava koji korisnik pristupa resursima, veoma nebezbedno i lako za kopiranje

Napisati prvi i poslednji red HTTP zahteva (HTTP verzija 1.1) koji se dobija kada se klikne na dugme Pošalji (kada se dodatno ništa ne unese u polja) u sledećoj formi:

```
<form action="http://localhost/Proba" method="GET">
<input type="text" name="polje1" value="">
<input type="text" name="polje2" value="test2">
<input type="submit">
</form>
```

Prvi: GET /Proba?polje1=&polje2=test2 HTTP/1.1

Poslednji: (prazno)

HTTP zahtev:

Pocinje redom: **METHOD/putanja HTTP/ verzija**

HTTP Metode:

GET - zahtevanje resursa od web servera, prikazani podaci u URL-u. **Parametri iz forme se za GET metodu smestaju u zaglavlje GET zahteva.** Posle znaka ? se zapisuje vrednost elementa forme (u obliku ključ=vrednost)

POST - šalje parametre forme i traži odgovor. **Vrednosti parametara forme smestaju se na kraj HTTP zahteva** (posle praznog reda) i posle vrednosti parametara nema "\r\n" karaktera na kraju reda.

HEAD - zahteva samo HTTP odgovor (response), bez slanja samog resursa. Možda želi HTTP odgovor zbog nekog testiranja.

PUT - omogućava klijentu da pošalje datoteku na web server

OPTIONS - od web servera se traži spisak metoda koje podržava

DELETE - omogućava klijentu da obriše resurs sa web servera

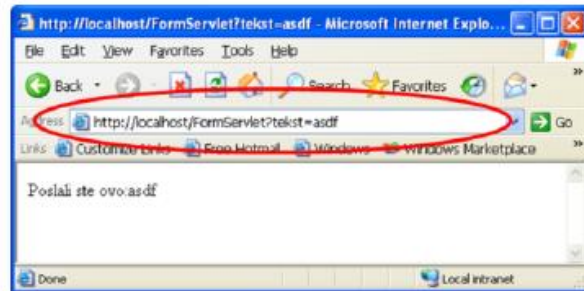
- dodatni redovi sadrže atribute oblika: Ime:vrednost

- prazan red na kraju. Ako je POST zahtev posle praznog reda idu parametri forme

Kod GET metode se parametri forme nalaze u heder delu HTTP request poruke

```
GET /FormServlet?tekst=asdf HTTP/1.1
...
```

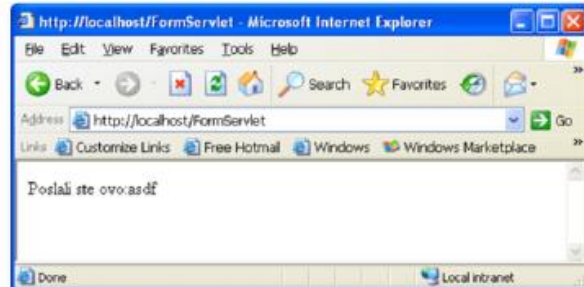
Kod HTTP **GET** metode, posle znaka ? zapisuju se vrednosti URL promenljivih tipa *ključ=vrednost&ključ=vrednost...*



```
POST /FormServlet HTTP/1.1
...
Content-length: 10
...
```

```
tekst=asdf&ključ=vrednost
```

Kod HTTP **POST** metode se vrednosti parametara forme smeštaju na kraju HTTP zahteva (posle praznog reda), i posle vrednosti parametara nema "r\n" karaktera na kraju reda



35/43

Kod POST metode se parametri forme nalaze u body delu HTTP request poruke

Atributi u HTTP zahtevu:

User-Agent --- identifikuje web browser

Accept --- definise koje tipove resursa navigator prihvata kao odgovor na ovaj zahtev

Accept-Language --- definise koje jezike ocekuje kao odgovor

Accept-Encoding --- definise koje kodiranje ocekuje kao odgovor

Accept-Charset --- definise koju kodnu stranu ocekuje

Cookie --- definise mehanizam pracenja sesije. Server salje cookie klijentu u okviru http response, klijent cuva primljeni cookie i salje ga uz svaki http request.

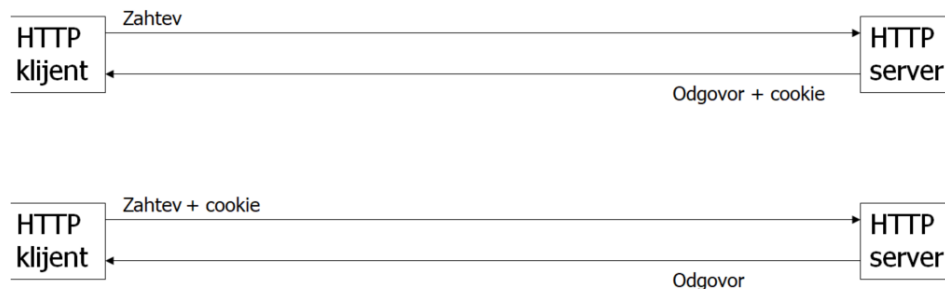
Refer --- definise URL sa kojeg se doslo na ovu stranicu: koristi se za statistiku, hotlinking

Connection --- HTTP1.1 "kaze" serveru da ne zatvara konekciju po isporuci resursa

q --- predstavlja **relative quality factor** tj. floating point vrednost "tezine" parametra - favorizovani charset je UTF-u ili ISO-8859-1, ali ukoliko oni nisu podrzani moze i ASCII, ako ni to onda bilo koji

Praćenje sesije korisnika ^{2/3}

- kada se koristi cookie mehanizam



HTTP odgovor:

Pocinje redom: **HTTP/verzija kod tekstualni_opis** -> (**HTTP/1.1 200 OK**)

dodatni redovi sadrze attribute: Ime: vrednost

prazan red

sledi sadrzaj datoteke(resursa) --- **Ovde je npr HTML, CSS, JS(vidi primere dole)**

Atributi u HTTP odgovoru:

Content-type --- definise tip odgovora

Cache-Control --- definise kako se kes na klijentu azurira

Location --- definise novu adresu kod redirekcije

Connection --- potvrda klijentu da li da zatvori konekciju ili da je ostavi otvorenu
npr. Connection: Keep-Alive

Sa GET dobavljamo informacije, a sa POST postavljamo nesto.

HTTP GET metoda šalje podatke preko URL-a (Uniform Resource Locator). Kada se koristi GET metoda, podaci se mogu videti u URL-u koji se šalje na server. HTTP POST metoda šalje podatke kao deo HTTP zahteva, a ne preko URL-a. Ovo znači da podaci nisu vidljivi u URL-u i da su sigurniji od GET metode. HTTP POST zahtev će biti poslat na server, a podaci će biti poslani u HTTP zahtevu, a ne u URL

Primeri HTTP GET i POST zahteva:

```
GET /hello.htm HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.foobar.com
Accept-Language: en-us
Accept-Encoding: gzip, deflate
Connection: Keep-Alive
```

```
POST /cgi-bin/process.cgi HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.foobar.com
Content-Type: application/x-www-form-urlencoded
Content-Length: length
Accept-Language: en-us
Accept-Encoding: gzip, deflate
Connection: Keep-Alive

licenseID=string&content=string&/paramsXML=string
```

Primeri HTTP odgovora 200, 400, 404:

```
HTTP/1.1 400 Bad Request
Date: Sun, 18 Oct 2012 10:36:20 GMT
Server: Apache/2.2.14 (Win32)
Content-Length: 230
Content-Type: text/html; charset=iso-8859-1
Connection: Closed

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html>
<head>
  <title>400 Bad Request</title>
</head>
<body>
  <h1>Bad Request</h1>
  <p>Your browser sent a request that this server could not understand.</p>
  <p>The request line contained invalid characters following the protocol string.</p>
</body>
</html>
```

```
HTTP/1.1 200 OK
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache/2.2.14 (Win32)
Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT
Content-Length: 88
Content-Type: text/html
Connection: Closed

<html>
  <body>
    <h1>Hello, World!</h1>
  </body>
</html>
```

```
HTTP/1.1 404 Not Found
Date: Sun, 18 Oct 2012 10:36:20 GMT
Server: Apache/2.2.14 (Win32)
Content-Length: 230
Connection: Closed
Content-Type: text/html; charset=iso-8859-1

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html>
<head>
  <title>404 Not Found</title>
</head>
<body>
  <h1>Not Found</h1>
  <p>The requested URL /t.html was not found on this server.</p>
</body>
</html>
```


GET metoda

```
GET /hello.htm HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.foobar.com
Accept-Language: en-us
Accept-Encoding: gzip, deflate
Connection: Keep-Alive

HTTP/1.1 200 OK
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache/2.2.14 (Win32)
Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT
ETag: "34aa387-d-1568eb00"
Vary: Authorization,Accept
Accept-Ranges: bytes
Content-Length: 88
Content-Type: text/html
Connection: Closed

<html>
  <body>
    <h1>Hello, World!</h1>
  </body>
</html>
```

HEAD metoda:

```
GET /hello.htm HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.foobar.com
Accept-Language: en-us
Accept-Encoding: gzip, deflate
Connection: Keep-Alive

HTTP/1.1 200 OK
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache/2.2.14 (Win32)
Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT
ETag: "34aa387-d-1568eb00"
Vary: Authorization,Accept
Accept-Ranges: bytes
Content-Length: 88
Content-Type: text/html
Connection: Closed
```


POST metoda:

```
POST /cgi-bin/process.cgi HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.foobar.com
Content-Type: text/xml; charset=utf-8
Content-Length: 88
Accept-Language: en-us
Accept-Encoding: gzip, deflate
Connection: Keep-Alive

<?xml version="1.0" encoding="utf-8"?>
<string xmlns="http://clearforest.com/">string</string>

HTTP/1.1 200 OK
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache/2.2.14 (Win32)
Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT
ETag: "34aa387-d-1568eb00"
Vary: Authorization,Accept
Accept-Ranges: bytes
Content-Length: 88
Content-Type: text/html
Connection: Closed

<html><body><h1>Request Processed Successfully</h1></body></html>
```

```
POST /edit/ HTTP/1.1
Host: example.org
User-Agent: Thingio/1.0
Authorization: Basic ZGFmZnk6c2VjZXJldA==
Content-Type: application/atom+xml;type=entry
Content-Length: nnn
Slug: First Post

<?xml version="1.0"?>
<entry xmlns="http://www.w3.org/2005/Atom">
  <title>Atom-Powered Robots Run Amok</title>
  <id>urn:uuid:1225c695-cfb8-4ebb-aaaa-80da344efa6a</id>
  <updated>2003-12-13T18:30:02Z</updated>
  <author><name>John Doe</name></author>
  <content>Some text.</content>
</entry>

HTTP/1.1 201 Created
Date: Fri, 7 Oct 2005 17:17:11 GMT
Content-Length: nnn
Content-Type: application/atom+xml;type=entry; charset="utf-8"
Location: http://example.org/edit/first-post.atom
ETag: "c180de84f991g8"

<?xml version="1.0"?>
<entry xmlns="http://www.w3.org/2005/Atom">
  <title>Atom-Powered Robots Run Amok</title>
  <id>urn:uuid:1225c695-cfb8-4ebb-aaaa-80da344efa6a</id>
  <updated>2003-12-13T18:30:02Z</updated>
  <author><name>John Doe</name></author>
  <content>Some text.</content>
  <link rel="edit"
    href="http://example.org/edit/first-post.atom"/>
</entry>
```

PUT metoda:

```
PUT /users/123 HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.foobar.com
Accept-Language: en-us
Connection: Keep-Alive
Content-type: application/json
Content-Length: 182

{"user_id": 123, "firstName": "John", "LastName": "Doe"}
```

```
HTTP/1.1 204 No Content
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache/2.2.14 (Win32)
Connection: Closed
```

DELETE metoda:

```
DELETE /users/123 HTTP/1.1
User-Agent: Mozilla/4.0 (compatible; MSIE5.01; Windows NT)
Host: www.foobar.com
Accept-Language: en-us
Connection: Keep-Alive
```

```
HTTP/1.1 200 OK
Date: Mon, 27 Jul 2009 12:28:53 GMT
Server: Apache/2.2.14 (Win32)
Last-Modified: Wed, 22 Jul 2009 19:15:56 GMT
ETag: "34aa387-d-1568eb00"
Vary: Authorization,Accept
Accept-Ranges: bytes
Content-Length: 88
Content-Type: text/html
Connection: Closed
```

```
<html><body><h1>Request Processed Successfully</h1></body></html>
```

Vrste WWW sadržaja:

Statički - nalaze se u okviru datoteka WWW servera. Klijent zahteva datoteku, server je učitava sa svog fajl-sistema i šalje je klijentu

Dinamički - sadržaj se generise po zahtevu i šalje klijentu. Klijent zahteva "datoteku", server je generise i šalje sadržaj klijentu; ne snima je u svoj fajl-sistem

Preuzimanje podataka sa formi:

- Parametri iz forme se za GET metodu smeštaju u zaglavlje GET zahteva.

HTML kod na klijentu:

```
<form method="get" action="FormServlet">
  <input type="text" name="tekst">
  <input type="submit" value="Posalji">
</form>
```

- GET HTTP zahtev:

GET /Putanja?tekst_polje=asdf HTTP/1.1

Praćenje sesije korisnika:

- HTTP protokol ne prati sesiju – veza klijenta i servera se zatvara po isporuci resursa.
- Koristi se *cookie* mehanizam:
 - Server šalje *cookie* klijentu u okviru http response
 - klijent čuva primljeni *cookie* i šalje ga uz svaki http request.
- Ako navigator ne prihvata *cookie*-je, koristi se URL Rewriting mehanizam

Isporuca resursa:

- Server - Mrežna aplikacija koja sluša na portu 8080 i isporučuje dinamički generisan http sadržaj
- Klijent - web browser ili postman aplikacija koja poziva resurs localhost:8080

HTTP verzije

HTTP/1.0, ostvaruje se konekcijama na sedmom OSI sloju(Aplikativnom). Kako bi se konekcija ostvarila prvobitno se ostvaruje komunikacija na svim nizim slojevima. Posle odgovora HTTP servera klijentu konekcija se zatvara i ukoliko klijent zeli/zatrazi nesto ponovo potrebno je ponovo izvršiti celokupnu konekciju. Nastaje problem OVERHEAD(bespotrebno trošenje resursa).

HTTP/1.1, ostvaruje se permanentna/keep-alive konekcija koja se ne zatvara posle svakog odgovora HTTP servera, kada klijent zavrsi sa upotrebom servera konekcija se zatvara. Problem je sta ukoliko klijent ne zatvori konekciju, problem se resava uvođenjem TIMEOUT-a koji posle određenog vremena automatski zatvara konekciju. Definisuje format upita za pristup podacima web stranice.

HTTP/2.0, poboljšanje performansi, ubrzanje rada, kompresije, enkripcije...

HTTP header-i

REQUEST header, sastoji se od ključne reči metode GET/POST..., potom /naziv sadržaja koji se zahteva i na kraju HTTP/verzija protokola. Šalju se i neki dodatni redovi koji opisuju da li se prihvataju kolačići, koji browser se koristi, ali još jedno važno polje je Host: ovde se stavlja mesto odakle želimo da zahtevamo neku stranicu.

RESPONSE header, HTTP/verzija protokola potom poruka o uspešno/gresci(poruke koje počinju sa 2 označavaju uspešnost operacije, sa 3 redirekcije, 4 greska sa strane klijenta, 5 greska sa strane servera). Potom se navodi koji sadržaj se šalje klijentu i posle praznog reda sam sadržaj(HTML u našem slučaju)

HTTP (Hypertext transfer protocol) Web-ov protokol aplikativnog sloja, stateless organizovan po klijent-server modelu gdje klijent zahtjeva, prima i prikazuje Web objekte, a server salje objekte kao odgovor na primljeni zahtjev. Klijent inicira TCP konekciju na portu 80, server je prihvata, razmjenjuju se HTTP poruke i TCP konekcija se zatvara. HTTP se zasniva na mehanizmu da je svaki objekat na Web-u (HTML dokument, slika, video) adresiran preko URL-a. U toku parsiranja html odgovora klijent moze naici na dodatni broj referenci.

Ako je u pitanju **nonperistant** HTTP klijent ce za svaku referencu morati ponovo zahtjevati konekciju, cekati da se ona odobri, poslati zahtjev i cekati odgovor nakon cega se raskida konekcija i to ce se desiti onoliko puta koliko referenci je pronasao dok u slucaju **persistant HTTP-a konekcija ostaje otvorena** pa se poruke istih klijent/server ucesnika salju u okviru jedne sesije sto znaci da klijent salje novi zahtjev cim u toku parsiranja otkrije novu referencu (ne mora ponovo uspostavljati konekciju jer nije ni prekidana) cime se dosta stedi na vremenu (trosi samo 1 dodatni RTT za prenos svih objekata referisanih web stranicom) za razliku od nonpersistant HTTP-a koji zahtjeva 2 dodatna RTT po SVAKOM objektu sto uzrokuje OS opterecenje za svaku TCP konekciju.(cesto pretrazivaci otvore paralelne TCP veze da se sto brze dobave objekti).

Sve sto moze da se uradi sa POST moze i sa GET, ali ako se salju velike količine podataka ili osetljivih informacija, HTTP POST metoda je bolja opcija. Ako tražite podatke sa servera, a podaci su manji i ne osetljivi, HTTP GET metoda može biti prikladnija opcija.

Ključna razlika je sto POST menja stanje servera, dok GET samo prikazuje informacije koje su na serveru. POST metoda se koristi kada klijent želi da promeni stanje servera, na primer, kada želi da pošalje formularske podatke, ili želi da kreira, ažurira ili obriše neki resurs na serveru.

1. Podaci se šalju na različite načine:

- POST metoda šalje podatke u telu zahteva, dok se GET metoda koristi za slanje parametara u URL-u.

2. Karakteristike podataka:

- POST metoda je pogodna za slanje velikih količina podataka, dok se GET preporučuje za manje količine podataka (zbog ograničenja dužine URL-a).
- POST metoda omogućava slanje različitih vrsta podataka, uključujući binarne podatke, dok se GET metoda koristi uglavnom za slanje teksta.

3. Sigurnost:

- POST metoda je sigurnija za slanje osetljivih informacija, jer se podaci šalju u telu zahteva koji nije vidljiv u URL-u, dok GET metoda prikazuje sve informacije u URL-u, što može biti rizično ako se šalju osetljivi podaci.

4. Cache:

- GET metoda je podložnija korišćenju cache memorije, dok se POST metoda obično ne koristi za cache-ovanje podataka.

5. Povratna vrednost:

- POST metoda obično ne vraća odgovor, dok GET metoda obično vraća odgovor u telu odgovora.

Ukratko, POST se koristi kada se šalju velike količine podataka, kada se šalju osetljive informacije ili kada se vrši promena stanja servera, dok se GET koristi kada se dobavljaju manje količine podataka i kada se ne vrši promena stanja servera.

HTTP je protokol koji je po dizajnu connectionless (bez konekcije) i stateless (bez stanja).

Connectionless znači da svaki zahtev (request) i odgovor (response) u HTTP protokolu predstavljaju zasebnu komunikaciju. To znači da se nakon završetka svakog zahteva veza između klijenta i servera prekida, bez održavanja trajne konekcije. Nakon što se odgovor pošalje nazad klijentu, veza se zatvara i server zaboravlja o tom zahtevu. Sledeći zahtev od strane istog ili drugog klijenta se smatra zasebnim zahtevom i ne zavisi od prethodnih zahteva.

Stateless znači da server ne pamti stanje (kontekst) između zahteva. Svaki zahtev se tretira izolovano i server ne zna ništa o prethodnim zahtevima klijenta. Ovo znači da server ne pamti informacije o prethodnim sesijama ili stanjima korisnika. Ako je potrebno čuvanje stanja, klijent mora eksplicitno proslediti informacije u svakom zahtevu, na primer putem HTTP kolačića (cookies) ili sesija.

---Ove karakteristike HTTP protokola čine ga skalabilnim i pogodnim za distribuirane i paralelne sisteme, ali takođe zahtevaju dodatne mehanizme kao što su kolačići ili sesije da bi se održalo stanje i identifikovali korisnici između različitih zahteva.

Azure

Azure je platforma za cloud. Sire gledano to je kolekcija integrisanih usluga koje ce pomoci da brze napredujemo i uštedimo. Razlika izmedju cuvanja dokumenta i hostovanja aplikacije je ta sto aplikacije zahtevaju jak hardver koji unapred mora biti dostupan za mnogo vise korisnika nego sto trenutno koristi aplikacija. Azure je servis koji dozvoljava da hostujemo aplikaciju u cloudu tako da ne moramo da kupujemo i održavamo svoje servere. Problem je sto u pocetku placamo jak hardver, a imamo mali broj klijenata, ali vremenom cemo imati veliki broj klijenata. Prednost Azure-om je sto nam dozvoljava da placamo onoliko resursa koliko aplikacija koristi, a dozvoljava i skaliranje kako bi se zadovoljilo bilo koje povecanje broja korisnika. Na ovaj nacin je aplikacija optimalno skalabilna i uvek dostupna.

Azure ima sledece opcije/module:

1. Virtuelne masine - daju kontrolu nad kompletnim virtualnim masinama, ukljucujuci i operative sisteme(IaaS - Infrastructure as a service)
2. Veb sajtovi - pruza vam širok spektar aplikacija, frameworka i templejta kako biste mogli da kreirate velike i skalabilne web aplikacije, ali i brzu prezentaciju veb sajtova, uz efikasan razvoj i testiranje
3. Cloud servisi - su platform-as-a-service(PaaS) opcije podesene za kreiranje skalabilnih i stabilnih aplikacija, ali sa mnogo vise fleksibilnosti nego web sajtovi
4. Software as a service(SaaS) - pristup softverskim aplikacijama koje su hostovane na cloud platformi. Korisnici mogu koristiti aplikaciju putem web pregledača ili klijentske aplikacije, a infrastruktura i održavanje aplikacije su odgovornost pružaoca usluge. Primeri SaaS usluga ukljucuju Gmail, Office 365, Salesforce, Dropbox, itd.

Svaki Azure model ima svoju ulogu, a mi mozemo kombinovati tehnologije ili ih koristiti zasebno
VM se najcesce koriste za:

1. Razvoj i testiranje - mozemo ih koristiti da kreiramo jeftina okruzenja za testiranje i razvoj platforme koje mozemo ugasiti kada nam vise ne budu potrebne
2. Prebacivanje aplikacija na Azure(Lift-and-shift) - odnosi se na to da bukvalno mozemo da prebacimo svoju aplikaciju. Uzmemo VM i prebacimo je na Azure. Moguce je da ce ovde biti potrebna dodatna podesavanja u zavisnosti od razlicitih sistema
3. Prosirivanje datacentra - koristimo Azure VM da prosirimo postojeći data centar i povežemo ih da rade kao da su u lokalnoj mrezi

PaaS - fokusirani iskljucivo na aplikacije i podatke

Azure veb sajtovi dozvoljavaju da kreiramo veb aplikaciju koja ce podržati mnogo istovremenih korisnika, sa brzim rastom, bez mnogo administracije i održavanja infrastrukture, koji radi non-stop. Ogracenja koja se javljaju: nemamo administratorska prava pristupa tj. ne mozemo da stavimo nas specifičan softver. **Azure VM** daju dosta fleksibilnosti, ukljucujuci i administratorska prava pristupa, tako da sigurno mozemo napraviti skalabilnu aplikaciju, ali cemo morati da se pobrinemo i oko administracije.

Usluge koje Azure nudi:

1. VM - pokretanje aplikacije i servisa u virtuelnom okruzenju u oblaku, bez potrebe za fizickim serverima
2. Azure App Service - razvoj, upravljanje i hosting web aplikacija u cloudu, ukljucujuci web aplikacije, mobilne aplikacije i API servise
3. Azure Kubernetes Service(AKS) - upravlja Kubernetes klasterima u oblaku i omogucava automatsko skaliranje i upravljanje kontejnerizovanim aplikacijama
4. Azure SQL Database - nudi fully managed relacijsku bazu podataka u oblaku, koja omogucava brzo upravljanje i skaliranje baze podataka
5. Azure Cosmos DB - nudi globalno distribuiranu, multi-model bazu podataka, koja omogucava skalabilnost i visoku dostupnost
6. Azure Blob Storage - nudi skalabilno i pouzdano skladište za sve vrste podataka, ukljucujuci tekst, slike, audio i video fajlove
7. Azure Active Directory - omogucava upravljanje identitetima i pristupom u oblaku, sto je kljucno za bezbednost i upravljanje korisnicima
8. Azure DevOps - pruza alate za razvoj, testiranje i isporuku softvera u oblaku, ukljucujuci alate za upravljanje kodom, izgradnju i testiranje aplikacija

Usluge skladištenja podataka(3 kljucne usluge):

1. Azure Blob Storage - Blob Storage je skalabilna usluga skladištenja podataka koja omogucava skladištenje velikih kolicina teksta, slika, audio i video datoteka. Blobovi su organizovani u kontejnere i moze im se pristupiti preko HTTP ili HTTPS protokola. Blob Storage nudi razlicite nivoe dostupnosti i replikacije kako bi se obezbedila visoka dostupnost podataka
2. Azure Queue Storage - Queue Storage je skalabilna usluga za skladištenje poruka koje aplikacije mogu koristiti za slanje poruka jedna drugoj. Usluga omogucava skladištenje velikih kolicina poruka u redovima, koje se mogu citati i obradjivati po redosledu. Nudi mogucnost za upravljanje vremenom zivota poruka i automatsko brisanje poruka nakon sto se obradjuju
3. Azure Table Storage - Table Storage je skalabilna usluga za skladištenje podataka strukture kljuc-vrednost. Omogucava skladištenje velikih kolicina podataka u tabelama koje se mogu citati i pisati. Usluga je idealna za skladištenje velikih kolicina podataka u aplikacijama koje ne zahtevaju relacijsku bazu podataka. Nudi automatsko skaliranje i replikaciju podataka kako bi se obezbedila visoka dostupnost

DevOps - metodologija razvoja softvera koja se bavi automatizacijom procesa razvoja, testiranja, isporuke i upravljanja softverom. Ima za cilj smanjenje vremena izmedju pisanja koda i implementacije softvera u proizvodno okruzenje, uz visok nivo pouzdanosti i kvaliteta.

Testiranje softvera

Najcesce vrste testiranja softvera:

1. Unit testiranje - fokusira se na testiranje pojedinačnih delova koda - jedinica. Cilj je da se utvrdi da li jedinice rade ispravno, pre nego sto se one integrisu u vece komponente. Unit testovi se mogu pisati u razlicitim programskim jezicima i obicno se izvrsavaju u razvojnom okruzenju.

Programer ima predstavu kako bi trebao da se ponasa kod koji testira. On unosi testove kako bi proverio da li kod radi onako kako je ocekivano. To je nacin da programer proveri svoje ocekivanje o tome kako kod treba da se ponasa. Kada se programer bavi debugovanjem koda koji ne radi kako treba, testovi mogu biti veoma korisni u razjasnjanju sta je tacno uzrok problema.

Razlozi za koriscenje iako programer zna rezultat:

1. Automatska provera - cak i ako programer nije siguran da li je ispravno implementirao neku funkciju on moze pisati testove za proveru funkcionalnosti funkcije.
 2. Sigurnost u izmeni - kada se u kod uvedu promene, postoji rizik da se izazovu nezeljene posledice na drugim delovima koda. Omogucava brzu promenu nakon izmena.
 3. Lakse odrzavanje koda - sigurnost da izmene ne uticu na postojeću funkcionalnost. Lakse je pronaci i otkloniti greske u kodu.
 4. Dokumentacija koda - programer detaljno razmislja o funkcionalnosti koda. To dovodi do boljeg razumevanja koda i dokumentacije koda koju je lakse deliti sa drugim programerima.
- **Mokovanje objekta** - ideja je da se umesto stvarnih objekata koji predstavljaju zavisnosti, koriste lazni objekti, koji se nazivaju mock objekti. Koriste se kada se koristi jedinica koda koja zavisi od drugih objekata, a koji mozda jos uvek **nisu implementirani ili nisu dostupni** u okruzenju za testiranje. Koriscenjem mock objekta, moze se kreirati **virtuelni objekat koji simulira ponasanje stvarnog objekta**. Koristi se **kada stvarni objekti nisu dostupni ili je testiranje njihovih funkcionalnosti prekomplikovano ili preksupo**.
 - Primer: imamo klasu "Kalkulator" koji omogucava sabiranje metodom "Dodaj" i broj trebamo da povucemo iz klase "BazaPodataka" metodom "DohvatiPodatak". Ako BazaPodataka jos uvek nije dostupna mozemo odraditi mock objekta. Postavljamo ocekivanu vrednost koju vraca metoda iz klase BazaPodataka (vrednost 5). Prosledjujemo mock objekat 'BazaPodataka'. Kada smo pozvali metodu 'Dodaj' na objektu 'Kalkulator', mock objekat 'BazaPodataka' se koristio umesto stvarnog objekta, tako da se ocekivano ponasanje moze testirati izolovano od stvarnih zavisnosti. U ovom primeru smo koristili metodu 'Setup' na mock objektu da bismo postavili ocekivanje, ali mock objekti mogu biti mnogo slozeniji i mogu imati mnogo vise funkcionalnosti.


```

public class Kalkulator
{
    private BazaPodataka baza;

    public Kalkulator(BazaPodataka baza)
    {
        this.baza = baza;
    }

    public int Dodaj(int a, int b)
    {
        int rezultat = a + b;
        int podatak = baza.DohvatiPodatak();
        return rezultat + podatak;
    }
}

public class BazaPodataka
{
    public int DohvatiPodatak()
    {
        return 42;
    }
}

```

```

// Kreiranje mock objekta
var mockBaza = new Mock<BazaPodataka>();
mockBaza.Setup(baza => baza.DohvatiPodatak()).Returns(5);

// Kreiranje instance Kalkulatora i pozivanje metode
var kalkulator = new Kalkulator(mockBaza.Object);
int rezultat = kalkulator.Dodaj(3, 4);

// Provera rezultata
Assert.AreEqual(12, rezultat);

```

- Drugi primer testa

```

// Kreiranje instance klase Kalkulator
var kalkulator = new Kalkulator();

// Pozivanje metode Oduzmi
var rezultat = kalkulator.Oduzmi(5, 3);

// Provera da li je rezultat tačan
Assert.AreEqual(2, rezultat);

```

2. Integraciono testiranje - fokusira se na testiranje nacina na koji komponente sistema rade zajedno. Integracioni testovi se obicno izvrsavaju nakon unit testova i pre testiranja sistema u celini. Cilj je da se utvrdi da li su sve komponente sistema integrisane na pravi nacin i da li rade zajedno bez gresaka.

3. Sistemsko testiranje - fokusira se na testiranje sistema u celini, nakon sto su sve komponente integrisane. Cilj je da se utvrdi da li sistem radi ispravno u stvarnom okruzenju, ukljucujuci i situacije u kojima se korisnik moze susresti sa greskama ili prekidima.

4. Performansno testiranje - fokusira se na testiranje performansi sistema u razlicitim uslovima opterecenja. Cilj je da se utvrdi da li sistem moze da podrzi ocekivani broj korisnika i da radi ispravno u situacijama sa velikim opterecenjem.

5. Sigurnosno testiranje - fokusira se na testiranje sistema kako bi se utvrdilo da li su podaci bezbedni i zasticeni od neautorizovanog pristupa ili napada. Cilj je da se utvrdi da li sistem ima odgovarajuce mere zastite, kao sto su autentifikacija, autorizacija i enkripcija.

6. Testiranje prihvatljivosti - da li sistem zadovoljava potrebe korisnika i da li je lak za koriscenje. Cilj je da se utvrdi da li korisnici mogu da koriste sistem bez problema i da li je jednostavan za navigaciju.

NuGet je upravljac paketima za razvoj softvera u .NET okruzenju, koji omogucava instaliranje, azuriranje i uklanjanje biblioteke i alata. Ne spada u vrstu testiranja.

QA(Quality Assurance) - testiranje da bi se osiguralo da funkcioniše u skladu sa ocekivanjima korisnika, da ispunjava zahteve za kvalitetom i da je spreman za izbacivanje u produkciju. Obuhvata funkcionalno, performansno, sigurnosno i testiranje prihvatljivosti.

GIT:

Git je sistem za kontrolu verzija koji se koristi za praćenje promena u kodu projekta tokom vremena. Omogućava saradnju više programera, praćenje istorije izmena i olakšava povratak na prethodne verzije koda.

Branch(grana) - verzija projekta koja se razvija nezavisno od ostalih grana. Svaka grana ima svoju kopiju celokupnog projekta sa svim izmenama i istorijom commit-ova. Grane omogućavaju paralelno razvijanje različitih funkcionalnosti, ispravki gresaka ili eksperimentisanje, a zatim se mogu spojiti kako bi se kombinovali rezultati na različitim granama.

git init - inicijalizovanje repozitorijuma u trenutnom direktorijumu

git branch ime_grane - kreiranje nove grane

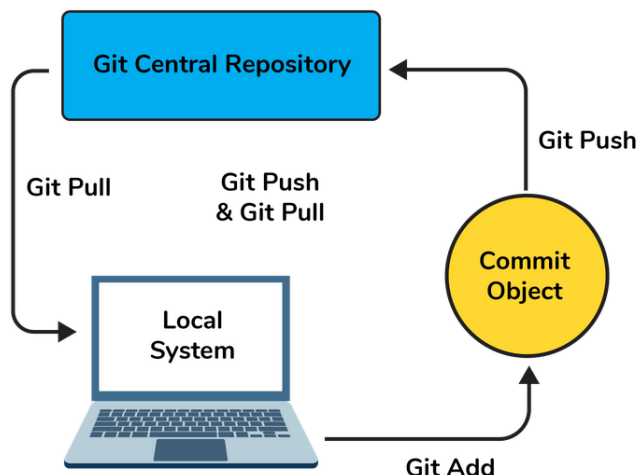
git checkout ime_grane - Trenutni radni direktorijum se prebacuje na drugu granu

git add ime_fajla - dodavanje izmene u indeks(staging area) u Gitu

git commit -m "Poruka commit-a" - pravljenje commit-a sa porukom

git push origin ime_grane - objavljivanje promene na udaljenom repozitorijumu određene grane

git pull origin ime_grane - da bismo povukli najnovije promene sa udaljenog repozitorijuma za granu



Git rollback se odnosi na povratak na prethodnu verziju ili promenu u Git repozitorijumu. Komanda koja se koristi za rollback u Git-u je **git revert**

git revert <commit> - pravi novi commit koji ponistava promene unete u odredjenom commit-u. Nakon ove komande Git ce napraviti novi commit koji se primenjuje na trenutnu granu, tako da se stanje projekta vrati na prethodno stanje. Ako zelimo da ponistimo promene iz commit-a sa hash vrednoscu "abc123" komanda je **git revert abc123**.

Git revert ne brise istoriju commit-ova, vec kreira novi commit koji ponistava promene. Ovo je siguran nacin za povratak na prethodno stanje i ocuvanje istorije promena u Git repozitorijumu.

git clone link_repozitorijuma - klonira postojeći repozitorijum na lokalni repozitorijum

Git config - za podesavanje Git konfiguracija na lokalnom ili globalnom nivou. Pomocu ove komande moze da se postavi korisnicko ime, email, konfigurise resavanje merge konflikta, konfigurise editor...

git config --global user.name "John Doe"

git config --global user.email "johndoe@gmail.com"

Ove informacije će biti korisne za autentifikaciju autora commit-ova

HEAD je pokazivac koji označava trenutnu verziju projekta(poslednji commit).

git checkout HEAD~1 prebacivanje na prethodni commit.

git status - pracenje stanja projekta, pracenje promena i otkrivanje eventualnih problema. Moze prikazati: trenutnu granu, promene u radnom direktorijumu, promene u indeksu(staging area) tj. promena nad nekim fajlovima, informacije o merge-u, informacije o nesnimljenim promenama...

Git merge - za spajanje promena iz jedne grane u drugu granu.

```
# Prebacite se na granu "main"
git checkout main

# Spojite granu "feature" u granu "main"
git merge feature
```

Merge conflict - kada Git ne može automatski spojiti promene iz dve grane zbog sukoba konflikta u kodu. To se desava kada je isti deo datoteke izmenjen na obe grane koje se spajaju.

Oznake <<<<<< **HEAD**, ===== i >>>>>> **feature** označavaju sukob spajanja. Potrebno je ručno rešiti sukob tako što biramo koje promene želimo da zadržimo.

git branch -d ime_grane - brisemo granu kada završimo rad na njoj i više nam nije potrebna

git reflog - lista zabeleženih akcija gde svaki red predstavlja jedan unos refloga i obično sadrži informacije: SHA-1 hash commit-a, HEAD pokazivaca, akcije(prebacivanje grane, izvršavanje commit-a) i vremena kada se desila akcija. Koristimo ga da bi smo se vratili na prethodne commit-ove, oporavak izgubljenih promena ili pracenje istorije kretanja kroz repozitorijum.

git pull origin master - azurira našu lokalnu granu 'master' sa najnovijim promenama sa udaljenog repozitorijuma. Istovremeno radi 'git fetch' i 'git merge'

git fetch - hvata sve najnovije promene, ali bez automatskog merge tih promena sa lokalnom granom. Umesto toga, azurira informacije o udaljenim granama i commit-ovima u našem lokalnom repozitorijumu omogućavajući nam da vidimo šta se promenilo u udaljenom repozitorijumu. Ako nismo spremni za merge možemo videti promene i proveriti konflikte pre spajanja.

git stash - koristi se za privremeno skladištenje nesnimanog radnog direktorijuma i indeksa

Version control system (VCS) - sistem za kontrolu verzija koji se koristi za upravljanje promenama u kodu ili fajlovima tokom vremena. Omogućava pracenje, organizovanje i upravljanje različitim verzijama datoteka i projekata.

Git repository - mesto gde se nalaze svi git fajlovi

git clean - uklanja untracked fajl iz radnog direktorijuma

git tag [commitID] - dodavanje taga commit-u

git commit -am "For All Changes" - izvršavanje commita svih promena koje su već indeksirane (staged) i za dodavanje poruke commita. To znaci da nije potrebno raditi 'git add' to jest eksplicitno dodati na indeks.

Ako ne radimo ovako potrebno je dodati na indeks tj. **git add** pa onda **git commit -m "poruka"**. Dodavanje na indeks (staging) je proces kojim se označavaju određene modifikacije datoteka za uključivanje u sledeći commit. Kada izvršite `git add` za određene datoteke, one se premeštaju sa radnog direktorijuma u "staging area" (indeks). Staging area je privremena oblast u kojoj se pripremaju promene koje će biti deo sledećeg commit-a. Kada izvršite `git commit`, Git će uzeti sve datoteke koje su se nalazile u staging area-i i izvršiti commit, čime će trajno zabeležiti promene u repozitorijumu. Samo datoteke koje su bile stage-ovane (dodate na indeks) će biti deo commit-a, dok će ostale modifikovane datoteke koje nisu bile stage-ovane biti ignorisane.

Rebasing(alternativa za merge) u Gitu je proces kojim se menja istorija commita tako da izgleda kao da ste napravili promene na najnovijem commitu, umesto da se samo dodaju novi commiti na postojeću granu. To se postiže tako što se uzima serija commita iz jedne grane i primenjuje na drugu granu. Rebasing se često koristi u situacijama kada želite da integrišete promene iz jedne grane u drugu, ili kada želite da očistite istoriju commita tako da izgleda linearno i čitljivo.

Git vs GitHub

Git	GitHub
This is a distributed version control system installed on local machines which allow developers to keep track of commit histories and supports collaborative work.	This is a cloud-based source code repository developed by using git.
This is maintained by "The Linux Foundation".	This was acquired by "Microsoft"
SVN, Mercurial, etc are the competitors	GitLab, Atlassian BitBucket, etc are the competitors.

Git is a software	GitHub is a service
Git can be installed locally on the system	GitHub is hosted on the web
Provides a desktop interface called git GUI	Provides a desktop interface called GitHub Desktop.
It does not support user management features	Provides built-in user management

Ostalo:

ASP.NET vs ASP.NET Core

1. ASP.NET je baziran na windows-only tehnologijama i koristi .NET Framework, dok je ASP.NET Core dizajniran da bude prenosim i može se pokretati na različitim platformama (windows, linux, macOS). ASP.NET Core koristi .NET Core
 2. ASP.NET Core je potpuno open-source, što znači da je dostupan javnosti i može se besplatno koristiti i prilagođavati, dok ASP.NET nije u potpunosti.
 3. Performanse: ASP.NET Core je dizajniran da bude brzi i skalabilniji u odnosu na ASP.NET. Ima manji memorijski i resursni utrosak, što ga čini pogodnim za moderna i cloud okruženja.
 4. Cross-platform podrška: ASP.NET Core podržava cross-platform izgradnju aplikacija, što znači da se može pokretati na različitim OS. Ovo omogućava razvoj i implementaciju aplikacija na širem spektru platformi.
 5. Modularnost: ASP.NET Core je izgrađen na modularnom principu, gdje su funkcionalnosti podjeljene u nekoliko paketa (nuget packages). Ovo omogućava programerima da koriste samo one pakete koji su im potrebni, što rezultira manjim veličinama aplikacija i boljom kontrolom nad zavisnostima.
 6. Cloud-ready: ASP.NET Core ima ugrađenu podršku za razne cloud platforme i mikroservise. Pruža integraciju sa Azure cloud servisima i olakšava razvoj i deploy aplikacija u cloud okruženjima.
- Osobine i ciljevi ASP.NET Core-a su usmereni ka modernom web razvoju, sa fokusom na performanse, skalabilnosti, prenosivost i otvorenost. On pruža razvojne alate i funkcionalnosti koje olakšavaju izgradnju moderne web aplikacije i mikroservisa. Ima cloud-ready funkcionalnosti i nudi više mogućnosti za razvoj modernih aplikacija, podržava cross-platform implementaciju.

ASP.NET Core vs .NET Core

ASP.NET Core zavisi od .NET Core platforme i koristi njene funkcionalnosti i biblioteke za izgradnju web aplikacija. Najčešće se koriste zajedno kako bi se postigla potpuna podrška za web razvoj.

1. Namena i fokus: .NET Core je otvoreni izvor, prenosiva, modularna platforma za razvoj softvera koja podržava različite tipove aplikacija, uključujući web aplikacije, desktop aplikacije i usluge. S druge strane, ASP.NET Core je web okvir (framework) koji se gradi na .NET Core platformi i specifično je dizajniran za izgradnju web aplikacija i usluga.
2. Obuhvat - širina funkcionalnosti i podrške: .NET Core obuhvata širi spektar funkcionalnosti i biblioteka koje su potrebne za razvoj različitih vrsta aplikacija. To uključuje podršku za razne jezike (C#, VB.NET, F#), obradu podataka, mrežno programiranje, sigurnost i mnoge druge funkcionalnosti. ASP.NET Core je fokusiran samo na web razvoj i pruža specifične funkcionalnosti za izgradnju web aplikacija kao što su rutiranje, MVC (Model-View-Controller) arhitektura, podrška za API-je, sigurnost...
3. Arhitektura i sintaksa: .NET Core ima generičku arhitekturu i pruža zajednički set biblioteka, jezika i alata za različite vrste aplikacija. ASP.NET Core koristi taj zajednički set biblioteka i alata, ali dodaje svoje specifične komponente i sintaksu za web razvoj. Na primer, u ASP.NET Core-u se koristi koncept ruta (routing) za mapiranje URL-ova na odgovarajuće kontrolere i akcije.
4. Ciljna publika: .NET Core je namenjen programerima koji žele da razvijaju različite vrste aplikacija na .NET platformi, dok je ASP.NET Core usmeren na programere koji žele da se fokusiraju na web razvoj i izgradnju web aplikacija.

Hackaton - takmicenje u kojem timovi programera, dizajnera zajedno razvijaju softverske projekte u odredjenom vremenskom periodu.

OWASP(Open Web Application Security Project) - organizacija koja se fokusira na poboljšanje sigurnosti aplikacija na webu. Definise 10 najvažnijih ranjivosti aplikacija. To su najcesce slabosti koje napadaci mogu iskoristiti da bi napali web aplikacije. Siroko su dokumentovane i pružaju smernice i preporuke za njihovo otkrivanje, prevenciju i popravku. Prepoznavanje i resavanje ovih ranjivosti je ključno za izgradnju sigurnih web aplikacija.

1. Injection (Injekcije): Ova ranjivost se javlja kada se nevalidni podaci ubacuju u upite ili komande koji se izvršavaju na strani servera.
2. Broken Authentication (Nedostatak autentifikacije): Ova ranjivost se odnosi na slabosti u mehanizmima autentifikacije i upravljanja sesijama, što može dovesti do krađe identiteta ili neovlašćenog pristupa.
3. Sensitive Data Exposure (Izlaganje osetljivih podataka): Ovde se misli na situacije u kojima osetljivi podaci, kao što su lozinke ili finansijski podaci, nisu adekvatno zaštićeni i mogu biti otkriveni napadačima.
4. XML External Entities (Eksterni entiteti XML-a): Ova ranjivost se javlja kada se aplikacija nepravilno obrađuje XML ulaz, što može dovesti do izvršavanja udaljenih zahteva ili otkrivanja osetljivih podataka.
5. Broken Access Control (Nedostatak kontrole pristupa): Ova ranjivost se odnosi na situacije u kojima aplikacija nepravilno upravlja pravima pristupa, omogućavajući napadačima da dobiju neovlašćen pristup resursima.
6. Security Misconfiguration (Nevažeća konfiguracija sigurnosti): Ova ranjivost se odnosi na situacije u kojima aplikacija ima neispravne ili nedovoljno sigurnosne podešavanja, što olakšava napadačima pronalaženje slabosti.
7. Cross-Site Scripting (XSS): Ova ranjivost se javlja kada se nevalidni podaci ubacuju u veb stranice i omogućavaju napadačima izvršavanje zlonamernog koda na strani korisnika.
8. Insecure Deserialization (Nesigurna deserijalizacija): Ova ranjivost se odnosi na situacije u kojima se podaci deserijalizuju na nebezbedan način, što omogućava napadačima da izvrše udaljeni kod.
9. Using Components with Known Vulnerabilities (Korišćenje komponenti sa poznatim ranjivostima): Ova ranjivost se javlja kada se koriste biblioteke ili komponente koje sadrže poznate sigurnosne propuste.
10. Insufficient Logging & Monitoring (Nedovoljno beleženje i praćenje): Ova ranjivost se odnosi na situacije u kojima aplikacija nema adekvatne mehanizme za praćenje i beleženje aktivnosti, što otežava otkrivanje i reagovanje na sigurnosne incidente.

Spear phishing je oblik napada putem elektronske pošte (e-mail) koji se ciljano usmerava prema određenim pojedincima ili organizacijama. Ovaj napad se razlikuje od tradicionalnog phishinga po tome što je prilagođen specifičnim metama i zahteva detaljnije istraživanje. Kod spear phishinga, napadači obično prikupljaju informacije o svojim ciljevima kako bi kreirali ubedljive i prilagođene poruke e-pošte. Ove poruke se često maskiraju kao legitimne i poznate izvore, kao što su banka, kompanija ili kolega, kako bi prevarile žrtvu da otkrije poverljive informacije, kao što su lozinke, korisnička imena ili bankovni podaci. Napadači koriste razne tehnike manipulacije i društvenog inženjeringa kako bi povećali šanse da žrtva nasledi njihovu prevaru. To može uključivati upotrebu personalizovanih podataka, lažnih hitnih situacija, imitaciju autoriteta ili upotrebu prevarantskih veza ili privitaka koji vode do zlonamernih veb stranica ili softvera. Cilj spear phishinga je obično krađa poverljivih informacija, kao što su finansijski podaci, korporativne tajne ili pristupni podaci. Ovaj oblik napada često cilja na pojedince u ključnim ulogama, kao što su menadžeri, zaposleni u finansijama ili IT stručnjaci, jer se pretpostavlja da poseduju važne informacije ili pristup sistemu.

Framework vs biblioteka

Razlika je u načinu na koji se koriste i kako organizuju razvoj aplikacija, koliko strukturu i kontrolu diktiraju programeru. Biblioteka pruža alate i funkcionalnosti koje programer može koristiti po potrebi, dok framework definiše strukturu i pravila za razvoj aplikacije.

Biblioteka je kolekcija već gotovih funkcionalnosti, alata ili komponenti koje programer može koristiti u svom kodu. Biblioteka pruža skup rešenja za određene probleme ili funkcionalnosti, ali ne diktira sveukupnu strukturu i tok aplikacije. Programer ima veću slobodu u izboru kako će koristiti biblioteku i kako će organizovati kod.

Framework je sveobuhvatniji i strukturiraniji set alata, pravila i konvencija koji definišu arhitekturu aplikacije. Framework obično definiše strukturu, protokole i obrasce ponašanja aplikacije. Programer mora pratiti specifične smernice i pravila koje framework postavlja kako bi razvijao aplikaciju. Framework pruža gotove module i funkcionalnosti za određene delove razvoja aplikacije i često ima unapred definisanu hijerarhiju ili tok rada.

Razlozi zbog kojih se koristi framework .NET zavise od različitih faktora kao što su zahtevi projekta, tehnološka infrastruktura, timski kapaciteti, podrška i druge specifične potrebe. Ako su vaši zahtevi u skladu sa mogućnostima i prednostima koje .NET pruža on se bira.

1. Višenamenski i širok spektar podrške: .NET omogućava razvoj različitih vrsta aplikacija, uključujući desktop, web, mobilne i cloud aplikacije. Takođe podržava više jezika poput C#, Visual Basic, F# i drugih, omogućavajući programerima da biraju jezik koji im najviše odgovara.
2. Integracija sa Microsoft tehnologijama: .NET je razvijen od strane Microsofta i nudi duboku integraciju sa drugim Microsoft tehnologijama kao što su Windows operativni sistem, SQL Server baza podataka, Azure cloud platforma i drugi alati i servisi koji su deo Microsoft ekosistema. To olakšava razvoj i integraciju aplikacija u Microsoft okruženju.
3. Obimna biblioteka klasa: .NET dolazi sa obimnom bibliotekom klasa (Class Library) koja sadrži mnoge gotove funkcionalnosti i komponente koje programeri mogu da koriste pri razvoju aplikacija. Ova biblioteka pruža rešenja za često korišćene zadatke poput obrade podataka, rad sa bazama podataka, manipulaciju fajlovima, rad sa mrežom i još mnogo toga.
4. Bezbednost i performanse: .NET je dizajniran sa fokusom na bezbednost i performanse. Ima ugrađene mehanizme zaštite podataka i prevenciju sigurnosnih propusta, kao i optimizacije za brzo izvršavanje koda. Takođe podržava višenitnost i asinhrono izvršavanje, što može poboljšati performanse aplikacija.
5. Velika zajednica i podrška: .NET ima veliku zajednicu programera i razvojnih timova koji pružaju podršku, resurse, biblioteke trećih strana i razne alate koji olakšavaju razvoj aplikacija. Postoje brojni forumi, blogovi, dokumentacija i onlajn resursi koji su dostupni za učenje i rešavanje problema.

Angular se smatra frameworkom jer pruža sveobuhvatni okvir za razvoj aplikacija. On definiše arhitekturu, pravila i konvencije za izgradnju aplikacija. Angular ima svoje specifičnosti, kao što je hijerarhijska struktura komponenti i upotreba TypeScript jezika. Programer mora slediti strukturu i pravila koje postavlja Angular.

React se smatra bibliotekom jer pruža komponente i alate za izgradnju korisničkog interfejsa. React ne diktira potpunu strukturu aplikacije i ostavlja programeru veću fleksibilnost u organizaciji koda. Programer može koristiti React komponente na određeni način, ali ima slobodu da odabere ostale delove arhitekture aplikacije.

Zasto bi neko koristio Angular:

1. Moćan i kompleksan: Angular je moćan i kompleksan razvojni okvir koji je posebno dizajniran za izgradnju skalabilnih i visoko performantnih veb aplikacija. Nudi sveobuhvatne mogućnosti za upravljanje strukturom, logikom i korisničkim interfejsom aplikacije.
2. Reaktivni pristup: Angular koristi reaktivni pristup, što znači da omogućava praćenje promena podataka i automatsko ažuriranje korisničkog interfejsa kada dođe do promena. Ovo olakšava upravljanje stanjem aplikacije i pruža bolje korisničko iskustvo.
3. Komponentna arhitektura: Angular se oslanja na komponentnu arhitekturu, što znači da aplikaciju gradite odvojenim komponentama koje su ponovno upotrebljive i lako održive. To olakšava organizaciju koda, testiranje i timski rad.
4. Velika zajednica i podrška: Angular ima veliku zajednicu programera koja pruža podršku, resurse, biblioteke i alate. Postoji obilje dokumentacije, tutorijala, foruma i blogova koji vam mogu pomoći u učenju i rešavanju problema.
5. Integracija sa drugim alatima i bibliotekama: Angular se lako integriše sa drugim popularnim bibliotekama i alatima, kao što su TypeScript, RxJS, Material Design, Redux i drugi. Ovo vam omogućava da iskoristite već postojeće resurse i alate za efikasan razvoj aplikacija.
6. Podrška za različite platforme: Angular može biti korišćen za izgradnju veb aplikacija, ali takođe podržava izgradnju mobilnih aplikacija putem alata kao što je Ionic framework. Ovo omogućava razvoj aplikacija za različite platforme koristeći iste veštine i alate.

Bezbednost aplikacije je vazna kako bi se:

1. Zastitili korisnici (licni, finansijski, poslovni podaci) od kradje, zloupotrebe ili neovlasćenog pristupa
2. Prevencija hakovanja i zlonamernih aktivnosti
3. Poverenje korisnika - ako misle da nisu sigurni bice obeshrabreni da koriste aplikaciju
4. Uskladenost sa propisima i regulativama - standardi bezbednosti: podaci o zdravstvenim kartonima, finansijski podaci...

Zasto zelimo da ubrzamo aplikaciju:

Bolje korisnicko iskustvo, efikasnost u obradi zahteva i skalabilnost (može da podrži veći broj korisnika ili opterećenje), smanjenje resursa koji se koriste, konkurentska prednost (nasa aplikacija ili neka druga).

Nacini za ubrzanje aplikacije:

Vazno je identifikovati kritične delove aplikacije koje uticu na performanse i da se primene odgovarajuće tehnike i alati. Pобољsanje performansi može biti postignuto optimizacijom hardverske infrastrukture.

1. Baza podataka - optimizacija upita, indeksiranje, normalizacija sema baze podataka
2. Serverska logika - smanjenje broja zahteva, optimizacija algoritama, koriscenje kesiranja rezultata ili resursa i eliminisanje suvisnih operacija
3. Mrežna komunikacija - optimizacija mreznog saobracaja, smanjenje broja zahteva ili koriscenje kompresije podataka, provera protokola koje koristimo
4. Klijentska strana - kesiranje resursa, minimalizacija zahteva za serverom, smanjenje velicine prenosa podataka, paralelno preuzimanje resursa i optimizacija prikaza
5. Hardverska infrastruktura - brzi serveri, balansiranje opterećenja, skaliranje horizontalno i vertikalno, upotreba kesa na nivou servera

Automatizacija je proces u kojem se koriste tehnološki alati i sistemi kako bi se smanjila ljudska interakcija i povećala efikasnost u izvršavanju određenih zadataka ili procesa. Automatizacija se može primeniti u različitim oblastima, kao što su proizvodnja, informacione tehnologije, finansije, logistika, marketing i mnoge druge.

Cilj automatizacije je da olakša i ubrza rutinske ili repetitivne zadatke, smanji mogućnost grešaka, poboljša efikasnost i produktivnost, smanji troškove i omogući fokusiranje ljudskih resursa na složenije ili kreativnije zadatke.

Uz pomoć različitih tehnologija kao što su softverski alati, robotika, veštačka inteligencija (AI) i mašinsko učenje (ML), automatizacija može obuhvatiti širok spektar aktivnosti, od jednostavnih zadataka kao što su automatsko slanje e-poruka ili generisanje izveštaja, do kompleksnih procesa kao što su proizvodnja na traci trake ili upravljanje kompleksnim sistemima u realnom vremenu.

Automatizacija može doneti brojne prednosti kao što su poboljšana efikasnost, povećana tačnost, smanjenje grešaka, brži odziv, optimizacija resursa i smanjenje troškova, a koristi se u raznim industrijskim sektorima kako bi se postigle konkurentne prednosti i unapredila poslovna performansa.

Postoje mnogi **tipovi arhitekture softvera** koji se koriste u razvoju aplikacija. Evo nekoliko često korišćenih tipova:

1. **Slojevita arhitektura (Layered Architecture):** Ova arhitektura deli aplikaciju na različite slojeve, kao što su prezentacioni sloj (UI), poslovni sloj (Business Logic) i sloj podataka (Data Access). Svaki sloj ima svoju odgovornost i komunicira sa susednim slojevima preko definisanih interfejsa.
2. **Mikroservisna arhitektura (Microservices Architecture):** Ova arhitektura se fokusira na izgradnju aplikacije kao skupa malih, nezavisnih servisa koji rade zajedno. Svaki mikroservis se fokusira na određeni poslovni proces i može se razvijati, testirati i deploy-ovati nezavisno.
3. **Čista arhitektura (Clean Architecture):** Ova arhitektura promoviše razdvajanje poslovnih pravila od specifičnosti okruženja. Sastoji se od unutrašnjih slojeva koji definišu logiku aplikacije, dok se spoljni slojevi bave komunikacijom sa spoljnim sistemima (baza podataka, korisnički interfejs itd.).
4. **P2P arhitektura (Peer-to-Peer Architecture):** Ova arhitektura omogućava direktan komunikaciju između računara (peera) u mreži, bez potrebe za centralnim serverom. Svaki peer može delovati kao klijent i server istovremeno.
5. **Cloud arhitektura (Cloud Architecture):** Ova arhitektura se odnosi na dizajn aplikacije koja se izvodi u cloud okruženju, kao što su cloud provajderi poput AWS, Azure ili Google Cloud. Ona koristi skalabilnost, raspoloživost i druge karakteristike cloud platformi kako bi podržala aplikaciju.
6. **CORS (Cross-Origin Resource Sharing):** CORS je mehanizam koji omogućava veb aplikacijama da zahtevaju resurse sa drugih domena, odnosno da premoste ograničenja bezbednosti pretraživača. To omogućava veb stranicama da pristupaju resursima na drugim domenima.
7. **MVC arhitektura (Model-View-Controller):** Ova arhitektura deli aplikaciju na tri glavna komponente - Model (koja predstavlja podatke i poslovnu logiku), View (koja prikazuje korisnički interfejs) i Controller (koji upravlja interakcijom između Modela i View-a).
8. **MVP arhitektura (Model-View-Presenter):** Slično kao MVC, MVP arhitektura takođe deli aplikaciju na Model, View i Presenter komponente. Presenter upravlja logikom aplikacije i komunikacijom između Modela i View-a.
9. **MVVM arhitektura (Model-View-ViewModel):** Ova arhitektura se često koristi u razvoju aplikacija zasnovanih na XAML (poput WPF i Xamarin). ViewModel predstavlja vezu između View-a i Modela, pružajući podatke i akcije potrebne za prikazivanje i upravljanje podacima.

10. Event-Driven arhitektura: Ova arhitektura se fokusira na razmenu događaja između komponenti sistema. Komponente šalju i primaju događaje, reagujući na njih na odgovarajući način.
11. Monolitna arhitektura: Ova arhitektura se odnosi na tradicionalni pristup gde je cela aplikacija smeštena u jednom monolitnom softverskom paketu. Sve komponente rade zajedno u jednom izvršnom okruženju.
12. Serverless arhitektura: Ova arhitektura se oslanja na cloud provajdere koji omogućavaju izvršavanje funkcionalnosti bez potrebe za održavanjem servera. Logika aplikacije se implementira kao "serverless" funkcije koje se izvršavaju samo kada su potrebne.

Literatura:

<https://github.com/PredragDj99>

<https://www.mcloud.rs/blog/kako-da-koristite-rest-api-vodic-za-pocetnike/>

<https://www.webprogramiranje.org/web-servisi-osnove/>

<https://gocoding.org/bs/%C5%A1ta-su-mirni-web-servisi/>

<https://www.webprogramiranje.org/async-await-sintaksa/>

<https://www.manuelradovanovic.com/2016/05/asinhrono-async-i-await-visenitno.html>

<https://startit.rs/nosql-igraliste-ili-tri-nosql-baze-podataka-i-njihove-karakteristike/>

<https://www.ucim-programiranje.com/2019/02/dizajn-klase/>

<https://www.programiz.com/dsa/data-structure-types>

https://www.codeblog.rs/clanci.php?p=strukture_podataka

https://www.codeblog.rs/clanci.php?p=uvod_u_objektno_orijentisano_programiranje

https://www.codeblog.rs/clanci.php?p=relacione_baze_podataka

<https://mbranko.github.io/webkurs/#/1>

<https://blog.extreme.rs/2015/03/23/sta-je-uopste-microsoft-azure/>

Sajstovi za učenje:

<https://intellipaat.com/blog/tutorials/>

<https://intellipaat.com/blog/interview-questions/>

<https://www.programiz.com/dsa/data-structure-types>

eLithmus - neko testiranje, ima resenih testova

<https://www.indiabix.com/c-sharp-programming/questions-and-answers/>

<https://github.com/Ebazhanov/linkedin-skill-assessments-quizzes/tree/main>

<https://books.goalkicker.com/>

<https://www.interviewbit.com/git-interview-questions/>

<https://www.simplilearn.com/tutorials/git-tutorial/git-interview-questions>

<https://www.tutorialspoint.com/codingground.htm>

<https://refactoring.guru/design-patterns/csharp>

https://www.codeblog.rs/clanci.php?p=relacione_baze_podataka

<https://www.javatpoint.com/microsoft-azure-interview-questions>

<https://www.geeksforgeeks.org/>

<https://www.programiz.com/dsa/data-structure-types>

<https://www.w3schools.com/>

<https://www.tutorialspoint.com/How-to-read-inputs-as-integers-in-Hash>

<https://www.hackerrank.com/> -- easy i medium radi

<https://app.codility.com/programmers/> -- zadaci za vezbu